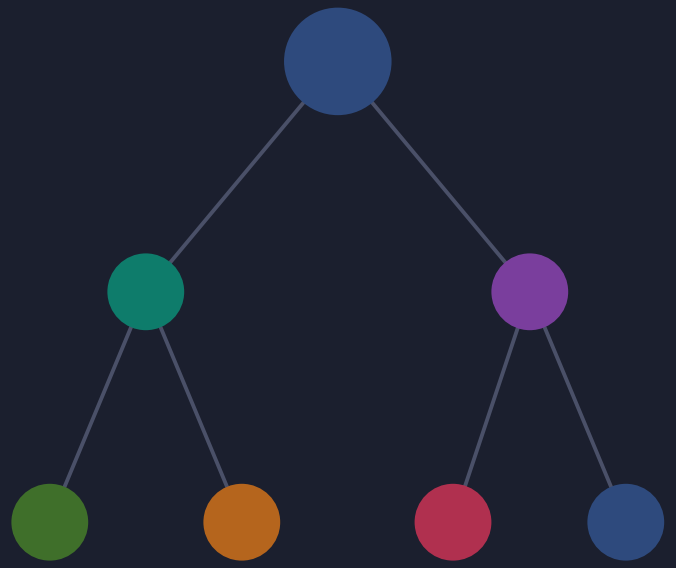
A COMPLETE TECHNICAL TUTORIAL

Machine Learning from Scratch



Foundations, Linear Regression, Logistic Regression, Decision Trees, Tuning Decision Trees and Cluster Analysis, explained in plain language with real Python code and real output

Foundations | **Linear Regression** | **Logistic Regression**

Decision Trees | **Tuning Trees** | **Clustering**

How to read this tutorial

Every idea is taught three ways: a picture of what is happening, the Python that makes it happen, and the output that proves it happened.

1 Concept slides

A plain language explanation of the idea, its moving parts and the assumptions hiding underneath. There are no equations anywhere in this deck; every mechanism is described in words and pictures instead.

2 Illustration slides

Drawn diagrams with annotation boxes that point at exactly the part of the picture being described. Read the numbered boxes in order, following each connector to its target.

3 Python slides

A dark code panel with line numbers. Coloured brackets on the right edge group lines together and link each group to an explanation box, so you always know which lines do what.

4 Output slides

A console panel showing the genuine output produced when this exact code was executed with scikit learn 1.9. Highlighted rows are linked to callouts that interpret every important number.

5 Chart slides

Native PowerPoint charts built from the real results, so you can click any point and inspect the value. Numbered notes beside each chart explain what the shape of the curve means.

6 Checkpoints

Short questions at the end of each section. Cover the answer box, commit to a choice, then check the reasoning. Retrieval like this is what makes the material stick.

The learning path through six sections

Each section reuses the vocabulary and the outputs of the sections before it, so read them in order the first time through.

1 Foundations

Data, splits, scaling, pipelines, overfitting, cross validation

2 Linear Regression

Predicting a number with a weighted sum of features

3 Logistic Regression

Turning a weighted sum into a probability for classes

4 Decision Trees

Learning a sequence of yes or no questions

5 Tuning Decision Trees

Controlling tree growth so it generalises

6 Cluster Analysis

Finding groups when there is no target at all

Supervised learning with a numeric target > supervised learning with a class target > rule based models and their tuning > unsupervised learning

What you will be able to do by the end

1 → 2 → 3 → 4 → 5 → 6

1 Frame

Recognise whether a business question is regression, classification or clustering, and choose the right family of model for it.

2 Prepare

Split data honestly, scale and encode features inside a pipeline, and avoid every common form of data leakage.

3 Fit

Train linear, logistic, tree and clustering models with scikit learn and read every attribute they learn.

4 Evaluate

Interpret MAE, RMSE, R squared, accuracy, precision, recall, ROC AUC, silhouette and adjusted Rand scores correctly.

5 Tune

Use validation curves, pruning, grid search, randomised search and nested cross validation to choose settings.

6 Explain

Translate coefficients, odds multipliers, split rules and cluster profiles into plain business language.

The toolkit used throughout

Everything runs in standard Python. These are the exact versions the outputs in this deck were produced with.

Library	Version	What we use it for
numpy	2.5.3	Fast arrays, random number generation, vectorised arithmetic
pandas	3.0.5	Tables of data with named columns, summaries, grouping
scikit learn	1.9.1	Every model, split, scaler, metric and search object (imported as sklearn)
scipy	1.18.1	Random distributions for search and the hierarchical clustering linkage
Built in datasets	sklearn	Breast cancer, iris and wine data ship with the library, so no downloads
Synthetic data	numpy	Exam scores, house prices and customers are simulated so we know the truth

Why simulate data?

When we generate the data ourselves we know the true rules that produced it. That lets us check whether a model recovered the truth, which real data can never tell us.

SECTION

1

Foundations of Machine Learning

The vocabulary, workflow and safeguards that every model in this tutorial depends on

- What learning from data actually means
- Supervised versus unsupervised problems
- Anatomy of a dataset and the machine learning workflow
- Honest evaluation: train and test splits, stratification
- Scaling, encoding, imputation and pipelines
- Overfitting, underfitting and the bias variance trade off
- Cross validation and baselines

What machine learning really is

Machine learning is the practice of letting an algorithm discover a useful rule from examples, instead of a person writing that rule by hand.

1 Learning from examples

We show the algorithm many past cases where we already know the answer. It searches for a pattern that links the inputs to those answers as closely as possible.

2 A model is a learned rule

The output of training is a model: a compact rule, such as a set of weights or a tree of questions, that can turn any new set of inputs into a prediction.

3 Generalisation is the goal

Memorising the training examples is easy and worthless. The only thing that matters is how well the rule performs on new cases it has never seen before.

4 Learning is optimisation

Every algorithm defines what a good rule looks like, such as small prediction errors, and then adjusts the rule step by step until it is as good as it can get.

5 Data quality sets the ceiling

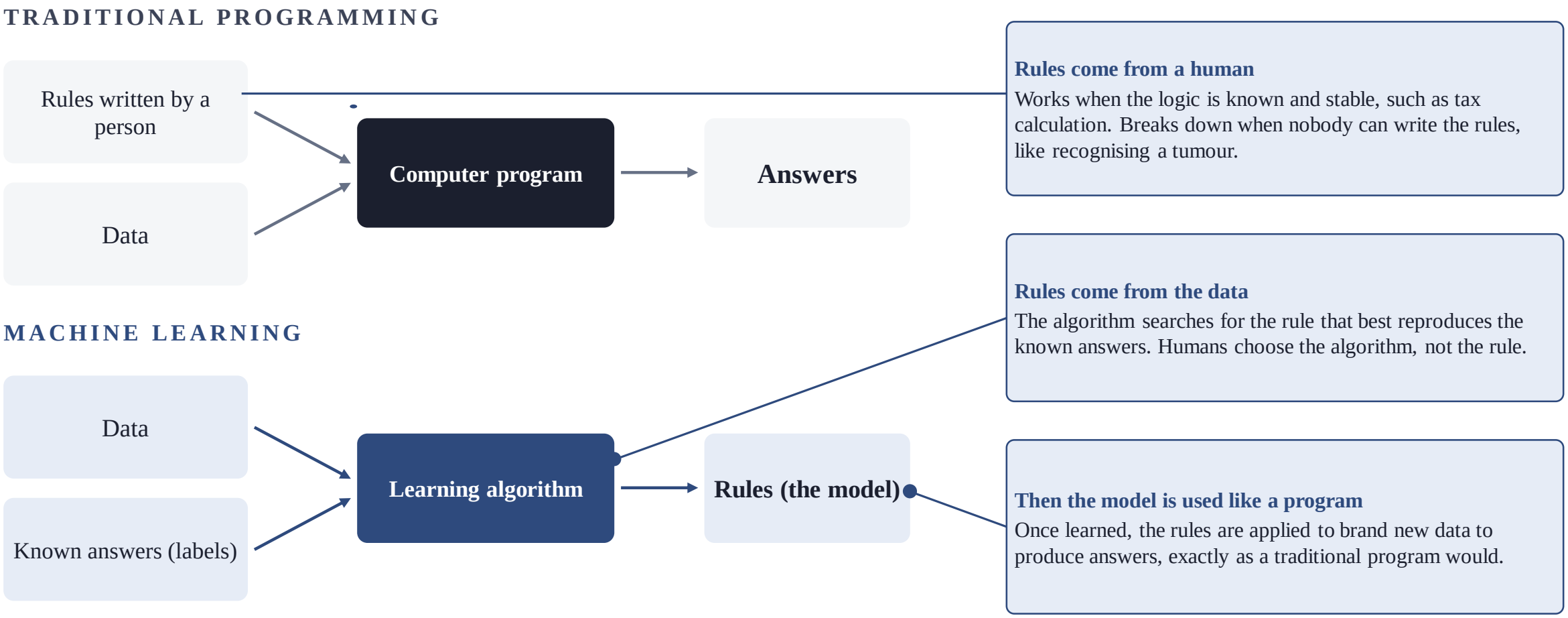
A model can only find patterns that exist in the data. Biased, mislabelled or unrepresentative examples produce a confident but wrong rule.

6 Uncertainty never disappears

Real outcomes contain noise that no feature explains. A good model captures the signal and accepts the noise, rather than chasing it.

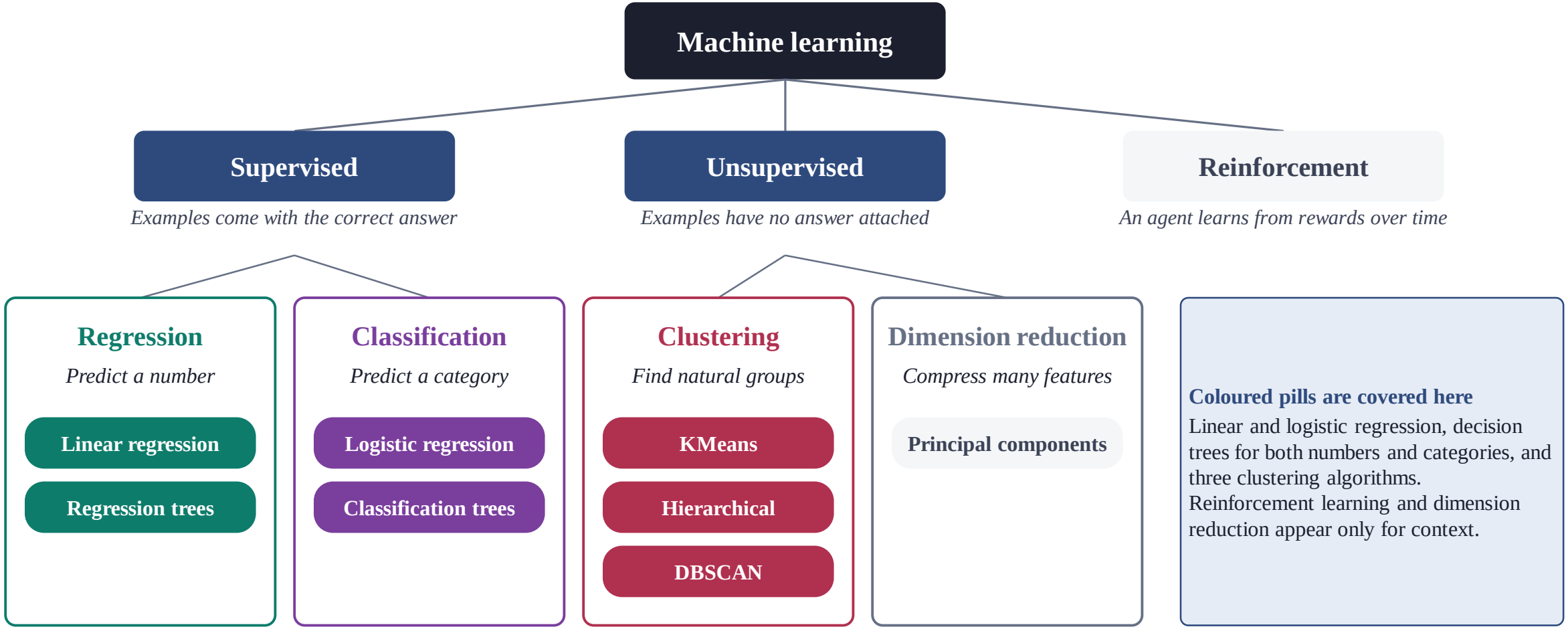
Traditional programming versus machine learning

The inputs and outputs swap places. In machine learning the rules become the output of the process.



The main families of machine learning

This tutorial covers both branches of supervised learning and the most important branch of unsupervised learning.



Regression versus classification

Both are supervised. The only difference is the type of answer the model must produce.

Regression: predict a quantity

- The target is a number on a continuous scale, such as a house price, an exam score or a delivery time.
- Predictions can take any value, including values never seen during training.
- Errors are measured as distances: how far was the prediction from the truth, on average?
- Typical metrics: mean absolute error, root mean squared error and R squared.
- In this deck: linear regression, ridge and lasso, and regression trees.

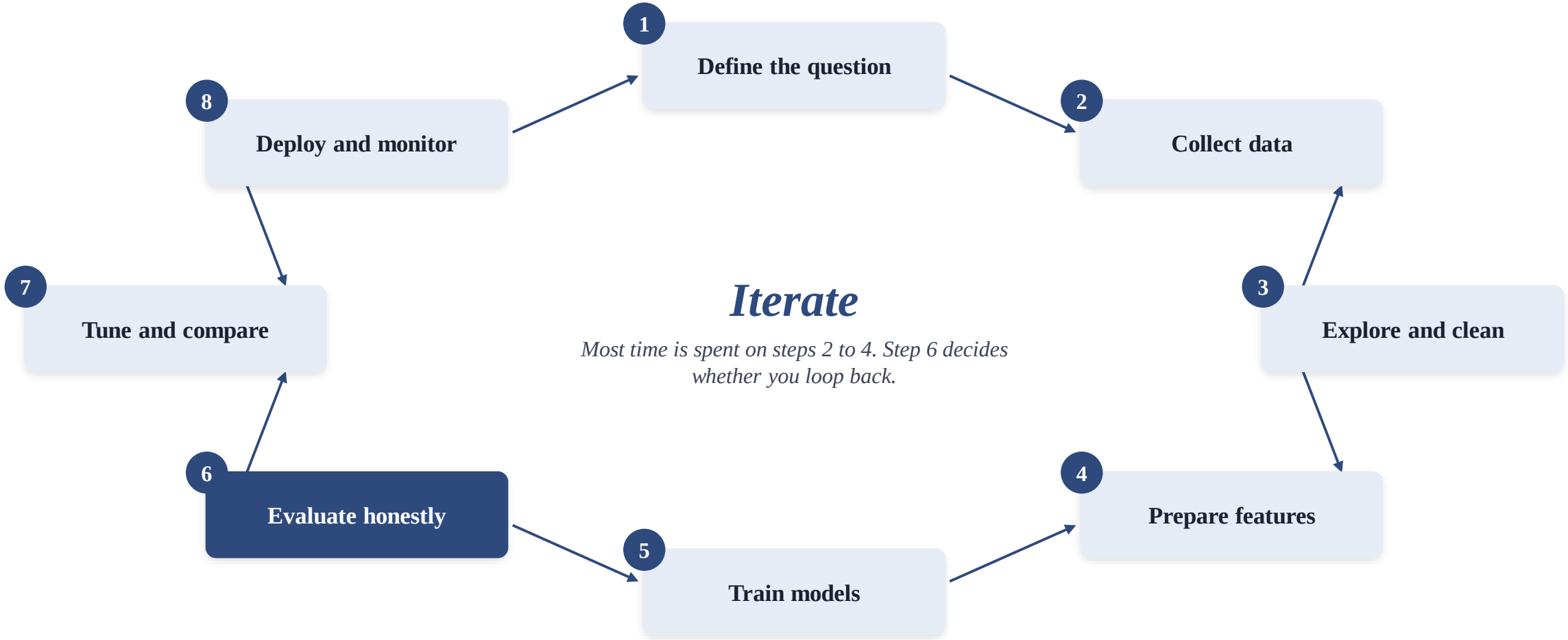
vs

Classification: predict a category

- The target is a label from a fixed set, such as pass or fail, malignant or benign, or one of three flower species.
- Good models output a probability for each class, then a decision rule converts it to a label.
- Errors are counted: how many cases ended up in the wrong class, and which kind of wrong?
- Typical metrics: accuracy, precision, recall, F1, ROC AUC and log loss.
- In this deck: logistic regression and classification trees.

The machine learning workflow is a loop, not a line

Real projects cycle through these stages many times. Evaluation results send you back to earlier steps.



Anatomy of a dataset

Every tabular dataset in this tutorial has the same shape: rows are cases, columns are measurements, and one column may be the answer.

hours_studied	sleep_hours	attendance	exam_score	passed
7.7	8.4	67.0	94.4	1
4.4	7.1	59.0	73.1	1
8.6	7.6	84.0	113.9	1
7.0	6.9	92.0	89.1	1
0.9	6.8	52.0	51.7	0

... 495 more rows

A row is one observation

Here, one student. Also called a sample, record or instance.

A column is one feature

A measured input, also called a variable, attribute or predictor. Here: sleep hours.

Purple columns are targets

exam_score is a numeric target for regression. passed is a 0 or 1 target for classification. Same rows, two possible questions.

Shape is rows by columns

This table has shape (500, 5). In code the features form a matrix called X and the target a vector called y, a convention used everywhere in scikit learn.

The kinds of values a feature can hold

Algorithms only understand numbers. Knowing the type of each column tells you how it must be converted.

1 Continuous numeric

Any value in a range, such as hours studied or floor area. Usable directly, though scaling often helps distance and gradient based models.

2 Discrete numeric

Whole number counts, such as bedrooms or visits per month. Treated as numeric because the order and the gaps both carry meaning.

3 Nominal categorical

Labels with no natural order, such as city or colour. Must be one hot encoded so the model does not invent a false ordering.

4 Ordinal categorical

Labels with an order but uneven gaps, such as small, medium, large. Can be mapped to ranked integers when the order matters.

5 Binary

Exactly two states, stored as 0 and 1. Our passed column and the breast cancer diagnosis are binary targets.

6 Dates and free text

Need feature engineering first: extract day of week or account age from dates, word counts or embeddings from text.

Setting up the Python environment

PYTHON

```python
import numpy as np
import pandas as pd
import sklearn
pd.set_option("display.width", 120)
print("numpy  ", np.__version__)
print("pandas ", pd.__version__)
print("sklearn", sklearn.__version__)
```

1

Import the core libraries

numpy handles arrays and random numbers, pandas handles labelled tables, and sklearn is the import name of the scikit learn package that holds every model.

2

Widen printed tables

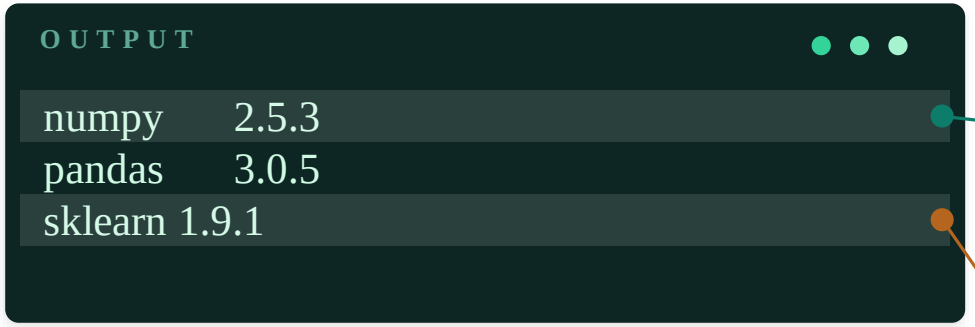
Tells pandas to print up to 120 characters per line, so wide tables are not wrapped awkwardly in the console.

3

Record the versions

Printing versions at the top of a notebook makes results reproducible: anyone rerunning the work can match the library versions exactly.

Output: confirming the library versions



numpy 2.5.3
The numerical engine underneath pandas and scikit learn. Every dataset becomes a numpy array before a model sees it.

scikit learn 1.9.1
All outputs in this deck were produced with this version. Older versions give the same ideas, though default settings occasionally differ.

Simulating a dataset whose truth we know

PYTHON

```python
rng = np.random.default_rng(42)
n = 500
df = pd.DataFrame({
        "hours_studied": rng.uniform(0, 10, n).round(1),
        "sleep_hours": rng.normal(7, 1, n).round(1),
        "attendance": rng.uniform(50, 100, n).round(0),
})
score = 20 + 5 * df.hours_studied + 2 * df.sleep_hours + 0.3 * df.attendance
df["exam_score"] = (score + rng.normal(0, 5, n)).round(1)
df["passed"] = (df.exam_score >= 70).astype(int)
print(df.head())
print("Shape:", df.shape)
```

1

Seeded random generator

A fixed seed of 42 makes the random numbers identical every run, so your output matches this slide exactly.

2

Three input features

Hours studied is uniform between 0 and 10, sleep is bell shaped around 7 hours, attendance is uniform between 50 and 100 percent.

3

The hidden true rule plus noise

Each extra study hour adds 5 points, each sleep hour adds 2, each attendance point adds 0.3. Random noise with a spread of 5 points mimics luck on the day.

4

A classification target

A student passes when the score reaches 70. Converting the comparison to integers gives 1 for pass and 0 for fail.

Output: the first five students

OUTPUT

	hours_studied	sleep_hours	attendance	exam_score	passed
0	7.7	8.4	67.0	94.4	1
1	4.4	7.1	59.0	73.1	1
2	8.6	7.6	84.0	113.9	1
3	7.0	6.9	92.0	89.1	1
4	0.9	6.8	52.0	51.7	0

Shape: (500, 5)

Column headers

Three features, then the two targets we created. The unnamed left column is the pandas row index, starting at 0.

Row 2: a score above 100

113.9 is possible because our simulated rule has no ceiling. Real exams cap at 100, a reminder that simulated data is a simplification.

Row 4: the only fail shown

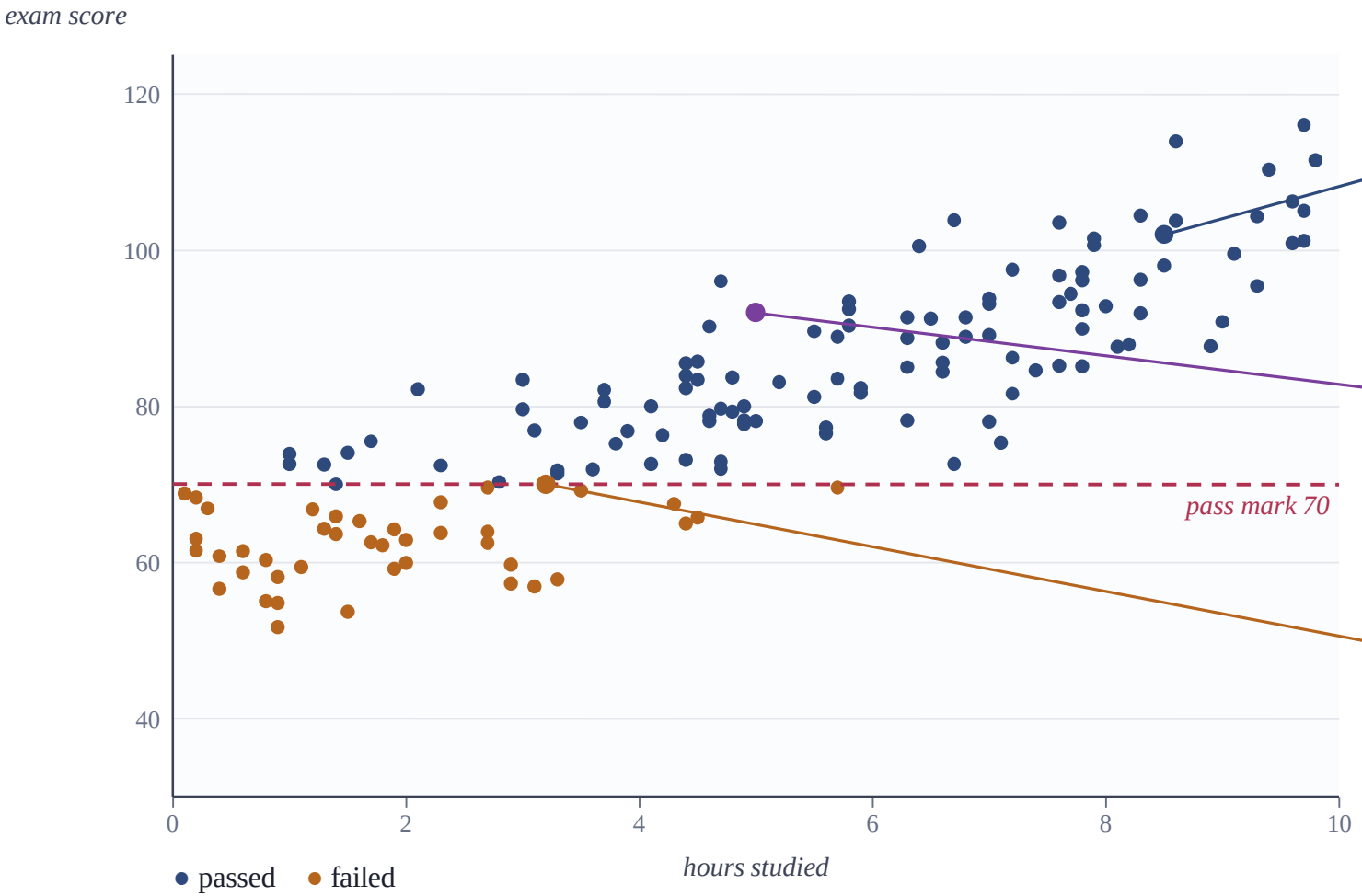
0.9 hours of study and 52 percent attendance produce 51.7, below 70, so passed is 0.

Shape (500, 5)

Five hundred students, five columns. Always print the shape early to confirm nothing was lost while loading.

Illustration: what the study data looks like

A scatter of the first 150 students. Each dot is one student, coloured by whether they passed.



Upward drift
More study hours push scores up. This is the signal a model should learn: about five points per hour.

Vertical spread
Students with equal hours score differently because sleep, attendance and noise also matter. One feature alone cannot explain everything.

Overlap near the line
Between about 2 and 5 hours, passes and fails mix. Classification models will be least certain in exactly this zone.

Summary statistics with describe()

One line of code, df.describe(), returns eight statistics for every numeric column.

OUTPUT

	hours_studied	sleep_hours	attendance	exam_score	passed
count	500.00	500.00	500.00	500.00	500.00
mean	4.96	6.94	75.30	80.43	0.70
std	2.87	1.02	14.58	15.99	0.46
min	0.10	3.40	50.00	43.50	0.00
25%	2.38	6.20	62.00	67.50	0.00
50%	5.00	7.00	75.00	81.05	1.00
75%	7.50	7.60	88.00	92.85	1.00
max	10.00	10.20	100.00	120.20	1.00

mean
The average. Hours studied averages 4.96, close to the midpoint of 0 to 10 as expected from a uniform draw.

std
Standard deviation: the typical distance from the mean. Exam scores spread by about 16 points.

min and max
Sanity checks. Sleep ranges from 3.4 to 10.2 hours, which is plausible. Impossible values here would signal data errors.

50% is the median
The middle value. When mean and median are close, the column is roughly symmetric. Large gaps signal skew or outliers.

passed mean is 0.70
For a 0 or 1 column the mean is the share of ones: 70 percent of students passed.

Reading summary statistics like an analyst

1 Count reveals missing data

Every column should show the same count as the number of rows. A smaller count means missing values that must be imputed or removed.

2 Mean versus median

Income style columns often have a mean far above the median because a few extreme values drag the mean upward. Trees cope well; linear models may need a log transform.

3 Quartiles describe the bulk

The 25 and 75 percent values bound the middle half of the data. The gap between them, the interquartile range, is a robust measure of spread.

4 Scale differences matter

Attendance spans 50 points while sleep spans 7. Distance based methods would let attendance dominate, which is why scaling exists.

5 Targets need checking too

A skewed or capped target changes which metric is meaningful. A class balance of 70 to 30 tells us accuracy alone could mislead.

6 Never skip this step

Most modelling disasters are visible in a summary table: impossible minimums, constant columns, or identifiers that were loaded as numbers.

Separating features from targets

PYTHON

```python
X = df[["hours_studied", "sleep_hours", "attendance"]]
y_reg = df["exam_score"]
y_clf = df["passed"]
print("X shape:", X.shape)
print("Mean exam score:", round(y_reg.mean(), 2))
print("Class balance:")
print(y_clf.value_counts(normalize=True).round(3))
```

1

X: the feature matrix

Double square brackets select a list of columns and return a DataFrame. Capital X signals a two dimensional table.

2

y: the target vectors

Single brackets return a one dimensional Series. We keep two targets so the same data can serve regression and classification.

3

Inspect before modelling

value_counts with normalize set to True returns proportions instead of counts, the quickest way to see class balance.

Output: shapes and class balance

```
OUTPUT
X shape: (500, 3)
Mean exam score: 80.43
Class balance:
passed
1       0.696
0       0.304
Name: proportion, dtype: float64
```

(500, 3)
Three feature columns. The targets were deliberately left out of X; including them would let the model cheat.

Mean score 80.43
A regression model that always predicts 80.43 is the simplest possible baseline. Any real model must beat it.

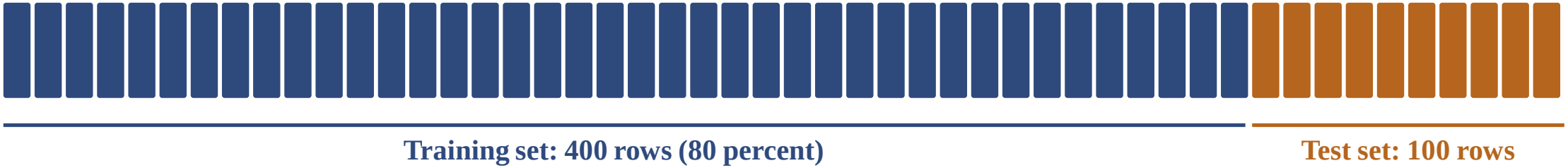
69.6 percent passed
The classes are imbalanced. A lazy model that always says pass would already be about 70 percent accurate.

Metadata line
pandas reports the Series name and data type. float64 means 64 bit decimal numbers.

Illustration: the train and test split

We hide part of the data from the model during training, then use it once at the end as a fair final exam.

Each block represents 10 students: 500 in total, shuffled randomly before splitting



Training set: 400 rows (80 percent)

Test set: 100 rows

The model learns here

Fitting, scaling statistics, feature selection and hyperparameter choices may all look at these rows freely.

The model is judged here

These rows are touched exactly once, after all decisions are frozen. The score estimates performance on future data.

Why not test on training rows?

A model that memorised the training data would score perfectly there and still fail in production. The held out score exposes that.

Why holding data back is non negotiable

1 The exam analogy

If students see the exact exam questions while revising, a perfect mark proves memory, not understanding. The test set is the unseen exam.

2 Optimism bias

Training error is almost always lower than error on new data, because the model was tuned to those exact rows. The gap grows with model flexibility.

3 Use the test set once

Every time you peek at test results and change something, the test set leaks into your decisions and stops being a fair judge.

4 Validation data for choices

Choosing between models or settings needs a separate validation set or cross validation inside the training data, covered later in this section.

5 Typical proportions

70 to 80 percent for training is common. Small datasets favour cross validation; very large ones can afford a smaller test share.

6 Time ordered data is different

For forecasting, the test set must come after the training period. Random shuffling would let the model peek at the future.

Splitting with train_test_split

PYTHON

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
        X, y_clf, test_size=0.2, random_state=42, stratify=y_clf)
print("Train rows:", len(X_train), " Test rows:", len(X_test))
print("Pass rate train:", round(y_train.mean(), 3))
print("Pass rate test: ", round(y_test.mean(), 3))
```

1 Import the splitter

train_test_split lives in the model_selection module alongside cross validation and search tools.

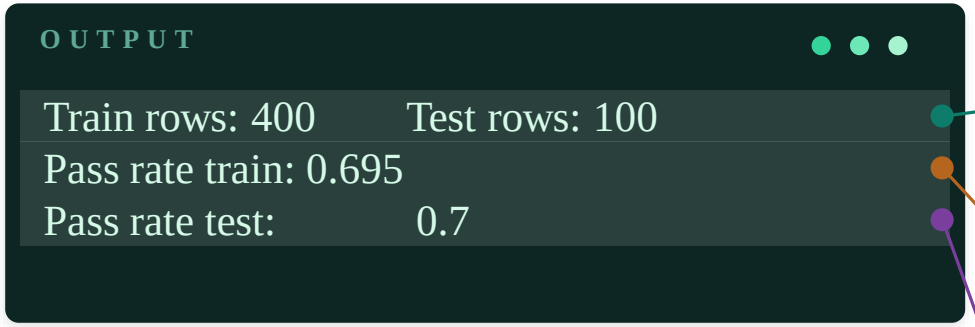
2 Four outputs in a fixed order

Training features, testing features, training target, testing target. test_size of 0.2 holds back 20 percent. random_state fixes the shuffle. stratify keeps the pass rate equal in both parts.

3 Verify the split

Comparing the pass rate in each part confirms stratification worked.

Output: a balanced split



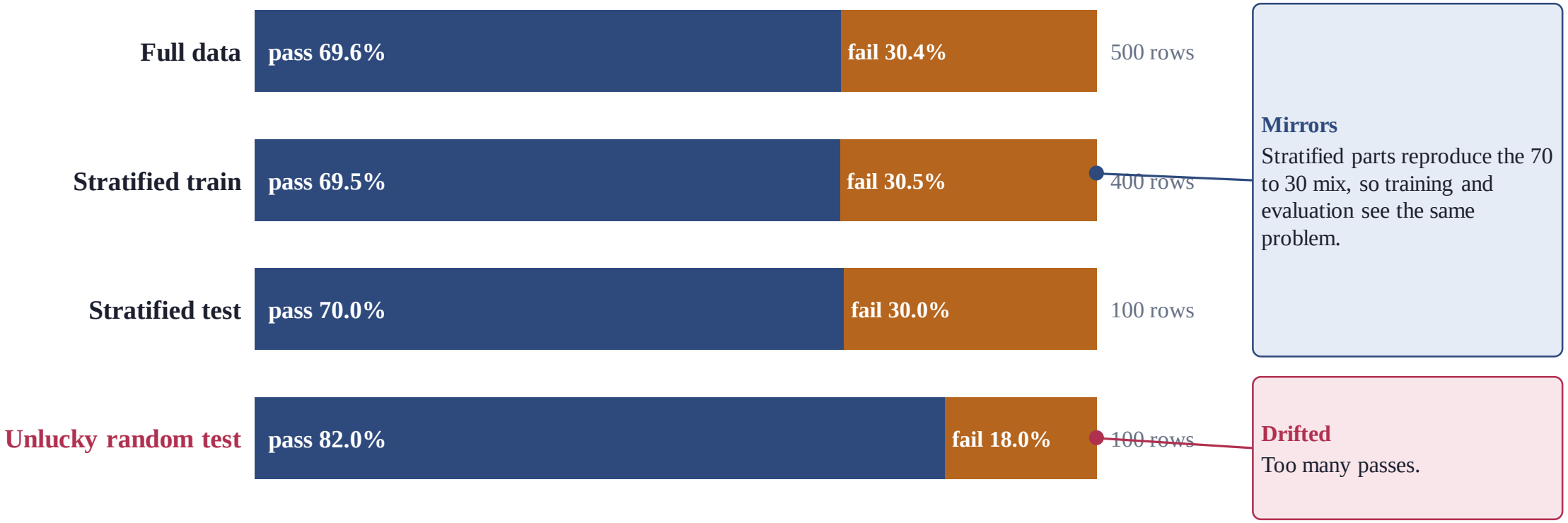
400 and 100
Exactly 80 and 20 percent of 500 rows.

Pass rate 0.695 in training
Matches the overall 0.696 almost perfectly.

Pass rate 0.7 in testing
Seventy passes and thirty fails. Without stratify, an unlucky shuffle could have produced 60 or 80 percent, making test scores unstable and misleading.

Illustration: what stratification protects against

Stratified splitting copies the class mix of the full dataset into both the training and the test portion.



The bottom bar is illustrative. With small test sets, a purely random split can drift several points away from the true mix, and every metric drifts with it. Always stratify classification splits.

Why feature scaling matters

Many algorithms compare features numerically. If one feature is measured in large units, it can drown out the others for no good reason.

1 Distance based methods

KMeans and nearest neighbours measure distances. A feature ranging over 50 units dominates one ranging over 7, regardless of importance.

2 Gradient based optimisation

Logistic regression is fitted by iterative steps. Features on wildly different scales make the search zigzag and converge slowly.

3 Regularisation fairness

Ridge, lasso and regularised logistic regression penalise large coefficients. Unscaled features get penalised unevenly purely because of their units.

4 Interpretable coefficients

After standardising, each coefficient means the effect of a one standard deviation change, so effects can be compared directly.

5 Trees do not care

Decision trees split on thresholds and only use the order of values, so scaling never changes a tree. We prove this in section 4.

6 Standardisation

StandardScaler subtracts each column mean and divides by its standard deviation, giving every feature a mean of 0 and a spread of 1.

Illustration: standardisation puts features on one ruler

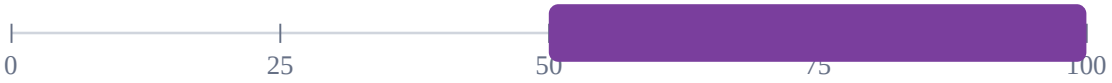
Before scaling, the three features live on different rulers. After scaling, they share a common ruler centred on zero.

BEFORE: raw units

hours_studied

sleep_hours

attendance



Sleep occupies a sliver of the raw ruler while attendance spans half of it.

AFTER: standardised units

hours_studied

sleep_hours

attendance



After scaling all three are centred on 0 with comparable widths.

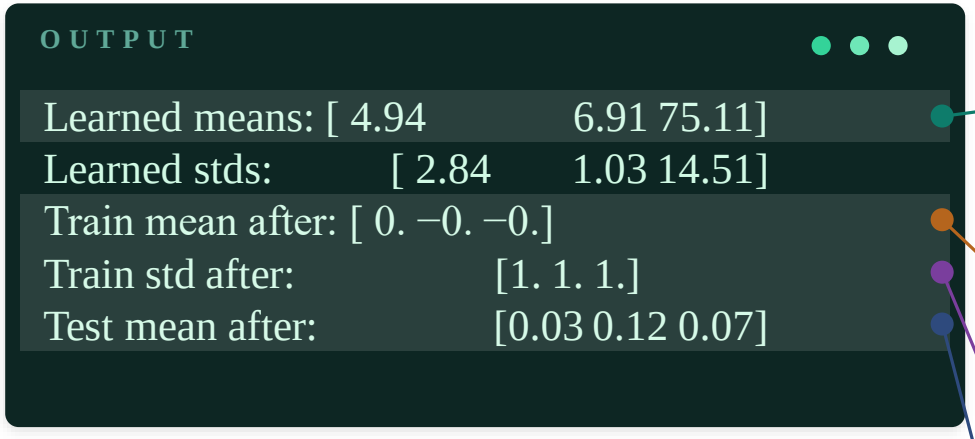
Standardising with StandardScaler

PYTHON

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
print("Learned means:", scaler.mean_.round(2))
print("Learned stds: ", scaler.scale_.round(2))
print("Train mean after:", X_train_s.mean(axis=0).round(2))
print("Train std after: ", X_train_s.std(axis=0).round(2))
print("Test mean after: ", X_test_s.mean(axis=0).round(2))
```

1. **Create the transformer**
   A transformer is an object with fit and transform methods. Creating it learns nothing yet.
2. **fit on training data only**
   fit computes the mean and standard deviation of each column using only the 400 training rows.
3. **transform both sets**
   The same training statistics are applied to the test rows. The test set never influences the scaling.
4. **Inspect what was learned**
   Attributes ending in an underscore, such as mean_ and scale_, are values learned during fit. This naming rule holds across all of scikit learn.

Output: learned statistics and their effect



Means learned from training rows

4.94 hours, 6.91 hours of sleep, 75.11 percent attendance.

Training means become 0

The minus zero values are floating point rounding of tiny numbers such as negative 0.0000001. They are mathematically zero.

Training spreads become 1

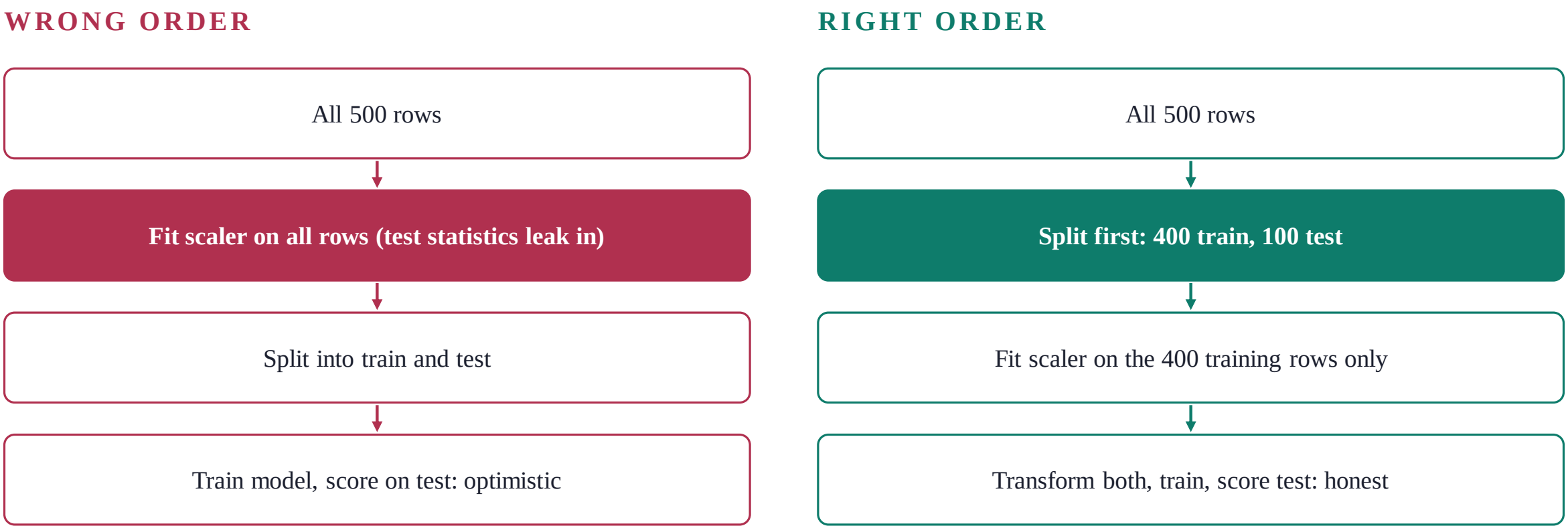
Every feature now has unit standard deviation.

Test means are not exactly 0

0.03, 0.12 and 0.07. This is correct and expected: the test rows were scaled with training statistics, as a future unseen row would be.

Illustration: data leakage, the silent accuracy inflator

Leakage happens when information from the test rows influences training. Scores look great and then collapse in production.



Leakage also comes from target derived features, duplicate rows across splits, and selecting features using the full dataset. Pipelines, shown next, prevent the common cases automatically.

Encoding categories and filling missing values

1 One hot encoding

Turns a column like city into several 0 or 1 columns, one per city. Each row has a single 1 marking its city. No artificial order is implied.

2 handle_unknown set to ignore

If a new city appears at prediction time, the encoder outputs all zeros instead of crashing. Essential for production models.

3 Dropping one level

For plain linear regression, one dummy column is redundant because the others imply it. drop set to first removes it to avoid perfect collinearity.

4 Median imputation

SimpleImputer replaces missing numbers with the training median, which resists outliers better than the mean.

5 Missing categories

A missing city can be filled with a new label such as Unknown, which lets the model learn whether missingness itself is informative.

6 ColumnTransformer

Applies different preparation to different columns in one object, then glues the results side by side into a single matrix.

Preparing mixed data with ColumnTransformer

PYTHON

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
raw = pd.DataFrame({
        "city": ["Leeds", "York", "Leeds", "Hull", None],
        "income": [32.0, None, 41.5, 28.0, 35.0]})
prep = ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"]),
        ("num", SimpleImputer(strategy="median"), ["income"])])
out = prep.fit_transform(raw.fillna({"city": "Unknown"}))
print(prep.get_feature_names_out())
print(np.asarray(out).round(1))
```

1. **Three preparation tools**
   An imputer for gaps, an encoder for categories, and a transformer that routes columns to the right tool.
2. **A tiny messy table**
   Five people, one missing city (None) and one missing income. Real data almost always looks like this.
3. **Route each column**
   Each entry is a name, a tool and the columns it handles. city goes to the one hot encoder, income goes to the median imputer.
4. **Fit, transform, inspect**
   fillna labels the missing city Unknown. get_feature_names_out lists the columns that come out of the transformer.

Output: a clean numeric matrix

```
OUTPUT

['cat__city_Hull' 'cat__city_Leeds' 'cat__city_Unknown' 'cat__city_York'
 'num__income']
[[ 0.   1.   0.   0.  32. ]
 [ 0.   0.   0.   1.  33.5]
 [ 0.   1.   0.   0.  41.5]
 [ 1.   0.   0.   0.  28. ]
 [ 0.   0.   1.   0.  35. ]]
```

Generated column names

Each name is prefixed with its transformer, cat or num, followed by a double underscore. Four city columns were created, one per distinct city.

First person: Leeds, 32.0

A single 1 in the Leeds column and the original income.

Second person: York, 33.5

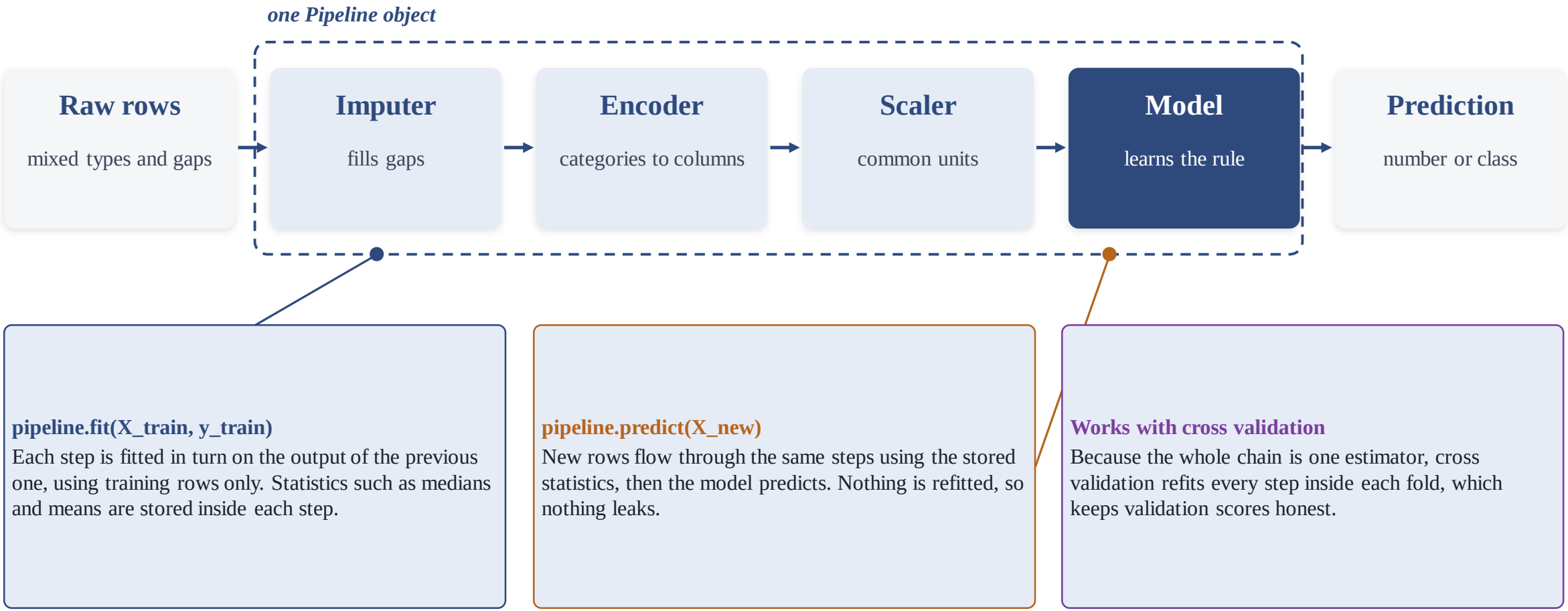
Their income was missing, so it was filled with the median of the four known incomes: 32, 41.5, 28 and 35 give 33.5.

Fifth person: Unknown city

The missing city became its own category rather than being silently dropped.

Illustration: a pipeline chains every step into one model

A pipeline bundles preparation and modelling, so fitting and predicting always apply the steps in the same order with the same learned statistics.



Building a pipeline with make_pipeline

PYTHON

```python
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)
print(model)
print("Steps:", [name for name, _ in model.steps])
print("Test accuracy:", round(model.score(X_test, y_test), 3))
```

1. **Imports**
   make_pipeline builds a Pipeline and names each step automatically from its class name in lower case.
2. **Scaler then classifier**
   Order matters: data flows left to right. The final step must be a model; every earlier step must be a transformer.
3. **One fit call**
   Fits the scaler on the training rows, transforms them, then fits logistic regression on the scaled result.
4. **Inspect and score**
   score on a classifier returns accuracy, the share of test rows predicted correctly.

Output: the pipeline and its accuracy

```
OUTPUT

Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
Steps: ['standardscaler', 'logisticregression']
Test accuracy: 0.94
```

The fitted object

Printing a pipeline shows every step and any settings changed from their defaults. Here everything is default.

Automatic step names

These names let you reach inside later: model[1] or model.named_steps with the step name returns the logistic regression object.

Accuracy 0.94

94 of 100 test students classified correctly. Compare with the 0.70 a model that always predicts pass would achieve. We build that baseline shortly.

Parameters versus hyperparameters

Parameters: learned from data

- Values the algorithm sets during fit by looking at the training rows.
- Examples: regression coefficients and intercept, tree split thresholds, cluster centroids.
- Stored as attributes ending in an underscore, such as coef_ or cluster_centers_.
- You never set these by hand; the optimisation finds them.

vs

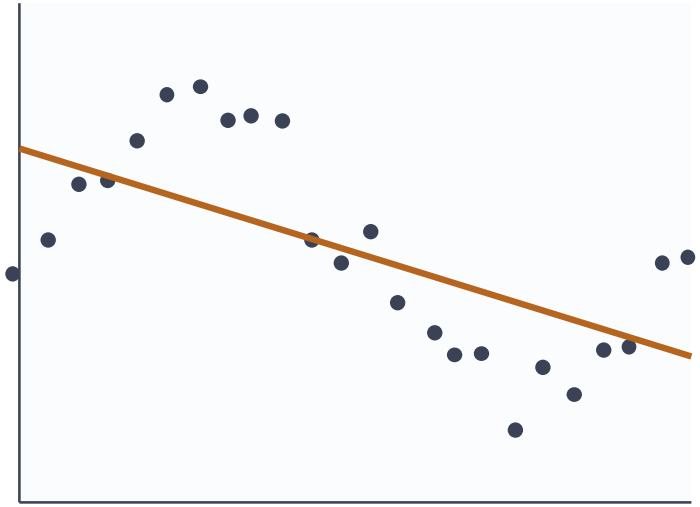
Hyperparameters: chosen by you

- Settings fixed before training that control how learning happens.
- Examples: max_depth of a tree, regularisation strength C or alpha, the number of clusters.
- Passed to the constructor, such as DecisionTreeClassifier(max_depth=3).
- Chosen by comparing validation performance, never test performance. Section 5 is devoted to this.

Illustration: underfitting, a good fit and overfitting

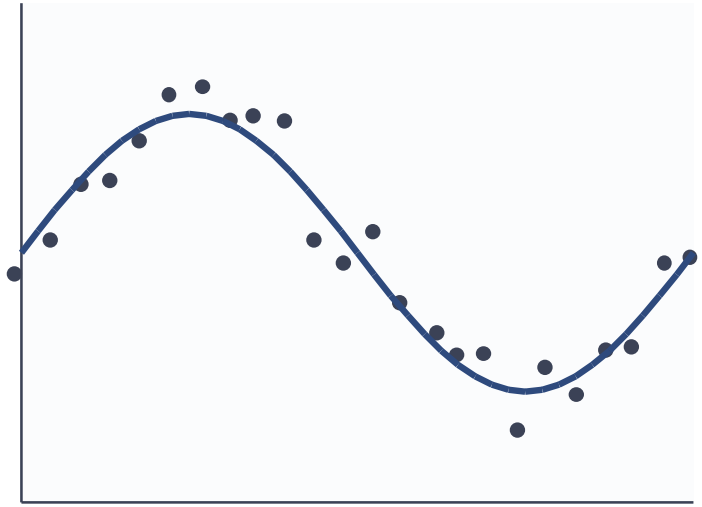
The same noisy wave fitted three ways. Model flexibility should match the true complexity of the pattern, no more and no less.

Underfit



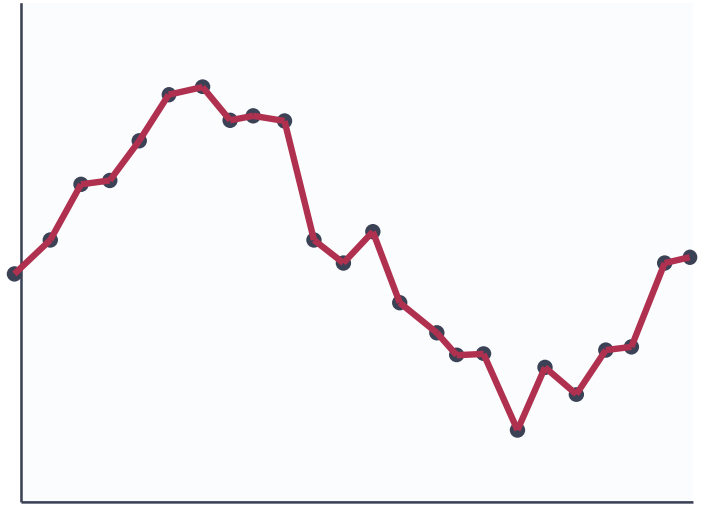
Too simple: a straight line misses the wave entirely. High error everywhere.

Good fit



Follows the wave and ignores the noise. Will predict new points well.

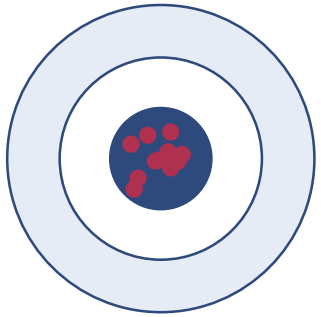
Overfit



Threads every point, including noise. Perfect on training, erratic on new data.

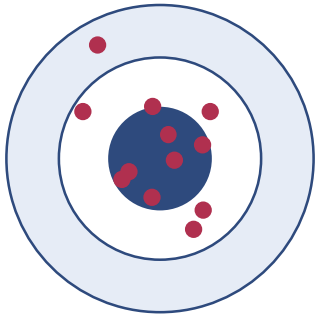
Illustration: bias and variance as a target

Imagine training the same model on many different samples of data. Each dot is one trained model's prediction; the bullseye is the truth.



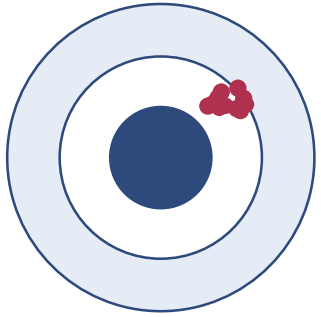
Low bias, low variance

The goal: accurate and consistent.



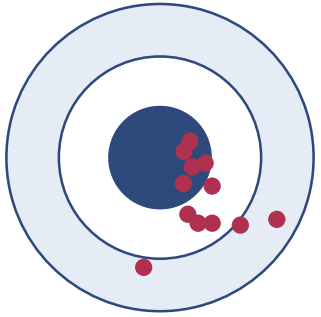
Low bias, high variance

Right on average, but each sample gives a wildly different model. Typical of deep trees.



High bias, low variance

Consistently wrong. Typical of models too simple for the pattern.



High bias, high variance

Wrong and unstable: the worst case.

Bias is systematic error from wrong assumptions. Variance is sensitivity to the particular training sample. Flexible models lower bias but raise variance; the art is finding the balance.

Measuring overfitting with polynomial curves

PYTHON

```python
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
x = np.sort(rng.uniform(0, 1, 60)).reshape(60, 1)
wave = np.sin(2 * np.pi * x).ravel() + rng.normal(0, 0.25, 60)
x_tr, x_te, w_tr, w_te = train_test_split(x, wave, test_size=0.4, random_state=0)
for d in [1, 3, 9, 15]:
    m = make_pipeline(PolynomialFeatures(d), LinearRegression()).fit(x_tr, w_tr)
    tr = mean_squared_error(w_tr, m.predict(x_tr))
    te = mean_squared_error(w_te, m.predict(x_te))
    print(f"degree {d:>2}: train MSE {tr:.3f}   test MSE {te:.3f}")
```

1 **Tools**

PolynomialFeatures creates powers of x so a linear model can bend. Mean squared error averages the squared prediction errors.

2 **A noisy wave**

60 points on a sine wave plus noise, split 60 to 40 so the test set is large enough to be stable.

3 **Increasing flexibility**

Degree 1 is a straight line, degree 15 can wiggle fifteen times. Each model is a two step pipeline.

4 **Error on both sets**

Comparing training and test error at each degree reveals where the model stops learning signal and starts memorising noise.

Output: the sweet spot appears at degree 3

```
OUTPUT
degree  1: train MSE 0.328   test MSE 0.202
degree  3: train MSE 0.064   test MSE 0.056
degree  9: train MSE 0.050   test MSE 0.107
degree 15: train MSE 0.051   test MSE 0.092
```

Degree 1 underfits

Large errors on both sets. A line cannot follow a wave. Test error is slightly lower here only by chance in which points landed in each set.

Degree 3 is best

Lowest test error, 0.056, close to the noise level we injected. The model captured the signal.

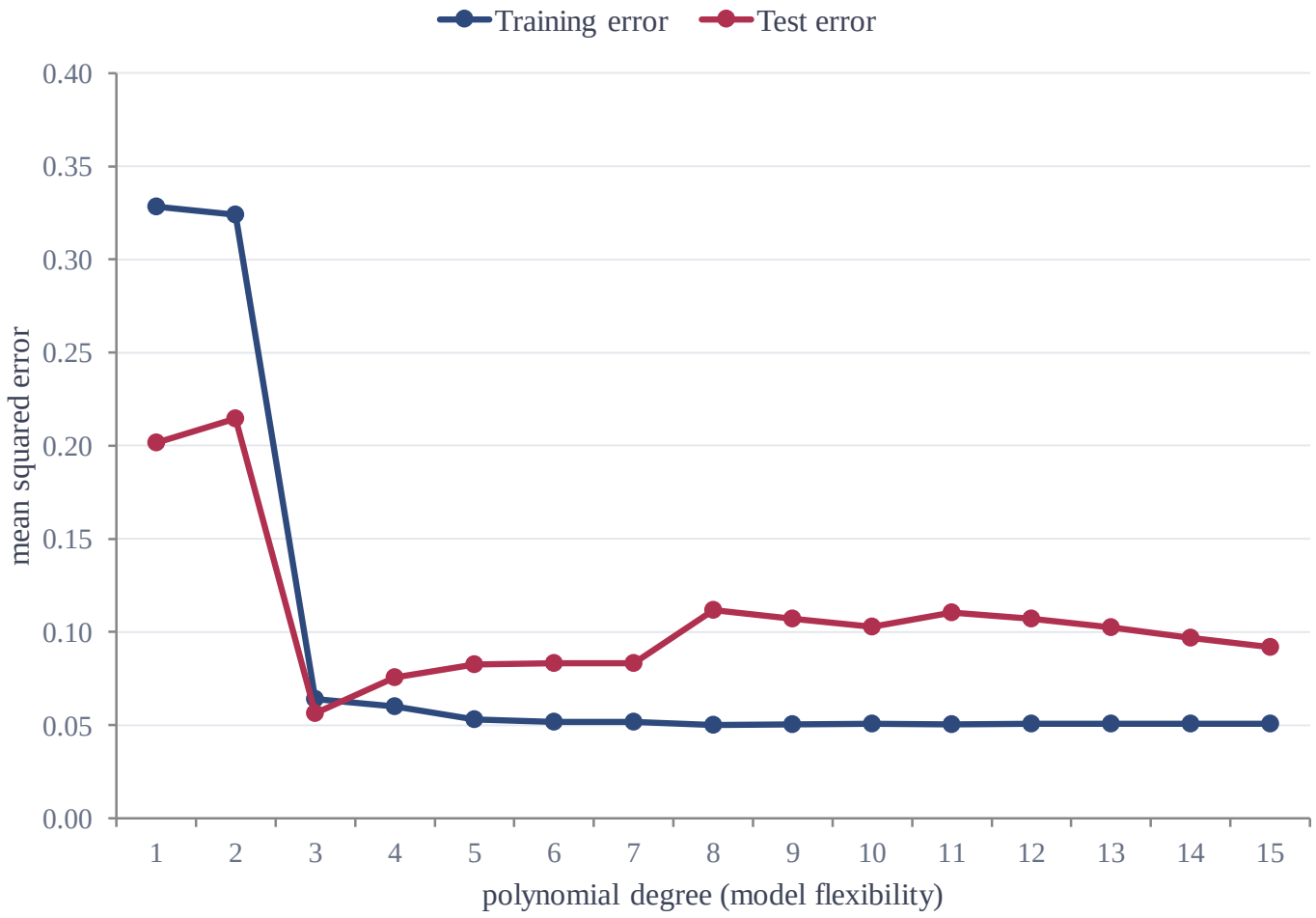
Degree 9 overfits

Training error keeps falling to 0.050 but test error nearly doubles to 0.107. The extra flexibility is spent fitting noise.

The telltale gap

Whenever training error is much lower than test error, suspect overfitting.

Chart: training and test error across fifteen degrees



1. **Left side: underfitting**
   At degree 1 and 2 both curves are high. The model is too rigid to follow the wave.

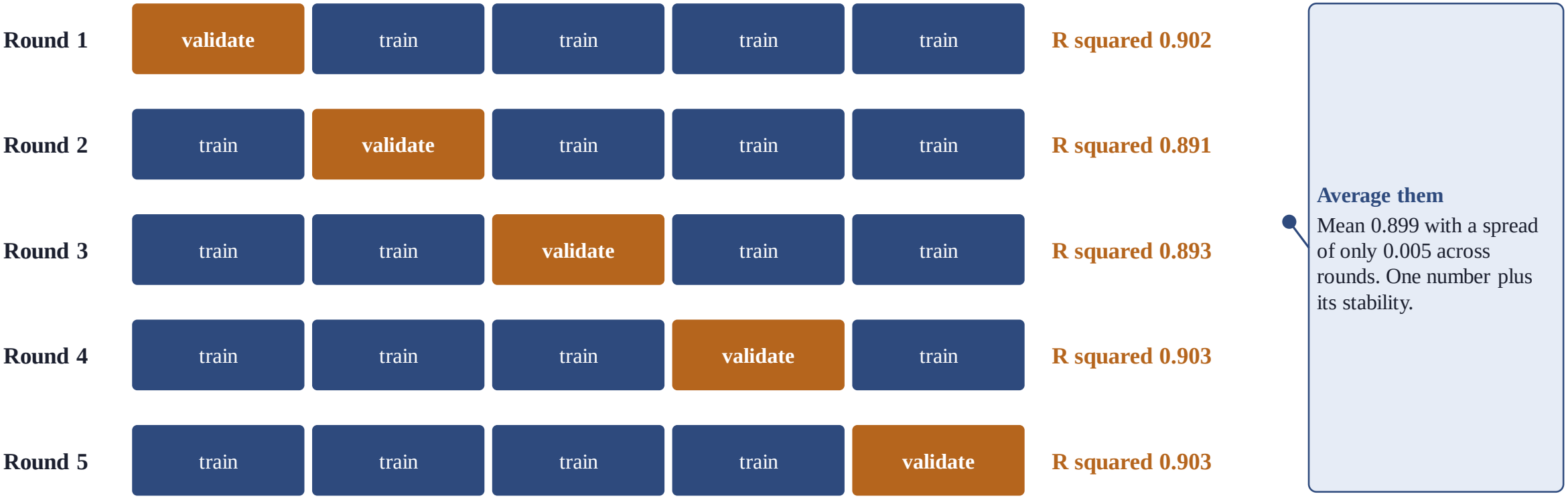
2. **The valley**
   Test error bottoms out at degree 3 with 0.056. This is the best trade off between bias and variance for this wave.

3. **Right side: overfitting**
   From degree 8 onward training error sits flat near 0.05 while test error doubles to about 0.11. The extra flexibility only fits noise.

4. **The universal shape**
   Swap degree for tree depth or regularisation strength and you get the same U shaped test curve. Section 5 uses exactly this idea to tune trees.

Illustration: K fold cross validation

Instead of one validation split, rotate through five. Every row is used for validation exactly once and for training four times.

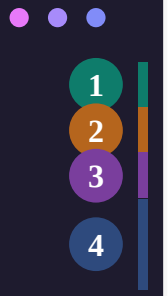


These five scores are the real output of the next code example: linear regression predicting exam_score.

Running five fold cross validation

PYTHON

```
1 from sklearn.model_selection import cross_val_score, KFold
2 folds = KFold(n_splits=5, shuffle=True, random_state=42)
3 scores = cross_val_score(LinearRegression(), X, y_reg, cv=folds, scoring="r2")
4 print("R2 per fold:", scores.round(3))
5 print("Mean R2:", round(scores.mean(), 3), " Std:", round(scores.std(), 3))
```



1 Imports

cross_val_score runs the whole fit and score loop for you. KFold defines how rows are assigned to folds.

2 Shuffled folds

shuffle set to True randomises rows before cutting five folds. Use StratifiedKFold for classification targets.

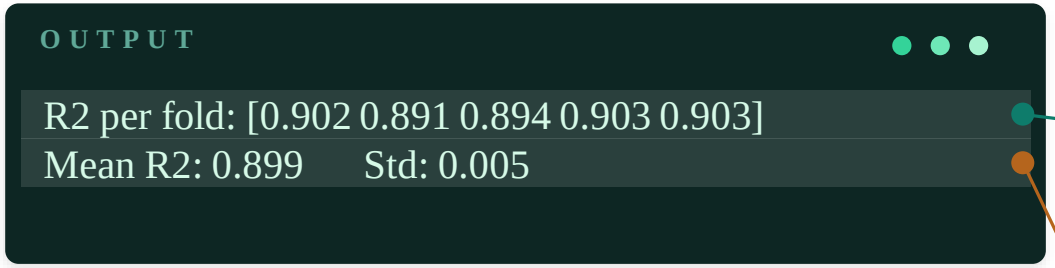
3 Five fits, five scores

A fresh LinearRegression is trained on four folds and scored with R squared on the fifth, five times over.

4 Summarise

Report the mean as the estimate and the standard deviation as its uncertainty.

Output: a stable R squared near 0.90



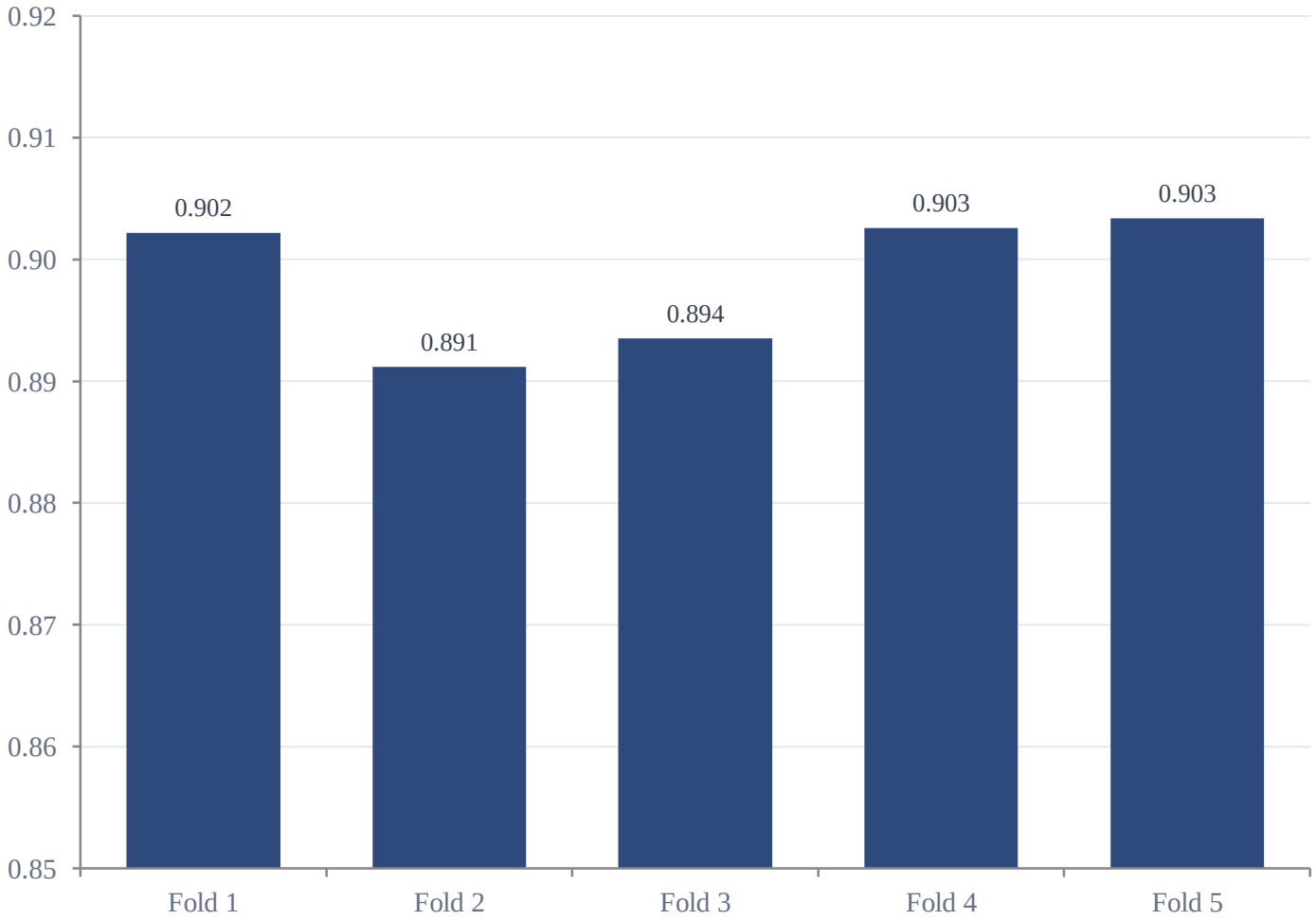
Five fold scores

Each value is the share of variation in exam scores explained on rows the model never trained on.

Mean 0.899, spread 0.005

About 90 percent of score variation is explained. The remaining 10 percent is the random noise we injected, which no model can predict. The tiny spread means the estimate is trustworthy.

Chart: fold by fold stability



1 Zoomed axis
The vertical axis starts at 0.85 to make small differences visible. On a full 0 to 1 axis these bars would look identical.

2 Consistent folds
No fold is an outlier. If one fold were far lower, it would suggest unusual rows or a data quality issue in that slice.

3 Why it matters
A single train and test split gives one of these bars. Cross validation shows the whole range, so you know how much luck is in any single score.

Baselines: the score to beat

PYTHON

```python
from sklearn.dummy import DummyClassifier, DummyRegressor
base_c = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline accuracy:", round(base_c.score(X_test, y_test), 3))
base_r = DummyRegressor(strategy="mean")
print("Baseline R2 (5 fold):",
      cross_val_score(base_r, X, y_reg, cv=folds).mean().round(3))
```

1 **Dummy models**

These ignore the features entirely. They exist only to set a floor.

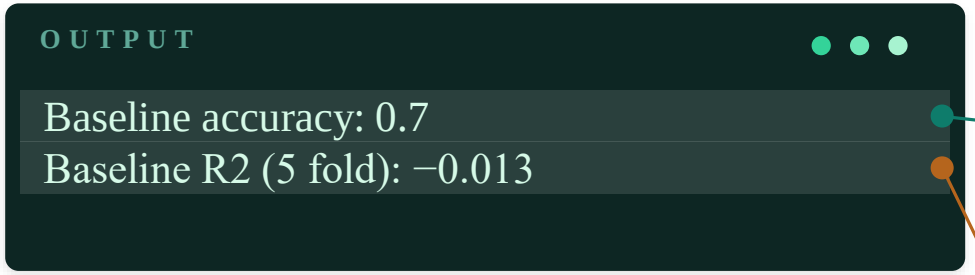
2 **Most frequent class**

Always predicts the most common label in training data, which is pass.

3 **Mean predictor**

Always predicts the training mean exam score, then we cross validate its R squared.

Output: what knowing nothing achieves



Accuracy 0.70 for free

Predicting pass for everyone is right 70 percent of the time. Our pipeline's 0.94 is a real gain of 24 points, not 94.

R squared near zero

Predicting the mean explains nothing, so R squared is about 0. It is slightly negative because the training mean differs a little from each validation fold's mean. Our model's 0.899 is genuine skill.

Map of evaluation metrics used in this tutorial

Task	Metric	What it answers	Better
Regression	MAE	On average, how many units is the prediction off by?	Lower
Regression	RMSE	Like MAE but punishes large misses more heavily, in the target's units	Lower
Regression	R squared	What share of the target's variation does the model explain?	Higher, max 1
Classification	Accuracy	What share of all cases got the right label?	Higher
Classification	Precision	Of the cases flagged positive, how many really were positive?	Higher
Classification	Recall	Of the truly positive cases, how many did we catch?	Higher
Classification	ROC AUC	How well do the probabilities rank positives above negatives?	Higher, max 1
Classification	Log loss	How honest and confident are the probabilities?	Lower
Clustering	Silhouette	Are points closer to their own cluster than to the next one?	Higher, max 1
Clustering	Adjusted Rand	How well do clusters match known groups, corrected for chance?	Higher, max 1

Six golden rules carried into every section

1 Split before anything else

Then fit every transformer on training rows only. Pipelines enforce this.

2 Always build a baseline

A dummy model tells you whether your model learned anything at all.

3 Prefer cross validation for choices

Single splits are noisy; five folds show the spread.

4 Touch the test set once

It is a final exam, not a practice paper.

5 Pick the metric before modelling

The business cost of each kind of error decides it, not convenience.

6 Match flexibility to the data

Watch the gap between training and validation scores for overfitting.

Check your understanding

A colleague scales all 10,000 rows, then splits into training and test sets and reports 97 percent accuracy. What is the main problem?

A Scaling is never needed

B Test statistics leaked into the scaler

C The test set is too small

D Accuracy cannot exceed 95 percent

Answer B and why

The scaler's means and standard deviations were computed using test rows, so information from the test set influenced training. The effect on a scaler is often small, but the same mistake with feature selection or target encoding can inflate scores dramatically. Split first, then fit the scaler inside a pipeline on training rows only.

Check your understanding

Training error keeps falling as model flexibility increases, but validation error starts rising. What is happening?

A Underfitting

B Data leakage

C Overfitting

D The baseline is too strong

Answer C and why

This widening gap is the signature of overfitting, seen in our polynomial experiment at degree 9 and above. The extra flexibility is being used to memorise noise in the training rows. The remedy is less flexibility, more data or regularisation.

SECTION

2

Linear Regression

Predicting a number as a weighted sum of features, and understanding exactly what every weight means

- The intuition: a line, a plane and a recipe
- How least squares finds the best line
- Assumptions and what breaks when they fail
- Fitting, coefficients and recovering a known truth
- MAE, RMSE, R squared and residual diagnostics
- Multicollinearity, ridge and lasso regularisation
- Curves with polynomial features, categories and prediction

The intuition behind linear regression

Linear regression predicts a number by starting from a baseline and adding a fixed amount for every unit of every feature.

1 A starting value

The intercept is the prediction when every feature equals zero. It anchors the line vertically and is often not meaningful on its own.

2 One dial per feature

Each feature has a coefficient: how much the prediction moves when that feature rises by one unit, while all other features stay fixed.

3 Straight line effects

The tenth extra square metre adds the same amount as the first. If reality bends, a plain linear model cannot follow unless we add curved features.

4 Learned from data

The algorithm chooses the intercept and coefficients that make the predictions as close as possible to the known prices in training data.

5 Fast and transparent

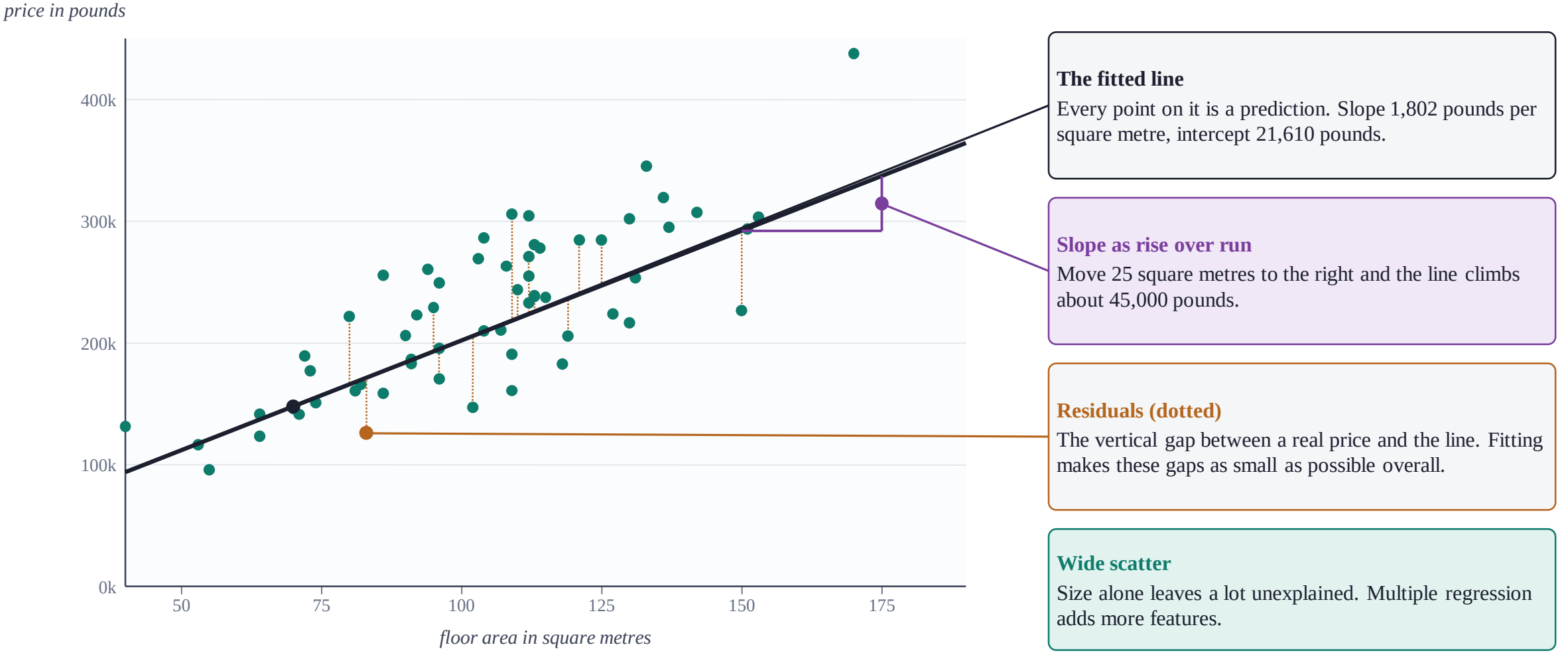
Fitting takes milliseconds even on large data, and every coefficient can be read and explained to a non technical audience.

6 Often a strong baseline

Even when a more complex model wins, linear regression tells you how much of the problem simple additive effects already explain.

Illustration: the anatomy of a fitted line

Real data: 60 houses, price against floor area, with the line fitted by simple linear regression on our training set.



How the best line is found: least squares in words

1 Step 1: propose a line

Any choice of intercept and slope gives a candidate line and therefore a predicted price for every training house.

2 Step 2: measure each miss

For every house, take the actual price minus the predicted price. This difference is the residual.

3 Step 3: square the misses

Squaring makes every miss positive so overestimates and underestimates cannot cancel, and it punishes big misses far more than small ones.

4 Step 4: add them up

The total of the squared residuals is a single score for the candidate line. Lower is better.

5 Step 5: find the minimum

For linear regression this minimum can be calculated directly with linear algebra in one step, so there is exactly one best line and no randomness.

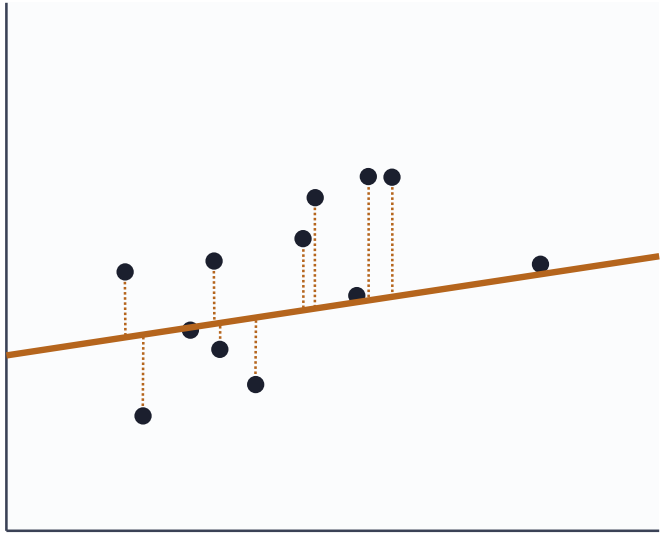
6 Why it is called ordinary least squares

It finds the least possible sum of squares. Ordinary distinguishes it from weighted or regularised variants that change the score being minimised.

Illustration: three candidate lines, one winner

The same twelve houses scored against three candidate lines. The total squared error decides which line least squares picks.

Too flat

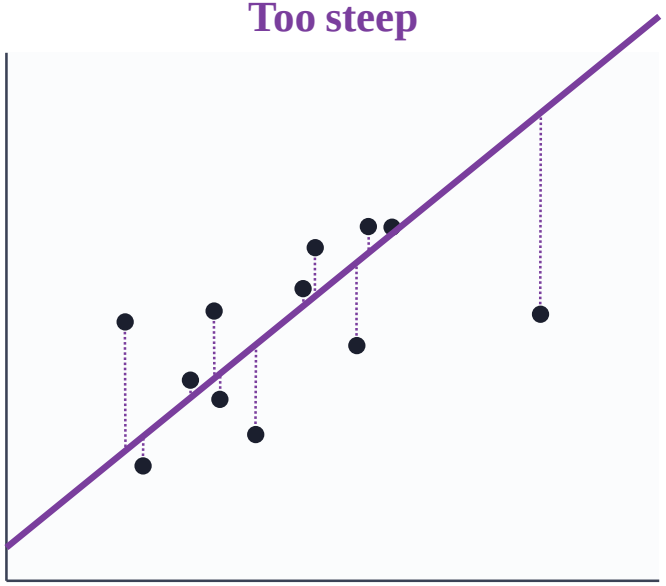


total squared error

29.6 billion

Misses large and small houses badly in opposite directions.

Too steep

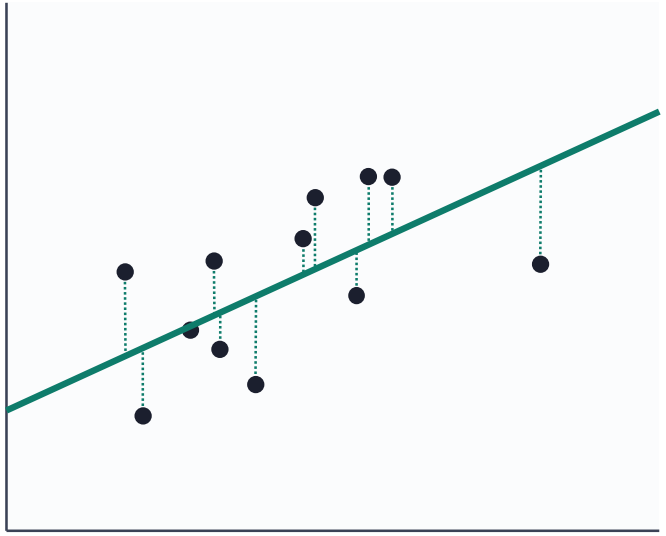


total squared error

36.1 billion

Overshoots big houses, undershoots small ones.

Least squares



total squared error

21.9 billion

The smallest possible total. This is the line scikit learn returns.

Simple versus multiple linear regression

Simple: one feature

- Prediction depends on a single input, such as floor area.
- Geometrically a straight line through a two dimensional scatter.
- The slope mixes the true effect of size with the effects of anything correlated with size.
- Easy to draw, easy to explain, usually too limited.

vs

Multiple: many features

- Prediction combines several inputs: area, bedrooms, age and distance.
- Geometrically a flat plane, or hyperplane, through a space with one axis per feature.
- Each coefficient is the effect of its feature holding all the others constant, which separates overlapping influences.
- Cannot be drawn beyond two features, but reads exactly like a recipe of additions.

The assumptions behind linear regression

Predictions can still be useful when these are bent, but coefficients and uncertainty estimates become untrustworthy.

1 Linearity

The true effect of each feature is a straight line. A curved relationship leaves systematic patterns in the residuals.

2 Independent errors

One house's error tells you nothing about another's. Repeated measures of the same customer or time series often violate this.

3 Constant spread

The size of errors does not grow with the prediction. If expensive homes have wider errors, the spread fans out, called heteroscedasticity.

4 Roughly normal residuals

Matters mostly for confidence intervals and hypothesis tests, less for raw prediction accuracy.

5 No perfect multicollinearity

No feature can be an exact combination of others, or the coefficients cannot be uniquely determined.

6 No dominating outliers

Squaring errors means one extreme house can drag the whole line toward itself. Check for influential points.

Simulating 800 houses with a known price formula

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(7)
n = 800
houses = pd.DataFrame({
        "size_sqm": rng.normal(110, 30, n).clip(40, 250).round(0),
        "bedrooms": rng.integers(1, 6, n),
        "age_years": rng.integers(0, 80, n),
        "distance_km": rng.uniform(0.5, 25, n).round(1)})
gross = 75000 + 1800 * houses.size_sqm + 9000 * houses.bedrooms
penalty = 900 * houses.age_years + 3500 * houses.distance_km
houses["price"] = (np.subtract(gross, penalty) + rng.normal(0, 18000, n)).round(0)
print(houses.head())
print("Rows:", len(houses))
```

1 2 3 4

1 Setup

A new seed, 7, gives an independent dataset of 800 homes.

2 Four features

Area centred on 110 square metres and clipped to a realistic range, 1 to 5 bedrooms, age 0 to 79 years, distance to the centre 0.5 to 25 km.

3 The true formula

Base 75,000 plus 1,800 per square metre plus 9,000 per bedroom, minus 900 per year of age and 3,500 per km. np.subtract performs the minus.

4 Noise and inspection

Random noise with a spread of 18,000 pounds stands for everything we did not measure, such as condition and view.

Output: the first five houses

OUTPUT

	size_sqm	bedrooms	age_years	distance_km	price
0	110.0	3	74	13.1	213607.0
1	119.0	3	67	11.7	195782.0
2	102.0	2	76	18.0	136969.0
3	83.0	2	28	18.9	126056.0
4	96.0	1	79	21.3	140286.0

Rows: 800

Four features and the target
price is what we will predict. The other four columns are inputs.

House 0: 213,607 pounds
110 square metres, 3 bedrooms but 74 years old and 13 km out. Age and distance pull the price down substantially.

House 3: the cheapest shown
Small at 83 square metres and far out at 18.9 km.

800 rows
We will use 600 for training and 200 for testing.

Output: summary of the housing data

houses.describe().round(1) gives a first feel for ranges and spread before modelling.

OUTPUT

	size_sqm	bedrooms	age_years	distance_km	price
count	800.0	800.0	800.0	800.0	800.0
mean	106.9	3.1	39.3	13.0	214235.6
std	28.1	1.4	22.6	7.1	65007.7
min	40.0	1.0	0.0	0.6	31597.0
25%	88.0	2.0	20.0	6.9	171739.0
50%	107.0	3.0	39.0	13.0	212490.0
75%	126.0	4.0	58.0	19.3	256793.8
max	186.0	5.0	79.0	24.9	427471.0

Mean price 214,236
Our baseline: predicting this for every house is the model to beat.

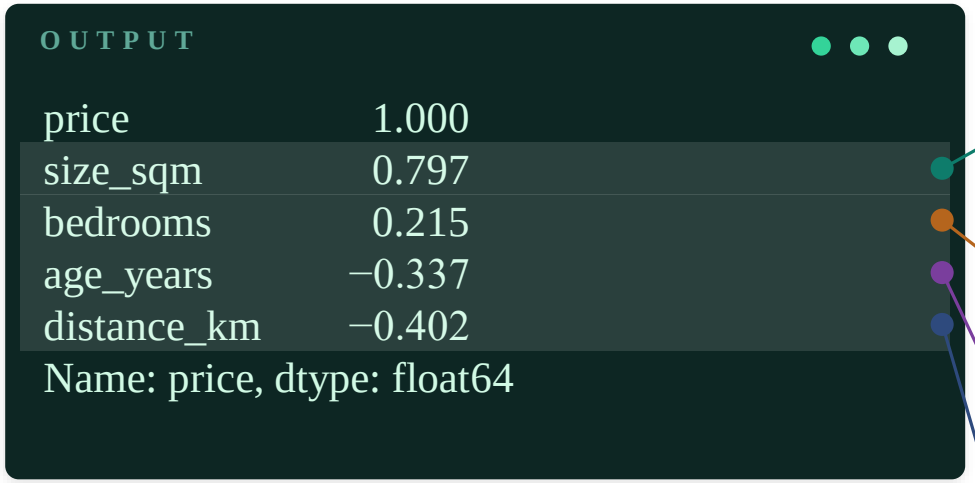
Price spread 65,008
The typical distance of a price from the average. A useful model must have errors well below this.

Cheapest house 31,597
Old, small and remote homes with bad luck in the noise. Worth checking such extremes in real data.

Area capped at 186
Our clip allowed 250 but no draw went that high. Predictions far outside the observed range are extrapolation and risky.

Output: how each feature correlates with price

houses.corr() measures straight line association between every pair of columns, from minus one to plus one.



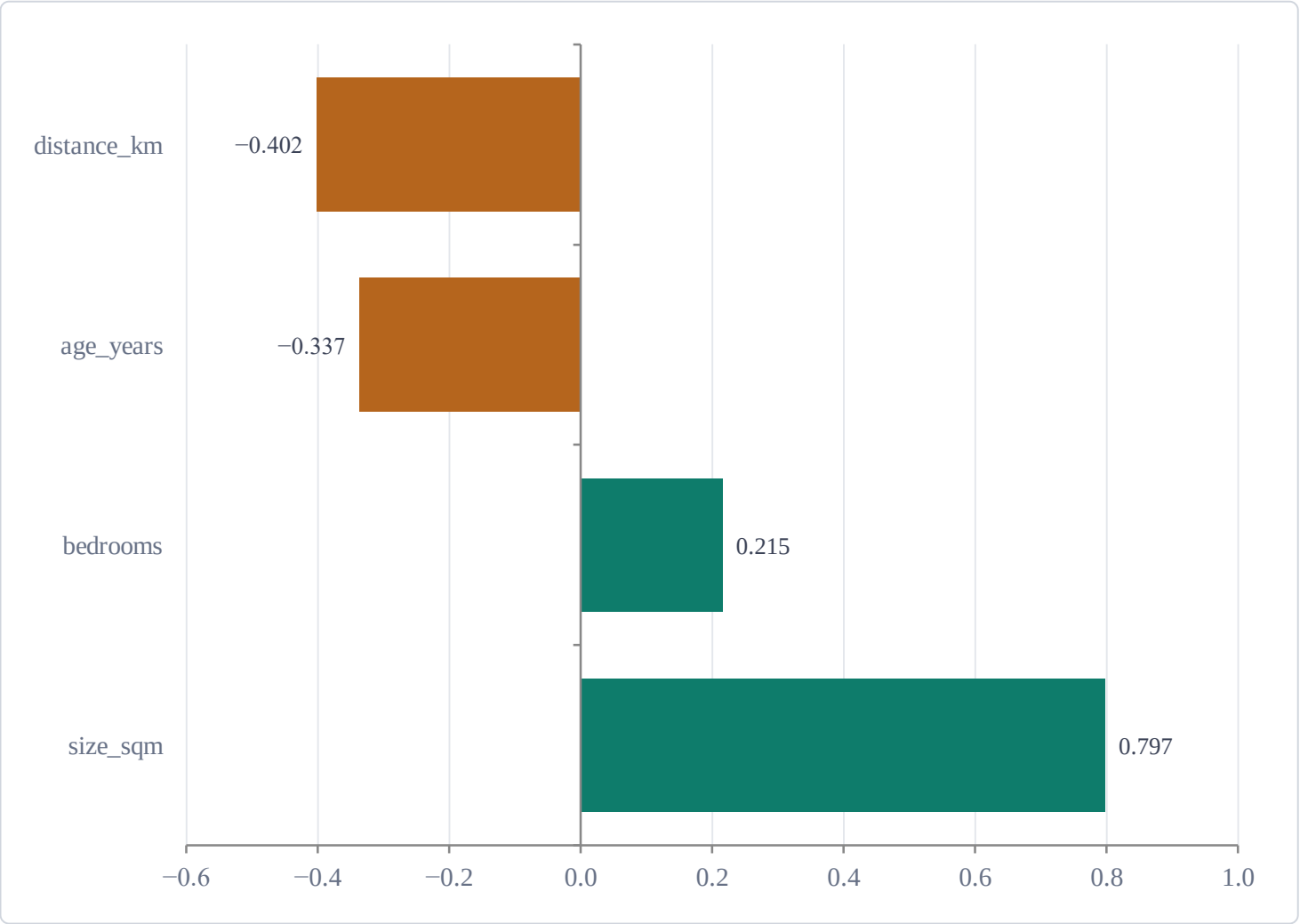
Area: 0.797
A strong positive association. Bigger homes cost more.

Bedrooms: 0.215
Weaker, although the true effect is 9,000 per room. Bedrooms vary less, so they explain less of the spread in price.

Age: minus 0.337
Older homes are cheaper. The sign matches our formula.

Distance: minus 0.402
The strongest negative relationship: each kilometre further out lowers the price.

Chart: correlation of each feature with price



1 Direction
Bars to the right mean price rises with the feature; bars to the left mean price falls.

2 Strength is not importance
Correlation ignores other features. Bedrooms look weak here, yet each bedroom truly adds 9,000 pounds.

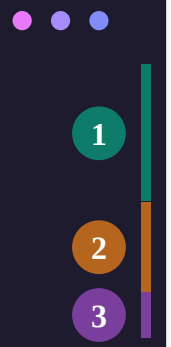
3 Only straight lines
Correlation detects linear association only. A perfect U shaped relationship can have a correlation near zero.

4 A screening tool
Use correlations to spot candidates and data errors, then let the multiple regression sort out joint effects.

Choosing features and splitting the data

PYTHON

```
1 features = ["size_sqm", "bedrooms", "age_years", "distance_km"]
2 X = houses[features]
3 y = houses["price"]
4 X_train, X_test, y_train, y_test = train_test_split(
5         X, y, test_size=0.25, random_state=42)
6 print("Training rows:", X_train.shape[0], " Testing rows:", X_test.shape[0])
```



1 Features and target

The four inputs form X and the price forms y.

2 A 75 to 25 split

No stratify argument this time: stratification needs classes, and price is continuous.

3 Output

Training rows: 600, Testing rows: 200, exactly as intended.

Fitting LinearRegression

PYTHON

```python
from sklearn.linear_model import LinearRegression
lin = LinearRegression()
lin.fit(X_train, y_train)
print("Intercept:", round(lin.intercept_, 1))
for name, coef in zip(features, lin.coef_):
        print(f"{name:<12} {coef:>10.1f}")
```

1

Create the model

LinearRegression has almost nothing to tune. By default it fits an intercept.

2

fit solves least squares

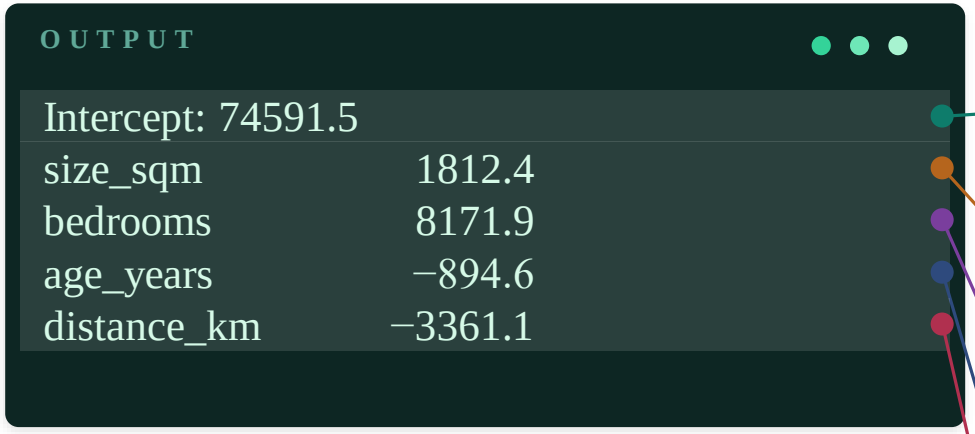
Using only the 600 training rows, it computes the one intercept and four coefficients with the smallest total squared error.

3

Read the learned parameters

intercept_ is a single number; coef_ is an array in the same order as the columns of X, so zip pairs each with its name.

Output: the learned recipe for price



Intercept 74,592
The predicted price of a zero area, zero bedroom, new house at the city centre. Meaningless physically, but it anchors the plane. Truth: 75,000.

1,812 per square metre
Holding the other three fixed. Truth: 1,800.

8,172 per bedroom
Truth: 9,000. Bedrooms vary over only five values, so this estimate is the least precise.

Minus 895 per year of age
Truth: minus 900.

Minus 3,361 per km
Truth: minus 3,500. All four signs and sizes were recovered from noisy data.

Did the model recover the truth?

Because we simulated the data, we can compare each learned value with the value that really generated the prices.

Parameter	True value	Learned value	Difference	Comment
Intercept	75,000	74,592	0.5%	Very close
size_sqm	1,800	1,812	0.7%	Area varies widely, so it is pinned down precisely
bedrooms	9,000	8,172	9.2%	Few distinct values give a noisier estimate
age_years	minus 900	minus 895	0.6%	Very close
distance_km	minus 3,500	minus 3,361	4.0%	Close; noise spread of 18,000 blurs it

Why not exact?

The 18,000 pound noise means a different sample of 600 houses would give slightly different coefficients. This sample to sample wobble is exactly the variance part of the bias variance trade off. More data would shrink it.

Interpreting coefficients without fooling yourself

1 Always state the units

Say 1,812 pounds per additional square metre, not a coefficient of 1,812. Units make the number meaningful.

2 Always say holding others fixed

The bedroom effect compares two homes of the same area, age and distance. That is different from comparing all homes with more bedrooms.

3 Size is not importance

A coefficient's magnitude depends on the feature's units. Per square metre and per km cannot be compared directly; standardised coefficients can.

4 Association is not causation

Unless the data come from an experiment, a coefficient shows association. Adding a bedroom by splitting a room will not necessarily add 8,172 pounds.

5 Beware extrapolation

The model has never seen a 400 square metre mansion. Its prediction there is a straight line guess beyond the evidence.

6 Correlated features share credit

When two features move together, the model can trade credit between them. We will see this break the coefficients shortly.

Making predictions on the test set

PYTHON

```
y_pred = lin.predict(X_test)
compare = pd.DataFrame({"actual": y_test.values[:6],
                        "predicted": y_pred[:6].round(0)})
compare["error"] = np.subtract(compare.actual, compare.predicted)
print(compare)
```

1 2 3

1 predict

Applies the learned recipe to each of the 200 test houses, producing an array of 200 predicted prices.

2 Side by side

A small table of the first six actual and predicted prices, rounded to whole pounds.

3 Error column

Actual minus predicted. Positive means the model undervalued the house; negative means it overvalued it.

Output: predictions next to reality

OUTPUT

	actual	predicted	error
0	286694.0	277841.0	8853.0
1	183065.0	182536.0	529.0
2	148175.0	172496.0	−24321.0
3	234461.0	238646.0	−4185.0
4	368295.0	331850.0	36445.0
5	166779.0	175523.0	−8744.0

Off by only 529
A near perfect prediction: this house happened to have little noise.

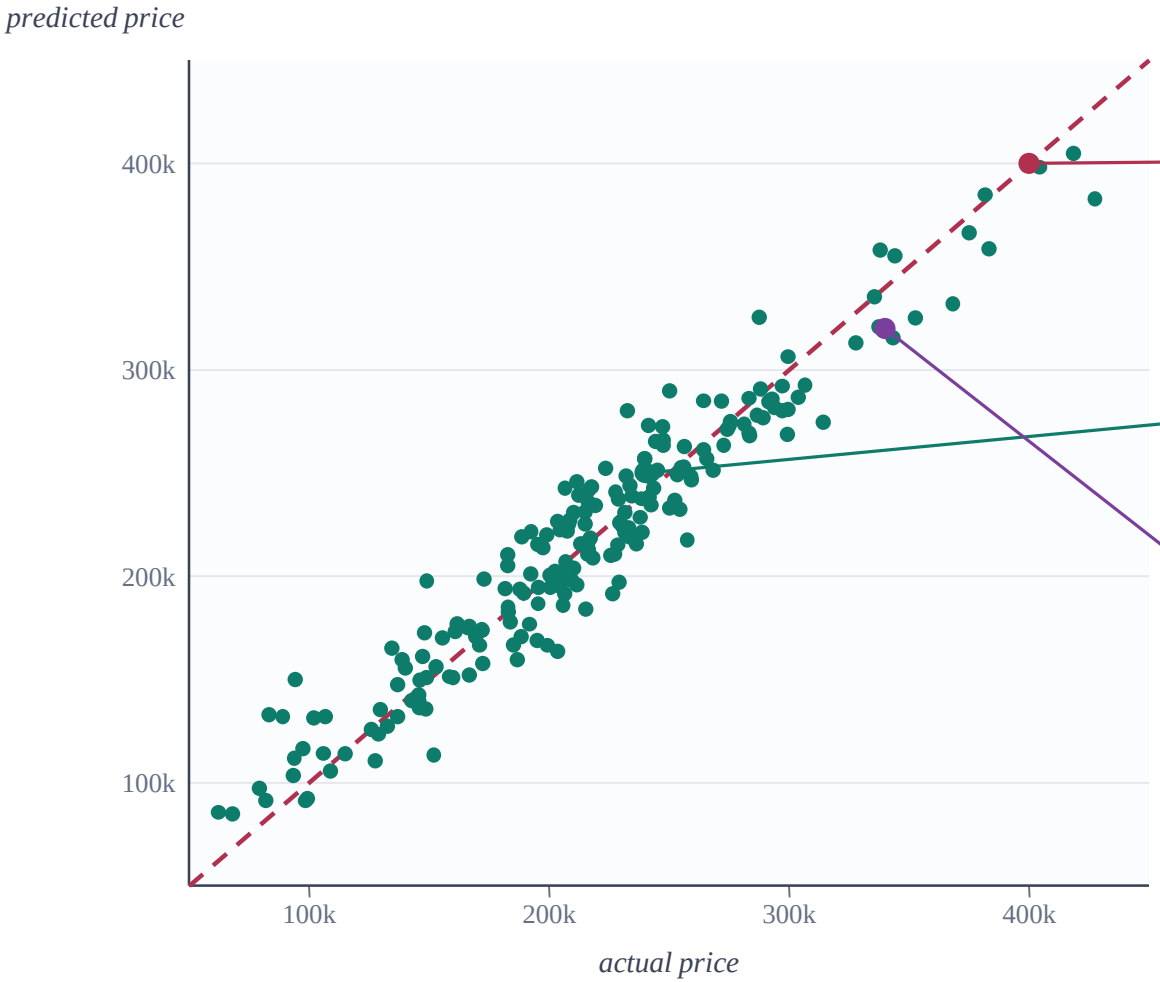
Overvalued by 24,321
Negative error: the model predicted more than the true price. Something unmeasured made this house cheaper.

Undervalued by 36,445
The model was too cautious here. Errors of this size are about two noise spreads, unusual but expected occasionally.

Reading the table
Errors are scattered in both directions with no obvious bias, which is what a well specified model produces.

Illustration: predicted versus actual prices

200 test houses. A perfect model would place every point on the diagonal line.



The diagonal is perfection
Points above it are overvalued, points below it undervalued.

A tight, even band
Points hug the diagonal across the whole price range, so accuracy does not degrade for cheap or expensive homes.

Constant width
The band does not widen at the top, so the constant spread assumption looks reasonable.

Band thickness is the noise
Its thickness reflects the 18,000 pound random noise no model can remove.

Regression metrics explained in plain language

1 Mean absolute error (MAE)

Take the size of every miss, ignoring direction, and average them. Reads directly as a typical error in pounds. Treats all misses proportionally.

2 Root mean squared error (RMSE)

Square each miss, average, then take the square root to return to pounds. Large misses count much more, so RMSE is always at least as big as MAE.

3 The gap between them

If RMSE is far above MAE, a few very large errors exist. If they are close, errors are fairly uniform in size.

4 R squared

The share of variation in the target that the model explains, compared with always predicting the mean. 1 is perfect, 0 is no better than the mean, negative is worse.

5 Compare train and test

Similar values on both sets suggest good generalisation. A much better training score signals overfitting.

6 Choose by business cost

If a few large mistakes are disastrous, focus on RMSE. If every pound of error costs the same, MAE matches the cost better.

Computing MAE, RMSE and R squared

PYTHON

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
for label, Xs, ys in [("train", X_train, y_train), ("test", X_test, y_test)]:
    p = lin.predict(Xs)
    mae = mean_absolute_error(ys, p)
    rmse = np.sqrt(mean_squared_error(ys, p))
    r2 = r2_score(ys, p)
    print(f"{label:<5} MAE {mae:>9.0f}  RMSE {rmse:>9.0f}  R2 {r2:.4f}")
```

1 Metric functions

Each takes the true values first and the predictions second.

2 Loop over both sets

Scoring training and test data side by side exposes any overfitting immediately.

3 Three metrics

RMSE is computed as the square root of mean squared error so it returns to pounds.

4 Formatted print

Fixed widths and decimal places line up the columns for easy comparison.

Output: strong and consistent performance

```
OUTPUT
train MAE    14270   RMSE    17785   R2 0.9218
test  MAE    14711   RMSE    18652   R2 0.9267
```

Training: MAE 14,270, R squared 0.922

On houses it learned from, the typical miss is about 14,000 pounds.

Test: MAE 14,711, R squared 0.927

Almost identical on unseen houses. No overfitting: a linear model with five parameters is far too simple to memorise 600 rows.

RMSE close to the noise

18,652 against our injected noise of 18,000. The model is near the best that is achievable on this data.

The test scores at a glance

14,711

MAE in pounds

A typical prediction is about 15,000 pounds away from the sale price, on homes averaging 214,000.

18,652

RMSE in pounds

Slightly larger than MAE, consistent with roughly normal errors and no extreme failures.

0.927

R squared

The model explains about 93 percent of the variation in price. The rest is the random noise we added.

How to report this to a stakeholder

On houses it has never seen, the model's price estimate is typically within about 15,000 pounds, and it captures over nine tenths of what drives price differences in this market.

Residual analysis: reading a model's mistakes

1 Mean near zero

If residuals average far from zero, the model is systematically too high or too low. Least squares with an intercept forces the training mean to zero.

2 No pattern against predictions

Plot residuals against predicted values. A shapeless horizontal cloud is good; curves or funnels reveal broken assumptions.

3 Roughly bell shaped

A histogram of residuals should be symmetric and peaked at zero. Long tails mean some cases are poorly modelled.

4 About two thirds within one spread

For bell shaped errors, roughly 68 percent of residuals fall within one standard deviation of zero.

5 Inspect the extremes

The largest residuals often point at data errors, unusual properties or a missing feature worth adding.

6 Check by segment

Residuals that are consistently positive for one area or price band reveal a group the model handles badly.

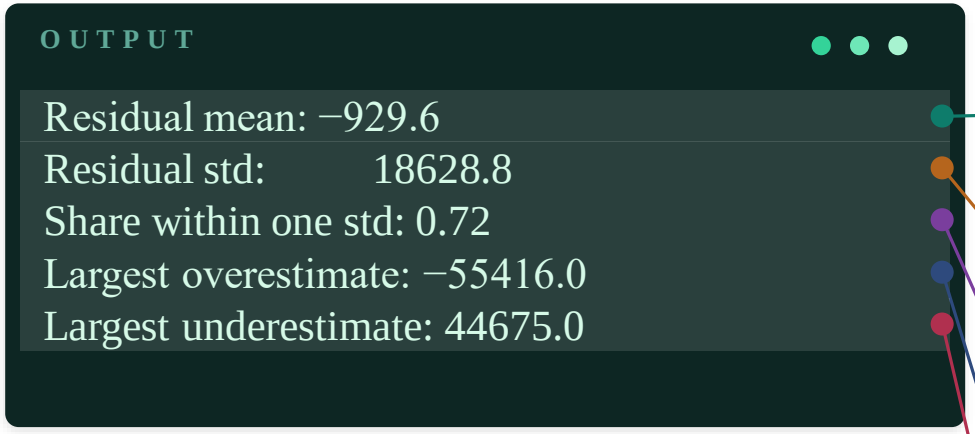
Summarising the test residuals

PYTHON

```python
residuals = np.subtract(y_test.values, y_pred)
print("Residual mean:", round(residuals.mean(), 1))
print("Residual std: ", round(residuals.std(), 1))
inside = np.abs(residuals) <= residuals.std()
print("Share within one std:", round(inside.mean(), 3))
print("Largest overestimate:", round(residuals.min(), 0))
print("Largest underestimate:", round(residuals.max(), 0))
```

1. **Residuals**
   Actual minus predicted for every test house.
2. **Centre and spread**
   The mean checks for bias; the standard deviation measures typical size.
3. **Share within one spread**
   A boolean mask, averaged, gives the proportion of houses whose error is within one standard deviation.
4. **Extremes**
   The minimum is the worst overestimate, the maximum the worst underestimate.

Output: residuals behave as theory predicts



Mean minus 930
Tiny compared with prices around 214,000, so there is no meaningful bias on the test set.

Spread 18,629
Matches the injected noise of 18,000.

72 percent within one spread
Close to the 68 percent expected for bell shaped errors.

Worst overestimate 55,416
About three spreads out: rare but plausible with 200 houses.

Worst underestimate 44,675
No single catastrophic miss dominates.

Illustration: residuals against predictions

The most important diagnostic plot for any regression model. We want a featureless horizontal band.



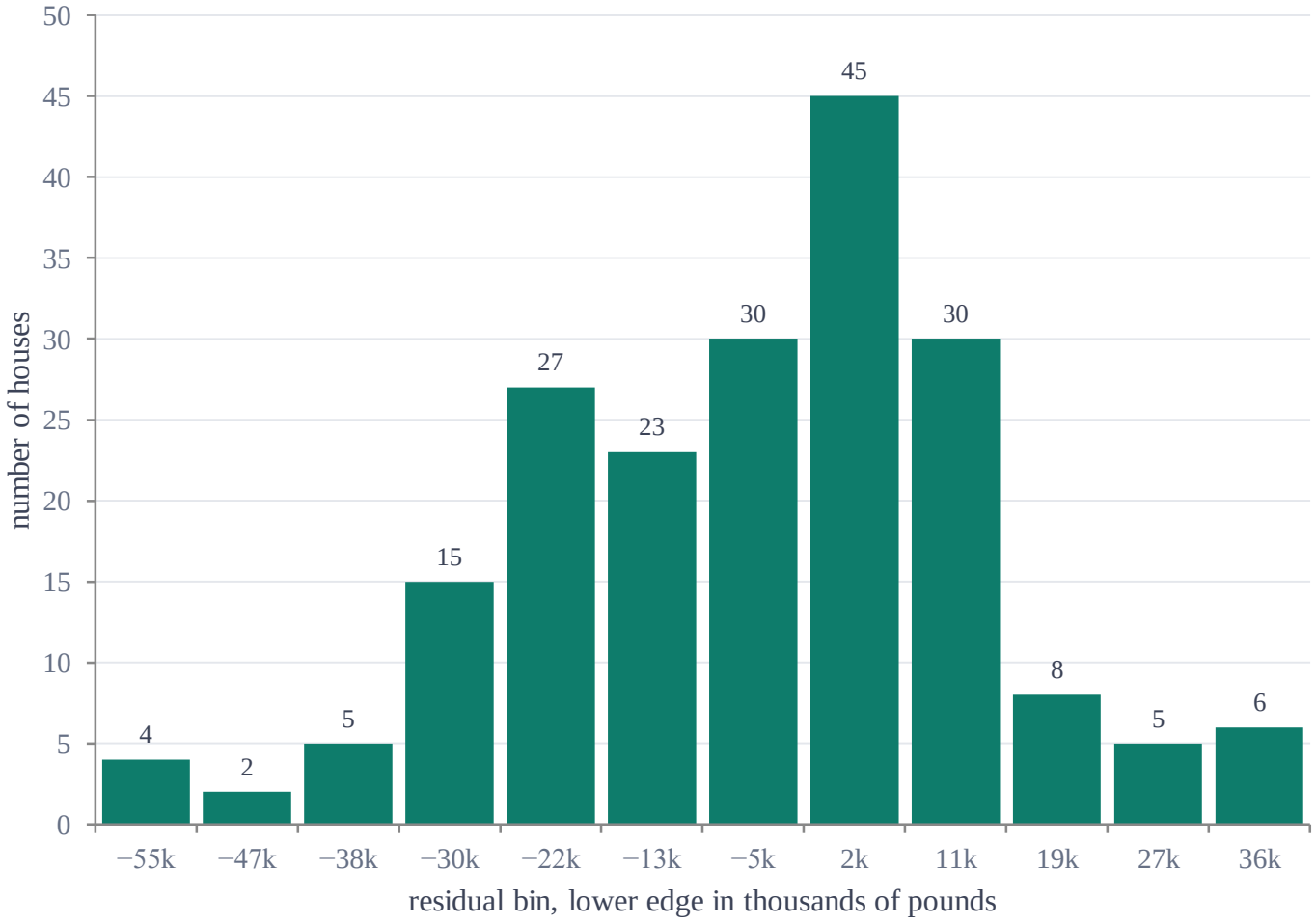
Zero line
Points above were undervalued, points below overvalued. They balance on both sides.

Shaded band: one spread
Most points sit inside the band of plus or minus 18,629 pounds.

No curve, no funnel
Spread is similar for cheap and expensive homes.

Verdict
The linear model is well specified for this data.

Chart: the distribution of test residuals



1 Peak near zero
The tallest bars sit around zero: most predictions are close.

2 Roughly symmetric
The left and right tails have similar weight, so over and under predictions are balanced.

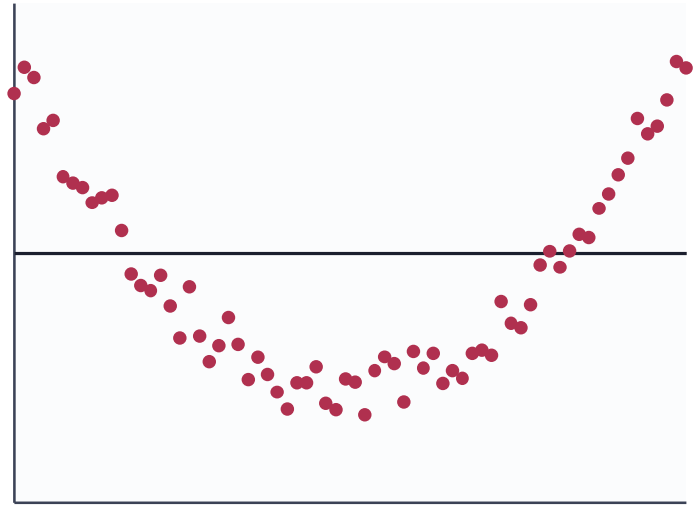
3 Thin tails
Only a handful of houses fall beyond 40,000 pounds of error in either direction.

4 What would worry us
Two humps would suggest a hidden group; a long one sided tail would suggest a skewed target that might need a log transform.

Illustration: residual patterns that signal trouble

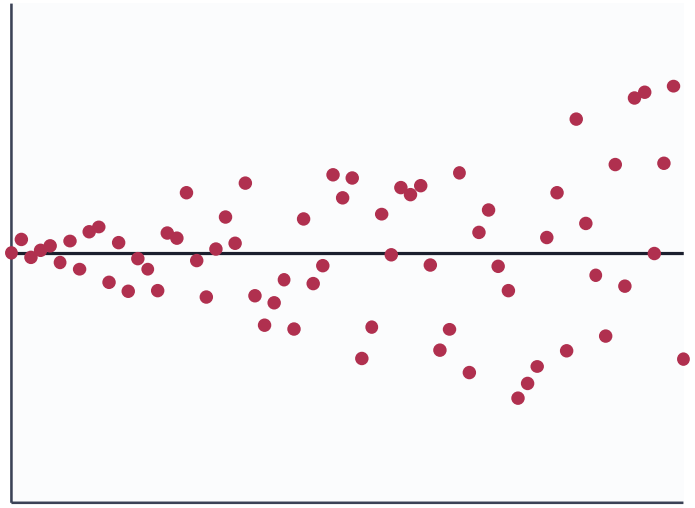
Three illustrative failure shapes. If your residual plot looks like any of these, the plain linear model needs changing.

Curve



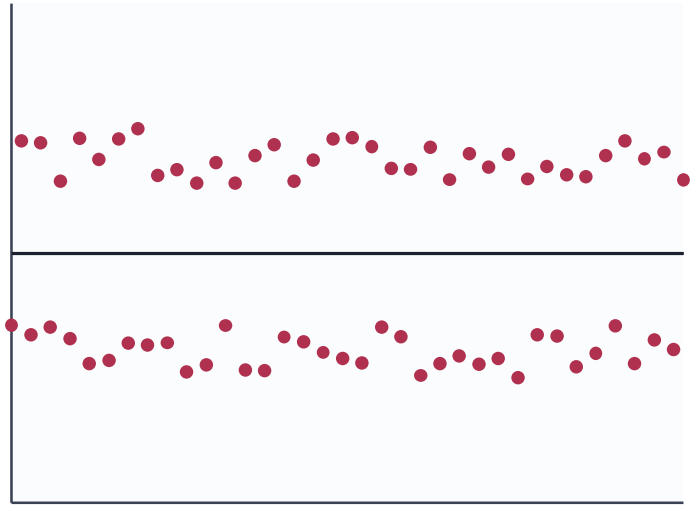
A U shape means the true relationship bends. Add polynomial terms or transform the feature.

Funnel



Spread grows with the prediction. Errors are larger for bigger values; consider predicting the logarithm of the target.

Two bands



Separate groups sit above and below zero, revealing a missing categorical feature such as area type.

Standardised coefficients for a fair comparison

PYTHON

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
std_lin = make_pipeline(StandardScaler(), LinearRegression()).fit(X_train, y_train)
effects = pd.Series(std_lin[1].coef_, index=features)
print("Price change for a one standard deviation increase:")
print(effects.reindex(effects.abs().sort_values(ascending=False).index).round(0))
```

1 Imports

A pipeline chaining StandardScaler and LinearRegression.

2 Fit on scaled features

Each feature now has mean 0 and spread 1 in training data, so every coefficient is measured in the same currency: pounds per standard deviation.

3 Rank by absolute size

std_lin[1] reaches the second step of the pipeline. Sorting by absolute value ranks the features by influence.

Output: which features move price the most

OUTPUT

Price change for a one standard deviation increase:

size_sqm	50782.0
distance_km	−23829.0
age_years	−20387.0
bedrooms	11625.0

dtype: float64

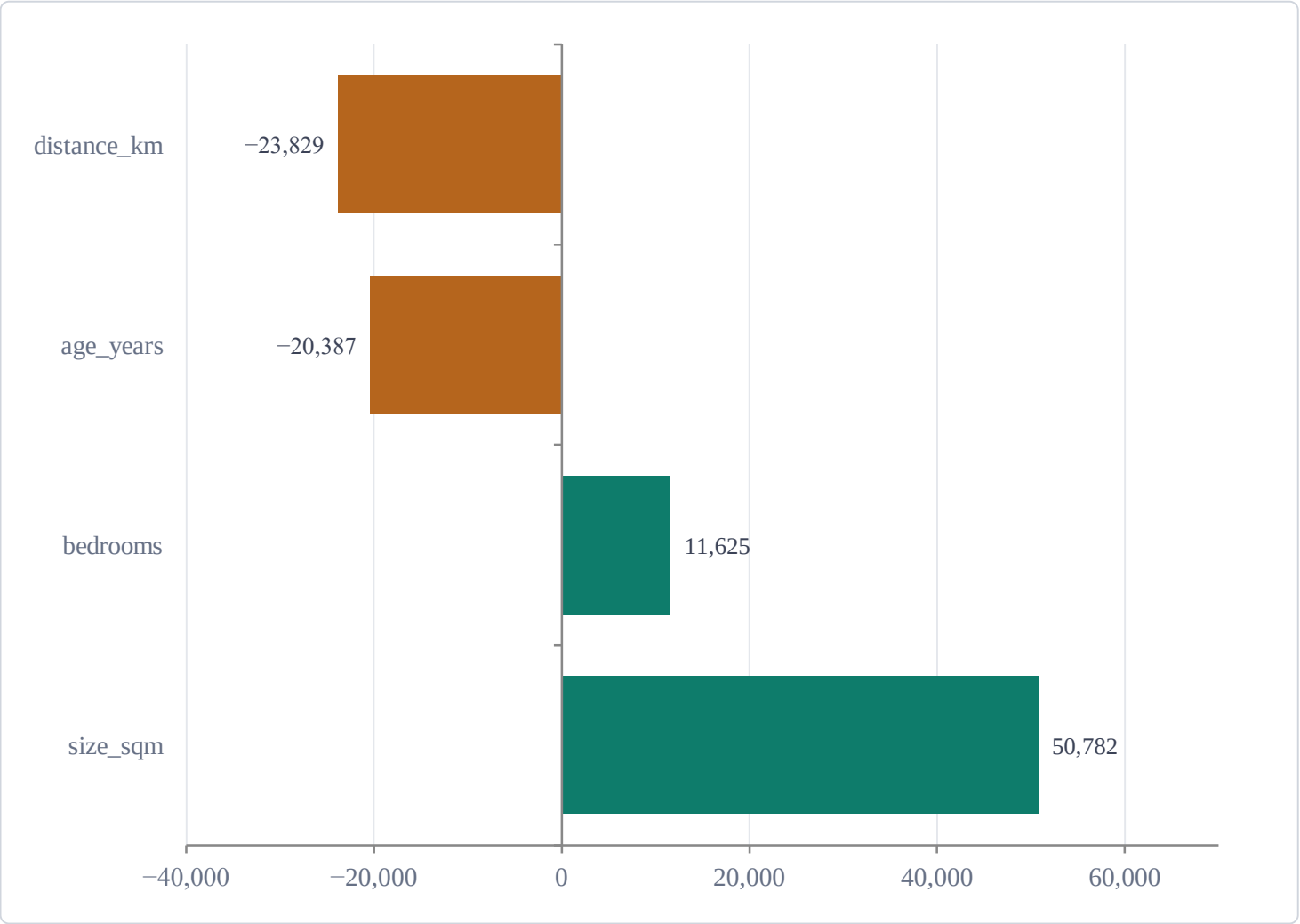
Area dominates: 50,782
A typical swing in floor area, about 28 square metres, moves price by about 51,000 pounds.

Distance: minus 23,829
A typical change of about 7 km lowers price by roughly 24,000.

Age: minus 20,387
About 23 extra years of age costs around 20,000.

Bedrooms: 11,625
Smallest standardised effect, even though the raw coefficient of 8,172 looked large. Units can mislead.

Chart: standardised effect of each feature



1. **Same currency**
   Every bar is pounds per one standard deviation of its feature, so lengths are directly comparable.

2. **Sign shows direction**
   Bars to the left reduce price.

3. **Ranking for stakeholders**
   Floor area first, then location, then age, then bedrooms: a story a business audience can act on.

4. **Predictions unchanged**
   Scaling only changes the units of coefficients. The predictions and R squared are identical to the unscaled model.

Simple regression on area alone

PYTHON

```python
simple = LinearRegression().fit(X_train[["size_sqm"]], y_train)
print("Slope per sqm:", round(simple.coef_[0], 1))
print("Intercept:", round(simple.intercept_, 1))
print("Simple R2 test:  ", round(simple.score(X_test[["size_sqm"]], y_test), 4))
print("Multiple R2 test:", round(lin.score(X_test, y_test), 4))
```

1 **A single feature**

Double brackets keep X two dimensional, which scikit learn always requires, even for one column.

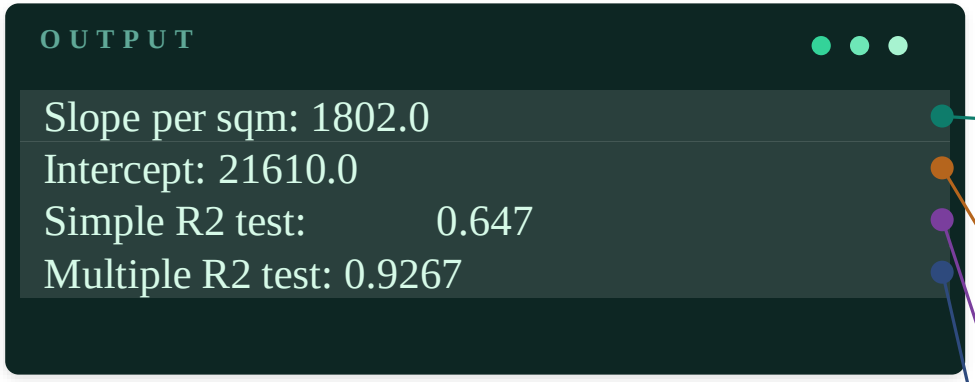
2 **Slope and intercept**

The line drawn on the earlier anatomy illustration.

3 **Compare fit quality**

R squared on test data for one feature versus all four.

Output: one feature explains far less



Slope 1,802
Very close to the multiple regression value, because area is uncorrelated with the other features in our simulation.

Intercept 21,610
Very different from 74,592: it now absorbs the average effects of the missing bedrooms, age and distance.

R squared 0.647
Area alone explains about 65 percent of price variation.

R squared 0.927
Adding three more features explains a further 28 percentage points. Omitted variables were the main source of error.

Multicollinearity: when features tell the same story

If two features are strongly correlated with each other, the model cannot tell which deserves the credit.

1 What it is

Two or more features carry nearly the same information, such as area in square metres and area in square feet.

2 What breaks

Coefficients become unstable: small changes in the data swing credit from one feature to the other, even flipping signs.

3 What survives

Predictions and R squared are usually fine, because the combined effect is still estimated well. Only interpretation suffers.

4 How to detect it

The variance inflation factor (VIF) asks how well each feature can be predicted from all the others. Values above about 5 to 10 are a warning.

5 How to fix it

Drop one of the duplicates, combine them into a single feature, or apply ridge regularisation which shares credit sensibly.

6 Why it matters here

Many real datasets contain overlapping measures, such as income and spending, or several size measurements of the same object.

Creating a duplicate feature and measuring VIF

PYTHON

```python
houses["size_sqft"] = (houses.size_sqm * 10.764 + rng.normal(0, 4, n)).round(0)
cols = features + ["size_sqft"]
def vif(frame, col):
    others = [c for c in frame.columns if c != col]
    r2 = LinearRegression().fit(frame[others], frame[col]).score(frame[others], frame[col])
    return 1 / np.subtract(1, r2)
for c in cols:
    print(f"{c:<12} VIF {vif(houses[cols], c):>8.1f}")
```

1

2

3

1 A near copy

Square feet is square metres times 10.764 plus a little measurement noise, so it is almost perfectly correlated with size_sqm.

2 VIF from scratch

Regress one feature on all the others and read the R squared. VIF is one divided by one minus that R squared, so it explodes as R squared approaches 1.

3 Every feature

Compute the VIF of each of the five columns.

Output: an enormous VIF for both area measures

OUTPUT

size_sqm	VIF	5879.1
bedrooms	VIF	1.0
age_years	VIF	1.0
distance_km	VIF	1.0
size_sqft	VIF	5879.2

VIF 5,879 for size_sqm
The other features predict square metres almost perfectly, through square feet. Anything above 10 is serious; this is extreme.

VIF 1.0 for the rest
Bedrooms, age and distance are unrelated to every other feature, the ideal situation.

VIF 5,879 for size_sqft
The problem is symmetric: both members of the pair are flagged.

Regularisation: ridge versus lasso

Both add a penalty for large coefficients to the least squares score. The strength is set by alpha, which you choose.

Ridge (L2 penalty)

- Penalises the sum of squared coefficients.
- Shrinks all coefficients towards zero smoothly but rarely makes any exactly zero.
- With correlated features it spreads credit evenly between them, which stabilises the estimates.
- Good default when you believe most features matter a little.

vs

Lasso (L1 penalty)

- Penalises the sum of absolute coefficients.
- Can push coefficients to exactly zero, performing automatic feature selection.
- With correlated features it tends to pick one and drop or shrink the other, somewhat arbitrarily.
- Good when you believe only a few features truly matter. Always scale features first for both.

Comparing OLS, ridge and lasso on collinear data

PYTHON

```python
from sklearn.linear_model import Ridge, Lasso
Xc_train, Xc_test = X_train.join(houses.size_sqft), X_test.join(houses.size_sqft)
models = {"OLS": LinearRegression(),
          "Ridge": make_pipeline(StandardScaler(), Ridge(alpha=10)),
          "Lasso": make_pipeline(StandardScaler(), Lasso(alpha=2000, max_iter=20000))}
for name, m in models.items():
    m.fit(Xc_train, y_train)
    coefs = m.coef_ if name == "OLS" else m[1].coef_
    print(f"{name:<6} R2 {m.score(Xc_test, y_test):.4f}  sqm {coefs[0]:>9.1f}  sqft {coefs[4]:>9.1f}")
```

1 **Add the duplicate**

join attaches size_sqft to the training and test feature tables by matching row index.

2 **Three models**

Plain least squares, ridge with alpha 10 and lasso with alpha 2,000, the latter two after scaling. Lasso's alpha is large because prices are large numbers.

3 **Compare**

Test R squared plus the two area coefficients. OLS coefficients are in raw units; ridge and lasso are per standard deviation.

Output: OLS splits credit absurdly, ridge shares it

OUTPUT

OLS	R2 0.9265	sqm	−175.6	sqft	184.7
Ridge	R2 0.9258	sqm	25018.7	sqft	25344.4
Lasso	R2 0.9182	sqm	13222.6	sqft	35545.4

OLS: minus 176 per square metre

Nonsense on its own: bigger homes cheaper? Yet plus 184.7 per square foot times 10.764 is about 1,988 per square metre. Net effect 1,812, still correct. Predictions fine, coefficients uninterpretable.

Ridge: an even split

About 25,000 each per standard deviation. Credit is shared sensibly and both signs are positive. R squared 0.926.

Lasso: uneven split

Lasso leaned towards square feet. With stronger alpha it would drop one feature entirely. R squared dips to 0.918 from the extra shrinkage.

Choosing ridge alpha with cross validation

PYTHON

```python
from sklearn.model_selection import cross_val_score
for a in [0.01, 1, 10, 100, 1000, 10000]:
    pipe = make_pipeline(StandardScaler(), Ridge(alpha=a))
    s = cross_val_score(pipe, Xc_train, y_train, cv=5, scoring="r2")
    print(f"alpha {a:>8}: mean R2 {s.mean():.4f}")
```

1 2 3

1 Cross validation

Each alpha is judged on five folds of training data only. The test set stays untouched.

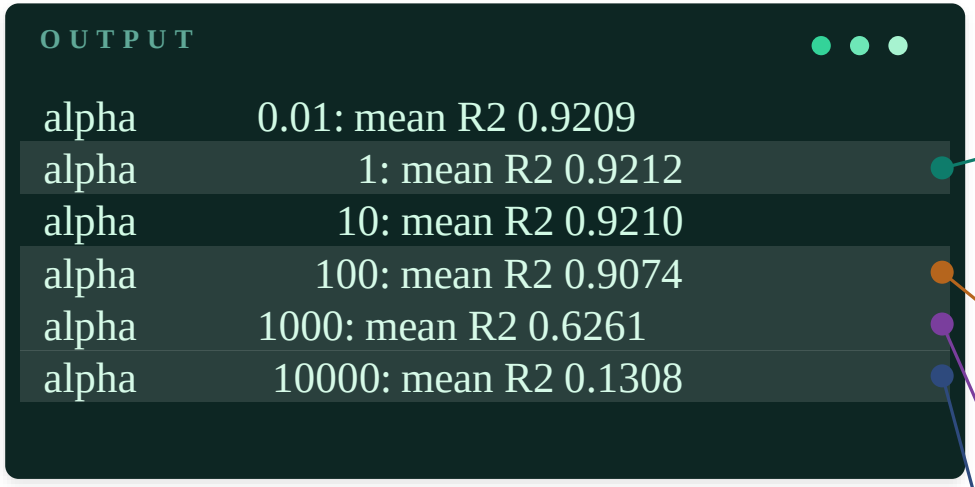
2 A wide sweep

Alpha from 0.01 to 10,000 on a roughly logarithmic grid, because its effect operates on a multiplicative scale.

3 Score and report

Mean R squared across folds for each alpha.

Output: small alphas are best for this data



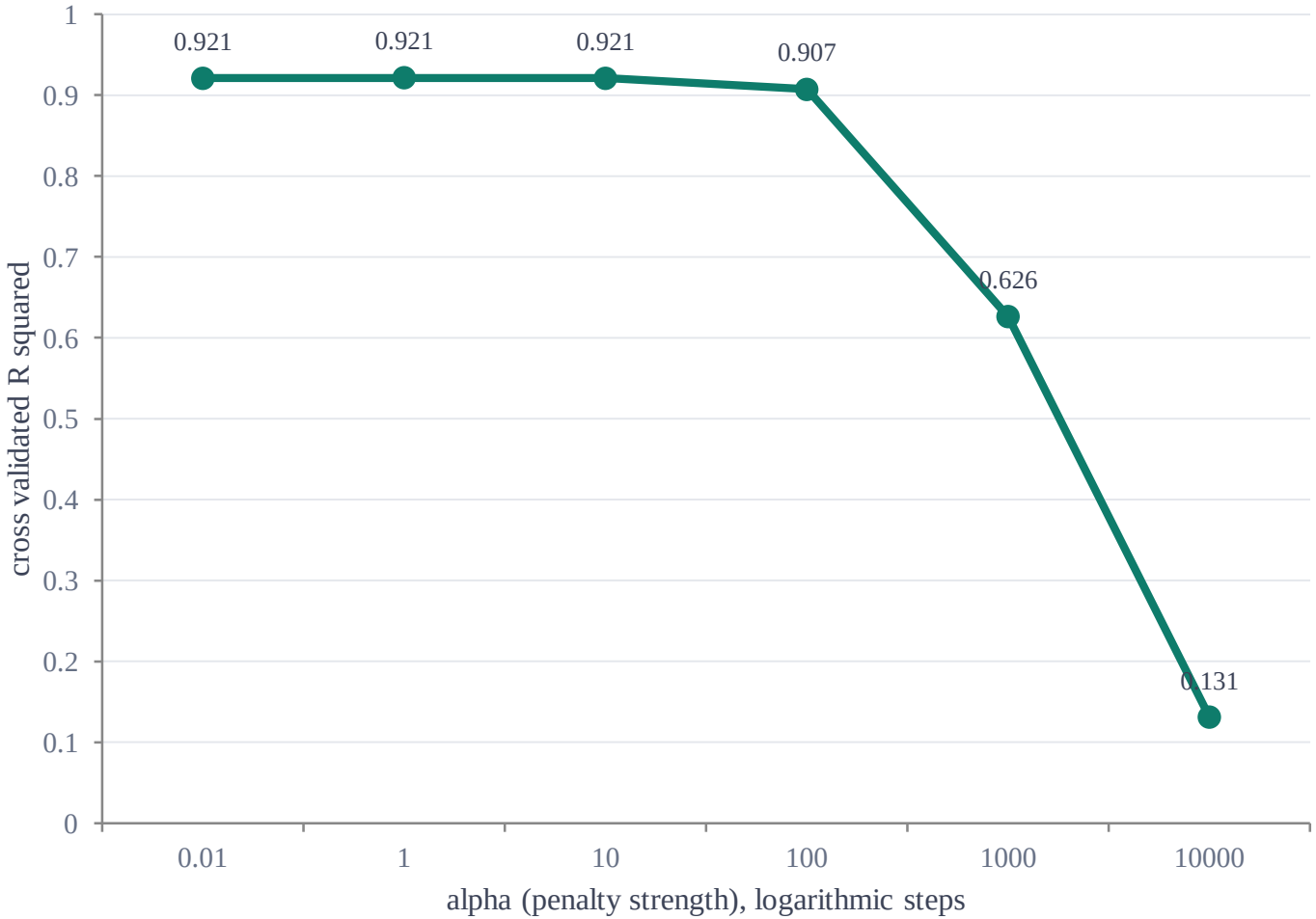
alpha 1: 0.9212, the best
A little shrinkage helps marginally. The differences between 0.01 and 10 are tiny.

alpha 100: starting to hurt
R squared falls to 0.907 as real effects get shrunk.

alpha 1,000: underfitting
0.626. The penalty now dominates and coefficients are crushed towards zero.

alpha 10,000: almost the mean
0.131. The model barely differs from predicting the average price.

Chart: cross validated R squared against ridge alpha



1 Flat on the left
When the penalty is weak, ridge behaves like ordinary least squares.

2 A cliff on the right
Beyond about 100 the penalty overwhelms the data and the model underfits.

3 The general lesson
Regularisation trades a little bias for less variance. With 600 rows and 5 features variance is already low, so little is gained here. With many features and few rows the curve would peak in the middle.

Bending a straight line: polynomial features

Linear regression is linear in its coefficients, not necessarily in its inputs. Adding powers of a feature lets the prediction curve.

1 The trick

PolynomialFeatures adds new columns such as speed squared and speed cubed. The model still adds weighted columns, but the result is a curve in speed.

2 Degree controls flexibility

Degree 2 allows one bend, degree 3 two bends. High degrees wiggle and overfit, exactly as the foundations experiment showed.

3 Interactions

With several features it also creates products such as area times bedrooms, letting one feature's effect depend on another.

4 Scale first

Powers of large numbers become enormous. Standardising before expanding keeps the numbers well behaved.

5 Interpretation gets harder

Individual coefficients of squared terms are hard to explain. Describe the shape of the curve instead.

6 Use validation

Choose the degree by test or cross validated score, never by training score, which always improves with degree.

Fuel economy that peaks at a middle speed

PYTHON

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
rng = np.random.default_rng(3)
speed = rng.uniform(20, 130, 300).reshape(300, 1)
peak = np.abs(np.subtract(speed.ravel(), 75))
mpg = 62 + rng.normal(0, 2.5, 300)
mpg = np.subtract(mpg, 0.006 * peak ** 2)
s_tr, s_te, m_tr, m_te = train_test_split(speed, mpg, random_state=1)
for deg in [1, 2, 3, 8]:
    model = make_pipeline(StandardScaler(), PolynomialFeatures(deg), LinearRegression()).fit(s_tr, m_tr)
    print(f"degree {deg}: train R2 {model.score(s_tr, m_tr):.3f}  test R2 {model.score(s_te, m_te):.3f}")
```

1 2 3 4

1 Imports

Scaling, polynomial expansion and linear regression will be chained in one pipeline.

2 A curved truth

Economy peaks at 75 km per hour and falls away on both sides, plus noise. The distance from the peak is squared to make the curve.

3 Split

Default test size of 25 percent.

4 Four degrees

Each degree gets its own pipeline; training and test R squared are printed for each.

Output: a straight line fails, a curve succeeds

```
OUTPUT
degree 1: train R2 0.001    test R2 −0.148
degree 2: train R2 0.834    test R2 0.768
degree 3: train R2 0.839    test R2 0.752
degree 8: train R2 0.844    test R2 0.744
```

Degree 1: R squared near zero
A straight line through a hump is flat, explaining nothing. Test R squared is even negative: worse than predicting the mean.

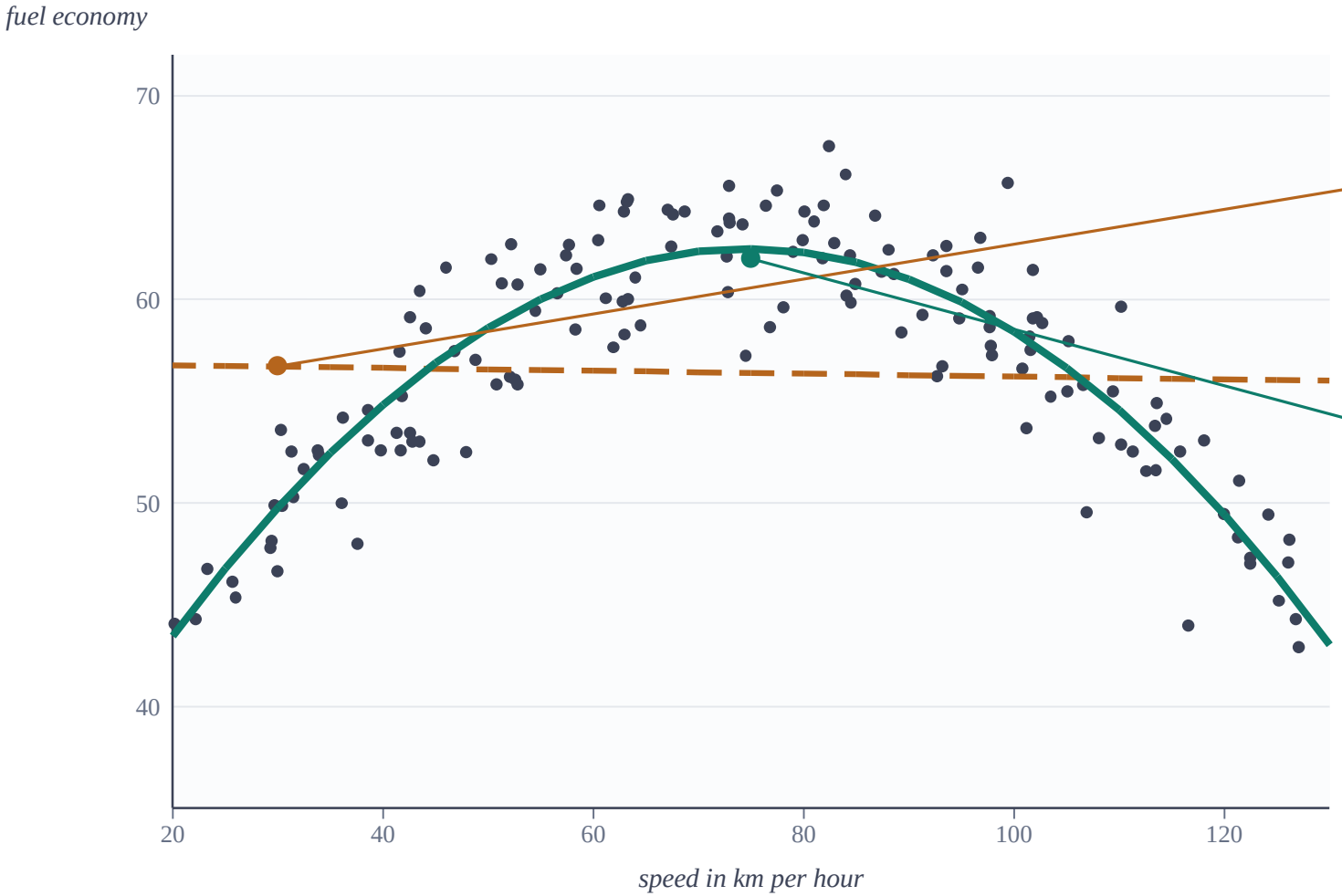
Degree 2: 0.768 on test
One bend is exactly the true shape. The best test score.

Degree 3: slightly worse
Training improves trivially, test falls a little: the extra flexibility begins fitting noise.

Degree 8: same story
Still decent thanks to scaling, but no better than degree 2. Simpler wins ties.

Illustration: straight line versus quadratic fit

150 cars. The dashed line is the degree 1 model, the solid curve is the degree 2 model, both fitted on the same training rows.



Degree 1 (dashed)
Flat because rising and falling halves cancel out. It cannot represent a peak.

Degree 2 (solid)
Bends once and finds the sweet spot near 75 km per hour, matching the true formula.

Still linear regression
The model is a weighted sum of speed and speed squared. Only the inputs changed, not the algorithm.

Predicting a new house and cross validating

PYTHON

```python
new_home = pd.DataFrame({"size_sqm": [95], "bedrooms": [3],
                         "age_years": [12], "distance_km": [4.5]})
print("Predicted price:", round(lin.predict(new_home)[0], 0))
cv = cross_val_score(LinearRegression(), X, houses.price.values, cv=5, scoring="neg_mean_absolute_error")
print("Cross validated MAE:", round(np.abs(cv).mean(), 0))
```

1 2 3

1 A new listing

95 square metres, 3 bedrooms, 12 years old, 4.5 km out. Column names must match training exactly.

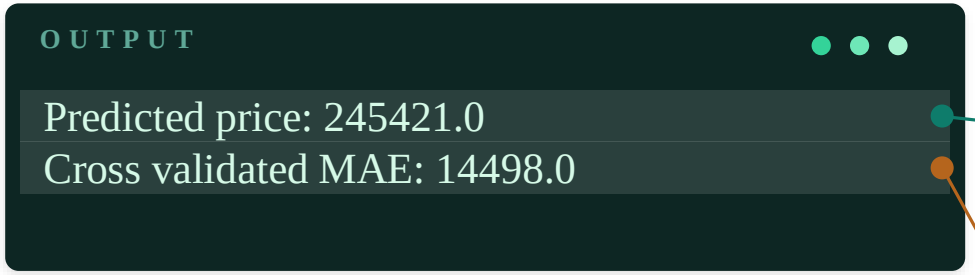
2 predict returns an array

Indexing with [0] extracts the single prediction.

3 Cross validated MAE

scikit learn maximises scores, so error metrics are offered negated. np.abs turns them back into positive pounds.

Output: a price and an honest error estimate

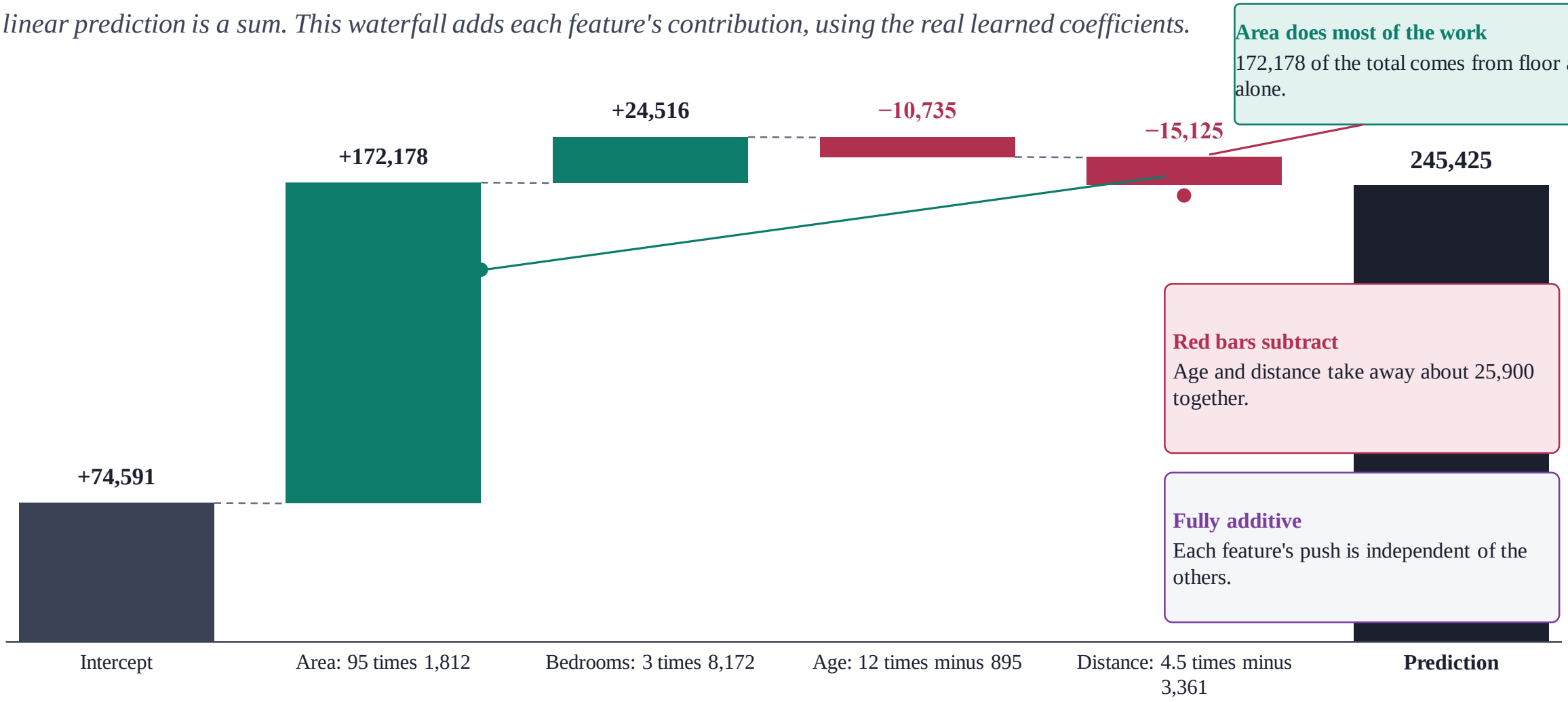


245,421 pounds
The true formula would give 246,450 before noise. The model is within about 1,000 pounds of the truth.

Cross validated MAE 14,498
The typical error to expect, so a fair statement is about 245,000, give or take 15,000.

Illustration: how the 245,421 pound prediction is built

A linear prediction is a sum. This waterfall adds each feature's contribution, using the real learned coefficients.



Adding a categorical feature: area type

PYTHON

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
houses["area"] = rng.choice(["City", "Suburb", "Rural"], n, p=[0.3, 0.5, 0.2])
houses["price"] = houses.price + houses.area.map({"City": 30000, "Suburb": 10000, "Rural": 0})
prep = ColumnTransformer([("num", "passthrough", features),
                          ("cat", OneHotEncoder(drop="first"), ["area"])])
area_model = make_pipeline(prep, LinearRegression())
Xa = houses[features + ["area"]]
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(Xa, houses.price, test_size=0.25, random_state=42)
area_model.fit(Xa_tr, ya_tr)
names = area_model[0].get_feature_names_out()
for nm, cf in zip(names, area_model[1].coef_):
    print(f"{nm:<20} {cf:>10.1f}")
print("Test R2:", round(area_model.score(Xa_te, ya_te), 4))
```

1 **Encoders**

OneHotEncoder converts the category into 0 or 1 columns, ColumnTransformer routes the columns.

2 **A new truth**

Each house is City, Suburb or Rural. City homes gain 30,000 and suburban homes 10,000 relative to rural.

3 **drop set to first**

Drops the first category alphabetically, City, which becomes the reference level. This avoids the dummy variable trap.

4 **Fit and read**

Coefficients are printed with their generated names, then the test R squared.

Output: category effects relative to City

OUTPUT

num__size_sqm	1818.2
num__bedrooms	8105.4
num__age_years	−895.2
num__distance_km	−3340.0
cat__area_Rural	−36949.8
cat__area_Suburb	−22912.7
Test R2: 0.9242	

Numeric effects unchanged

Area, bedrooms, age and distance keep the same coefficients as before, since area type is unrelated to them.

Rural: minus 36,950

A rural home is worth about 37,000 less than an identical City home. The true gap is 30,000; the difference is sampling noise.

Suburb: minus 22,913

Versus a true gap of 20,000. The order Rural, Suburb, City is recovered.

Test R squared 0.924

Strong fit despite the new source of variation, because the model now measures it.

Reading categorical coefficients and the dummy trap

1 Reference level

With drop set to first, every category coefficient is a difference from the dropped category, here City. Change the reference and the numbers change, but predictions do not.

2 The dummy variable trap

If all three dummy columns were kept alongside an intercept, any one column would equal one minus the others: perfect multicollinearity.

3 Regularised models

Ridge and lasso can keep all dummies because the penalty resolves the ambiguity, so drop is often left unset there.

4 Rare categories

A category seen only a handful of times gets a noisy coefficient. Consider grouping rare levels into Other.

5 Unseen categories

Set handle_unknown to ignore so new categories at prediction time are treated as the reference rather than causing an error.

6 Ordinal alternatives

When categories have a natural order with roughly even steps, a single integer column can be simpler and just as accurate.

An end to end linear regression checklist

1

Explore

Describe, plot and correlate. Check ranges, skew, missing values and outliers.

2

Prepare

Split first. Encode categories, impute gaps and scale inside a pipeline.

3

Fit

Start with plain least squares as the baseline. Read every coefficient with units.

4

Evaluate

MAE, RMSE and R squared on test data, compared with training and a dummy model.

5

Diagnose

Residual plots, VIF and segment checks. Look for curves, funnels and groups.

6

Improve

Polynomial terms for curves, ridge or lasso for collinearity, new features for patterns.

Check your understanding

A model has training R squared 0.93 and test R squared 0.92. A colleague says it must be overfitting because 0.93 is so high. Are they right?

A Yes, anything above 0.9 overfits

B No, train and test agree closely

C Yes, the test score is lower

D It depends only on the intercept

Answer B and why

Overfitting is diagnosed by a gap between training and test performance, not by a high score. Here the two scores are almost identical, which is exactly what our housing model showed. A high R squared simply means the features explain most of the variation.

Check your understanding

Two features have a VIF above 5,000 and one of them has a negative coefficient that contradicts common sense. What is the best response?

A Delete the target

B Report the negative effect as a finding

C Drop one duplicate or use ridge

D Increase the test size

Answer C and why

Extreme VIF values mean the features are near copies, so least squares can split credit between them arbitrarily, as our OLS fit did with minus 176 per square metre. Dropping a duplicate or using ridge, which shares credit evenly, restores interpretable coefficients without hurting predictions.

SECTION

3

Logistic Regression

Turning a weighted sum into a calibrated probability, then into a decision

- Why linear regression fails for classes
- Log odds, the sigmoid curve and probabilities
- How the model is trained and what it learns
- Confusion matrix, precision, recall and F1
- ROC curves, AUC, log loss and choosing a threshold
- Coefficients as odds multipliers, and regularisation with C
- Class imbalance, multiclass problems and cross validation

The classification problem we will solve

Given 30 measurements of cell nuclei from a breast mass image, predict whether the mass is malignant or benign.

1 The dataset

569 real cases from the Wisconsin diagnostic breast cancer study, shipped with scikit learn. Each row is one patient's fine needle aspirate image.

2 The features

Radius, texture, perimeter, area, smoothness and more, each summarised three ways: the mean, the standard error and the worst value across nuclei.

3 The target

0 means malignant, 1 means benign. Note the coding: the positive class 1 is the healthy outcome, which matters when reading precision and recall.

4 Why probabilities

A clinician needs more than a label. A 51 percent and a 99 percent prediction of benign deserve very different follow up.

5 Unequal error costs

Missing a malignant case is far worse than a false alarm that triggers a biopsy. Our choice of threshold must reflect that.

6 An educational example

This is a teaching dataset. Real diagnostic models require clinical validation, larger samples and regulatory review.

Loading the breast cancer data

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print("Shape:", X.shape)
print("Classes:", [str(c) for c in data.target_names])
print(y.value_counts().rename({0: "malignant (0)", 1: "benign (1)"}))
print(X.columns[:6].tolist())
```

1. **Load as a DataFrame**
   as_frame set to True returns pandas objects with named columns instead of bare numpy arrays.
2. **Features and target**
   data.data holds the 30 features; data.target holds the 0 or 1 diagnosis.
3. **Inspect**
   Shape, class names, class counts and the first six feature names. rename makes the counts self explanatory.

Output: 569 patients, 30 features, two classes

OUTPUT

```
Shape: (569, 30)                                                                    1
Classes: ['malignant', 'benign']                                                    2
target
benign (1)          357                                                             3
malignant (0)       212
Name: count, dtype: int64
['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness', 'mean compactness']   4
```

1 (569, 30)

A small but rich dataset: many features per case.

2 Class order

Index 0 is malignant, index 1 is benign. Probability columns from the model follow this same order.

3 357 benign, 212 malignant

Roughly 63 to 37. Mildly imbalanced, so we will stratify every split.

4 Mean features

The first ten columns are means across nuclei; the next ten are standard errors; the last ten are worst values.

Why not just use linear regression on 0 and 1?

1 Outputs escape the range

A straight line keeps rising and falling forever, so it predicts values below 0 and above 1, which cannot be probabilities.

2 Wrong error shape

Least squares punishes a confident correct prediction of 1.3 for a true 1, which makes no sense for classes.

3 Constant effects are unrealistic

Moving from 50 to 60 percent probability should be easier than moving from 98 to 108 percent, which is impossible. Effects must taper near the extremes.

4 Outliers tilt the line

A few extreme cases far from the boundary drag a regression line and shift the cutoff for everyone else.

5 Logistic regression fixes all four

It keeps a weighted sum inside the model but passes it through an S shaped curve that always lands between 0 and 1.

6 Let us prove the first problem

The next code fits a plain linear regression to the diagnosis and counts the impossible outputs.

Fitting linear regression to a class label

PYTHON

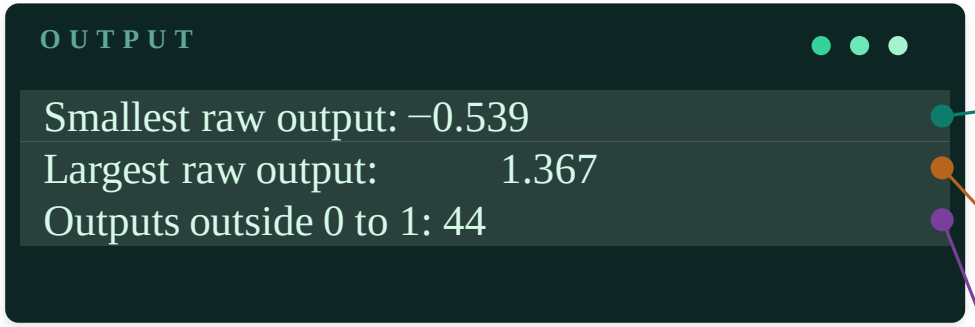
```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y)
straight = LinearRegression().fit(X_train, y_train)
raw = straight.predict(X_test)
print("Smallest raw output:", round(raw.min(), 3))
print("Largest raw output: ", round(raw.max(), 3))
print("Outputs outside 0 to 1:", int(((raw < 0) | (raw > 1)).sum()))
```

1. **Stratified split**
   75 to 25 with stratify set to y, so both parts keep the 63 to 37 class mix. We reuse this split for the whole section.

2. **The wrong tool**
   A LinearRegression fitted to the 0 or 1 target, then raw predictions for the 143 test patients.

3. **Count impossible values**
   The vertical bar means or; we count outputs below 0 or above 1.

Output: 44 impossible probabilities



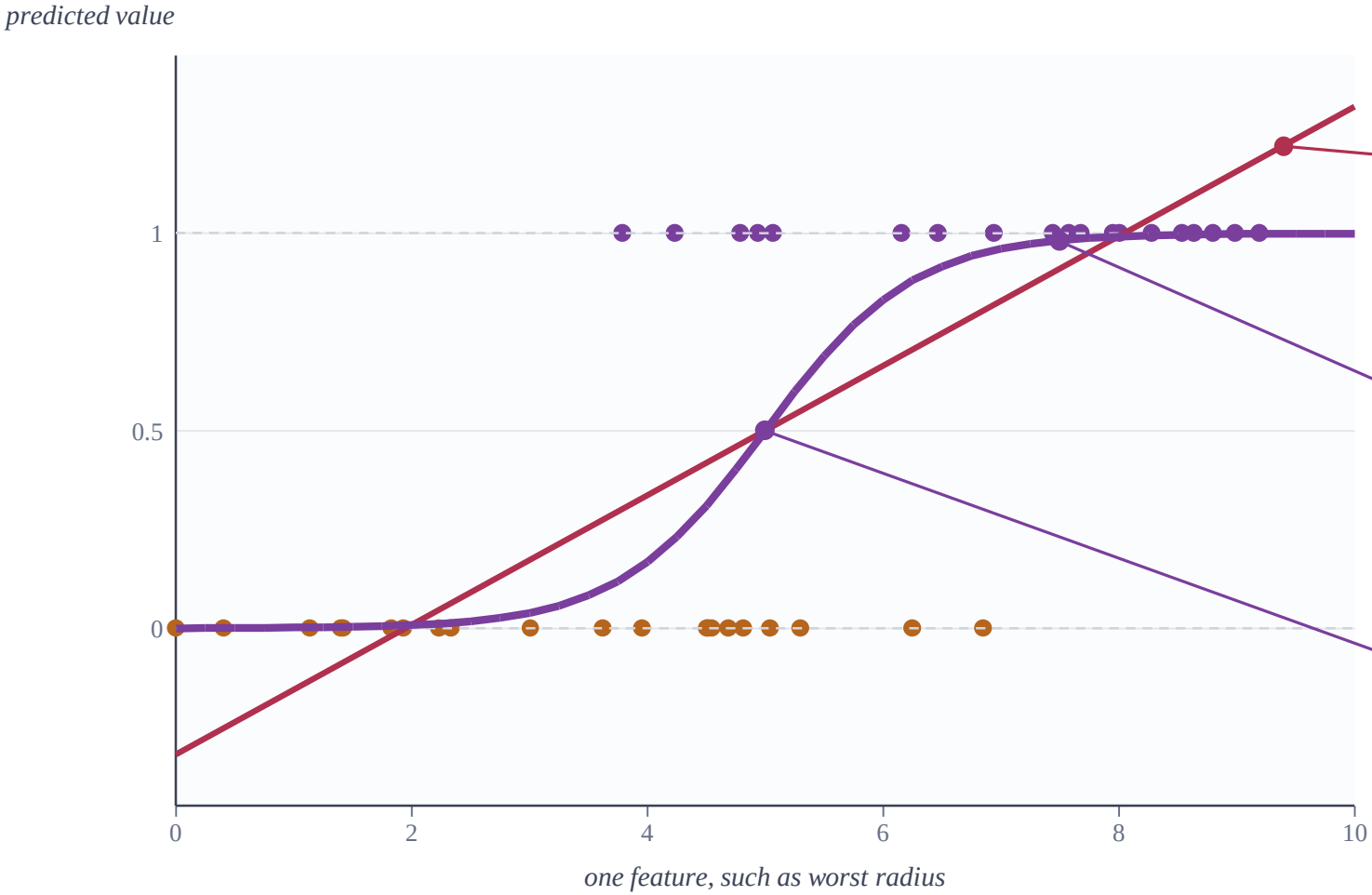
Minus 0.539
A negative probability of being benign. Meaningless.

1.367
A 137 percent chance of being benign. Equally meaningless.

44 of 143 outputs
Almost a third of predictions fall outside the valid range. This is why classification needs a dedicated model.

Illustration: a straight line versus an S curve

Illustrative data: one feature on the horizontal axis, class 0 or 1 on the vertical. Each model's output is drawn as a line.



Straight line (red)
Shoots below 0 on the left and above 1 on the right: impossible probabilities.

S curve (purple)
Flattens towards 0 and 1 but never crosses them. Every output is a valid probability.

Steepest in the middle
Around the boundary small feature changes move the probability a lot; far from it, the model is already confident.

Logistic regression in four moves

Inside, logistic regression is linear regression with a squashing step bolted onto the end.

1

Weighted sum

Multiply every feature by its coefficient and add the intercept, exactly as in linear regression. The result can be any number.

2

Read it as log odds

That number is interpreted as the log odds of the positive class: zero means even chances, positive favours class 1, negative favours class 0.

3

Squash with the sigmoid

The S shaped sigmoid converts log odds into a probability between 0 and 1. Large positive becomes near 1, large negative near 0.

4

Apply a threshold

By default, probability of at least 0.5 becomes class 1. You can move the threshold to trade one kind of error for the other.

Illustration: one patient's journey through the model

Real numbers for the third test patient, taken from the outputs we produce later in this section.

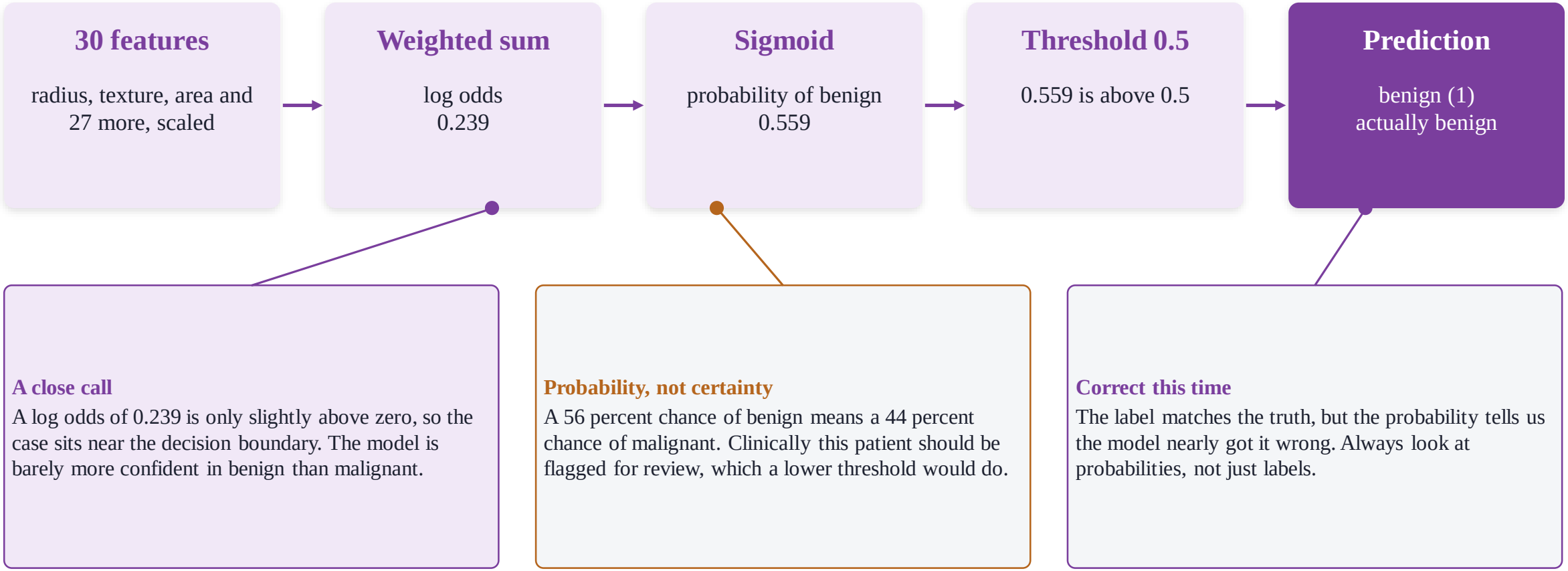
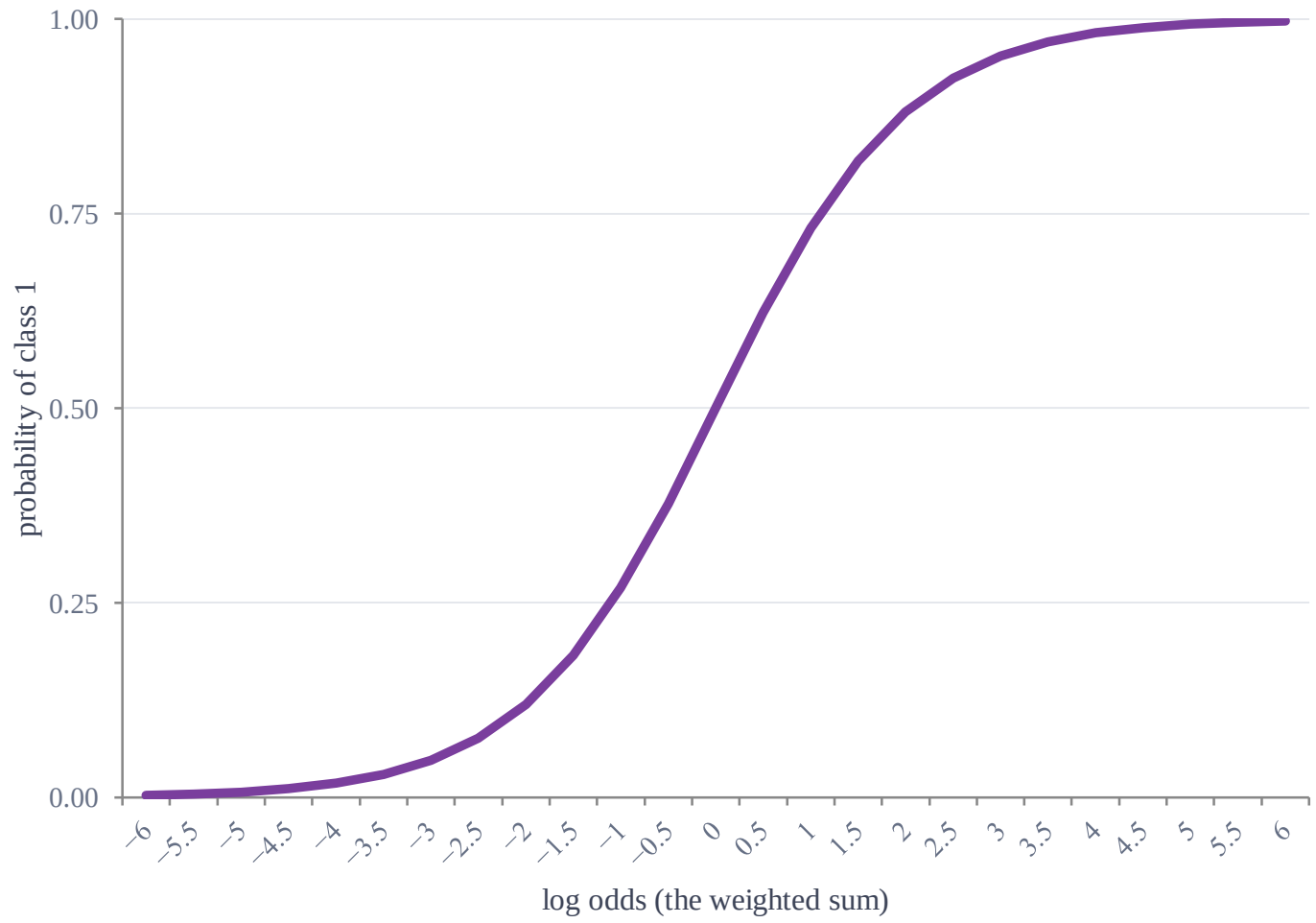


Chart: the sigmoid curve that maps log odds to probability



1 Zero maps to 0.5
When the weighted sum is exactly zero the model is undecided.

2 Symmetric
Log odds of plus 2 gives 0.88; minus 2 gives 0.12. Equal distances from zero give mirror image probabilities.

3 Saturation
Beyond about plus or minus 5 the curve is almost flat at 0 or 1. Extra evidence barely changes an already confident prediction.

4 Monotonic
A higher weighted sum always means a higher probability, so ranking cases by score and by probability gives the same order.

Probability, odds and log odds side by side

Odds compare the chance of an event happening with the chance of it not happening. Log odds stretch odds onto a symmetric scale.

Probability	Odds	Read as	Log odds
0.10	0.11	1 in favour for every 9 against	−2.20
0.25	0.33	1 in favour for every 3 against	−1.10
0.50	1.00	Even chances	0.00
0.75	3.00	3 in favour for every 1 against	1.10
0.90	9.00	9 in favour for every 1 against	2.20
0.99	99.0	99 in favour for every 1 against	4.60

Why log odds?

Probabilities are trapped between 0 and 1; odds are trapped above 0. Log odds run freely from minus infinity to plus infinity, which is exactly the range a weighted sum can produce. That is why the weighted sum is read as log odds.

How logistic regression is trained

1 The goal: likely labels

Choose coefficients that make the labels we actually observed as probable as possible. This is called maximum likelihood.

2 Log loss is the score

Equivalently, minimise log loss: a penalty that is near zero for confident correct predictions and grows without limit for confident wrong ones.

3 No one step solution

Unlike least squares there is no direct formula. A solver such as lbfgs, the default, improves the coefficients step by step until changes become tiny.

4 Convergence

max_iter caps the number of steps. If the solver stops early scikit learn warns that it did not converge; scaling features or raising max_iter fixes it.

5 Regularised by default

scikit learn adds an L2 penalty with strength set by C, default 1. Smaller C means a stronger penalty and smaller coefficients.

6 Convex problem

The log loss surface has a single bowl shaped minimum, so every run from any start reaches the same answer.

Training a logistic regression pipeline

PYTHON

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
clf.fit(X_train, y_train)
print("Train accuracy:", round(clf.score(X_train, y_train), 4))
print("Test accuracy: ", round(clf.score(X_test, y_test), 4))
print("Iterations used:", clf[1].n_iter_[0])
```



1 Imports

The scaler and the classifier will be chained.

2 Scaled pipeline

Scaling helps the solver converge quickly and makes regularisation treat every feature fairly. max_iter is raised generously.

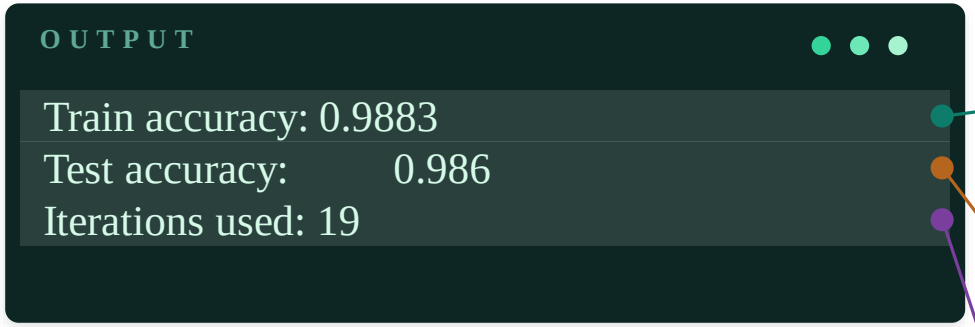
3 Fit

The scaler learns 30 means and spreads; the classifier learns 30 coefficients and an intercept.

4 Accuracy and iterations

n_iter_ reports how many solver steps were needed.

Output: 98.6 percent accuracy on unseen patients



Training 0.9883
Five of 426 training cases are misclassified.

Test 0.986
Only 2 of 143 test patients are misclassified. Training and test agree, so there is no sign of overfitting.

19 iterations
Thanks to scaling, the solver converged almost instantly, far below the cap of 5,000.

Looking at probabilities, not just labels

PYTHON

```python
proba = clf.predict_proba(X_test)
pred = clf.predict(X_test)
view = pd.DataFrame({"P(malignant)": proba[:6, 0].round(4),
                     "P(benign)": proba[:6, 1].round(4),
                     "predicted": pred[:6],
                     "actual": y_test.values[:6]})
print(view)
```

1 **predict_proba**

Returns one row per patient and one column per class, in the order of the class labels: malignant first, benign second.

2 **predict**

Applies the default threshold of 0.5 to the benign column.

3 **A readable table**

The first six test patients with both probabilities, the predicted label and the true label.

Output: confidence varies enormously

OUTPUT

	P(malignant)	P(benign)	predicted	actual
0	0.0310	0.9690	1	1
1	0.9996	0.0004	0	0
2	0.4406	0.5594	1	1
3	0.0611	0.9389	1	1
4	0.8245	0.1755	0	0
5	1.0000	0.0000	0	0

Patient 1: 0.9996 malignant
Overwhelming evidence. Columns always sum to 1.

Patient 2: 0.56 benign
The close call from the journey illustration. Correct, but barely.

Patient 4: 0.82 malignant
Confident and correct.

Patient 5: 1.0000
Rounded to four decimals; the true value is slightly below 1. A logistic model never outputs exactly 0 or 1.

Verifying the sigmoid by hand

PYTHON

```
1  np.set_printoptions(suppress=True)
2  z = clf.decision_function(X_test)[:4]
3  print("Log odds (decision_function):", z.round(3))
4  print("Sigmoid of log odds:         ", (1 / (1 + np.exp(np.negative(z)))).round(4))
5  print("predict_proba column 1:      ", proba[:4, 1].round(4))
```

1 Readable printing

Suppresses scientific notation so small numbers print as plain decimals.

2 decision_function

Returns the raw weighted sum, the log odds of class 1, before any squashing.

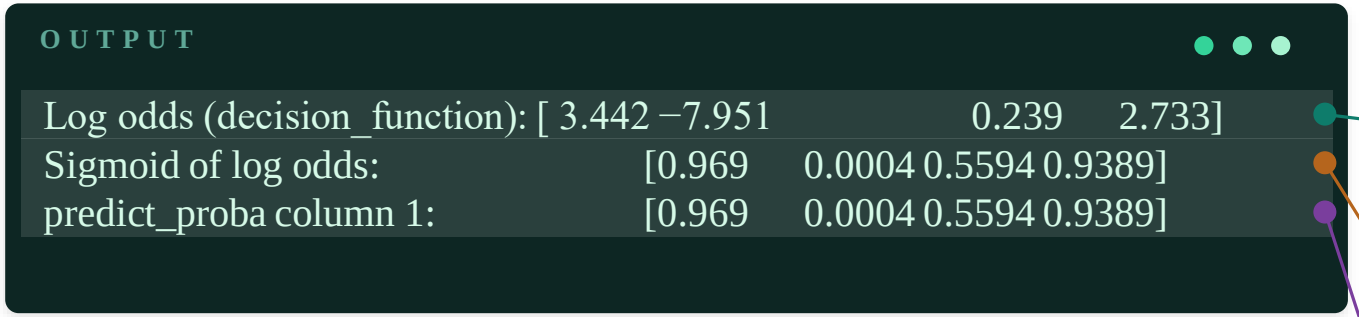
3 Apply the sigmoid ourselves

One divided by one plus the exponential of the negated log odds. np.negative avoids a minus sign.

4 Compare

If our hand computation matches predict_proba we have proved what the model does internally.

Output: the sigmoid explains predict_proba exactly



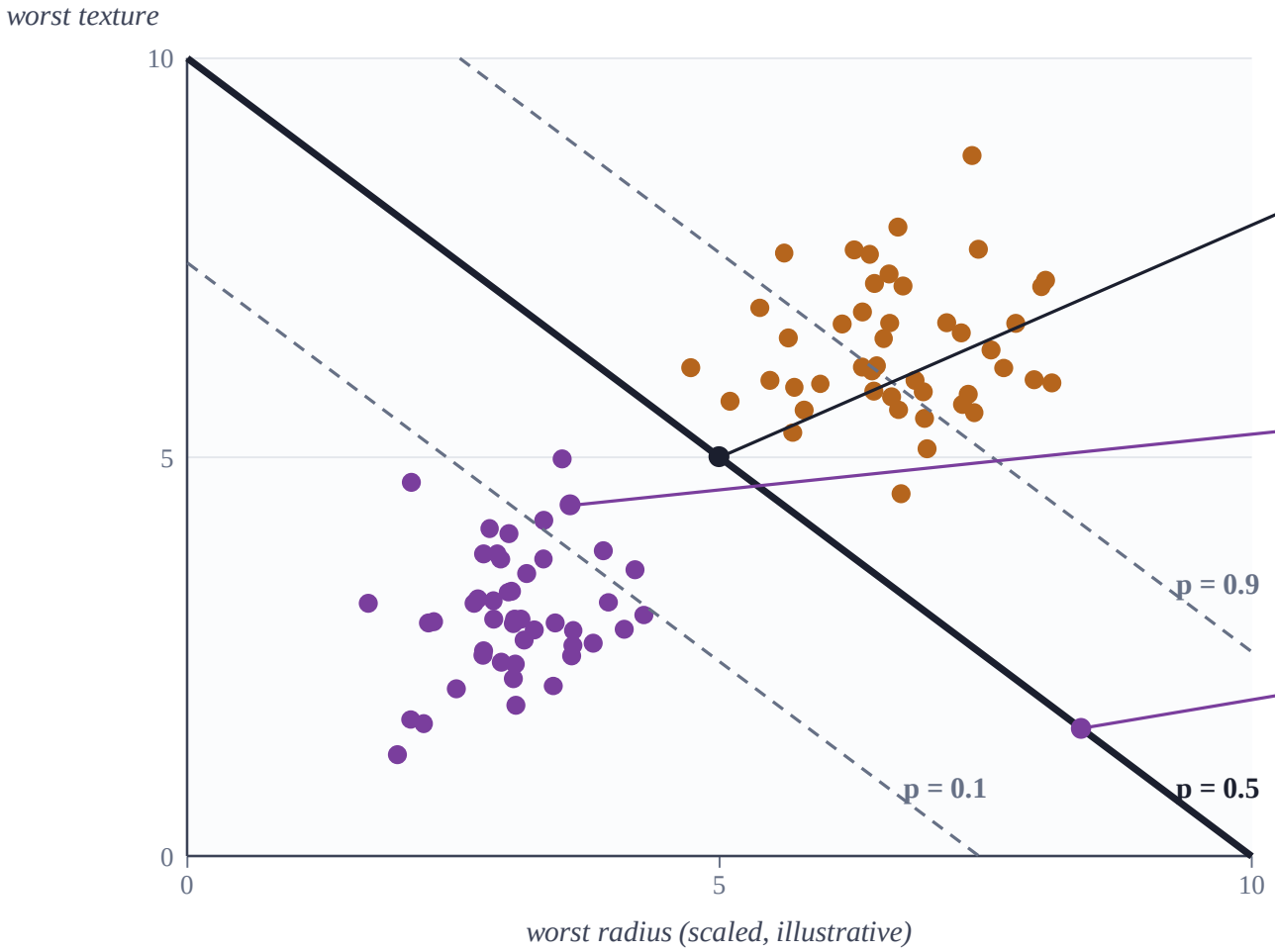
Log odds
3.442 is strongly benign, minus 7.951 strongly malignant, 0.239 barely benign, 2.733 fairly benign.

Our sigmoid
0.969, 0.0004, 0.5594 and 0.9389.

Identical to the model
No hidden magic: logistic regression is a weighted sum passed through the sigmoid.

Illustration: the decision boundary is a straight line

Illustrative two feature version. Every point on the boundary has probability exactly 0.5; parallel lines mark other probability levels.



The solid line: probability 0.5

Where the weighted sum equals zero. Patients on one side are predicted benign, on the other malignant.

Dashed lines: 0.1 and 0.9

Probability changes smoothly as you move across the boundary. Points far from it get confident probabilities.

Always straight

With raw features, logistic regression can only draw a straight boundary, or a flat plane in many dimensions.

Curved boundaries need polynomial features, or a different model such as a decision tree.

The confusion matrix vocabulary

Every prediction lands in one of four boxes. Decide first which class is positive: here scikit learn treats class 1, benign,as positive.

1 True positive

Predicted positive and really positive. Here: predicted benign, truly benign.

2 True negative

Predicted negative and really negative. Here: predicted malignant, truly malignant.

3 False positive

Predicted positive but really negative, a false alarm. Here: predicted benign but truly malignant, the dangerous error in this problem.

4 False negative

Predicted negative but really positive, a miss. Here: predicted malignant but truly benign, leading to an unnecessary biopsy.

5 Rows are truth

In scikit learn's confusion_matrix, rows are actual classes and columns are predicted classes, both in label order 0 then 1.

6 Name the positive class

Because the labels here make benign the positive class, think carefully: in medical language, a positive test usually means disease is present.

Building the confusion matrix

PYTHON

```
1 from sklearn.metrics import confusion_matrix
2 cm = confusion_matrix(y_test, pred)
3 print(pd.DataFrame(cm, index=["actual malignant", "actual benign"],
4                    columns=["pred malignant", "pred benign"]))
```

1

2

1 Compute

confusion_matrix takes true labels first, predicted labels second.

2 Label it

Wrapping the array in a DataFrame with descriptive row and column names prevents the most common misreading.

Output: only two mistakes in 143 patients

OUTPUT

	pred malignant	pred benign
actual malignant	52	1
actual benign	1	89

52 malignant caught
52 of 53 malignant cases correctly predicted malignant.

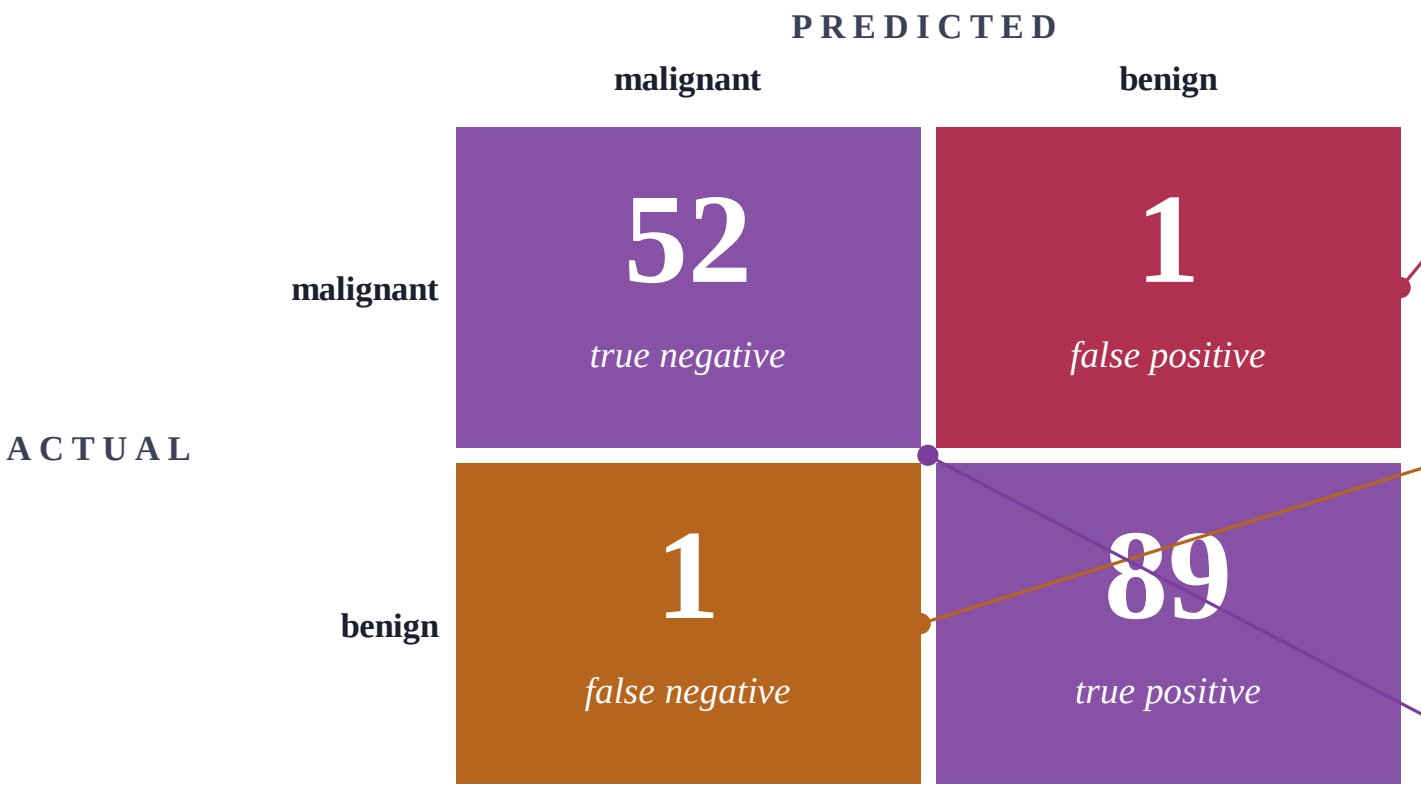
1 malignant missed
One malignant case was predicted benign. This is the error we most want to eliminate.

1 false alarm
One benign case predicted malignant. It would lead to further tests but no harm from a missed cancer.

89 benign correct
The diagonal, 52 plus 89, gives 141 correct out of 143: the 0.986 accuracy.

Illustration: the confusion matrix, annotated

The same real counts, laid out as a grid with each cell's meaning for this medical problem.



The dangerous cell

A malignant tumour predicted benign. With benign as the positive class this is a false positive. One patient here.

The costly cell

A benign case predicted malignant: an unneeded biopsy and anxiety, but recoverable. One patient here.

The diagonal is success

141 of 143 correct. Precision, recall and every other class metric are ratios built from these four numbers.

Precision, recall, F1 and support explained

1 Precision

Of everything the model labelled as a class, what share truly belonged to it? High precision means few false alarms for that class.

2 Recall

Of everything that truly belonged to a class, what share did the model find? High recall means few misses. Also called sensitivity.

3 The tension

Labelling more cases as malignant raises malignant recall but lowers malignant precision. Thresholds move you along this trade off.

4 F1 score

A single number that balances precision and recall, using a harmonic mean that is pulled down sharply if either one is poor.

5 Support

How many true cases of each class are in the evaluation set. Small support means the metric itself is noisy.

6 Macro and weighted averages

Macro treats every class equally; weighted multiplies by support. A big gap between them flags poor performance on a small class.

A full classification report

PYTHON

```
1 from sklearn.metrics import classification_report
2 print(classification_report(y_test, pred, target_names=data.target_names, digits=3))
```

1 Import

classification_report computes precision, recall and F1 for every class in one call.

2 Readable names and precision

target_names replaces 0 and 1 with words; digits set to 3 prints three decimal places.

Output: every class is predicted well

OUTPUT

	precision	recall	f1 score	support
malignant	0.981	0.981	0.981	53
benign	0.989	0.989	0.989	90
accuracy			0.986	143
macro avg	0.985	0.985	0.985	143
weighted avg	0.986	0.986	0.986	143

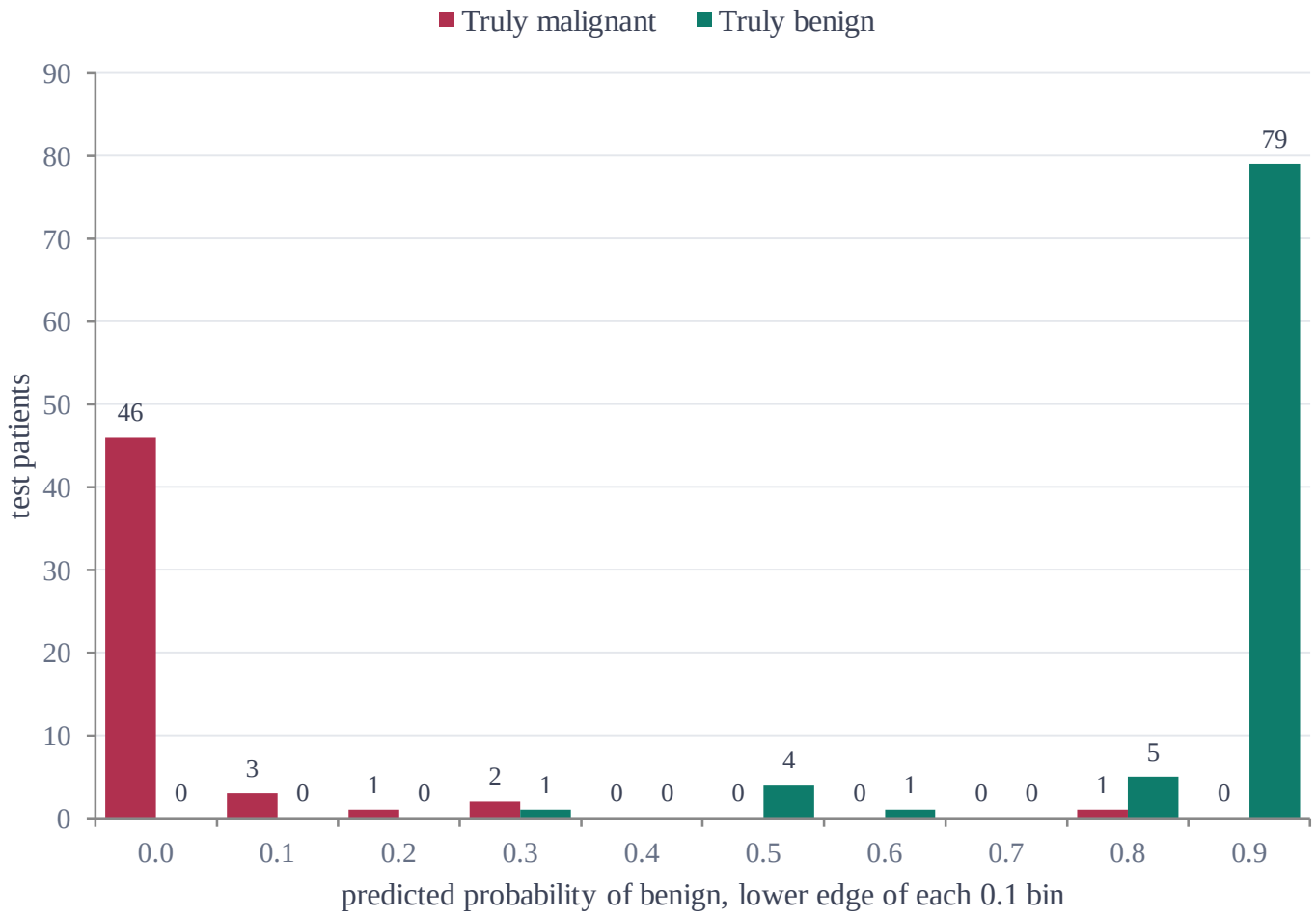
Malignant row
Precision 0.981: of 53 cases called malignant, 52 were. Recall 0.981: of 53 truly malignant cases, 52 were found.

Benign row
Precision and recall 0.989, based on 90 benign patients.

Accuracy 0.986 over 143
The overall share correct. Shown once because it is not a per class figure.

Macro average 0.985
Close to the weighted average, so neither class is neglected.

Chart: how confidently the model separates the classes



1 Two clean piles
Malignant cases pile up near 0, benign cases near 1. This separation is what a high ROC AUC measures.

2 The uncertain middle
Only a handful of patients fall between 0.2 and 0.8. These are where errors happen and where human review adds the most value.

3 Thresholds slice this picture
Moving the threshold left or right decides which middle cases are called benign. Few cases live there, so results are not very sensitive to it.

ROC curves and AUC in plain language

1 Every threshold at once

A ROC curve plots the true positive rate against the false positive rate as the threshold sweeps from 1 down to 0.

2 True positive rate

Recall of the positive class: the share of truly benign cases called benign at that threshold.

3 False positive rate

The share of truly malignant cases wrongly called benign at that threshold.

4 Area under the curve

AUC summarises the curve in one number. 1 is perfect separation, 0.5 is random guessing.

5 A ranking interpretation

AUC equals the chance that a randomly chosen benign case receives a higher benign probability than a randomly chosen malignant case.

6 Threshold free

AUC judges the probabilities' ranking, independent of the cutoff you finally choose. Use it to compare models, then choose a threshold separately.

Computing ROC AUC and log loss

PYTHON

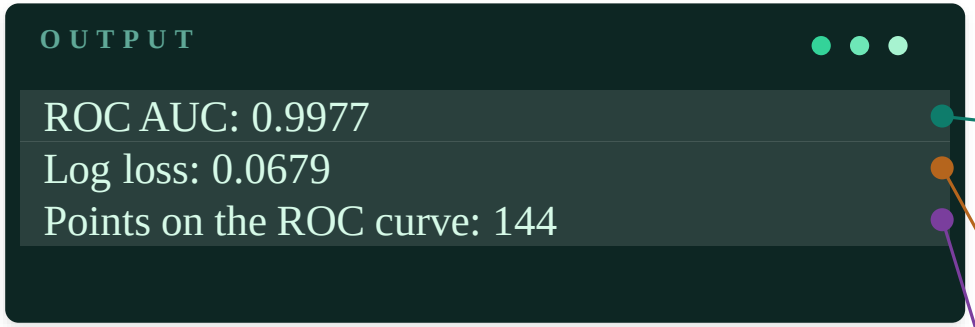
```python
from sklearn.metrics import roc_auc_score, roc_curve, log_loss
p_benign = proba[:, 1]
print("ROC AUC:", round(roc_auc_score(y_test, p_benign), 4))
print("Log loss:", round(log_loss(y_test, proba), 4))
fpr, tpr, thr = roc_curve(y_test, p_benign, drop_intermediate=False)
print("Points on the ROC curve:", len(fpr))
```

1. **Positive class probabilities**
   Both metrics need probabilities, not labels. Column 1 is the probability of benign.

2. **Two threshold free metrics**
   roc_auc_score uses the benign column; log_loss uses the full two column probability table.

3. **The curve itself**
   roc_curve returns the false positive rate, true positive rate and threshold at every distinct score. drop_intermediate set to False keeps every point.

Output: near perfect ranking



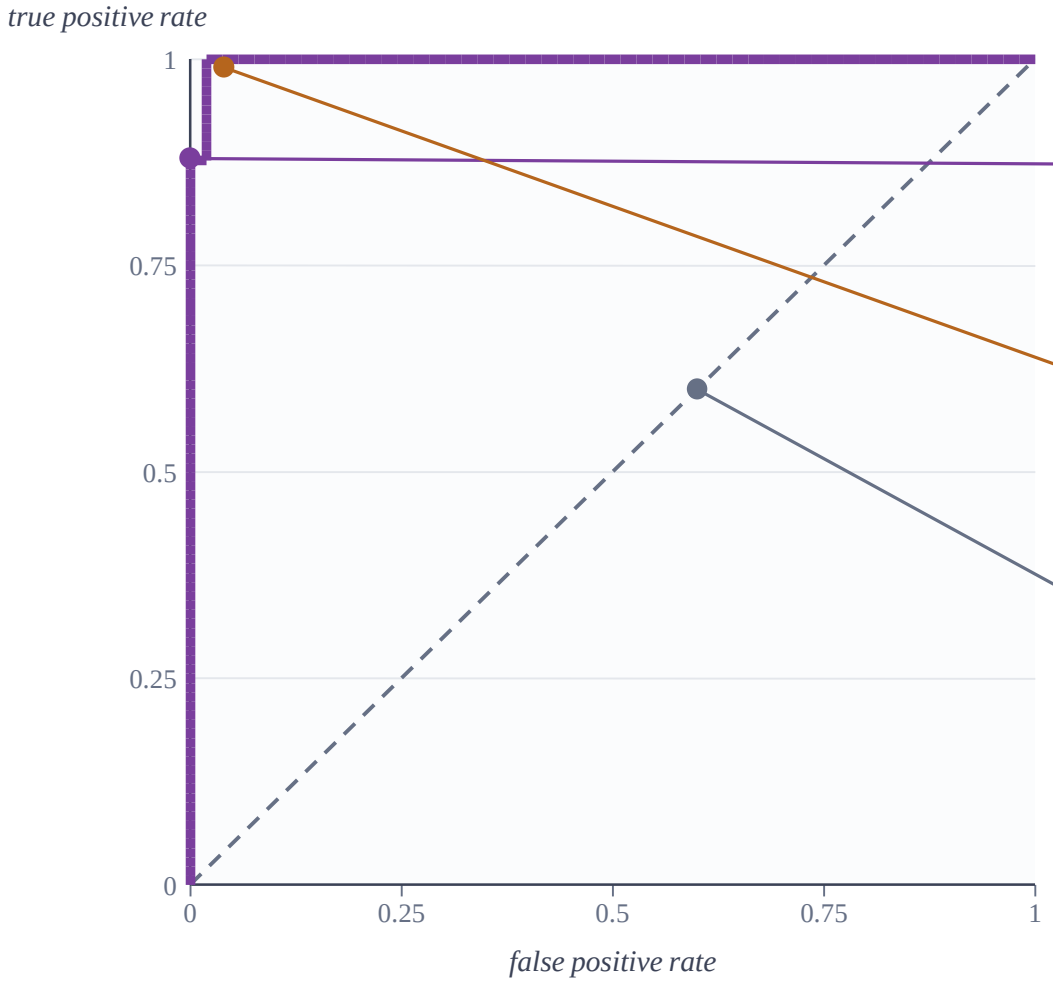
ROC AUC 0.9977
If we pick a random benign and a random malignant patient, the model ranks them correctly 99.8 percent of the time.

Log loss 0.0679
Low: the probabilities are both correct and appropriately confident. Always predicting 0.63 for everyone would score about 0.66.

144 curve points
One for each distinct score threshold plus the starting corner.

Illustration: the real ROC curve for our model

Each point is one threshold. The curve hugs the top left corner, the signature of an excellent classifier.



Straight up the left edge
The model finds 88 percent of benign cases with zero false positives, and all of them after just one false positive.

Top left corner is perfection
100 percent of positives found with no false alarms. Our curve nearly touches it.

Dashed diagonal: a coin flip
A model with no skill traces this line, giving AUC 0.5.

Area under the purple curve: 0.9977
Almost the whole unit square.

Log loss: rewarding honest confidence

1 What it measures

For each case, look at the probability the model gave to the true class. Log loss is large when that probability is small.

2 Confident and right

Giving 0.99 to the true class costs almost nothing.

3 Unsure

Giving 0.5 to the true class costs a moderate amount, the same as a coin flip.

4 Confident and wrong

Giving 0.01 to the true class is punished very heavily. One such case can dominate the average.

5 Why use it

Accuracy ignores confidence. Two models with equal accuracy can differ hugely in how trustworthy their probabilities are; log loss tells them apart.

6 Lower is better

Zero is perfect. Compare against the log loss of predicting the base rate for everyone to judge whether probabilities are informative.

Choosing a threshold for malignant cases

PYTHON

```python
from sklearn.metrics import precision_score, recall_score
p_malig = proba[:, 0]
is_malig = (y_test.values == 0).astype(int)
print("cutoff  flagged  recall_malig  precision_malig")
for t in [0.1, 0.2, 0.3, 0.5, 0.7, 0.9]:
    flag = (p_malig >= t).astype(int)
    r = recall_score(is_malig, flag)
    p = precision_score(is_malig, flag)
    print(f"{t:>5}   {flag.sum():>5}   {r:>11.3f}   {p:>13.3f}")
```

1. **Focus on malignancy**
   We now treat malignant as the class to catch: its probability is column 0, and is_malig marks the true malignant patients with 1.
2. **Sweep cutoffs**
   For each cutoff, flag every patient whose malignant probability reaches it.
3. **Measure both errors**
   Recall: share of malignant patients flagged. Precision: share of flagged patients who are truly malignant.

Output: the threshold trade off in numbers

OUTPUT

cutoff	flagged	recall_malig	precision_malig
0.1	64	1.000	0.828
0.2	58	0.981	0.897
0.3	58	0.981	0.897
0.5	53	0.981	0.981
0.7	50	0.943	1.000
0.9	46	0.868	1.000

Cutoff 0.1: catch every cancer

Recall 1.000: all 53 malignant cases flagged, at the cost of 11 false alarms among 64 flags.

Cutoff 0.5: the default

Recall 0.981, precision 0.981. One cancer missed.

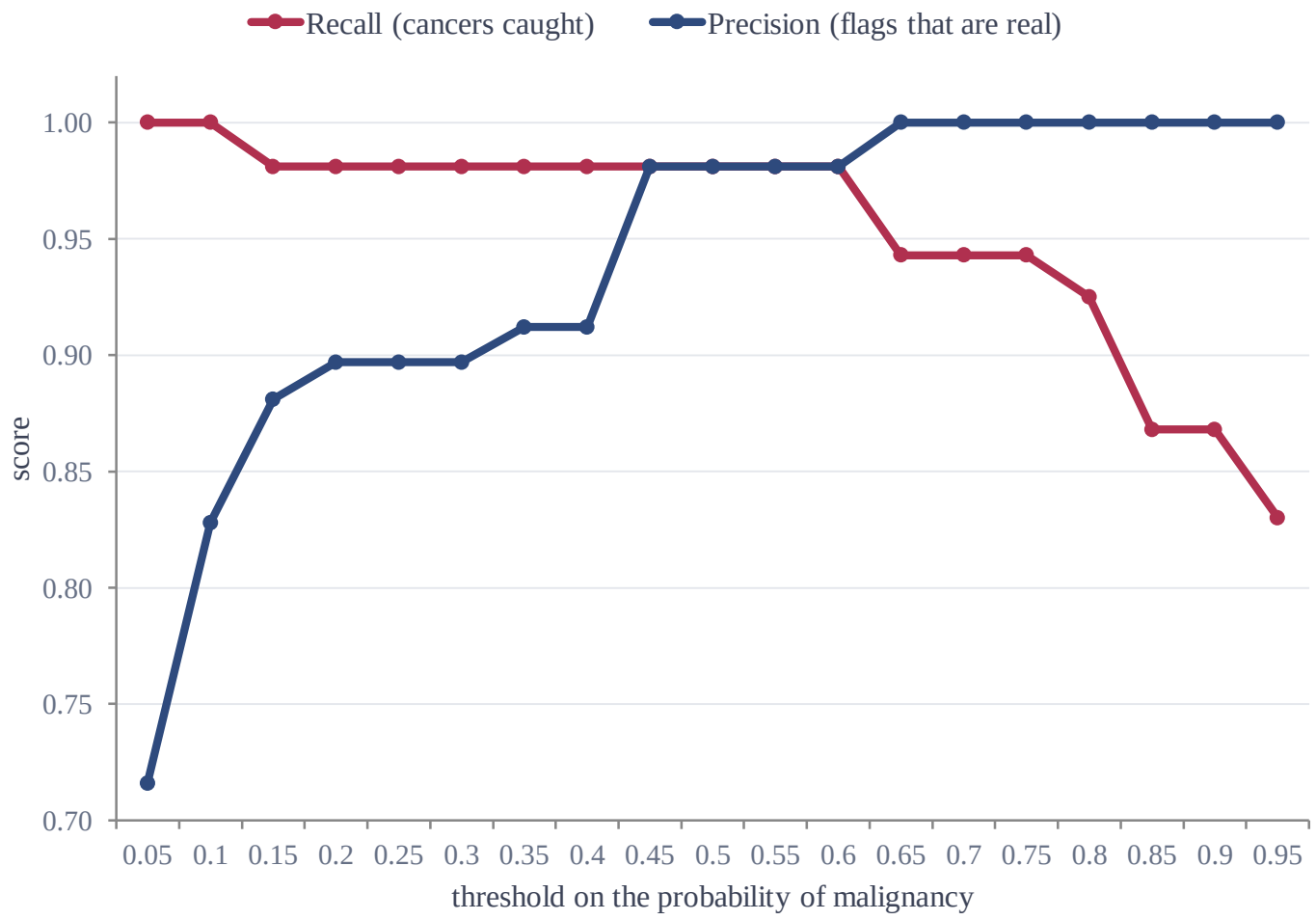
Cutoff 0.9: very cautious flagging

Every flag is correct, but 7 cancers are missed. Unacceptable for screening.

Cutoffs 0.2 and 0.3

Identical results: no patient's probability falls between them. Thresholds only matter where cases actually lie.

Chart: recall and precision for malignancy across thresholds



1 Opposite directions

Raising the threshold makes flags rarer: precision rises, recall falls.

2 Where to sit

For cancer screening, prioritise recall. A threshold of 0.1 catches every malignant case in this test set while keeping precision at 0.83.

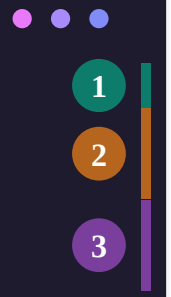
3 Decide with costs

The right threshold comes from the relative cost of a missed cancer versus an extra biopsy, agreed with domain experts, and validated on separate data.

Reading coefficients as odds multipliers

PYTHON

```python
coefs = pd.Series(clf[1].coef_[0], index=X.columns)
odds = np.exp(coefs)
table = pd.DataFrame({"coef": coefs.round(3), "odds_multiplier": odds.round(3)})
order = coefs.abs().sort_values(ascending=False).index
print(table.loc[order].head(8))
```



1 Coefficients

clf[1] is the logistic step. coef_ has one row for a binary problem; row 0 holds the 30 coefficients, in log odds of benign per standard deviation.

2 Exponentiate

Raising e to a coefficient converts it into an odds multiplier for a one standard deviation increase.

3 Rank by size

Sort by absolute coefficient and show the eight most influential features.

Output: every top feature pushes towards malignant

OUTPUT

	coef	odds_multiplier
worst texture	−1.250	0.286
radius error	−1.070	0.343
worst symmetry	−0.957	0.384
worst concave points	−0.942	0.390
area error	−0.941	0.390
worst area	−0.926	0.396
worst radius	−0.917	0.400
worst concavity	−0.797	0.451

worst texture: minus 1.25

One standard deviation more texture multiplies the odds of benign by 0.286, cutting them by about 71 percent.

radius error: odds times 0.343

More variable nucleus size points strongly to malignancy.

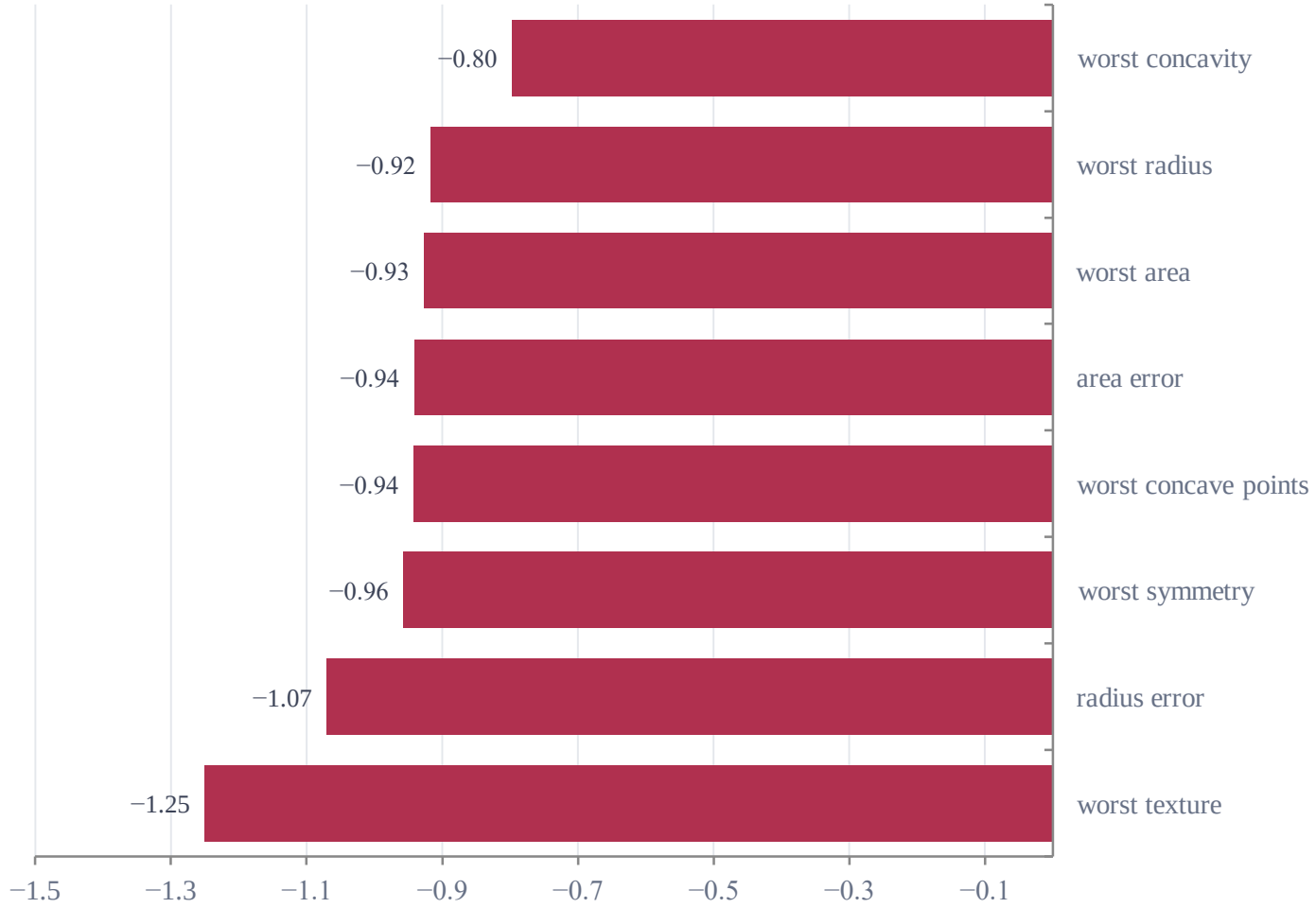
worst concave points

Irregular, indented cell outlines are a classic pathology sign of malignancy.

All negative

Because benign is class 1, negative coefficients mean the feature raises the risk of malignancy. Every one of the top eight does.

Chart: the eight strongest coefficients



1. **Per standard deviation**
   The model was trained on scaled features, so these compare like with like.

2. **Size and shape dominate**
   Texture, radius, area and concavity measures drive predictions, matching medical intuition.

3. **Shared credit**
   Radius, perimeter and area are almost perfectly correlated. Individual coefficients among them are unstable, as we saw with VIF in linear regression; interpret the group, not each member.

Interpreting logistic coefficients responsibly

1 Effects multiply odds

A coefficient adds to the log odds, so it multiplies the odds. It does not add a fixed amount of probability.

2 Probability effects vary

The same coefficient moves probability a lot near 0.5 and hardly at all near 0 or 1, because of the sigmoid's shape.

3 Odds multiplier above 1

Raises the odds of class 1. Below 1 lowers them. Exactly 1 means no effect.

4 Scaled units

With standardised inputs, effects are per standard deviation. Report the original unit equivalent when communicating.

5 Regularisation shrinks

Because of the default penalty, coefficients are pulled toward zero and are slightly smaller than unpenalised estimates.

6 Correlation is not causation

Large coefficients identify strong predictors, not causes. Correlated groups share credit unpredictably.

Regularisation strength C

In scikit learn's LogisticRegression, C is the inverse of regularisation strength. This is the opposite direction to alpha in ridge.

1 Small C

Strong penalty. Coefficients are squeezed toward zero, probabilities move toward the base rate, and the model may underfit.

2 Large C

Weak penalty. Coefficients can grow large to separate the training data perfectly, and the model may overfit.

3 Default C equals 1

A sensible starting point when features are standardised, but always worth tuning with cross validation.

4 Perfect separation

When classes can be split perfectly, unpenalised coefficients would grow forever. Regularisation keeps them finite and the model stable.

5 L2 penalty, the default

Shrinks all coefficients smoothly, like ridge. Keeps every feature in the model.

6 L1 penalty

Can set coefficients exactly to zero, like lasso, selecting a subset of features. Needs a compatible solver such as liblinear or saga.

Sweeping C from very strong to very weak penalty

PYTHON

```python
for C in [0.001, 0.01, 0.1, 1, 10, 100]:
    m = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=10000))
    m.fit(X_train, y_train)
    print(f"C {C:>6}: train {m.score(X_train, y_train):.3f}  test {m.score(X_test, y_test):.3f}"
          f"  largest coef {np.abs(m[1].coef_).max():.2f}")
```

1

2

1 Six values of C

From 0.001 to 100, each a factor of ten apart, every model in its own scaled pipeline.

2 Three measurements

Training accuracy, test accuracy and the largest absolute coefficient, which shows how hard the penalty is squeezing.

Output: the bias variance trade off again

OUTPUT

```
C    0.001: train 0.892    test 0.916    largest coef 0.08
C     0.01: train 0.958    test 0.937    largest coef 0.23
C      0.1: train 0.984    test 0.979    largest coef 0.55
C        1: train 0.988    test 0.986    largest coef 1.25
C       10: train 0.991    test 0.972    largest coef 3.95
C      100: train 0.998    test 0.944    largest coef 14.73
```

C 0.001: underfit

Coefficients crushed to at most 0.08. Training accuracy only 0.892.

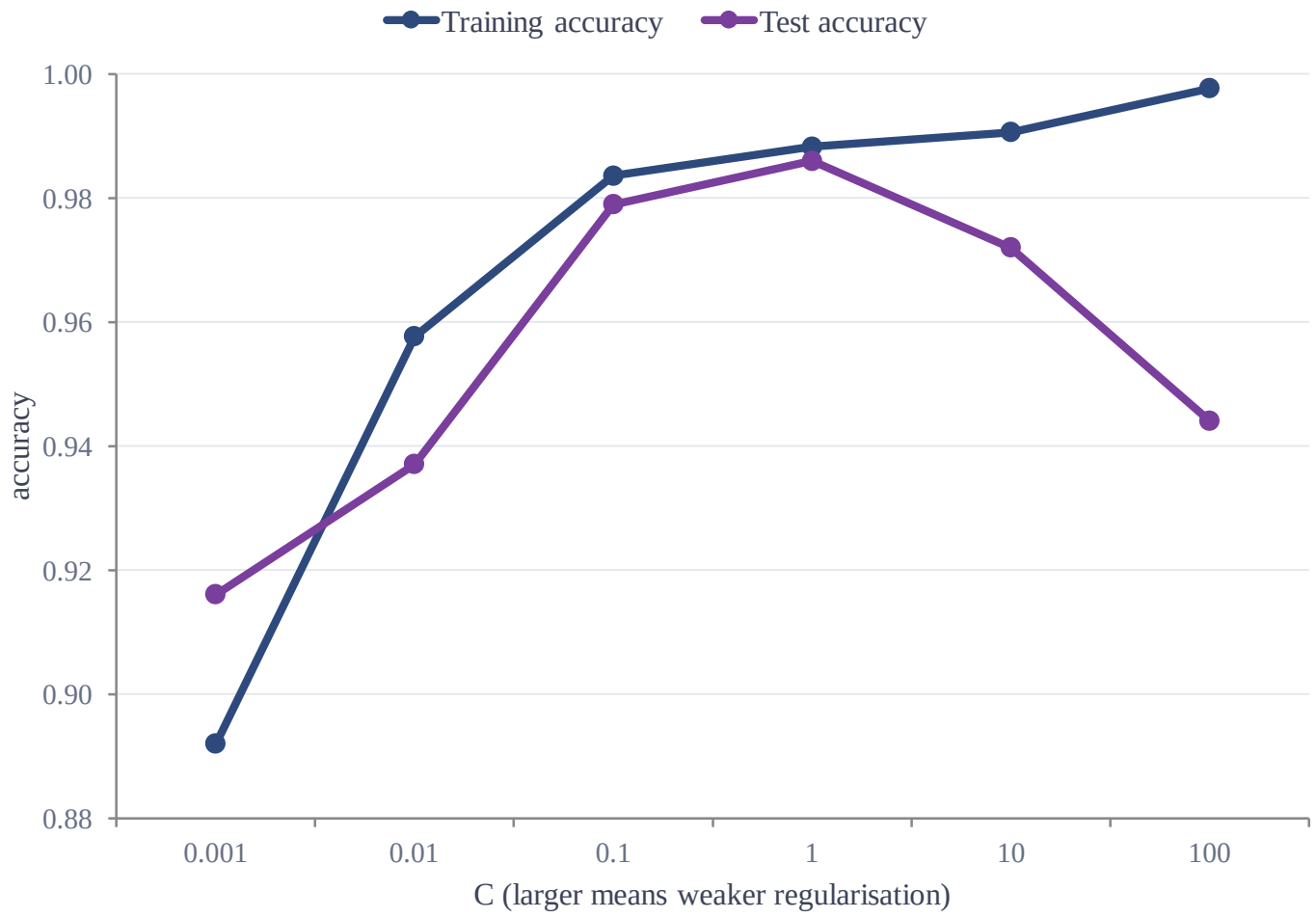
C 1: best test score

0.986 on test with a largest coefficient of 1.25: the default happens to be ideal here.

C 100: overfit

Training accuracy climbs to 0.998 while test drops to 0.944. A coefficient of 14.73 shows the model bending to individual training cases.

Chart: training and test accuracy against C



1. **Training only rises**
   Weaker penalties always fit training data better.

2. **Test peaks then falls**
   The peak at C equal to 1 marks the balance point.

3. **Same shape as before**
   Polynomial degree, ridge alpha and now C all produce this pattern. The tuning section formalises how to find the peak honestly with cross validation instead of the test set.

L1 regularisation for automatic feature selection

PYTHON

```python
for C in [0.01, 0.05, 0.1, 0.5, 1]:
    l1 = make_pipeline(StandardScaler(),
                       LogisticRegression(penalty="l1", solver="liblinear", C=C))
    l1.fit(X_train, y_train)
    kept = int((l1[1].coef_ != 0).sum())
    print(f"C {C:>5}: features kept {kept:>2} of 30   test acc {l1.score(X_test, y_test):.3f}")
```

1 2 3

1 L1 penalty

penalty set to l1 with the liblinear solver, which supports it. Five values of C are tried.

2 Count survivors

Coefficients that are exactly zero have been removed from the model. Counting the non zero ones shows how many features are kept.

3 Accuracy

Test accuracy for each sparsity level.

Output: five features already give 96 percent

OUTPUT

```
C  0.01: features kept  4 of 30  test acc 0.909
C  0.05: features kept  6 of 30  test acc 0.958
C   0.1: features kept  7 of 30  test acc 0.965
C   0.5: features kept 12 of 30  test acc 0.986
C     1: features kept 16 of 30  test acc 0.993
```

4 features: 0.909
A very strong penalty keeps only four features and still classifies 91 percent correctly.

7 features: 0.972
A compact model that a clinician could inspect feature by feature.

16 features: 0.993
Higher than the L2 model on this split. Half the features carry essentially all the signal.

Class imbalance: when one class is rare

1 The accuracy trap

If 95 percent of cases are negative, predicting negative always scores 95 percent accuracy while finding zero positives.

2 Why models ignore rare classes

Minimising average log loss is dominated by the majority. Pushing every probability down costs little on the rare class.

3 class_weight set to balanced

Weights each case inversely to its class frequency, so the rare class carries as much total influence as the common one.

4 Threshold moving

An alternative: keep the model but lower the threshold for the rare class, as we did for malignancy.

5 Better metrics

Report recall, precision, F1 or balanced accuracy for the rare class, plus ROC AUC or average precision.

6 Stratify everything

Use stratified splits and stratified folds so every subset contains enough rare cases to evaluate.

Testing class_weight on a rare event problem

PYTHON

```python
from sklearn.datasets import make_classification
from sklearn.metrics import recall_score
Xi, yi = make_classification(n_samples=4000, n_features=8, weights=[0.95],
                             class_sep=0.8, random_state=0)
Xi_tr, Xi_te, yi_tr, yi_te = train_test_split(Xi, yi, stratify=yi, random_state=0)
print("Rare class share:", round(yi.mean(), 3))
for cw in [None, "balanced"]:
    m = LogisticRegression(class_weight=cw, max_iter=2000).fit(Xi_tr, yi_tr)
    pr = m.predict(Xi_te)
    print(f"class_weight={cw!s:<9} accuracy {m.score(Xi_te, yi_te):.3f}  rare recall {recall_score(yi_te, pr):.3f}")
```

1

2

3

1 A synthetic rare event

make_classification generates 4,000 cases with 8 features where about 5 percent belong to class 1. class_sep sets how separable the classes are.

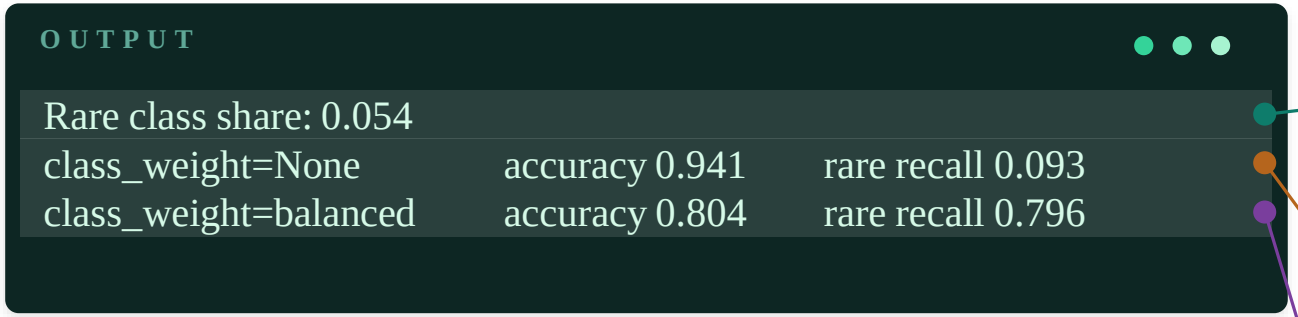
2 Stratified split

Keeps the 5 percent rate in both parts.

3 Two models

Unweighted versus balanced, each reporting accuracy and recall of the rare class.

Output: accuracy hides a model that misses 91 percent



5.4 percent rare

About 54 rare cases in the 1,000 test rows.

Unweighted: accuracy 0.941, recall 0.093

Looks great by accuracy, yet finds only 9 percent of the rare cases. Almost useless for detection.

Balanced: accuracy 0.804, recall 0.796

Accuracy falls because of more false alarms, but four in five rare cases are now caught. For fraud or disease, this is the far better model.

More than two classes: multinomial logistic regression

1 One score per class

The model learns a separate weighted sum for each class, so three classes means three sets of coefficients.

2 Softmax instead of sigmoid

The softmax function turns the class scores into probabilities that are all positive and sum exactly to 1.

3 Largest probability wins

predict returns the class with the highest probability.

4 Automatic in scikit learn

With the default lbfgs solver, LogisticRegression handles any number of classes without extra settings.

5 Coefficient shape

coef_ has one row per class. Each row describes what raises the score of that class relative to the others.

6 Same evaluation tools

Confusion matrices become larger grids; classification reports list every class; ROC AUC can be averaged one class versus the rest.

Classifying three iris species

PYTHON

```
from sklearn.datasets import load_iris
iris = load_iris()
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(iris.data, iris.target,
                                              stratify=iris.target, random_state=1)
multi = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(Xr_tr, yr_tr)
print("Classes:", [str(c) for c in iris.target_names])
print("Test accuracy:", round(multi.score(Xr_te, yr_te), 3))
print(pd.DataFrame(multi.predict_proba(Xr_te[:4]).round(3), columns=iris.target_names))
print("Row sums:", multi.predict_proba(Xr_te[:4]).sum(axis=1).round(3))
```

1

2

3

1 The iris data

150 flowers, 4 measurements, 3 species, 50 of each. A stratified split keeps all species represented.

2 Same pipeline

Nothing changes in the code for three classes.

3 Inspect

Accuracy, then the probability table for four test flowers, and a check that each row sums to 1.

Output: three probabilities per flower

OUTPUT

```
Classes: ['setosa', 'versicolor', 'virginica']
Test accuracy: 0.974
      setosa  versicolor  virginica
0      0.001       0.222      0.777
1      0.976       0.024      0.000
2      0.981       0.019      0.000
3      0.981       0.019      0.000
Row sums: [1. 1. 1. 1.]
```

Accuracy 0.974
37 of 38 test flowers classified correctly.

Flower 0: virginica 0.777
Some doubt with versicolor at 0.222. These two species overlap in measurement space.

Flower 1: setosa 0.976
Setosa is easy to separate from the others.

Rows sum to 1
The softmax guarantee: probabilities across classes always total exactly one.

Cross validating the final pipeline

PYTHON

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
auc = cross_val_score(clf, X, y, cv=skf, scoring="roc_auc")
acc = cross_val_score(clf, X, y, cv=skf, scoring="accuracy")
print("AUC per fold:", auc.round(4))
print("Mean AUC:", round(auc.mean(), 4), "  Mean accuracy:", round(acc.mean(), 4))
```



1 Stratified folds

Five folds, each with the same class mix as the full dataset.

2 Two metrics

The whole pipeline, scaler included, is refitted inside every fold, so there is no leakage.

3 Report

Fold scores plus their mean.

Output: performance holds across the whole dataset

```
OUTPUT
AUC per fold: [0.9951 0.9977 0.9851 1.     0.9987]
Mean AUC: 0.9953    Mean accuracy: 0.9737
```

AUC per fold

Between 0.985 and 1.000. One fold separated the classes perfectly.

Mean AUC 0.9953, accuracy 0.9737

Slightly below our single test split's 0.986 accuracy. That split was a little lucky; the cross validated figure is the more trustworthy estimate to report.

A logistic regression checklist

1

Define the positive class

Know which label is 1 and which errors are costly.

2

Stratify and scale

Stratified splits, scaling inside a pipeline, raise max_iter if warned.

3

Fit and inspect

Probabilities first, labels second. Check convergence and coefficients.

4

Evaluate fully

Confusion matrix, precision, recall, F1, ROC AUC and log loss.

5

Choose a threshold

From business costs, tested on validation data, not left at 0.5 by habit.

6

Tune and validate

C, penalty type and class_weight by cross validation.

Check your understanding

A fraud model has 99 percent accuracy, but fraud is 1 percent of transactions. What should you check first?

A The intercept

B Recall on the fraud class

C The number of iterations

D The scaler's mean

Answer B and why

A model that never predicts fraud also scores 99 percent accuracy. Recall on the rare class reveals whether any fraud is caught at all, exactly as our rare event experiment showed recall jumping from 0.093 to 0.796 with balanced class weights.

Check your understanding

A logistic coefficient on scaled data is 0.69. What does it mean?

A Probability rises by 0.69

B Odds roughly double per standard deviation

C The feature causes the outcome

D The class threshold becomes 0.69

Answer B and why

Coefficients add to log odds, so they multiply the odds. Raising e to 0.69 gives about 2.0, so each standard deviation increase roughly doubles the odds of class 1. The change in probability depends on where the case starts on the sigmoid.

SECTION

4

Decision Trees

Learning a flowchart of yes or no questions directly from data

- The intuition and the anatomy of a tree
- Impurity: Gini and entropy, computed by hand
- Greedy recursive splitting step by step
- Reading a fitted tree: rules, node tables and decision paths
- How a tree carves up feature space
- Feature importance, overfitting and instability
- Regression trees and the step function

The intuition: a game of twenty questions

A decision tree classifies a case by asking a sequence of simple questions, each about one feature, until it reaches an answer.

1 One question at a time

Each question compares a single feature with a threshold, such as is petal length at most 2.45 cm? The answer sends the case left or right.

2 Questions are learned

The algorithm chooses which feature to ask about and which threshold to use, picking the question that best separates the classes.

3 Answers live in leaves

When no more questions are asked, the case lands in a leaf. The leaf predicts the most common class among training cases that ended there.

4 Reads like rules

Every path from the top to a leaf is an if then rule a person can follow, which makes trees exceptionally easy to explain.

5 Nonlinear by nature

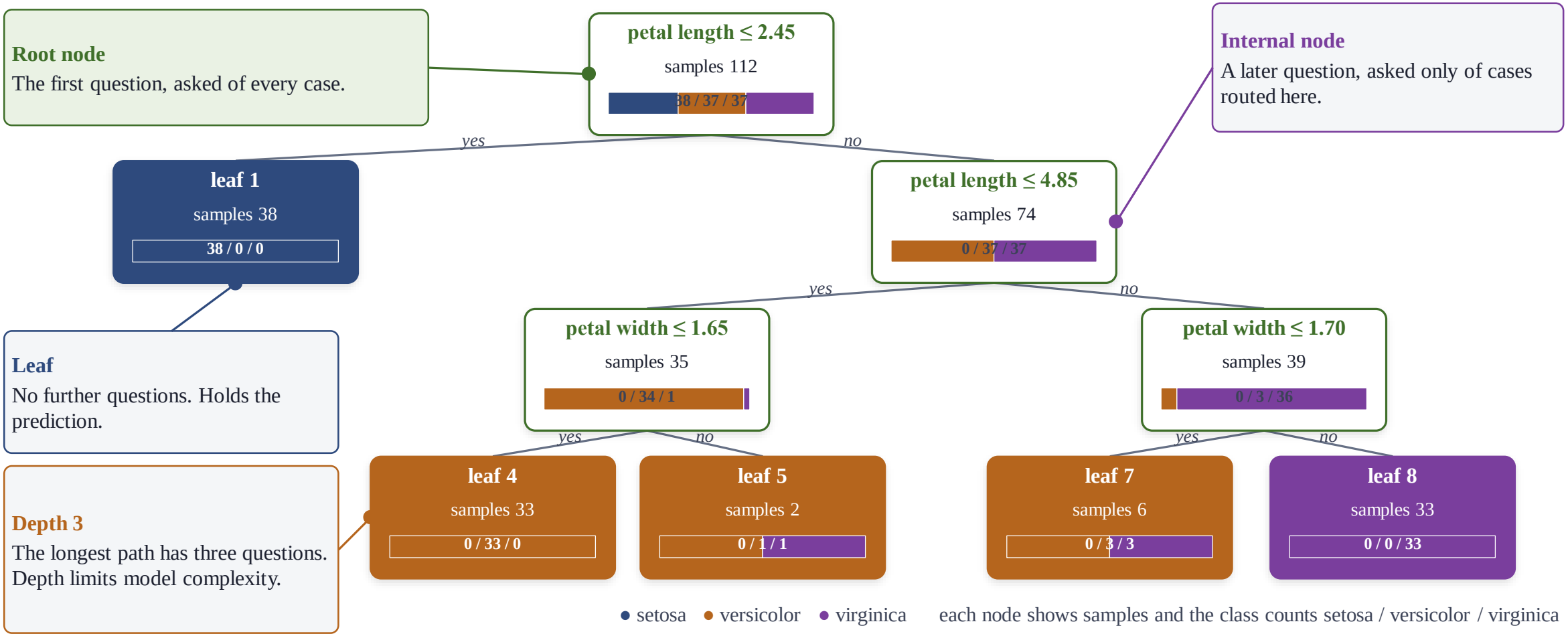
Different branches can ask completely different questions, so trees capture interactions and thresholds that linear models miss.

6 The foundation of ensembles

Random forests and gradient boosting, two of the strongest tabular methods, are built from many trees. Mastering one tree comes first.

Illustration: the anatomy of a decision tree

This is the real tree we fit to the iris flowers later in the section. Every part has a name.



How a tree grows: greedy recursive splitting

Trees are built top down. At every node the algorithm makes the locally best choice and never revisits it.

1

Start with all rows

The root holds every training case, with its own class mix and impurity.

2

Try every split

For each feature, sort the values and test every threshold between neighbouring distinct values.

3

Score each split

Measure how much the weighted impurity of the two children drops compared with the parent.

4

Keep the best

Choose the feature and threshold with the largest impurity decrease and split the rows.

5

Repeat on children

Apply the same procedure independently to each child, recursively.

6

Stop

When a node is pure, too small, or a limit such as max_depth is reached, it becomes a leaf.

Impurity: how mixed is a node?

A split is good if it produces children that are purer than their parent. Two standard measures quantify purity.

1 Pure node

Contains a single class. Impurity is zero; there is nothing left to separate.

2 Maximally mixed node

Classes appear in equal proportions. Impurity is at its highest for that number of classes.

3 Gini impurity

The chance that two cases drawn at random from the node, with replacement, belong to different classes. Two equal classes give 0.5; three give 0.667.

4 Entropy

The average number of yes or no questions needed to identify a random case's class. Two equal classes give 1 bit; three give 1.585 bits.

5 They usually agree

Both are zero when pure and largest when evenly mixed. Gini is slightly faster to compute and is scikit learn's default.

6 Impurity decrease

Parent impurity minus the size weighted average impurity of the children. The split with the largest decrease wins.

Computing Gini and entropy by hand

PYTHON

```python
def gini(counts):
    p = np.array(counts) / np.sum(counts)
    return np.subtract(1, np.sum(p ** 2))
def entropy(counts):
    p = np.array([c for c in counts if c > 0]) / np.sum(counts)
    return np.sum(p * np.log2(1 / p))
for counts in [[37, 37, 38], [37, 0, 0], [0, 33, 4], [10, 10, 0], [50, 1, 1]]:
    print(f"{str(counts):<14} gini {gini(counts):.3f}  entropy {entropy(counts):.3f}")
```

1

2

3

1 Gini

Turn counts into proportions, square and add them, then subtract from one. np.subtract performs the subtraction.

2 Entropy

Ignore empty classes, then add each proportion times the logarithm base 2 of its reciprocal.

3 Five example nodes

From perfectly mixed to perfectly pure, including counts taken from our iris tree.

Output: impurity for five class mixes

OUTPUT

[37, 37, 38]	gini 0.667	entropy 1.585
[37, 0, 0]	gini 0.000	entropy 0.000
[0, 33, 4]	gini 0.193	entropy 0.494
[10, 10, 0]	gini 0.500	entropy 1.000
[50, 1, 1]	gini 0.075	entropy 0.274

Evenly mixed three classes
The maximum: Gini 0.667, entropy 1.585 bits. This is roughly our iris root node.

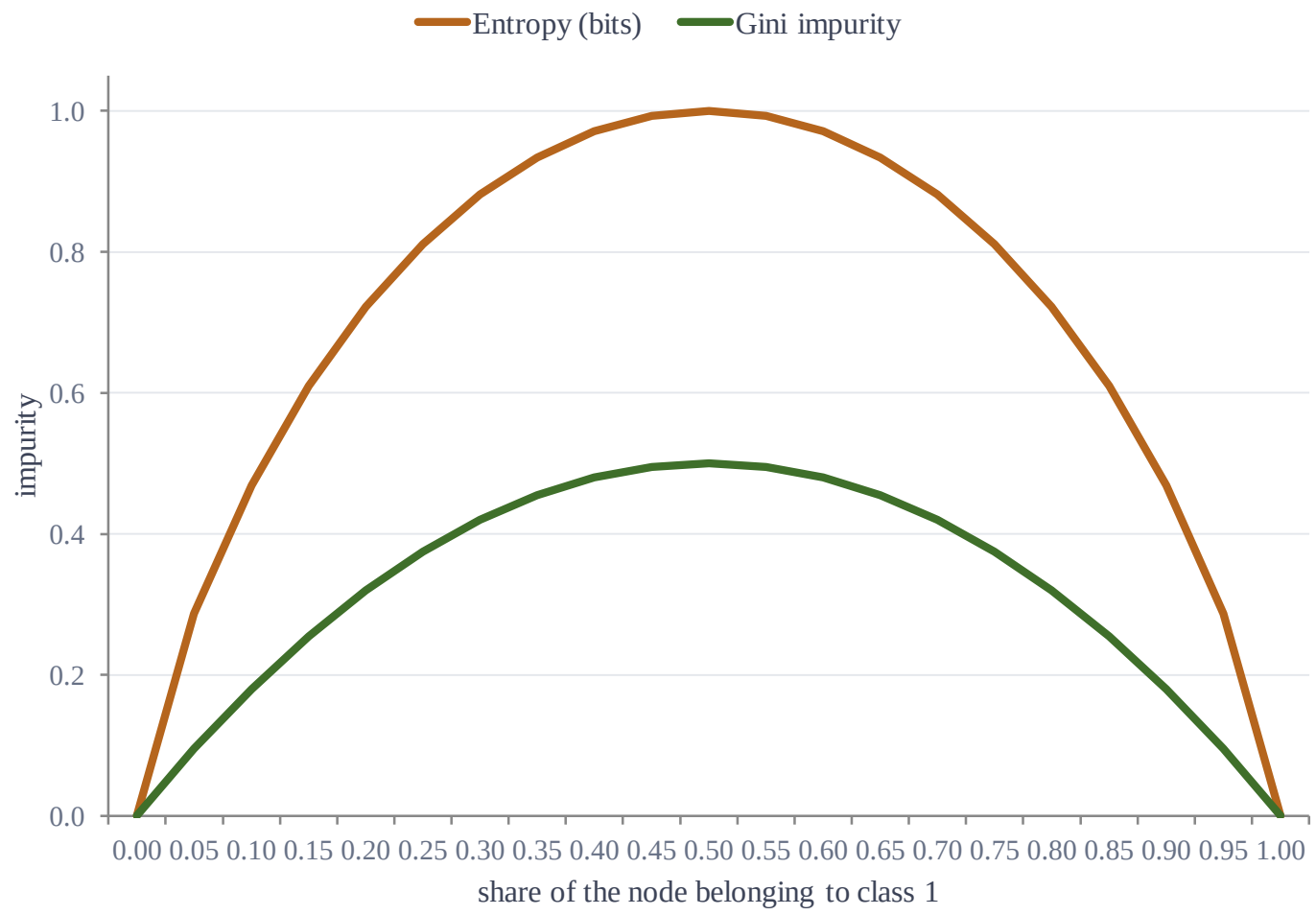
Pure
Both measures are exactly zero. The setosa leaf looks like this.

Mostly one class
33 versus 4: low impurity, 0.193 Gini.

Two classes, even
Gini 0.5 and entropy 1 bit: the maximum for two classes.

Nearly pure
One stray case of each other class barely raises impurity.

Chart: Gini and entropy for a two class node



1. **Zero at both ends**
   A node with only one class has no impurity under either measure.

2. **Peak in the middle**
   A 50 to 50 mix is the hardest node to predict.

3. **Same shape, different scale**
   Entropy is more sharply curved, so it rewards moving away from even mixes a little more strongly. In practice both usually choose the same splits.

Scoring one candidate split

PYTHON

```python
parent = [0, 37, 38]
left, right = [0, 36, 4], [0, 1, 34]
w_left = sum(left) / sum(parent)
w_right = sum(right) / sum(parent)
child = w_left * gini(left) + w_right * gini(right)
print("Parent gini:", round(gini(parent), 4))
print("Weighted child gini:", round(child, 4))
print("Impurity decrease:", round(np.subtract(gini(parent), child), 4))
```

1

A parent and two children

A node with 37 versicolor and 38 virginica, split into a left child of 40 cases and a right child of 35.

2

Weighted child impurity

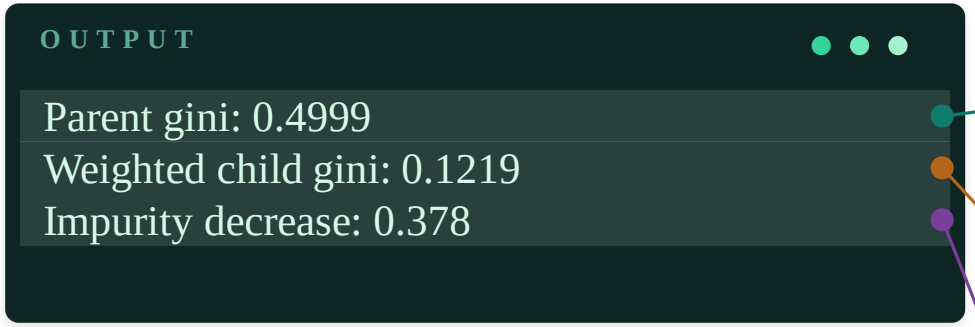
Each child's Gini counts in proportion to its share of the parent's cases.

3

The decrease

This is the number the tree compares across every candidate feature and threshold.

Output: the split removes three quarters of the impurity



Parent 0.4999
Almost exactly 50 to 50 between two classes.

Children 0.1219
After the split each side is dominated by one species.

Decrease 0.378
The algorithm computes this for every possible threshold of every feature and keeps the largest.

Illustration: what the scored split looks like

The same numbers drawn as a split: one parent node, a question, and two much purer children.

Weighted average
40 of 75 cases go left and 35 go right, so the children's combined impurity is about 0.53 times 0.180 plus 0.47 times 0.056, giving 0.122.

Parent: 75 cases



37 versicolor, 38 virginica
Gini 0.500

Decrease 0.378
0.500 minus 0.122. Among all candidates the tree keeps the split with the largest decrease.

a candidate question

Left: 40 cases



36 versicolor, 4 virginica
Gini 0.180

Right: 35 cases



1 versicolor, 34 virginica
Gini 0.056

Right child is nearly pure.

Fitting a classification tree to the iris flowers

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
iris = load_iris()
Xi_tr, Xi_te, yi_tr, yi_te = train_test_split(iris.data, iris.target,
                                              stratify=iris.target, random_state=42)
tree = DecisionTreeClassifier(max_depth=3, random_state=42).fit(Xi_tr, yi_tr)
print(export_text(tree, feature_names=iris.feature_names))
```

1 Imports

DecisionTreeClassifier builds the tree; export_text prints it as indented rules.

2 Data and split

150 flowers with 4 measurements, split 75 to 25 with stratification so all three species appear in both parts.

3 A shallow tree

max_depth set to 3 allows at most three questions per path. random_state makes tie breaking between equally good splits reproducible.

4 Print the rules

feature_names replaces column numbers with readable names.

Output: the tree written as rules

OUTPUT

```
|── petal length (cm) <= 2.45
|       |── class: 0
|── petal length (cm) >           2.45
|               |── petal length (cm) <= 4.85
|       |               |── petal width (cm) <= 1.65
|       |       |       |── class: 1
|       |       |── petal width (cm) >           1.65
|       |       |       |── class: 1
|       |── petal length (cm) >           4.85
|       |               |── petal width (cm) <= 1.70
|       |       |       |── class: 1
|       |       |── petal width (cm) >           1.70
|       |       |       |── class: 2
```

Root question
Petal length at most 2.45 cm sends a flower left. Every left flower is setosa, class 0.

Second level
For the other flowers, petal length at most 4.85 cm separates mostly versicolor from mostly virginica.

Same class on both sides
Both children of this split predict class 1. The split was kept because it purified the counts, even though the majority label is unchanged.

Class 2 is virginica
Long, wide petals. Indentation shows depth: three bars means the third question.

Illustration: the fitted iris tree, drawn

Each box shows its question, the number of training flowers reaching it and the class mix as a coloured bar.

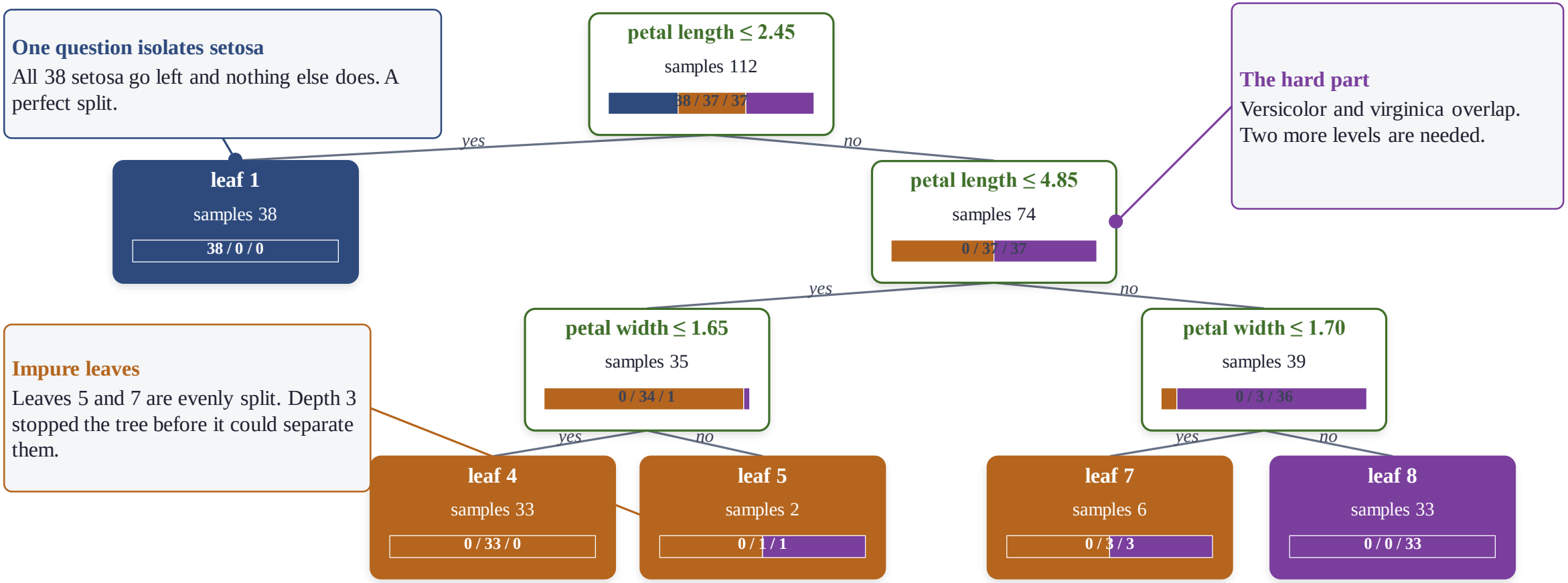
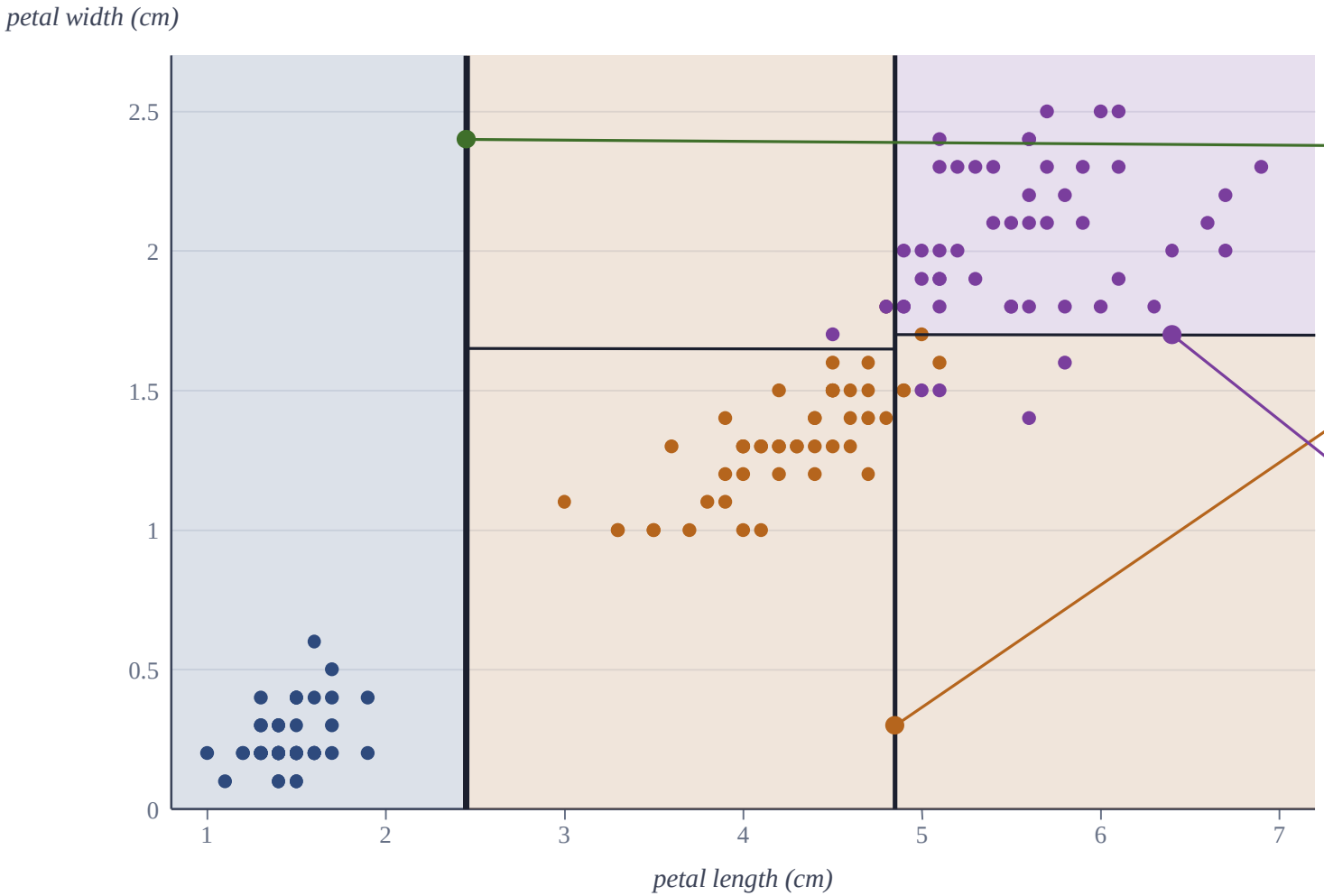


Illustration: how the tree carves up feature space

All 150 flowers by petal length and width. Every split is a straight cut parallel to an axis, producing rectangular regions.



Cut 1: length 2.45
The root split, a full height line. Setosa alone on the left.

Cut 2: length 4.85
Splits the remaining region into two halves.

Cut 3: width 1.70
Long petals that are also wide are virginica.

Boxes, never diagonals
Trees can only cut parallel to an axis. A diagonal boundary needs many small steps to approximate.

Size, accuracy and feature importance

PYTHON

```python
print("Depth:", tree.get_depth(), "  Leaves:", tree.get_n_leaves())
print("Train accuracy:", round(tree.score(Xi_tr, yi_tr), 3))
print("Test accuracy: ", round(tree.score(Xi_te, yi_te), 3))
imp = pd.Series(tree.feature_importances_, index=iris.feature_names)
print(imp.sort_values(ascending=False).round(3))
```

1. **Tree size**
   get_depth and get_n_leaves describe how complex the fitted tree is.
2. **Accuracy**
   Training versus test accuracy, the usual check for overfitting.
3. **Importance**
   feature_importances_ sums each feature's impurity decreases across all its splits, weighted by node size, and normalises them to total 1.

Output: petal length does almost everything

```
OUTPUT
Depth: 3        Leaves: 5
Train accuracy: 0.964
Test accuracy:          0.895
petal length (cm)               0.951
petal width (cm)                0.049
sepal width (cm)                0.000
sepal length (cm)               0.000
dtype: float64
```

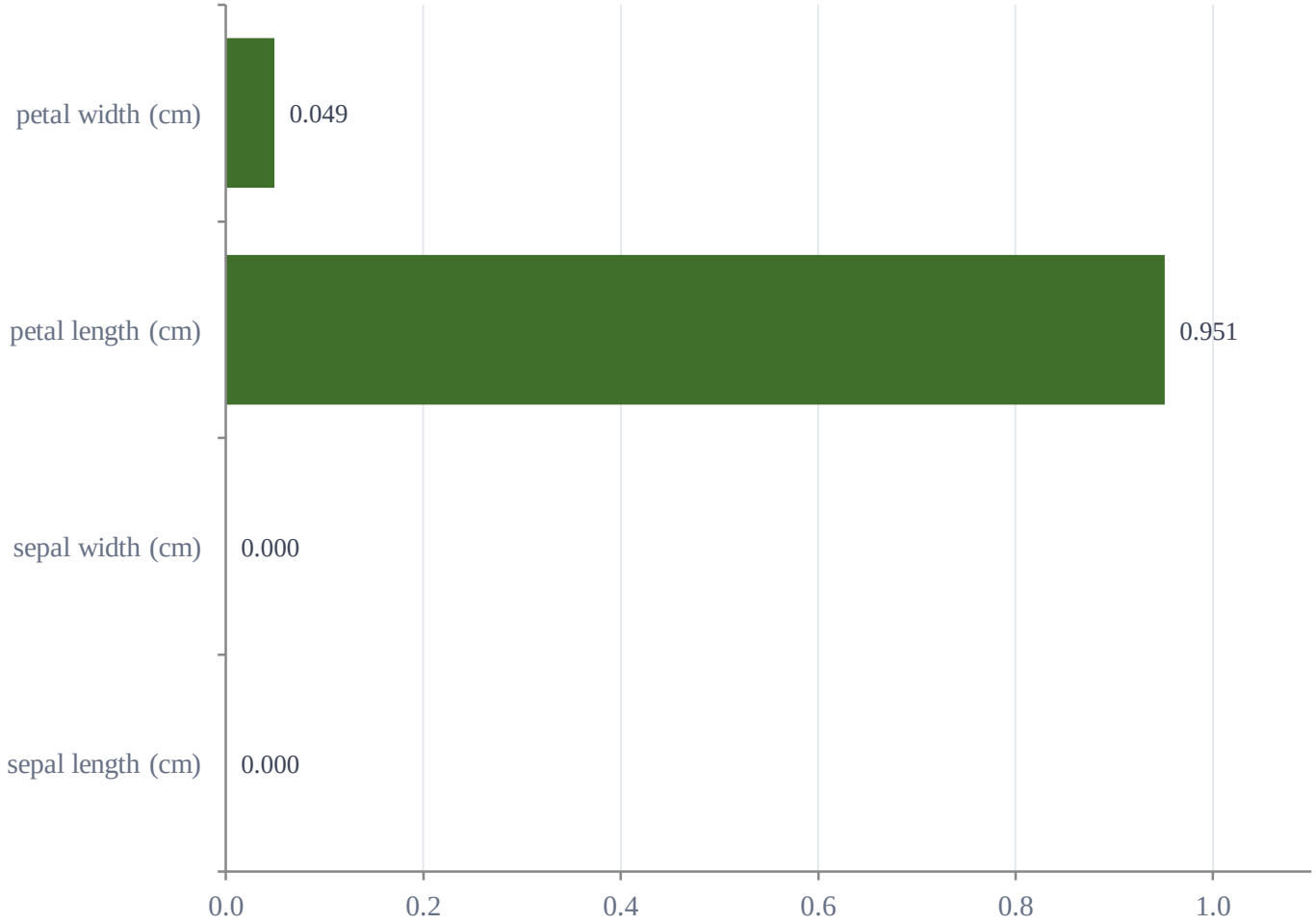
Depth 3, 5 leaves
Four split nodes and five leaves, as drawn.

Test accuracy 0.895
Lower than training at 0.964. With only 38 test flowers each error costs 2.6 points, so this gap is partly noise.

Petal length 0.951
Used at the root and second level, where most impurity is removed.

Sepal measurements 0
Never used for a split, so zero importance. They are not useless, just redundant once petals are known.

Chart: impurity based importance in the iris tree



1. **Sums to one**
   Importances are shares of the total impurity removed by the tree.

2. **Depends on the tree**
   A different random seed or depth can shift importance between correlated features like petal length and width.

3. **Not a direction**
   Importance says how much a feature was used, not whether larger values raise or lower a class.

Feature importance: uses and traps

1 How it is computed

Every time a feature is used to split, record the impurity decrease multiplied by the share of samples at that node. Add up per feature and normalise.

2 Useful for

Quick screening of which inputs the tree relies on, and for spotting leakage when one suspicious feature dominates.

3 Bias toward many values

Continuous features and high cardinality identifiers offer more thresholds, so they can look important by chance.

4 Correlated features

When two features carry the same signal the tree uses one and starves the other, which then looks unimportant.

5 Training data only

Impurity importance is measured on training data, so it can reward features that only help the tree overfit.

6 A better alternative

Permutation importance shuffles one feature on validation data and measures the drop in score. Slower, but less biased.

Following one flower through the tree

PYTHON

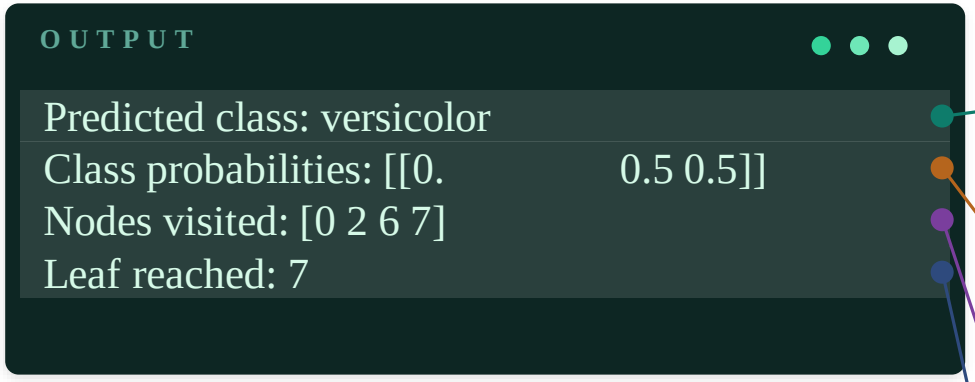
```python
flower = np.array([[6.1, 2.8, 4.9, 1.6]])
print("Predicted class:", iris.target_names[tree.predict(flower)[0]])
print("Class probabilities:", tree.predict_proba(flower).round(3))
path = tree.decision_path(flower).indices
print("Nodes visited:", path)
print("Leaf reached:", tree.apply(flower)[0])
```

1. **A new flower**
   Sepal 6.1 by 2.8, petal 4.9 by 1.6 cm. Note the double brackets: one row, four columns.

2. **Prediction and probabilities**
   Probabilities are simply the class shares in the leaf the flower lands in.

3. **The route taken**
   decision_path lists every node visited; apply returns only the final leaf.

Output: a tie broken by class order



Predicted versicolor
But only just. See the probabilities.

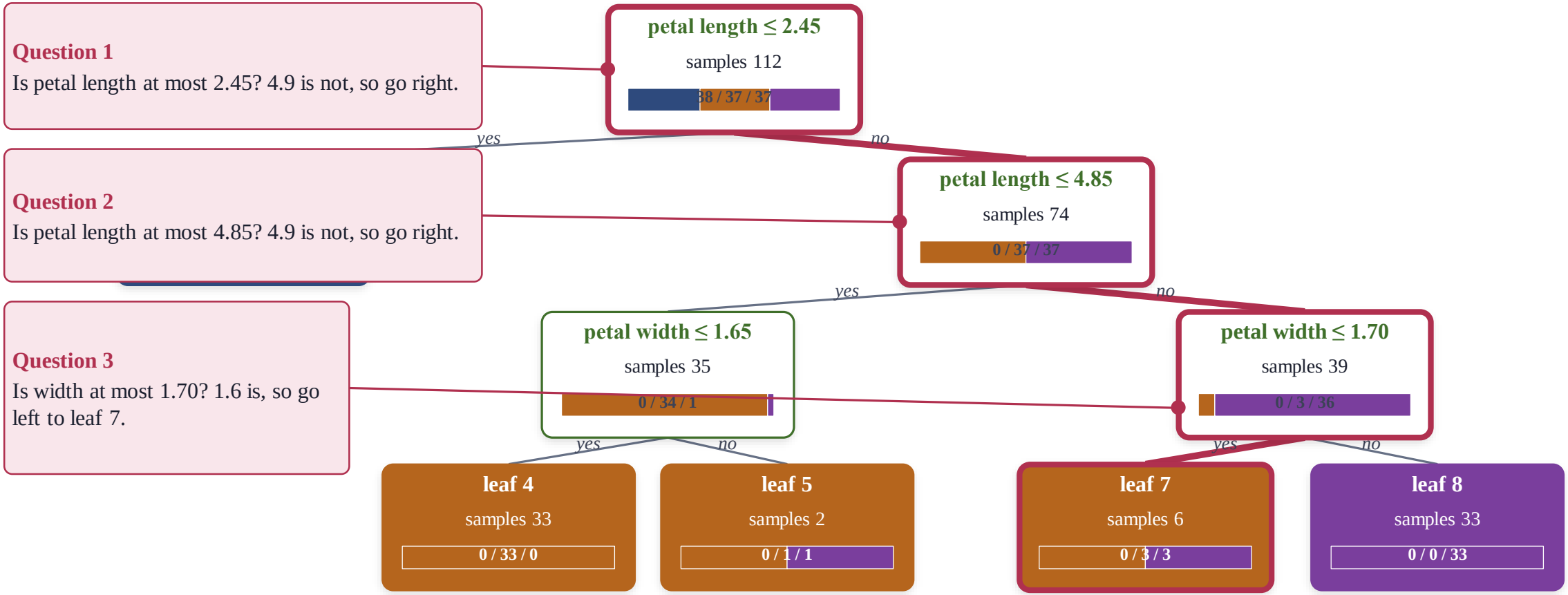
0.5 and 0.5
The flower reached leaf 7, which holds 3 versicolor and 3 virginica. When classes tie, scikit learn picks the first class in label order.

Nodes 0, 2, 6, 7
Root, then petal length above 2.45, then petal length above 4.85, then petal width at most 1.70.

Leaf 7
An impure leaf. A deeper tree might separate these six training flowers.

Illustration: the decision path, highlighted

The red route shows the three questions asked about the flower with petal length 4.9 cm and petal width 1.6 cm.



Inspecting the raw tree structure

PYTHON

```python
t = tree.tree_
print("node   feature          threshold  samples  class_counts")
for node in range(t.node_count):
        feat = iris.feature_names[t.feature[node]] if t.children_left[node] != t.children_right[node] else "LEAF"
        thr = f"{t.threshold[node]:.2f}" if feat != "LEAF" else ""
        counts = (t.value[node][0] * t.n_node_samples[node]).round(0).astype(int)
        print(f"{node:>4}  {feat:<18} {thr:>9}  {t.n_node_samples[node]:>7}  {counts}")
```

1

2

3

1 The tree_ object

Every fitted tree stores parallel arrays: feature, threshold, children, sample counts and class values, one entry per node.

2 Leaves have no children

A node is a leaf when its left and right child pointers are equal, both set to a sentinel value.

3 Rebuild the counts

value stores class proportions in this version, so multiplying by the node's sample count recovers whole numbers.

Output: the node table behind the drawing

OUTPUT

node	feature	threshold	samples	class_counts
0	petal length (cm)	2.45	112	[38 37 37]
1	LEAF		38	[38 0 0]
2	petal length (cm)	4.85	74	[0 37 37]
3	petal width (cm)	1.65	35	[0 34 1]
4	LEAF		33	[0 33 0]
5	LEAF		2	[0 1 1]
6	petal width (cm)	1.70	39	[0 3 36]
7	LEAF		6	[0 3 3]
8	LEAF		33	[0 0 33]

Node 0
112 training flowers, 38, 37 and 37 per class.

Node 1
The pure setosa leaf.

Node 5
A leaf with just two flowers, one of each class: the kind of tiny leaf that signals overfitting risk.

Node 7
The impure leaf where our test flower landed.

From a leaf to a prediction

1 Classification label

The majority class among training cases in the leaf. Ties go to the class that comes first in label order.

2 Classification probability

The share of each class in the leaf. A leaf of 33 virginica gives probability 1.0; a leaf of 3 and 3 gives 0.5 each.

3 Coarse probabilities

Every case in a leaf gets the identical probability, so a small tree outputs only a handful of distinct values.

4 Overconfident tiny leaves

A leaf with one training case predicts probability 1.0 on the strength of a single example. min_samples_leaf guards against this.

5 Regression prediction

For a regression tree the leaf predicts the mean target of its training cases. The whole model is a staircase.

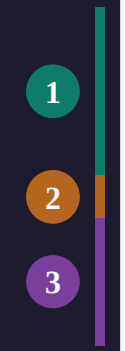
6 Fast inference

Prediction needs only as many comparisons as the depth of the tree, regardless of the number of training rows.

Letting a tree grow without limits

PYTHON

```
from sklearn.datasets import load_breast_cancer
bc = load_breast_cancer()
Xb_tr, Xb_te, yb_tr, yb_te = train_test_split(bc.data, bc.target,
                                              stratify=bc.target, random_state=42)
full = DecisionTreeClassifier(random_state=42).fit(Xb_tr, yb_tr)
print("Fully grown depth:", full.get_depth(), "  leaves:", full.get_n_leaves())
print("Train accuracy:", round(full.score(Xb_tr, yb_tr), 4))
print("Test accuracy: ", round(full.score(Xb_te, yb_te), 4))
```



1 Back to breast cancer

426 training patients and 143 test patients, the same proportions as the logistic regression section.

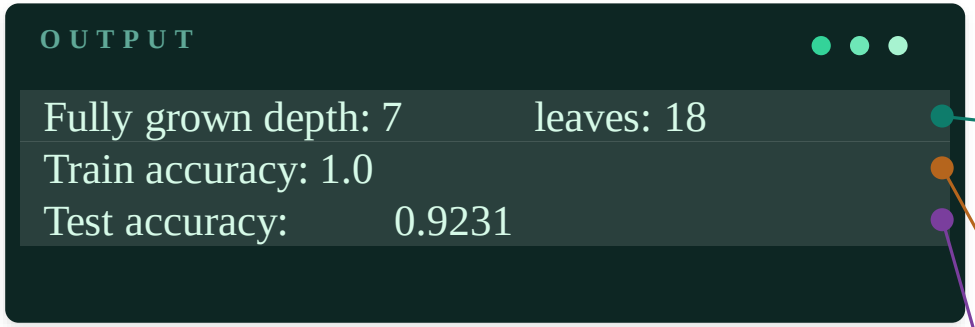
2 No limits

With default settings the tree keeps splitting until every leaf is pure.

3 Size and accuracy

Depth, leaf count and the familiar pair of accuracies.

Output: perfect on training, worse on test



Depth 7, 18 leaves
The tree grew until it had isolated every training patient into a pure leaf.

Training accuracy 1.0
Every training case classified correctly. This is memorisation, not proof of skill.

Test accuracy 0.9231
11 errors on 143 patients, against 2 for logistic regression. The unrestricted tree has overfitted.

Why unrestricted trees overfit

1 Unlimited flexibility

A tree can keep splitting until each leaf contains one case, which always yields perfect training accuracy.

2 Splits on tiny samples

Deep nodes are chosen using only a handful of cases, so their thresholds reflect noise rather than real patterns.

3 Greedy and myopic

Each split is best only for the current node; the tree never looks ahead or reconsiders, so early mistakes propagate.

4 High variance

Small changes to the training data can produce a completely different tree, as we will demonstrate shortly.

5 Low bias

Given enough depth, a tree can represent almost any pattern, which is why trees respond so well to the controls in section 5.

6 The remedies

Limit depth, require larger leaves, prune with cost complexity, or average many trees in a random forest.

Comparing split criteria

PYTHON

```python
for crit in ["gini", "entropy", "log_loss"]:
    m = DecisionTreeClassifier(criterion=crit, random_state=42).fit(Xb_tr, yb_tr)
    print(f"{crit:<9} depth {m.get_depth():>2}  leaves {m.get_n_leaves():>2}  test acc {m.score(Xb_te, yb_te):.4f}")
```

1 2

1 Three criteria

gini is the default. entropy and log_loss are the same criterion under two names in recent versions.

2 Fit and compare

Each fully grown tree is summarised by depth, leaves and test accuracy.

Output: small differences, no clear winner

OUTPUT

gini	depth	7	leaves 18	test acc 0.9231
entropy	depth	6	leaves 15	test acc 0.9301
log_loss	depth	6	leaves 15	test acc 0.9301

Gini: 18 leaves, 0.9231
The baseline full tree.

Entropy: 15 leaves, 0.9301
Slightly smaller and one more test patient correct. On a different split the order could reverse.

log_loss equals entropy
Identical results, confirming they are the same criterion. Choose the criterion by cross validation, not by one test split.

Proving that trees ignore feature scaling

PYTHON

```python
from sklearn.preprocessing import StandardScaler
raw_tree = DecisionTreeClassifier(max_depth=4, random_state=0).fit(Xb_tr, yb_tr)
sc = StandardScaler().fit(Xb_tr)
scaled_tree = DecisionTreeClassifier(max_depth=4, random_state=0).fit(sc.transform(Xb_tr), yb_tr)
print("Raw features test acc:   ", round(raw_tree.score(Xb_te, yb_te), 4))
print("Scaled features test acc:", round(scaled_tree.score(sc.transform(Xb_te), yb_te), 4))
same = (raw_tree.predict(Xb_te) == scaled_tree.predict(sc.transform(Xb_te))).mean()
print("Share of identical predictions:", same)
```

1

2

3

1 Raw features

A depth 4 tree on the original units.

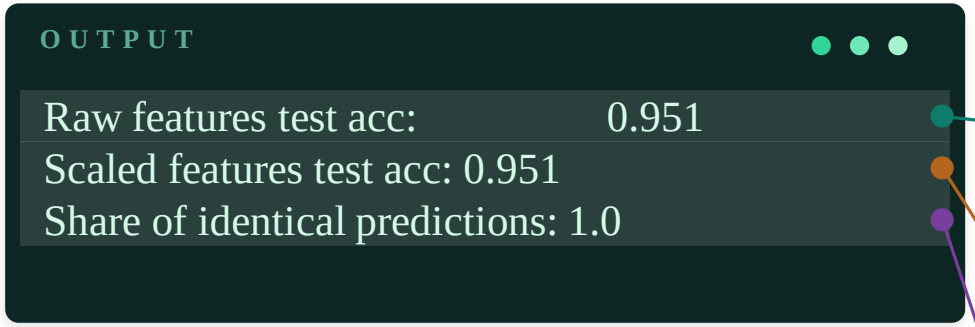
2 Scaled features

The same tree settings on standardised features.

3 Compare everything

Not only accuracy but every single prediction.

Output: identical trees, identical predictions



0.951 raw
A depth limit of 4 already beats the unrestricted tree's 0.923, an early hint of section 5.

0.951 scaled
Exactly the same.

100 percent identical
Standardising preserves the order of values, and a threshold split only depends on order. Thresholds move, decisions do not.

Demonstrating instability

PYTHON

```python
rng = np.random.default_rng(0)
for trial in range(5):
    rows = rng.choice(len(Xb_tr), size=int(0.9 * len(Xb_tr)), replace=False)
    m = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xb_tr[rows], yb_tr[rows])
    root = bc.feature_names[m.tree_.feature[0]]
    second = bc.feature_names[m.tree_.feature[1]]
    print(f"sample {trial}: root = {root:<22} next = {second}")
```

1

2

3

1 Five resamples

Each time, draw a random 90 percent of the training rows without replacement.

2 Same settings

Identical depth 3 trees with identical random_state; only the data changes.

3 Which features lead?

tree_.feature[0] is the root feature; feature[1] is the root's left child.

Output: four different root features from five samples

OUTPUT

```
sample 0: root = worst perimeter          next = worst concave points
sample 1: root = mean concave points      next = worst area
sample 2: root = worst area               next = worst concave points
sample 3: root = worst radius             next = worst concave points
sample 4: root = worst radius             next = worst concave points
```

worst perimeter
The first sample chooses perimeter at the root.

mean concave points
Removing a different 10 percent of rows changes the very first question.

worst radius
Radius, perimeter and area measure nearly the same thing, so tiny data changes flip which one wins.

The lesson
Do not over interpret the exact structure of one tree. Random forests exploit this instability by averaging many varied trees.

Strengths and weaknesses of a single decision tree

Strengths

- Readable rules that non specialists can follow and audit.
- No scaling needed and robust to monotonic transformations and outliers in features.
- Captures interactions and thresholds automatically.
- Handles numeric and encoded categorical features together, and multiclass problems natively.
- Fast to train and extremely fast to predict.

vs

Weaknesses

- Overfits badly without limits; needs tuning.
- Unstable: small data changes can produce a different tree.
- Axis parallel boundaries approximate diagonal patterns with jagged staircases.
- Coarse probabilities, since every case in a leaf shares one value.
- Usually less accurate than ensembles of trees or a well specified linear model.

Regression trees: predicting numbers with steps

1 Same growth procedure

Recursive binary splitting, but impurity is replaced by the variance, or squared error, of the target within each node.

2 Best split

The split that most reduces the size weighted squared error of the children compared with the parent.

3 Leaf prediction

The average target value of the training cases in the leaf.

4 A staircase

The prediction is piecewise constant: flat within each region, jumping between regions. More leaves means more, smaller steps.

5 No extrapolation

Predictions can never go above the largest or below the smallest leaf average seen in training.

6 Depth controls smoothness

Shallow trees give a few coarse steps; deep trees follow every noisy wiggle.

Fitting regression trees of increasing depth

PYTHON

```python
from sklearn.tree import DecisionTreeRegressor
x = np.sort(rng.uniform(0, 5, 200)).reshape(200, 1)
wave = np.sin(x).ravel() + rng.normal(0, 0.15, 200)
for depth in [1, 2, 4, 8, None]:
    reg = DecisionTreeRegressor(max_depth=depth, random_state=0).fit(x, wave)
    steps = len(np.unique(reg.predict(x)))
    print(f"max_depth {str(depth):<5} distinct predicted values {steps:>3}  train R2 {reg.score(x, wave):.3f}")
```

1 2 3

1 A noisy sine curve

200 points between 0 and 5 with random noise added.

2 Five depths

From a single split to an unlimited tree.

3 Count the steps

The number of distinct predicted values equals the number of leaves actually used.

Output: from two steps to two hundred

OUTPUT

max_depth 1	distinct predicted values	2	train R2 0.772
max_depth 2	distinct predicted values	4	train R2 0.868
max_depth 4	distinct predicted values	16	train R2 0.951
max_depth 8	distinct predicted values	112	train R2 0.988
max_depth None	distinct predicted values	200	train R2 1.000

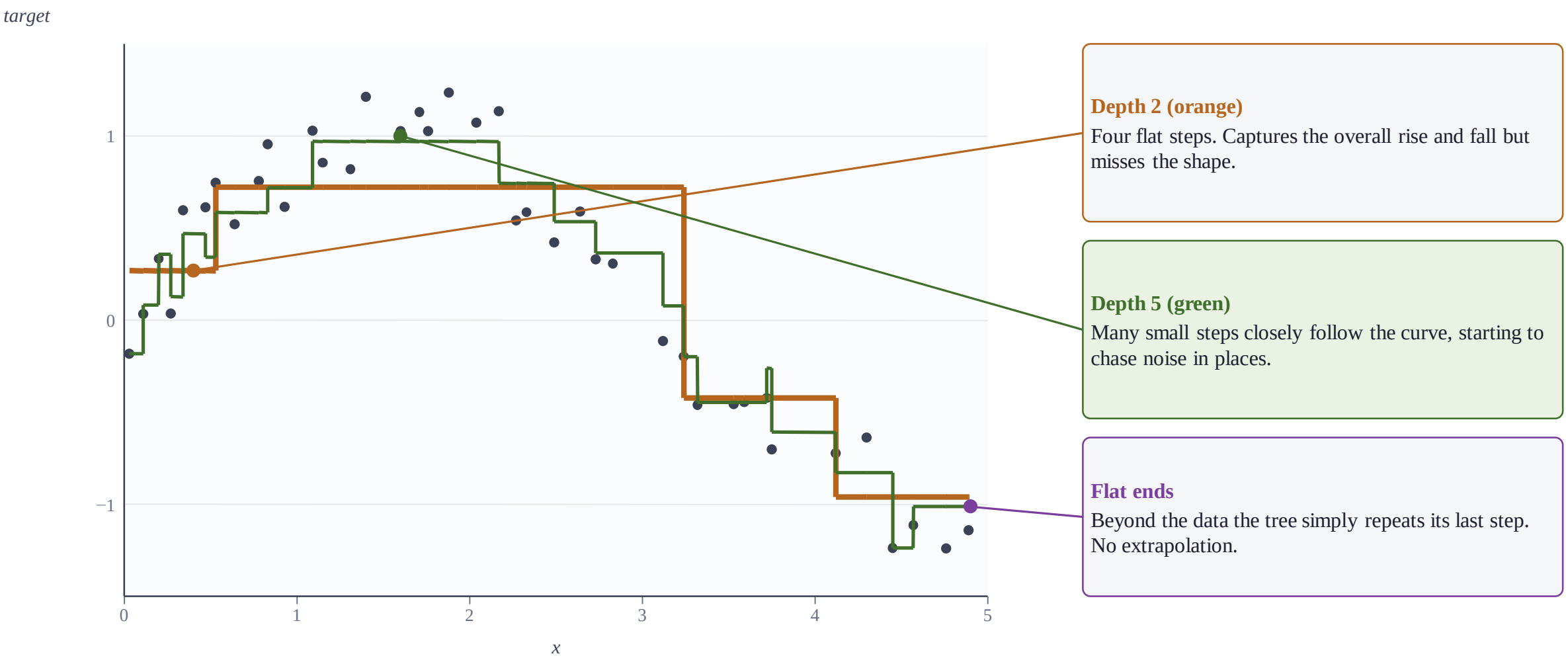
Depth 1: 2 values
One split, two flat levels. R squared 0.772 already, because the sine has two broad halves.

Depth 4: 16 values
Sixteen steps trace the curve well: R squared 0.951.

Unlimited: 200 values
One leaf per point and training R squared 1.000. Every bit of noise is memorised.

Illustration: a regression tree is a staircase

Every fifth point of the noisy sine data, with the predictions of a depth 2 tree and a depth 5 tree.



Depth 2 (orange)
Four flat steps. Captures the overall rise and fall but misses the shape.

Depth 5 (green)
Many small steps closely follow the curve, starting to chase noise in places.

Flat ends
Beyond the data the tree simply repeats its last step. No extrapolation.

Which features did the full cancer tree use?

PYTHON

```
1 imp = pd.Series(full.feature_importances_, index=bc.feature_names)
2 print("Features used by the full tree:", int((imp > 0).sum()), "of", len(imp))
3 print(imp.sort_values(ascending=False).head(6).round(3))
```

1

2

1 Importances

One value per breast cancer feature from the unrestricted tree.

2 Used and top six

Count features with non zero importance, then list the six largest.

Output: one feature carries most of the tree

OUTPUT

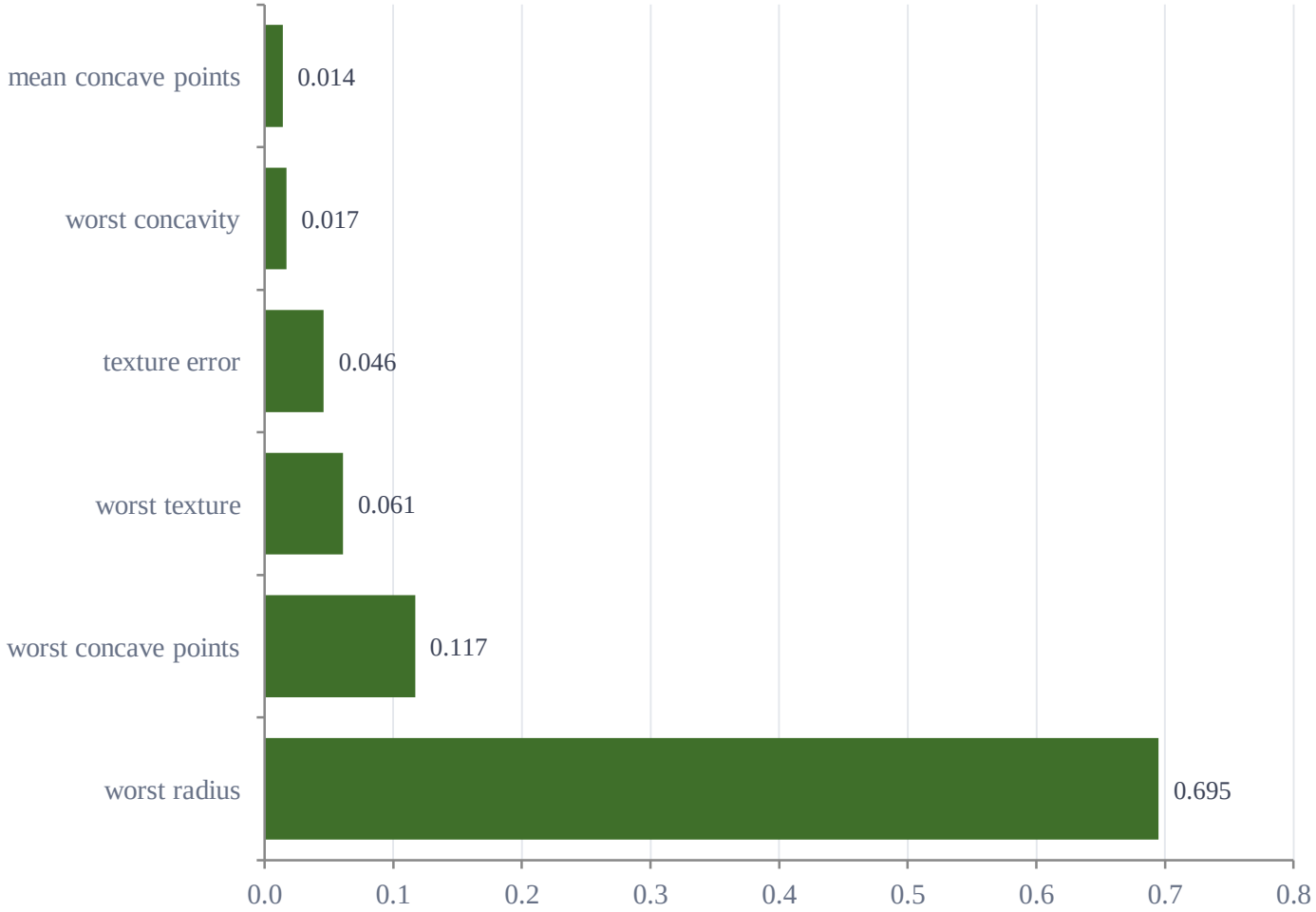
```
Features used by the full tree: 12 of 30
worst radius            0.695
worst concave points    0.117
worst texture           0.061
texture error           0.046
worst concavity         0.017
mean concave points     0.014
dtype: float64
```

12 of 30 used
The other 18 features never appear in any split. Trees perform implicit feature selection.

worst radius 0.695
The root split removes most of the impurity, so it dominates importance.

texture error 0.046
Used deep in the tree on few patients. Such features often reflect overfitting rather than signal.

Chart: the six most important features in the full tree



1. **Concentrated**
   A tree concentrates importance in its root feature far more than logistic regression spreads its coefficients.

2. **Compare with logistic**
   Logistic regression ranked worst texture first; the tree ranks worst radius. Different models can lean on different but correlated features.

3. **Unstable ranking**
   Given the instability demonstration, a different sample might put worst perimeter or worst area at the top instead.

Practical details: categories, missing values and ties

1 Categorical features

scikit learn trees need numeric inputs. One hot encoding works; ordinal encoding also works for trees because a threshold can isolate groups of codes.

2 High cardinality

Hundreds of categories create hundreds of one hot columns. Grouping rare levels or ordinal codes is often better.

3 Missing values

Recent scikit learn versions let trees send missing values down whichever branch reduces impurity most, learned during fit.

4 Thresholds are midpoints

A split threshold sits halfway between two neighbouring training values, which is why we saw 2.45 rather than a measured value.

5 Ties and random_state

When several splits are equally good the choice depends on feature order and random_state. Fix it for reproducibility.

6 Visualising

plot_tree from sklearn.tree draws the fitted tree with matplotlib; export_text gives the rule listing we used.

Decision tree checklist

1

Encode

Turn categories into numbers; no scaling required.

2

Start shallow

Fit a depth 2 or 3 tree to read the main rules.

3

Grow and compare

Fit a full tree and compare train and test accuracy for overfitting.

4

Inspect

Rules, node table, decision paths and importances.

5

Check stability

Refit on resamples; distrust exact structure.

6

Tune

Control growth with the hyperparameters of section 5.

Check your understanding

A tree's leaf contains 4 training cases: 3 of class A and 1 of class B. What does predict_proba return for a new case reaching it?

A 1.0 for A

B 0.75 for A and 0.25 for B

C 0.5 for each

D It depends on the depth

Answer B and why

Tree probabilities are simply the class shares among training cases in the leaf: 3 of 4 is 0.75. Every case reaching this leaf receives exactly the same probabilities, which is why tree probabilities are coarse.

Check your understanding

You standardise all features before fitting a decision tree. What happens to its predictions?

A They improve

B They get worse

C They stay identical

D The tree cannot be fitted

Answer C and why

A split only depends on the order of values within a feature. Standardising keeps that order, so the same splits are chosen with transformed thresholds, and every prediction is identical, exactly as our experiment showed with 100 percent agreement.

SECTION

5

Tuning Decision Trees

Turning an overfitting tree into a model that generalises, using honest validation

- The hyperparameters that control tree growth
- Validation curves for depth and leaf size
- Pre pruning versus cost complexity post pruning
- Grid search and randomised search with cross validation
- Final evaluation on the untouched test set
- Learning curves and nested cross validation
- Class weights and comparing against other models

Why trees must be tuned

Section 4 ended with a fully grown tree scoring 100 percent on training data and 92 percent on test data. Tuning closes that gap.

1 Complexity is a dial

Every tree hyperparameter limits how far the tree can grow, moving it between underfitting and overfitting.

2 The goal

Find the settings that maximise performance on unseen data, not on training data, which always prefers the biggest tree.

3 Honest measurement

Settings must be judged with cross validation on training data. Using the test set to choose would leak it into the model.

4 Smaller is often better

A tuned tree is usually much smaller, easier to read and more stable than an unrestricted one.

5 Diminishing returns

Tuning helps a single tree a lot, but it cannot remove the axis parallel and instability limits. Ensembles go further.

6 Reproducibility

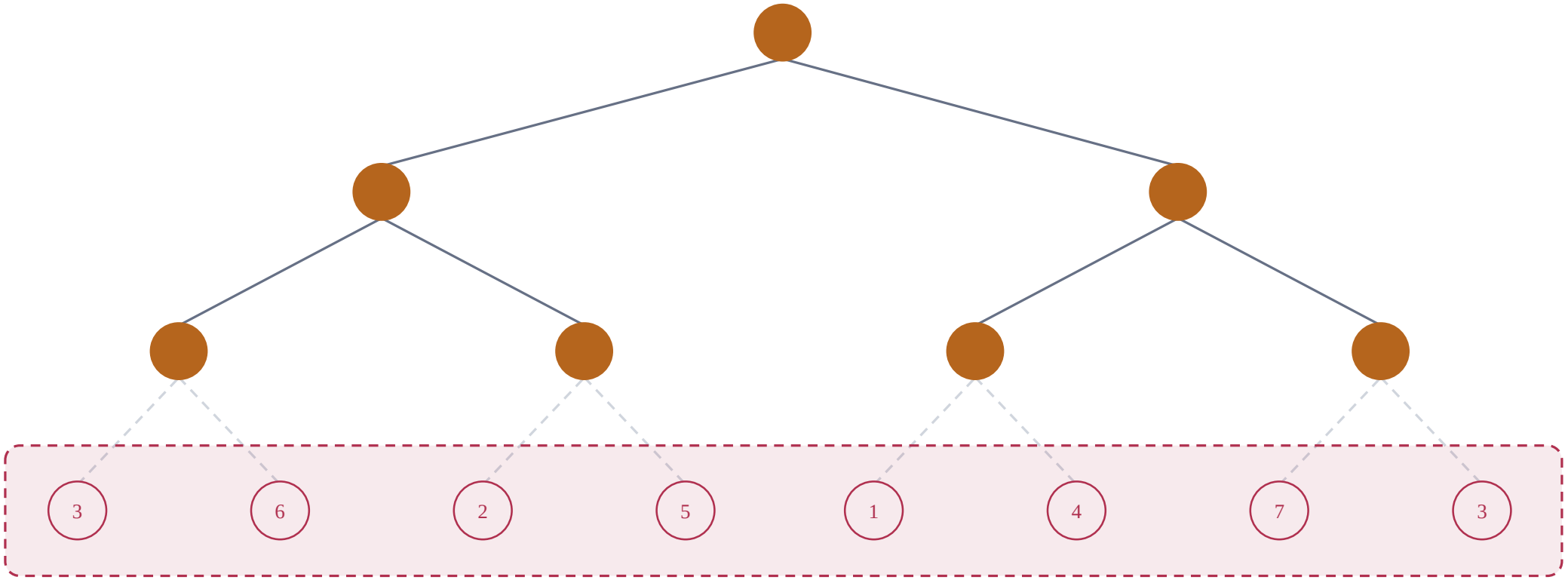
Fix random_state for the tree, the folds and any random search, so results can be repeated exactly.

The hyperparameters of DecisionTreeClassifier

Hyperparameter	Default	What it controls	Effect when made stricter
max_depth	None	The maximum number of questions on any path	Shallower, simpler tree
min_samples_split	2	Fewest cases a node needs before it may split	Fewer splits near the bottom
min_samples_leaf	1	Fewest cases allowed in each child after a split	Larger leaves, smoother probabilities
max_leaf_nodes	None	Total number of leaves; best splits are grown first	A compact tree of fixed size
min_impurity_decrease	0.0	Smallest impurity drop a split must achieve	Weak splits are refused
max_features	None	Number or share of features considered at each split	More randomness, less variance
ccp_alpha	0.0	Strength of cost complexity pruning after growth	Weak branches are cut away
criterion	gini	Impurity measure used to score splits	Usually a small difference
class_weight	None	Extra weight for rare classes when scoring splits	More attention to minority class

Illustration: how the main limits stop a tree from growing

A schematic fully grown tree. Each hyperparameter removes a different part of it.



max_depth 2 stops growth above this band

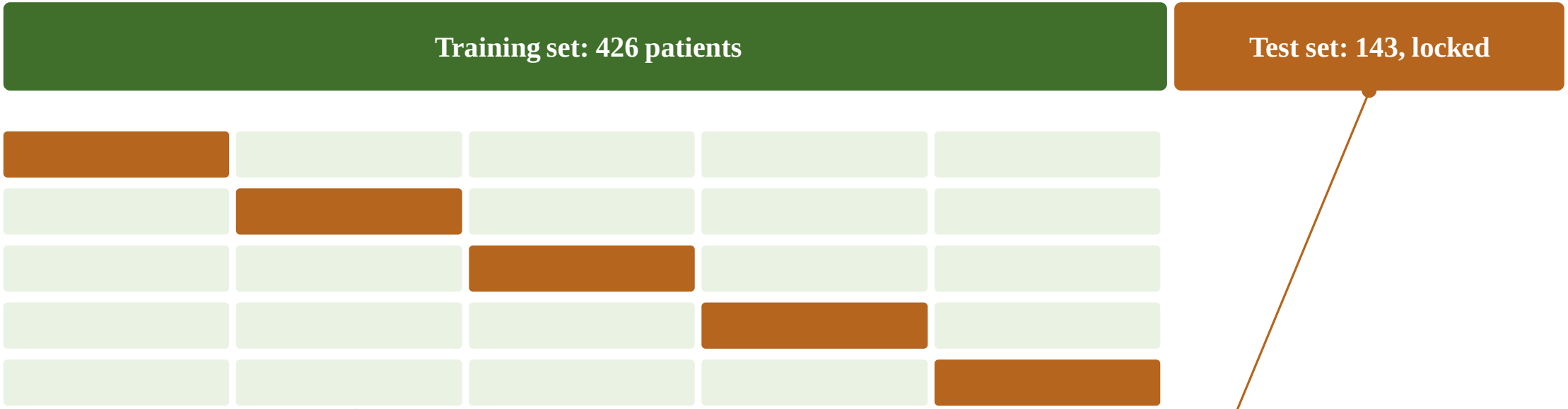
Numbers in the faded leaves are sample counts.

min_samples_leaf 5 would forbid leaves of 1 to 4.

max_leaf_nodes 4 keeps only the four best leaves.

Illustration: where tuning happens in the data

Hyperparameters are chosen with cross validation inside the training set. The test set is opened once, at the very end.



Inside: choose settings

Each candidate is trained on four folds and scored on the fifth, five times. The best mean score wins.

Outside: confirm once

The winning settings are refitted on all 426 training patients and scored once on the 143 test patients.

A validation curve for max_depth

PYTHON

```
import numpy as np, pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
bc = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
        bc.data, bc.target, test_size=0.25, stratify=bc.target, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
for d in [1, 2, 3, 4, 5, 6, 8, 10, None]:
        m = DecisionTreeClassifier(max_depth=d, random_state=0)
        s = cross_val_score(m, X_train, y_train, cv=cv)
        tr = m.fit(X_train, y_train).score(X_train, y_train)
        print(f"max_depth {str(d):<4}  train {tr:.3f}  cv {s.mean():.3f}  sd {s.std():.3f}")
```

1

2

3

4

1 Data

The breast cancer training and test split from the earlier sections, with the same 75 to 25 proportions.

2 Folds

Stratified five fold splitting with shuffling, reused for every experiment so comparisons are fair.

3 One depth at a time

Cross validated accuracy plus training accuracy for depths 1 to 10 and unlimited.

4 Report

sd is the standard deviation of the five fold scores.

Output: training accuracy rises, validation plateaus

OUTPUT

max_depth 1	train 0.923	cv 0.899	sd 0.028
max_depth 2	train 0.958	cv 0.908	sd 0.024
max_depth 3	train 0.977	cv 0.911	sd 0.024
max_depth 4	train 0.988	cv 0.908	sd 0.027
max_depth 5	train 0.995	cv 0.913	sd 0.019
max_depth 6	train 0.998	cv 0.906	sd 0.012
max_depth 8	train 1.000	cv 0.913	sd 0.012
max_depth 10	train 1.000	cv 0.913	sd 0.012
max_depth None	train 1.000	cv 0.913	sd 0.012

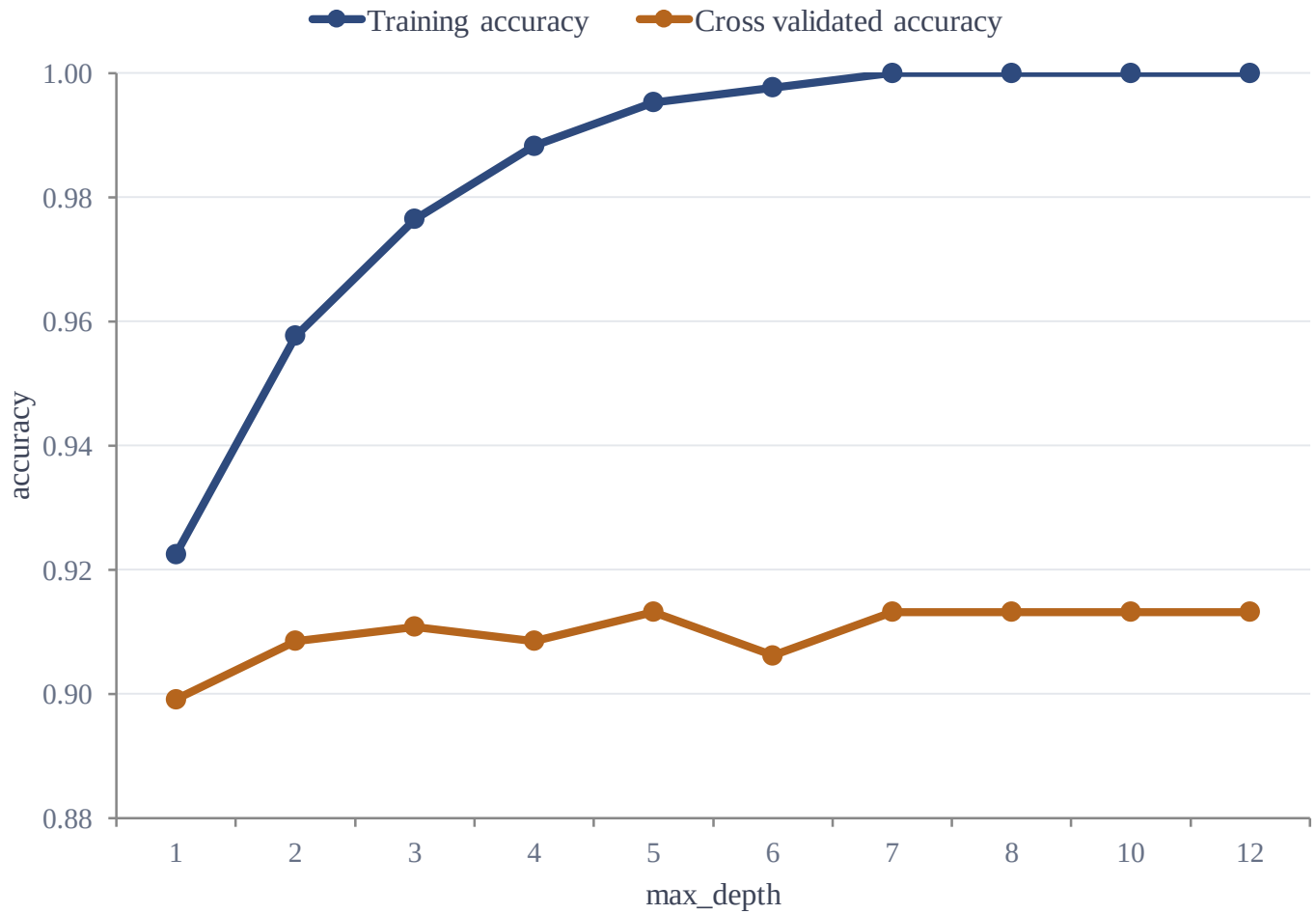
Depth 1: 0.899
A single question already gets 90 percent right. Strong signal in one feature.

Depth 3: 0.911
Close to the best validation score with a small, readable tree.

Depth 5: 0.913
Training accuracy 0.995, validation no better than depth 3.

Unlimited: 0.913
Training hits 1.000 while validation stays flat. Extra depth is spent memorising.

Chart: the validation curve for max_depth



1 The widening gap
The distance between the lines is overfitting. It grows steadily with depth.

2 A flat validation line
Cross validated accuracy barely moves after depth 2, so the simplest tree on the plateau is the sensible choice.

3 Noise in the plateau
Differences of 0.002 to 0.005 are smaller than the fold standard deviations of about 0.02, so they are not meaningful.

A validation curve for min_samples_leaf

PYTHON

```
for leaf in [1, 2, 5, 10, 20, 40, 80]:
        m = DecisionTreeClassifier(min_samples_leaf=leaf, random_state=0)
        s = cross_val_score(m, X_train, y_train, cv=cv)
        m.fit(X_train, y_train)
        print(f"min_samples_leaf {leaf:>3}: leaves {m.get_n_leaves():>3}  depth {m.get_depth():>2}  cv {s.mean():.3f}")
```

1

2

3

1 Leaf size

Seven values from 1, the default, to 80 cases per leaf. Depth is left unlimited so leaf size is the only limit.

2 Score and refit

Cross validated accuracy, then a full fit to measure the resulting tree's size.

3 Report

Leaves, depth and validation accuracy side by side.

Output: larger leaves shrink the tree automatically

OUTPUT

```
min_samples_leaf     1: leaves   18   depth   7   cv 0.913
min_samples_leaf     2: leaves   16   depth   6   cv 0.906
min_samples_leaf     5: leaves   12   depth   6   cv 0.918
min_samples_leaf    10: leaves   10   depth   5   cv 0.916
min_samples_leaf    20: leaves    8   depth   4   cv 0.906
min_samples_leaf    40: leaves    6   depth   4   cv 0.899
min_samples_leaf    80: leaves    4   depth   3   cv 0.899
```

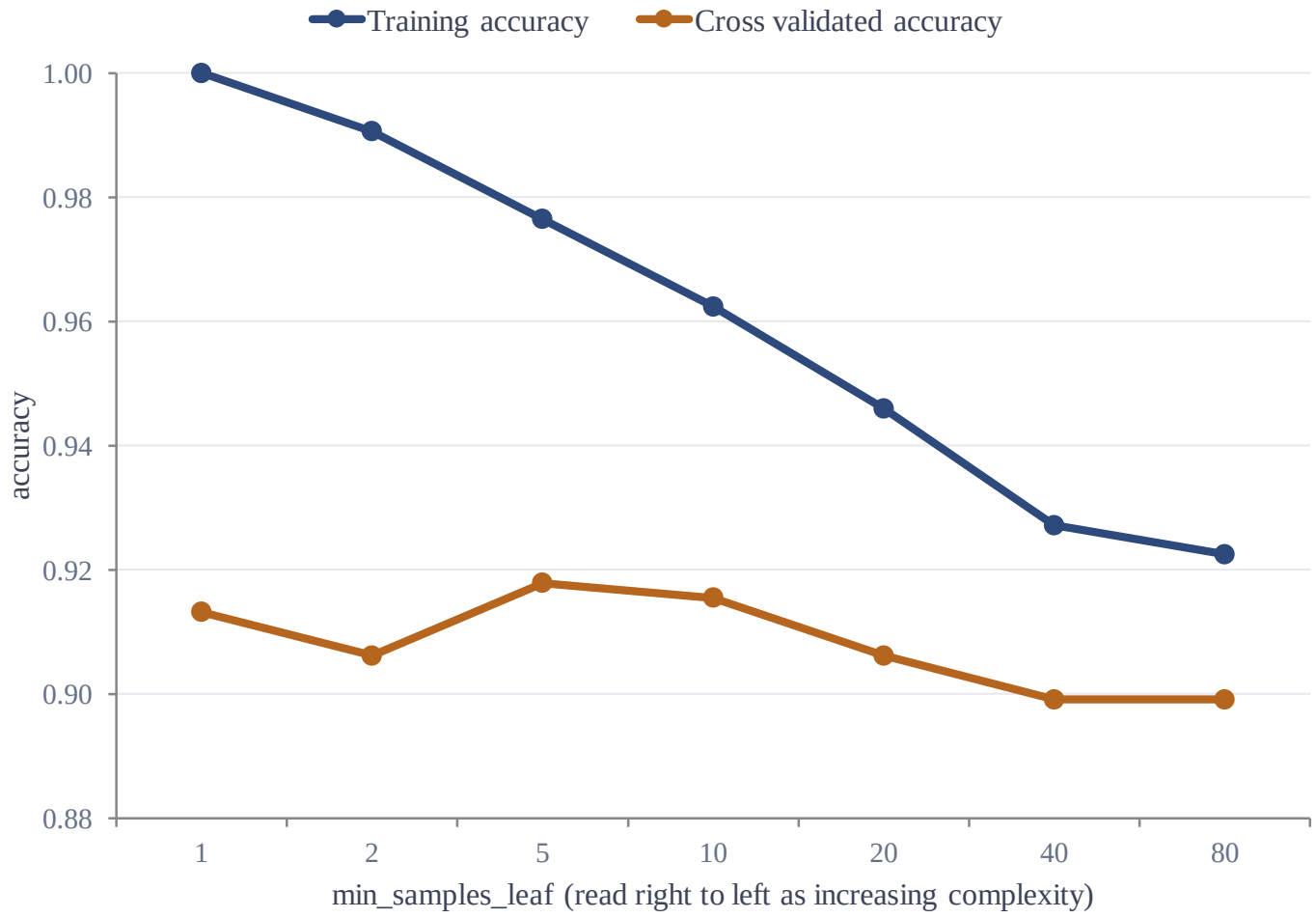
1 case per leaf
18 leaves, depth 7: the unrestricted tree.

5 per leaf: best, 0.918
12 leaves. Forbidding tiny leaves removes the noisiest splits.

10 per leaf
10 leaves, still 0.916. Similar accuracy with a much smaller tree.

80 per leaf
Only 4 leaves: the tree is now too coarse and underfits slightly.

Chart: accuracy against minimum leaf size



1. **Mirror image of depth**
   Here complexity falls as the value rises, so the overfitting gap is on the left.

2. **Smoother probabilities**
   Larger leaves average more cases, so predicted probabilities become more reliable, not just labels.

3. **A robust default**
   Values between 5 and 20 often work well for datasets of a few hundred to a few thousand rows.

Two more growth limits

PYTHON

```python
for split in [2, 10, 30, 60]:
    m = DecisionTreeClassifier(min_samples_split=split, random_state=0).fit(X_train, y_train)
    print(f"min_samples_split {split:>3}: leaves {m.get_n_leaves():>3}")
for cap in [4, 8, 16]:
    m = DecisionTreeClassifier(max_leaf_nodes=cap, random_state=0)
    print(f"max_leaf_nodes {cap:>3}: cv {cross_val_score(m, X_train, y_train, cv=cv).mean():.3f}")
```

1

2

1 min_samples_split

A node must contain at least this many cases to be split. Larger values stop growth earlier.

2 max_leaf_nodes

Caps the number of leaves. The tree grows best first, always expanding the split with the largest impurity decrease.

Output: fewer leaves with each constraint

```
OUTPUT
min_samples_split      2: leaves  18
min_samples_split     10: leaves  14
min_samples_split     30: leaves  10
min_samples_split     60: leaves   9
max_leaf_nodes     4: cv 0.904
max_leaf_nodes     8: cv 0.906
max_leaf_nodes    16: cv 0.906
```

Split threshold 2: 18 leaves
Any node with two or more cases may split, the default.

Split threshold 60: 9 leaves
Half the leaves disappear.

4 leaves: 0.904
Best first growth keeps only the most valuable splits.

16 leaves: 0.906
Adding leaves beyond 8 gives nothing.

Pre pruning versus post pruning

Pre pruning: stop early

- Limits applied while the tree grows: max_depth, min_samples_leaf, min_samples_split, max_leaf_nodes, min_impurity_decrease.
- Fast, because the unwanted branches are never built.
- Short sighted: a split that looks weak now might enable excellent splits below it, and early stopping never finds out.
- Several interacting settings to tune together.

vs

Post pruning: grow then cut

- Grow the full tree first, then remove branches that add complexity without enough improvement.
- In scikit learn: minimal cost complexity pruning through ccp_alpha.
- Sees the whole tree before deciding, so it can keep a weak split that enables strong ones beneath it.
- A single setting to tune, with a natural sequence of candidate values.

Cost complexity pruning explained in words

1 A score with a penalty

Every possible subtree is scored as its total training impurity plus alpha times its number of leaves. Leaves now cost something.

2 Alpha is the price of a leaf

At alpha zero, leaves are free and the full tree wins. As alpha rises, extra leaves must earn their keep.

3 The weakest link

For each internal node, compute how much impurity its branch removes per extra leaf. The branch with the smallest value is cut first.

4 A nested sequence

Repeating this gives a sequence of ever smaller subtrees, each optimal for a range of alpha values.

5 The pruning path

cost_complexity_pruning_path returns exactly the alpha values where the tree changes, so there is no guessing a grid.

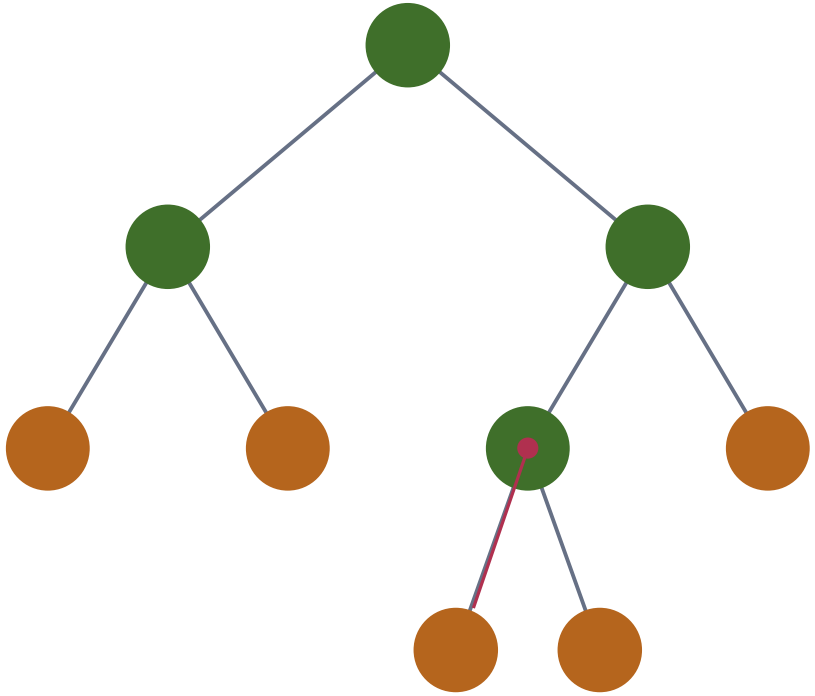
6 Choose by validation

Cross validate each candidate alpha and pick the one with the best score, or the largest alpha within one standard error of it.

Illustration: pruning the weakest link

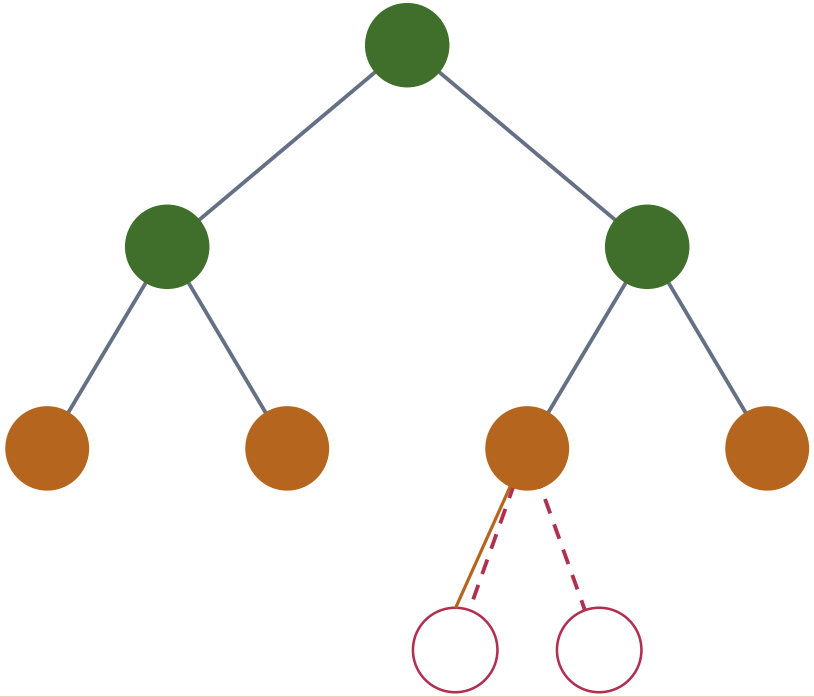
As alpha rises, branches that remove little impurity for the leaves they add are collapsed back into a single leaf.

Before: 5 leaves



raise alpha

After: 4 leaves



The weak branch
Its split removed very little impurity for the extra leaf it created.

Collapsed into a leaf
Its parent now predicts directly. Training impurity rises slightly, complexity falls.

Computing the cost complexity pruning path

PYTHON

```python
base = DecisionTreeClassifier(random_state=0)
path = base.cost_complexity_pruning_path(X_train, y_train)
alphas, impurities = path.ccp_alphas, path.impurities
print("Number of candidate alphas:", len(alphas))
print("First five alphas:", alphas[:5].round(5))
print("Three largest alphas:", np.flip(alphas)[:3].round(4))
```

1. **Grow, then trace**
   cost_complexity_pruning_path grows the full tree on the training data and records every alpha at which the pruned tree changes.

2. **Two arrays**
   ccp_alphas holds the breakpoints; impurities holds the total leaf impurity of the subtree at each one.

3. **Inspect**
   np.flip reverses the array so we can show the largest alphas first without negative indexing.

Output: thirteen candidate trees

OUTPUT

```
Number of candidate alphas: 13
First five alphas: [0.          0.00232 0.00313 0.00423 0.00457]
Three largest alphas: [0.3254 0.0547 0.0217]
```

13 alphas
Thirteen distinct subtrees, from the full 18 leaf tree down to a single root.

Starts at 0
Alpha 0 is the unpruned tree. Small alphas trim the weakest leaves first.

Largest 0.3254
At this alpha even the root split is not worth its cost, leaving a tree that predicts the majority class for everyone. We exclude it from tuning.

Cross validating every pruning level

PYTHON

```python
rows = []
for a in alphas[:np.argmax(alphas)]:
        m = DecisionTreeClassifier(ccp_alpha=a, random_state=0)
        rows.append((a, cross_val_score(m, X_train, y_train, cv=cv).mean(),
                                m.fit(X_train, y_train).get_n_leaves()))
res = pd.DataFrame(rows, columns=["alpha", "cv_acc", "leaves"])
best = res.loc[res.cv_acc.idxmax()]
print(res.round(4).tail(8).to_string(index=False))
print("Best alpha:", round(best.alpha, 5), " leaves:", int(best.leaves), " cv:", round(best.cv_acc, 4))
```

1

2

3

1 One tree per alpha

Every alpha except the largest, using np.argmax to stop before it. Each gets a cross validated score and a leaf count.

2 Collect and pick

A DataFrame of results; idxmax finds the row with the best mean accuracy.

3 Show the end of the path

The last eight candidates, where pruning becomes aggressive, then the winner.

Output: the best pruned tree has 8 leaves

OUTPUT

alpha	cv_acc	leaves
0.0046	0.9203	9
0.0053	0.9226	8
0.0056	0.9226	7
0.0063	0.9202	6
0.0081	0.9179	5
0.0149	0.9061	4
0.0217	0.9085	3
0.0547	0.9038	2

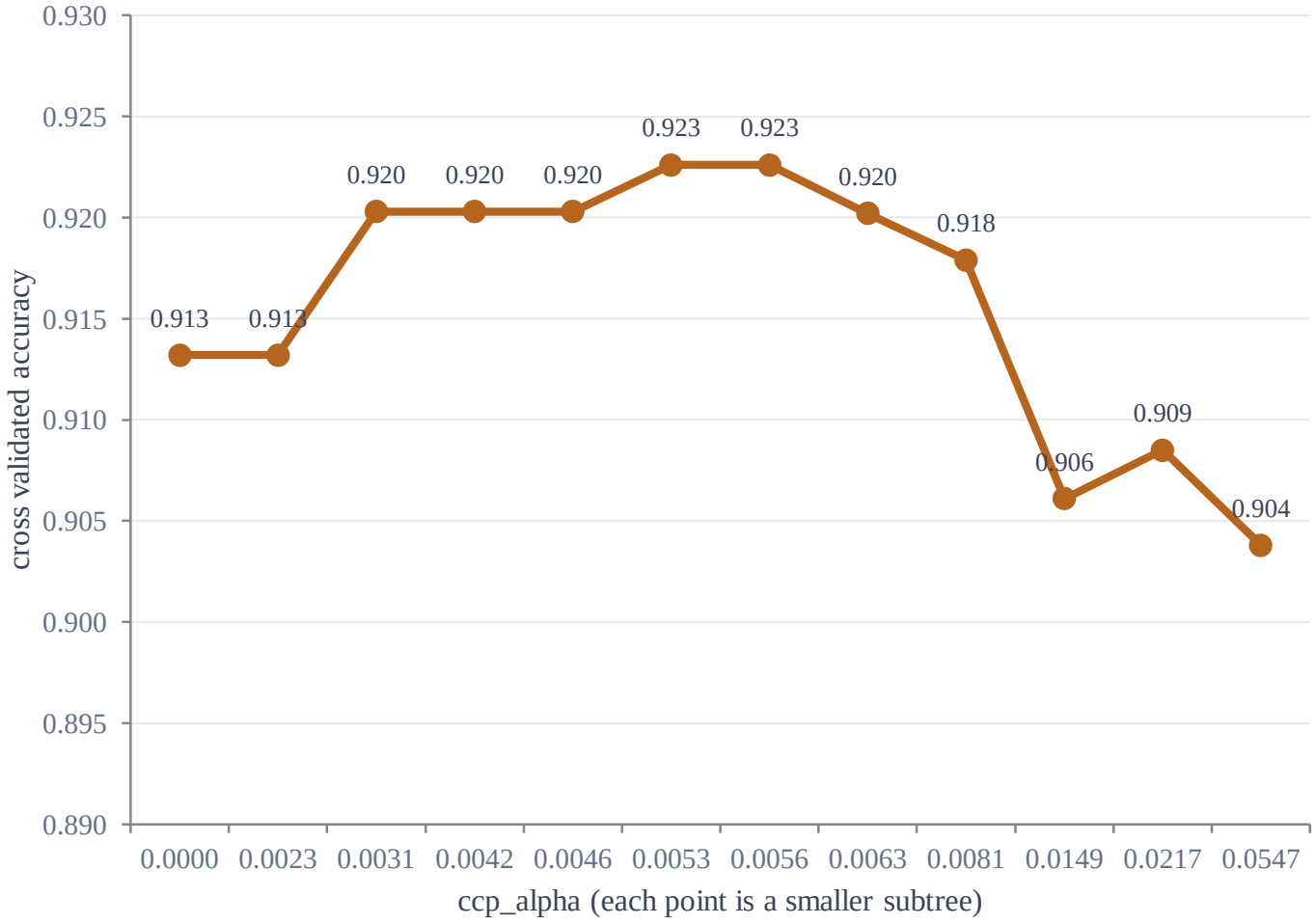
Best alpha: 0.00532 leaves: 8 cv: 0.9226

Alpha 0.0053, 8 leaves, 0.9226
The winner. Better than the full tree's 0.913 with fewer than half the leaves.

5 leaves: 0.9179
Nearly as good and simpler still. The one standard error rule might prefer it.

2 leaves: 0.9038
A single split. Heavy pruning starts to underfit.

Chart: pruning strength against validation accuracy



1. **Left: the full tree**
   Small alphas barely change the overfitted tree.

2. **The peak**
   Moderate pruning removes noisy branches and validation accuracy rises.

3. **Right: over pruning**
   Large alphas cut real structure and accuracy falls.

4. **The same U shape again**
   Pruning is simply another way of moving along the bias variance trade off.

Grid search: trying every combination

1 Hyperparameters interact

The best min_samples_leaf depends on max_depth. Tuning them one at a time can miss the best pair.

2 The grid

List candidate values for each hyperparameter. GridSearchCV tries every combination: 6 depths by 4 leaf sizes by 2 criteria gives 48.

3 Cross validated scoring

Each combination is evaluated with the folds you supply, so 48 candidates with 5 folds means 240 model fits.

4 Automatic refit

After the search, the best combination is refitted on the whole training set and stored as best_estimator_.

5 Full results table

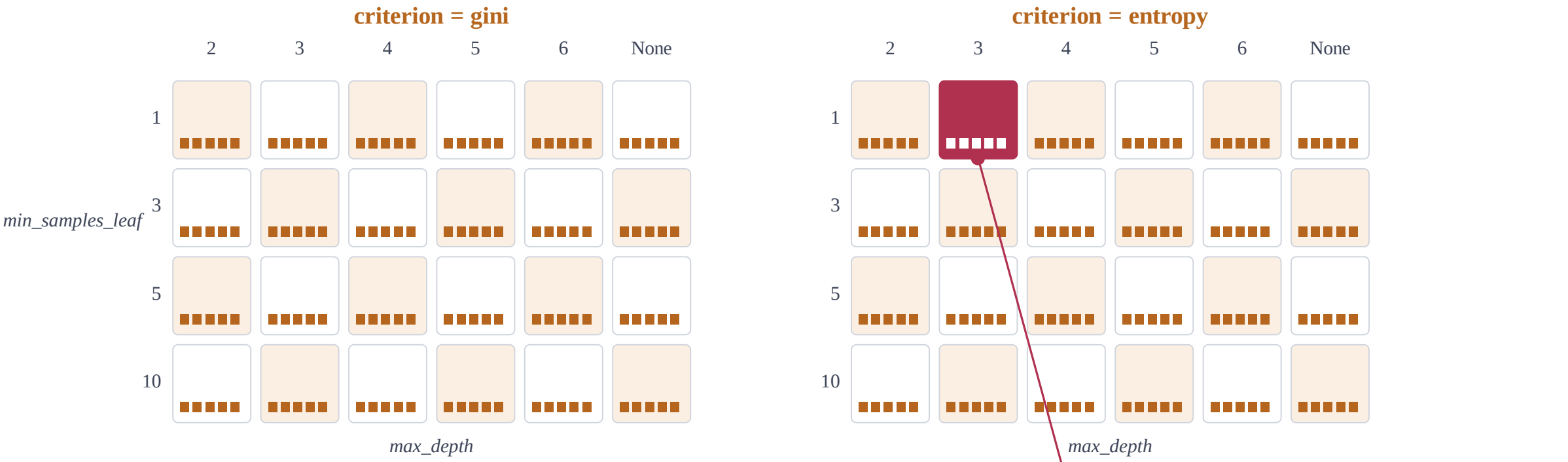
cv_results_ records the mean and spread of every candidate's scores, plus fit times, for inspection.

6 Cost grows fast

Every extra hyperparameter multiplies the number of fits. Large grids become expensive quickly.

Illustration: the 48 cell search grid

Every cell is one candidate tree, scored by five fold cross validation. The real best cell is highlighted.



Five small bars per cell
The five folds each candidate is scored on.

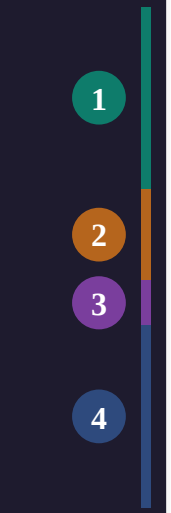
48 cells, 240 fits
Plus one refit of the winner.

Winner: entropy, depth 3, leaf 1
Mean accuracy 0.9273.

Running GridSearchCV

PYTHON

```python
from sklearn.model_selection import GridSearchCV
grid = {"max_depth": [2, 3, 4, 5, 6, None],
            "min_samples_leaf": [1, 3, 5, 10],
            "criterion": ["gini", "entropy"]}
search = GridSearchCV(DecisionTreeClassifier(random_state=0), grid,
                                  cv=cv, scoring="accuracy", return_train_score=True)
search.fit(X_train, y_train)
print("Candidates evaluated:", len(search.cv_results_["params"]))
print("Model fits performed:", len(search.cv_results_["params"]) * 5)
print("Best parameters:", search.best_params_)
print("Best cv accuracy:", round(search.best_score_, 4))
```



1 Define the grid

A dictionary mapping each hyperparameter name to its list of candidate values.

2 Configure the search

The base estimator, the grid, our fixed folds and the scoring metric. return_train_score records training accuracy too, for diagnosing overfitting.

3 Search

One call runs all 240 fits and then refits the best candidate on all training data.

4 Results

best_params_ and best_score_ summarise the winner.

Output: a depth 3 entropy tree wins

OUTPUT

```
Candidates evaluated: 48
Model fits performed: 240
Best parameters: {'criterion': 'entropy', 'max_depth': 3, 'min_samples_leaf': 1}
Best cv accuracy: 0.9273
```

1 **48 candidates**

Every combination of 6 depths, 4 leaf sizes and 2 criteria.

2 **240 fits**

48 times 5 folds. The final refit is extra.

3 **Best parameters**

Entropy, max_depth 3, min_samples_leaf 1. A small tree of at most 8 leaves.

4 **Mean cv accuracy 0.9273**

Up from 0.913 for the unrestricted tree.

Inspecting the top of the results table

PYTHON

```
r = pd.DataFrame(search.cv_results_)
cols = ["param_criterion", "param_max_depth", "param_min_samples_leaf",
        "mean_train_score", "mean_test_score", "rank_test_score"]
print(r[cols].sort_values("rank_test_score").head(6).round(4).to_string(index=False))
```

1

2

3

1 All results

cv_results_ is a dictionary of arrays; a DataFrame makes it sortable.

2 Useful columns

Parameter values, mean training score, mean validation score and rank.

3 Top six

Sorted by rank, with the index hidden for readability.

Output: many candidates are nearly tied

OUTPUT

param_criterion	param_max_depth	param_min_samples_leaf	mean_train_score	mean_test_score	rank_test_score	
entropy	3	1	0.9718	0.9273	1	1
gini	3	10	0.9595	0.9250	2	2
entropy	3	3	0.9712	0.9226	3	
gini	5	3	0.9842	0.9202	4	3
entropy	3	10	0.9601	0.9202	4	
entropy	3	5	0.9666	0.9202	4	

1 Rank 1: 0.9273

Training 0.9718: a modest gap of about 4.5 points.

2 Rank 2: gini, depth 3, leaf 10

0.9250, statistically indistinguishable and with larger, more reliable leaves.

3 Ties at rank 4

Three candidates share 0.9202. Depth 3 appears in five of the top six.

Grid search versus randomised search

GridSearchCV

- Tries every combination of listed values.
- Exhaustive and easy to reason about.
- Cost multiplies with each new hyperparameter.
- Wastes effort on unimportant hyperparameters, testing every value of them at every value of the important ones.
- Best for two or three hyperparameters with short lists.

vs

RandomizedSearchCV

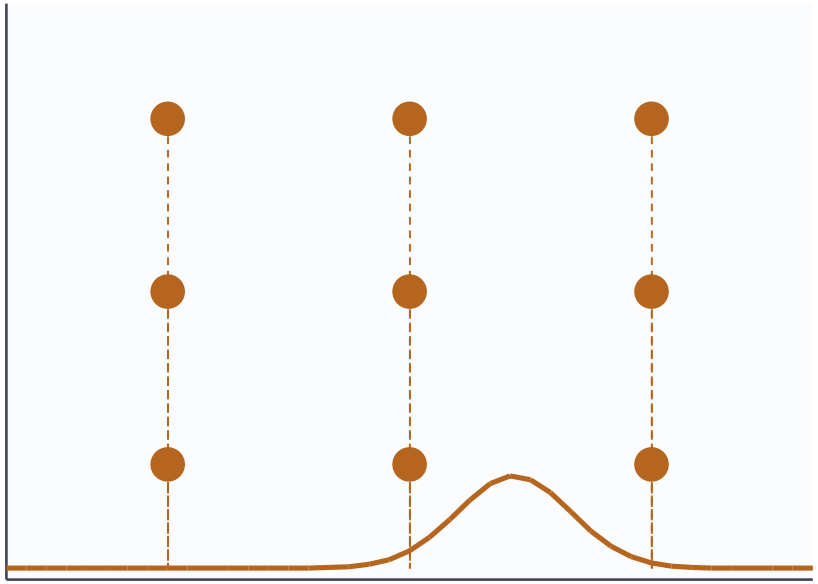
- Samples a fixed number of combinations from distributions you specify.
- Budget is set directly by n_iter, independent of how many hyperparameters there are.
- Explores many more distinct values of each hyperparameter for the same cost.
- Can search continuous ranges such as ccp_alpha and max_features.
- Best for four or more hyperparameters or expensive models.

Illustration: why random search explores better

Nine trials each. Suppose only the horizontal hyperparameter matters. The grid tests 3 distinct values of it; random search tests 9.

unimportant one

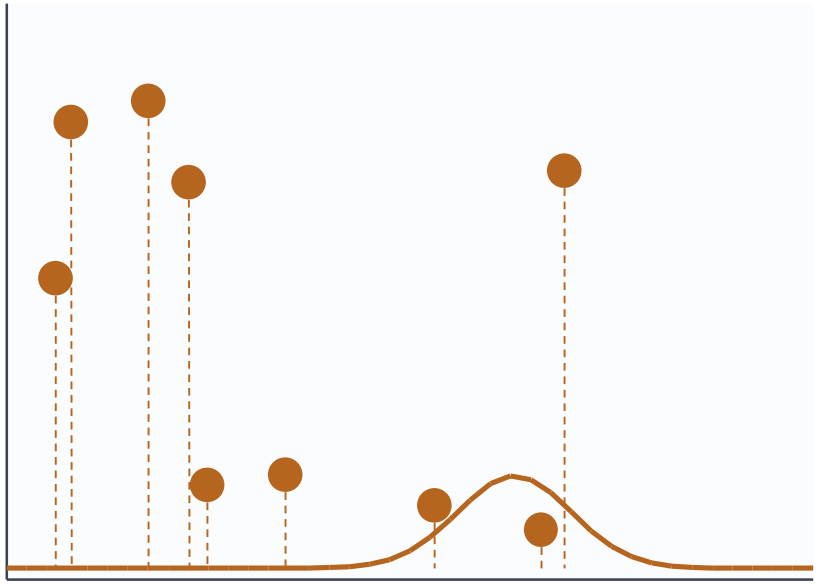
Grid search



important hyperparameter

unimportant one

Random search



important hyperparameter

orange curve: validation score along the important axis

Only 3 columns are tested, and the peak near 0.63 falls between them.

9 distinct values of the important axis: one lands close to the peak.

Running RandomizedSearchCV over four hyperparameters

PYTHON

```python
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
dist = {"max_depth": randint(2, 12), "min_samples_leaf": randint(1, 30),
        "max_features": uniform(0.3, 0.7), "ccp_alpha": uniform(0, 0.02)}
rand = RandomizedSearchCV(DecisionTreeClassifier(random_state=0), dist, n_iter=60,
                          cv=cv, random_state=0).fit(X_train, y_train)
print("Best cv accuracy:", round(rand.best_score_, 4))
for k, v in rand.best_params_.items():
    print(f"  {k:<17} {v:.4f}" if isinstance(v, float) else f"  {k:<17} {v}")
```

1

2

3

4

1 Distributions

randint draws whole numbers; uniform with loc and scale draws decimals, so max_features ranges from 0.3 to 1.0.

2 Search space

Depth 2 to 11, leaf size 1 to 29, feature share 0.3 to 1.0 and pruning alpha 0 to 0.02.

3 60 random candidates

n_iter fixes the budget at 60 candidates, 300 fits, regardless of how large the space is.

4 Report

Floats are printed to four decimals, integers as they are.

Output: randomness finds a better tree

OUTPUT

```
Best cv accuracy: 0.9367
    ccp_alpha          0.0096
    max_depth          11
    max_features       0.5226
    min_samples_leaf   4
```

0.9367 cross validated
Higher than the grid's 0.9273, using hyperparameters the grid never considered.

ccp_alpha 0.0096
Moderate pruning does the complexity control.

max_depth 11
Deep is allowed because pruning and leaf size already restrain the tree.

max_features 0.52
Each split considers a random half of the features, adding randomness that reduces variance: the core idea of random forests.

After the search: refit and evaluate once

1 Refit on all training data

With refit set to True, the default, the best settings are retrained on all 426 training rows, not just four folds.

2 best_estimator_ is ready

It behaves like any fitted model, with predict and predict_proba.

3 One test evaluation

Score it on the locked test set exactly once. This number is your estimate of real world performance.

4 Expect some drop

best_score_ is optimistic because it was the maximum over many candidates. Test scores are often slightly lower, though chance can push them either way.

5 Do not go back

If you tune again after seeing test results, the test set is no longer independent. Nested cross validation avoids this trap.

6 Report the full picture

Settings, cross validated score with spread, test score, confusion matrix and class level metrics.

Final evaluation of the grid search winner

PYTHON

```python
from sklearn.metrics import classification_report, confusion_matrix
final = search.best_estimator_
pred = final.predict(X_test)
print("Test accuracy:", round(final.score(X_test, y_test), 4))
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=bc.target_names, digits=3))
```

1

2

1 The refitted tree

best_estimator_ from the grid search: entropy, depth 3, trained on all training rows.

2 Three views of quality

Accuracy, the confusion matrix and the per class report, all on the 143 test patients.

Output: 95.1 percent on unseen patients

OUTPUT

```
Test accuracy: 0.951
[[48  5]
 [ 2 88]]
              precision    recall  f1 score   support

   malignant       0.960     0.906     0.932        53
      benign       0.946     0.978     0.962        90

    accuracy                          0.951       143
   macro avg       0.953     0.942     0.947       143
weighted avg       0.951     0.951     0.951       143
```

Test accuracy 0.951
Up from 0.923 for the unrestricted tree, with a far smaller model.

5 malignant missed
Row 1 is actual malignant: 48 caught, 5 predicted benign.

Malignant recall 0.906
The weakest number, and the most important clinically. A threshold or class_weight adjustment would be the next step.

Still behind logistic
Logistic regression reached 0.986 on the same split.

Learning curves: would more data help?

1 What they show

Training and validation scores as the training set grows, with model settings held fixed.

2 High variance signature

A large gap between training and validation that narrows as data grows. More data will help.

3 High bias signature

Both curves low and close together. More data will not help; a more flexible model will.

4 Converging curves

When validation stops rising, collecting more data of the same kind has little value.

5 Noisy on small data

Early points use few rows and few validation cases, so expect jumps.

6 learning_curve

scikit learn's function trains on increasing fractions of each training fold and returns both score matrices.

Computing a learning curve for the tuned tree

PYTHON

```
from sklearn.model_selection import learning_curve
sizes, tr, va = learning_curve(DecisionTreeClassifier(**search.best_params_, random_state=0),
                               X_train, y_train, cv=cv, train_sizes=[0.1, 0.25, 0.5, 0.75, 1.0])
for s, a, b in zip(sizes, tr.mean(axis=1), va.mean(axis=1)):
    print(f"train size {s:>3}: train acc {a:.3f}  validation acc {b:.3f}")
```

1

Import

learning_curve lives beside cross_val_score.

2

Five training sizes

From 10 percent to 100 percent of each training fold, using the grid search winner's settings via double star unpacking.

3

Average over folds

Mean training and validation accuracy at each size.

Output: the gap narrows as data grows

OUTPUT

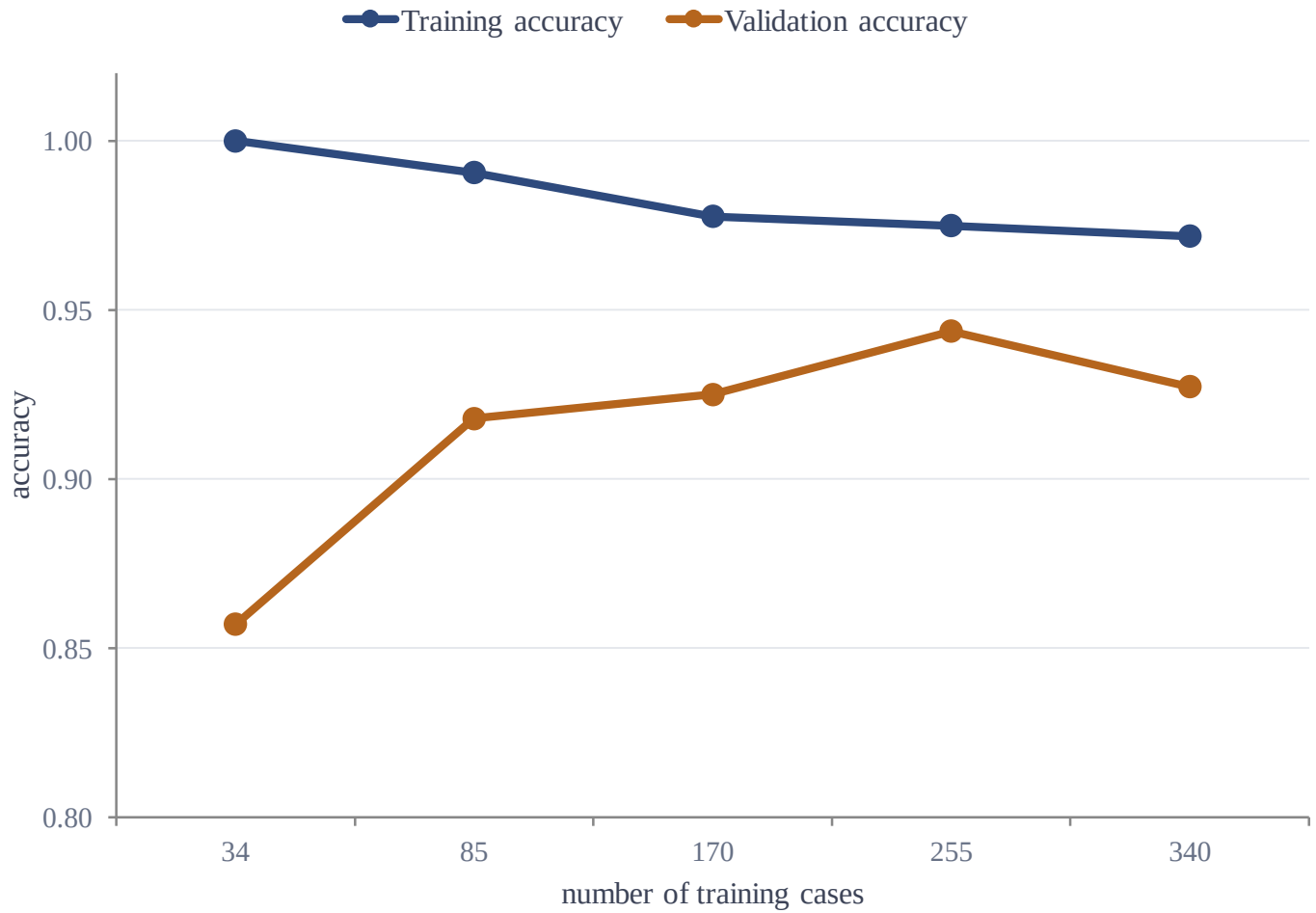
```
train size   34: train acc 1.000   validation acc 0.857
train size   85: train acc 0.991   validation acc 0.918
train size 170: train acc 0.978   validation acc 0.925
train size 255: train acc 0.975   validation acc 0.944
train size 340: train acc 0.972   validation acc 0.927
```

34 cases: gap of 14 points
Training 1.000 against validation 0.857: classic high variance.

170 cases: gap of 5 points
Validation rises to 0.925.

340 cases: gap of 4.5 points
Curves are converging; the dip from 0.944 to 0.927 is fold noise. More data would still help a little.

Chart: the learning curve of the tuned tree



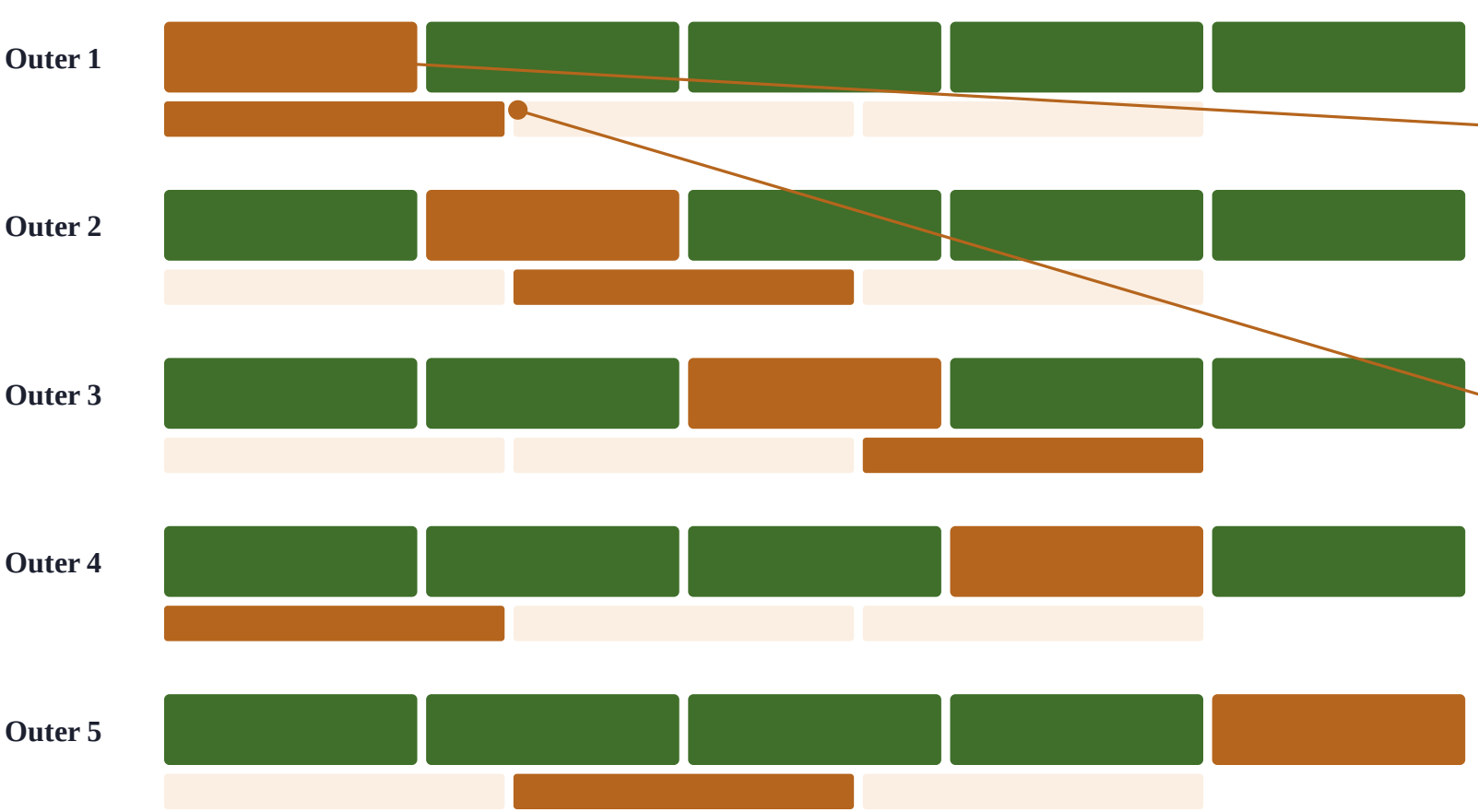
1. **Training falls slowly**
   With more cases the tree can no longer fit every one, which is healthy.

2. **Validation rises**
   More data means more reliable splits.

3. **The remaining gap**
   A few points remain between the curves: a moderate variance problem that an ensemble of trees would address.

Illustration: nested cross validation

An outer loop estimates performance; an inner loop does the tuning. The outer test fold never influences the choice of settings.



Orange: outer test fold

Scores the tuned model. Never seen during tuning.

Thin bars: inner folds

A full grid search with three folds runs on each outer training portion.

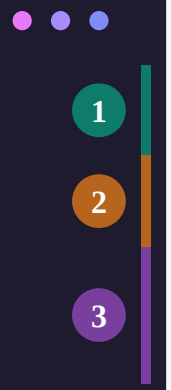
The result

Five outer scores estimate how well the whole tuning procedure generalises, not one fixed tree.

Running nested cross validation

PYTHON

```python
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=2)
tuner = GridSearchCV(DecisionTreeClassifier(random_state=0),
                     {"max_depth": [2, 3, 4, 5, 6], "min_samples_leaf": [1, 5, 10]}, cv=inner)
nested = cross_val_score(tuner, bc.data, bc.target, cv=outer)
print("Nested scores:", nested.round(4))
print("Honest estimate:", round(nested.mean(), 4), "sd", round(nested.std(), 4))
```



1 Two sets of folds

Five outer folds over all 569 patients, three inner folds for tuning.

2 A tuner as the model

A GridSearchCV object is itself an estimator, so it can be passed to cross_val_score.

3 Outer evaluation

Each outer fold runs a complete search on its training portion and scores the winner on its test portion.

Output: an honest estimate of tuned tree accuracy

OUTPUT

```
Nested scores: [0.9123 0.9474 0.9386 0.9386 0.9027]
Honest estimate: 0.9279 sd 0.0173
```

Five outer scores

From 0.903 to 0.947. Each comes from a different search on different data.

0.928, sd 0.017

The expected accuracy of the whole procedure: a grid search over depth limited trees. Close to the grid's 0.9273, so that score was not badly biased here.

Class weights for imbalanced data

PYTHON

```
from sklearn.datasets import make_classification
from sklearn.metrics import recall_score, balanced_accuracy_score
Xi, yi = make_classification(n_samples=3000, n_features=10, weights=[0.92], random_state=3)
a_tr, a_te, b_tr, b_te = train_test_split(Xi, yi, stratify=yi, random_state=3)
for cw in [None, "balanced"]:
    m = DecisionTreeClassifier(max_depth=5, class_weight=cw, random_state=0).fit(a_tr, b_tr)
    p = m.predict(a_te)
    print(f"class_weight={cw!s:<9} rare recall {recall_score(b_te, p):.3f}  balanced acc {balanced_accuracy_score(b_te, p):.3f}")
```

1

2

3

1 A rare class

3,000 synthetic cases with about 8 percent in class 1, split with stratification.

2 Two trees

Identical depth 5 trees, unweighted and balanced.

3 Metrics for rare classes

Recall of the rare class and balanced accuracy, which averages the recall of both classes.

Output: balanced weights catch more rare cases

OUTPUT

```
class_weight=None        rare recall 0.803    balanced acc 0.897
class_weight=balanced    rare recall 0.869    balanced acc 0.928
```

Unweighted: recall 0.803

Already decent because this synthetic problem is fairly separable.

Balanced: recall 0.869

Six more percentage points of rare cases caught, and balanced accuracy rises from 0.897 to 0.928. Weighting changes which splits look best, steering the tree toward the minority class.

Comparing tuned and untuned trees with logistic regression

PYTHON

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
logit = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
for name, m in [("Default tree", DecisionTreeClassifier(random_state=0)),
                ("Tuned tree", search.best_estimator_), ("Logistic", logit)]:
    s = cross_val_score(m, X_train, y_train, cv=cv)
    print(f"{name:<13} cv {s.mean():.4f}   test {m.fit(X_train, y_train).score(X_test, y_test):.4f}")
```

1

2

3

1 A strong baseline

The scaled logistic regression pipeline from section 3.

2 Three contenders

A default tree, the grid search winner and logistic regression.

3 Same folds, same test set

A fair comparison uses identical folds and the same untouched test rows.

Output: tuning helps, but logistic regression wins here

OUTPUT

Default tree	cv 0.9132	test 0.9371
Tuned tree	cv 0.9273	test 0.9510
Logistic	cv 0.9741	test 0.9860

Default tree
cv 0.913, test 0.937. A different random_state from section 4 gave a slightly different full tree.

Tuned tree
cv 0.927, test 0.951: a clear improvement from tuning alone.

Logistic regression
cv 0.974, test 0.986. Here the classes are separated well by a weighted sum of many features, which suits a linear model far better than axis aligned cuts.

When one tuned tree is not enough

1 Random forests

Train hundreds of deep trees on bootstrap samples, each split choosing from a random subset of features, and average them. Variance falls dramatically.

2 Gradient boosting

Add small trees one at a time, each correcting the errors of the ensemble so far. Often the most accurate method on tabular data.

3 Same hyperparameters

max_depth, min_samples_leaf and max_features reappear in both, so everything in this section transfers.

4 Trade off

Ensembles lose the single readable tree. Feature importance and partial dependence plots partly restore interpretability.

5 Why learn single trees

They are the building block, a useful explainable model in their own right and an excellent first model for exploring rules in data.

6 Know your data

As we saw, when the signal is a smooth combination of many features, a linear model can still beat every tree.

A practical recipe for tuning a decision tree

1

Lock the test set

Split once, stratified, and put the test rows away.

2

Explore one at a time

Validation curves for depth and leaf size show the scale of sensible values.

3

Search jointly

Grid search over a small grid, or random search over wider ranges and ccp_alpha.

4

Prefer simplicity

Among near ties, choose the smaller tree or larger leaves.

5

Evaluate once

Refit, then score the test set a single time; report cv and test results.

6

Estimate honestly

Use nested cross validation when the reported number matters.

Check your understanding

Your grid search reports a best cross validated accuracy of 0.93. You then try ten more grids, each time checking test accuracy, and keep the one with the best test score of 0.96. What is wrong?

A Nothing

B The test set has been used for tuning

C Grid search cannot exceed 0.93

D You should have used more folds

Answer B and why

Choosing between grids by test score turns the test set into a validation set. The 0.96 is now optimistically biased and no longer an honest estimate. Tune with cross validation only, evaluate the test set once, or use nested cross validation.

Check your understanding

Training accuracy is 1.00 and validation accuracy is 0.91 at every depth above 5. Which change is most likely to help?

A Increase max_depth

B Set min_samples_leaf to about 5 or 10

C Remove cross validation

D Lower the test size

Answer B and why

The large and constant gap is overfitting. Requiring larger leaves stops the tree from memorising individual cases, exactly as our leaf size curve peaked at 5 and 10 with a smaller, better generalising tree.

SECTION

6

Cluster Analysis

Discovering natural groups in data when there is no answer column at all

- Unsupervised learning and the meaning of similarity
- KMeans: the algorithm, inertia and its assumptions
- Choosing the number of clusters: elbow and silhouette
- Why scaling decides the result: customer segmentation
- Hierarchical clustering, linkage and dendrograms
- DBSCAN: density, noise and non convex shapes
- Validating, profiling and deploying clusters

What clustering is, and what it is not

Clustering groups cases so that members of a group resemble each other more than they resemble members of other groups.

1 No target column

There is no right answer to learn from. The algorithm only sees features and must find structure on its own.

2 Similarity must be defined

Every method depends on a notion of distance or density. Change the features or their scaling and the clusters change.

3 Many valid answers

The same data can be grouped sensibly in several ways. The best clustering is the one that is useful for your purpose.

4 Evaluation is harder

Without labels, we judge clusters by internal compactness and separation, by stability, and by whether they make business sense.

5 Not classification

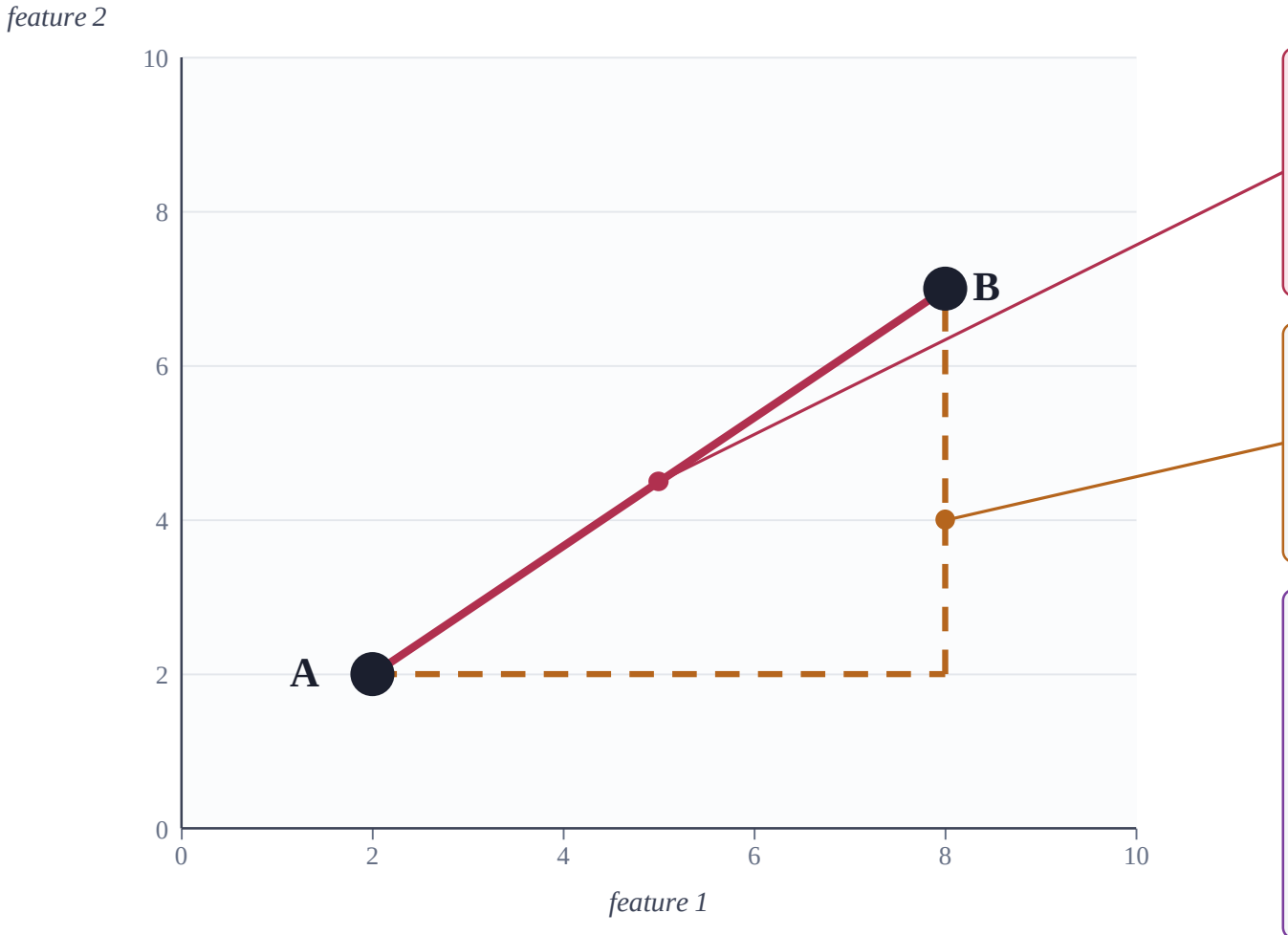
Cluster numbers are arbitrary names. Cluster 0 in one run may be cluster 2 in the next; only the grouping matters.

6 Often a first step

Segments feed targeted marketing, anomaly detection, data exploration, or become features for supervised models.

Illustration: distance is the heart of similarity

Two customers described by two features. How far apart they are depends on the distance measure and on the units.



Euclidean distance (solid)

The straight line between the points, about 7.8 units here. Used by KMeans, Ward linkage and DBSCAN by default.

Manhattan distance (dashed)

Travel along the axes only: 6 plus 5 gives 11. Less sensitive to a single large difference.

Units change everything

If feature 1 were income in pounds and feature 2 visits per month, a gap of 6,000 pounds would swamp a gap of 5 visits, and visits would be ignored. Scaling puts every feature on a comparable footing before distances are computed.

Four families of clustering algorithms

1 Centroid based: KMeans

Each cluster is represented by its centre. Points join the nearest centre. Fast and simple; prefers round clusters of similar size.

2 Hierarchical: agglomerative

Starts with every point alone and repeatedly merges the closest pair of groups, building a tree of merges called a dendrogram.

3 Density based: DBSCAN

Clusters are dense regions separated by sparse ones. Finds arbitrary shapes and labels isolated points as noise.

4 Model based: Gaussian mixtures

Assumes data come from a blend of bell shaped distributions and gives each point a probability of belonging to each one.

5 Covered in depth here

KMeans, agglomerative clustering with four linkage rules, and DBSCAN, each with real code and output.

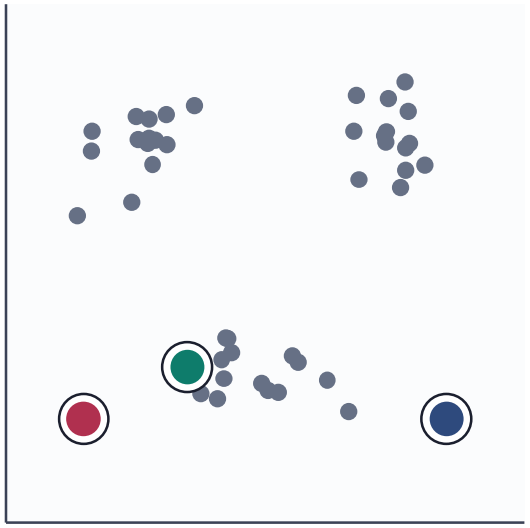
6 No universal winner

The right family depends on the expected shape of the groups, the amount of noise and whether you know how many groups to expect.

Illustration: KMeans in four moves

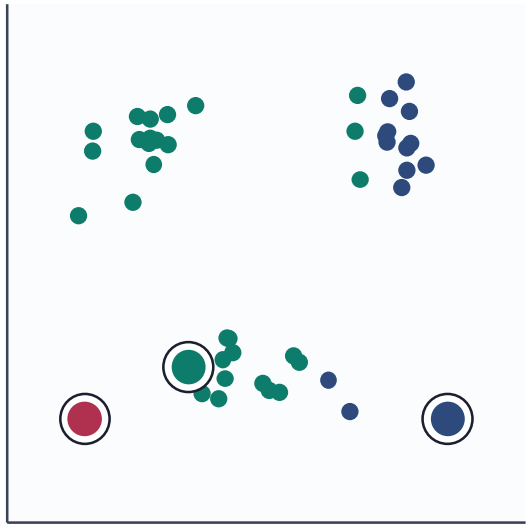
Illustrative data. KMeans alternates between assigning points to the nearest centre and moving each centre to the middle of its points.

1. Place k centres



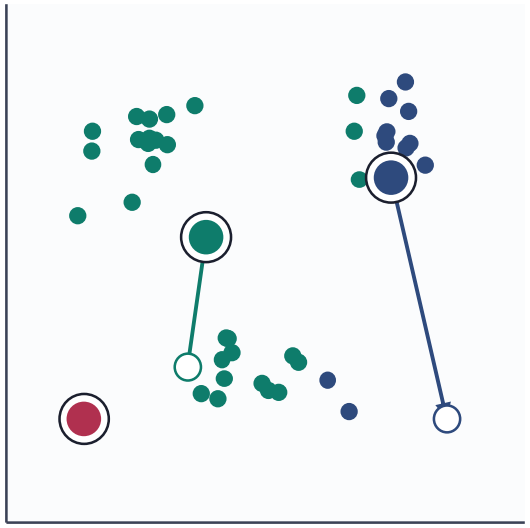
Start with k guesses for the centres, here deliberately poor ones.

2. Assign to nearest



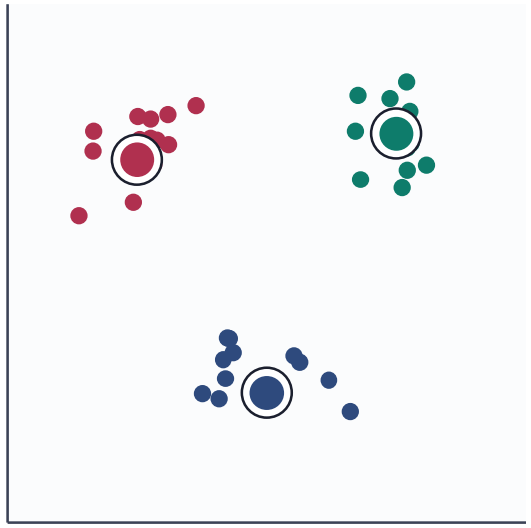
Each point takes the colour of its closest centre.

3. Move centres to means



Each centre jumps to the average position of its points. Arrows show the moves.

4. Repeat until stable



Assign and move again. When no point changes cluster, stop.

What KMeans optimises and what it assumes

1 Inertia

The sum, over all points, of the squared distance to their own cluster centre. KMeans tries to make this as small as possible.

2 Always decreases

Each assign step and each move step can only lower inertia, so the algorithm always converges, usually in a few iterations.

3 Local, not global

It converges to a local minimum that depends on the starting centres. Running several starts and keeping the best fixes most problems.

4 Round, similar sized clusters

Because it uses distance to a centre, KMeans carves space into convex regions and struggles with elongated, curved or very unequal groups.

5 You choose k

The number of clusters is a hyperparameter. Inertia always falls as k rises, so it cannot choose k by itself.

6 Sensitive to scale and outliers

Squared distances let large valued features and extreme points dominate. Scale features and consider removing outliers first.

Clustering 600 synthetic points with KMeans

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X, true_labels = make_blobs(n_samples=600, centers=4, cluster_std=0.9, random_state=11)
km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
print("Cluster sizes:", np.bincount(km.labels_))
print("Centroids:")
print(km.cluster_centers_.round(2))
print("Inertia:", round(km.inertia_, 1), "  Iterations:", km.n_iter_)
```

1 Imports

make_blobs generates labelled groups for practice; KMeans is in sklearn.cluster.

2 Four hidden groups

600 points around 4 centres with a spread of 0.9. We keep true_labels only to check the result later; KMeans never sees them.

3 Fit

n_clusters set to 4, n_init set to 10 runs with different starting centres, keeping the lowest inertia.

4 Learned attributes

labels_ gives each point's cluster, cluster_centers_ the centres, inertia_ the final score, n_iter_ the iterations needed.

Output: four clusters of about 150 points

OUTPUT

```
Cluster sizes: [149 150 150 151]
Centroids:
[[−1.7    −0.28]
 [−9.76 −0.21]
 [−6.42 −9.57]
 [−0.7     4.44]]
Inertia: 959.4          Iterations: 2
```

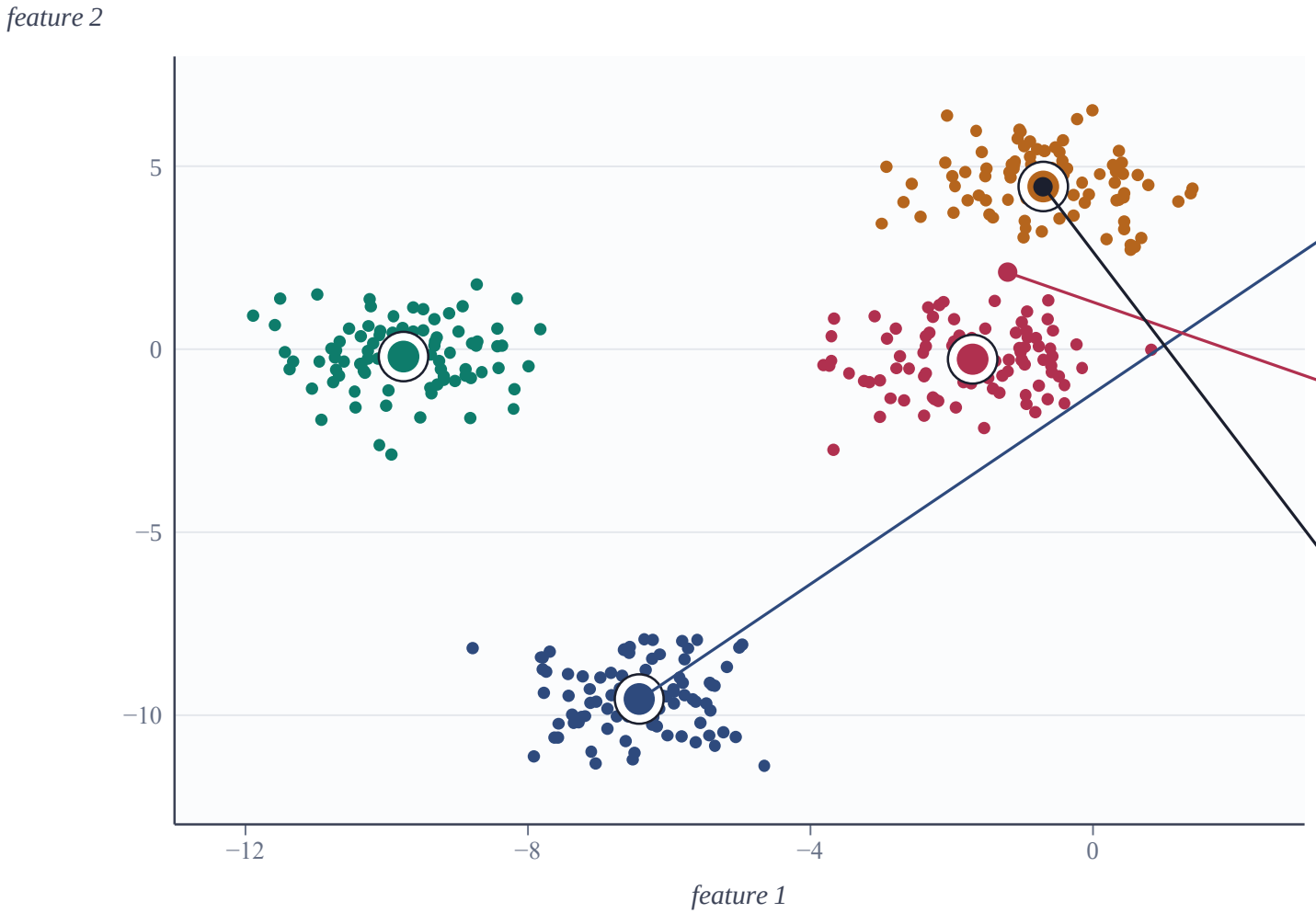
Sizes 149, 150, 150, 151
make_blobs created exactly 150 per group; one point drifted between two clusters.

Four centres
Each row is one centre's coordinates. These are the cluster summaries.

Inertia 959.4, 2 iterations
Well separated groups converge almost immediately.

Illustration: the four clusters KMeans found

Ninety points per cluster plotted from the real output, coloured by assigned cluster, with the learned centres as rings.



Isolated groups

Clusters 1 and 2 sit far from everything. Their points have high silhouette values.

Neighbouring groups

Clusters 0 and 3 are close together. Points between them are the least certain assignments.

Rings are centres

Each ring is a centre from cluster_centers_. Every point belongs to the ring it is closest to; boundaries are straight lines halfway between centres.

Choosing the number of clusters

1 Domain knowledge first

If marketing can only run three campaigns, three segments may be the right answer whatever a metric says.

2 The elbow method

Plot inertia against k. Inertia always falls, but the drop slows sharply once k passes the true number of groups. Look for the bend.

3 Silhouette score

Measures how much closer points are to their own cluster than to the next nearest one. Pick the k with the highest average.

4 Davies Bouldin index

Compares each cluster's spread with its distance to the most similar other cluster. Lower is better.

5 Stability

Cluster on random subsamples. A good k gives similar clusters every time; a bad one reshuffles.

6 Usefulness

Can each cluster be described in a sentence and acted upon? If not, more clusters are not more insight.

Computing inertia for k from 1 to 10

PYTHON

```python
for k in range(1, 11):
    model = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(f"k {k:>2}: inertia {model.inertia_:>9.1f}")
```

1 Ten models

One KMeans per value of k, each with ten random starts.

2 Record inertia

Lower inertia means points are closer to their centres.

Output: a sharp bend at k equals 4

OUTPUT

```
k  1: inertia  24570.5
k  2: inertia  10118.0
k  3: inertia   2709.0
k  4: inertia    959.4
k  5: inertia    864.2
k  6: inertia    774.0
k  7: inertia    713.5
k  8: inertia    618.4
k  9: inertia    575.1
k 10: inertia    512.5
```

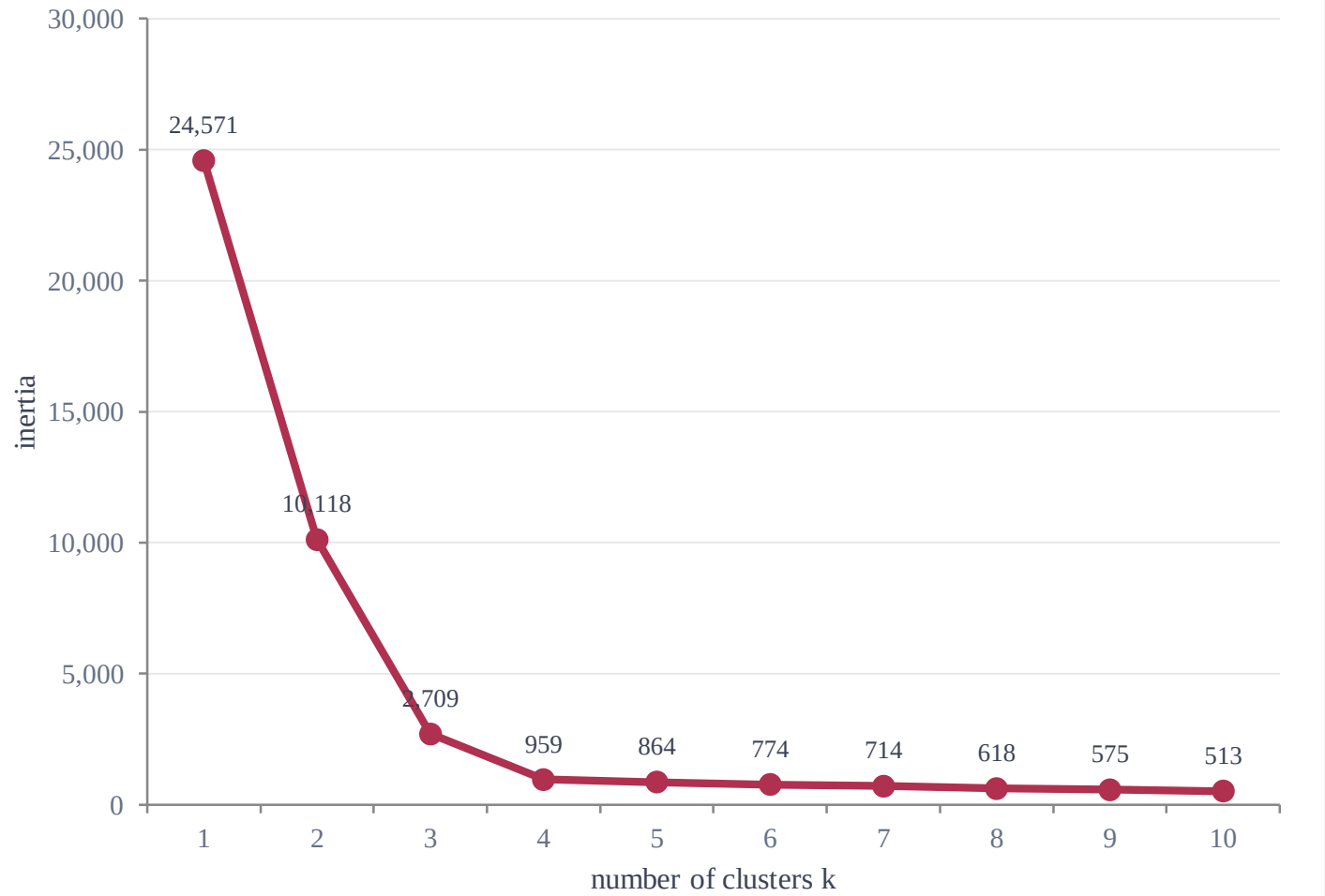
k 1: 24,571
One centre for everything: the total spread of the data.

k 3: 2,709
Still large: two true groups are being forced to share a centre.

k 4: 959
The last big drop. From here each extra cluster only splits a real group.

k 10: 513
Lower, but only because more centres always fit tighter. Not better clustering.

Chart: the elbow plot



1 Steep then flat
The curve falls steeply until k equals 4, then flattens into a gentle slope.

2 The elbow
The bend at 4 is unusually clear because the synthetic groups are well separated.

3 Real data is rarely this clean
Often the curve bends gradually and the elbow is ambiguous. Combine it with silhouette scores and judgement.

The silhouette score in plain language

1 Cohesion

For one point, compute its average distance to the other members of its own cluster. Small is good.

2 Separation

Compute its average distance to the members of the nearest other cluster. Large is good.

3 The point's silhouette

Separation minus cohesion, divided by whichever of the two is larger. The result lies between minus 1 and plus 1.

4 Reading the value

Near plus 1: well inside its cluster. Near 0: on the border between two clusters. Negative: probably in the wrong cluster.

5 The overall score

The average silhouette over all points summarises the whole clustering. Compare it across values of k.

6 Cost

It needs distances between all pairs of points, which is slow for very large datasets; use a random sample.

Silhouette and Davies Bouldin for k from 2 to 8

PYTHON

```python
from sklearn.metrics import silhouette_score, davies_bouldin_score
print(" k  silhouette  davies_bouldin")
for k in range(2, 9):
    lab = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)
    print(f"{k:>2}  {silhouette_score(X, lab):>10.3f}  {davies_bouldin_score(X, lab):>14.3f}")
```

1 **Two internal metrics**

Neither needs true labels; both use only the features and the cluster assignments.

2 **Fit and label**

fit_predict fits KMeans and returns the labels in one call.

3 **Score**

Silhouette starts at k equals 2 because it needs at least two clusters.

Output: both metrics agree on four clusters

OUTPUT

k	silhouette	davies_bouldin
2	0.572	0.779
3	0.732	0.375
4	0.743	0.368
5	0.648	0.661
6	0.572	0.856
7	0.399	1.080
8	0.328	1.177

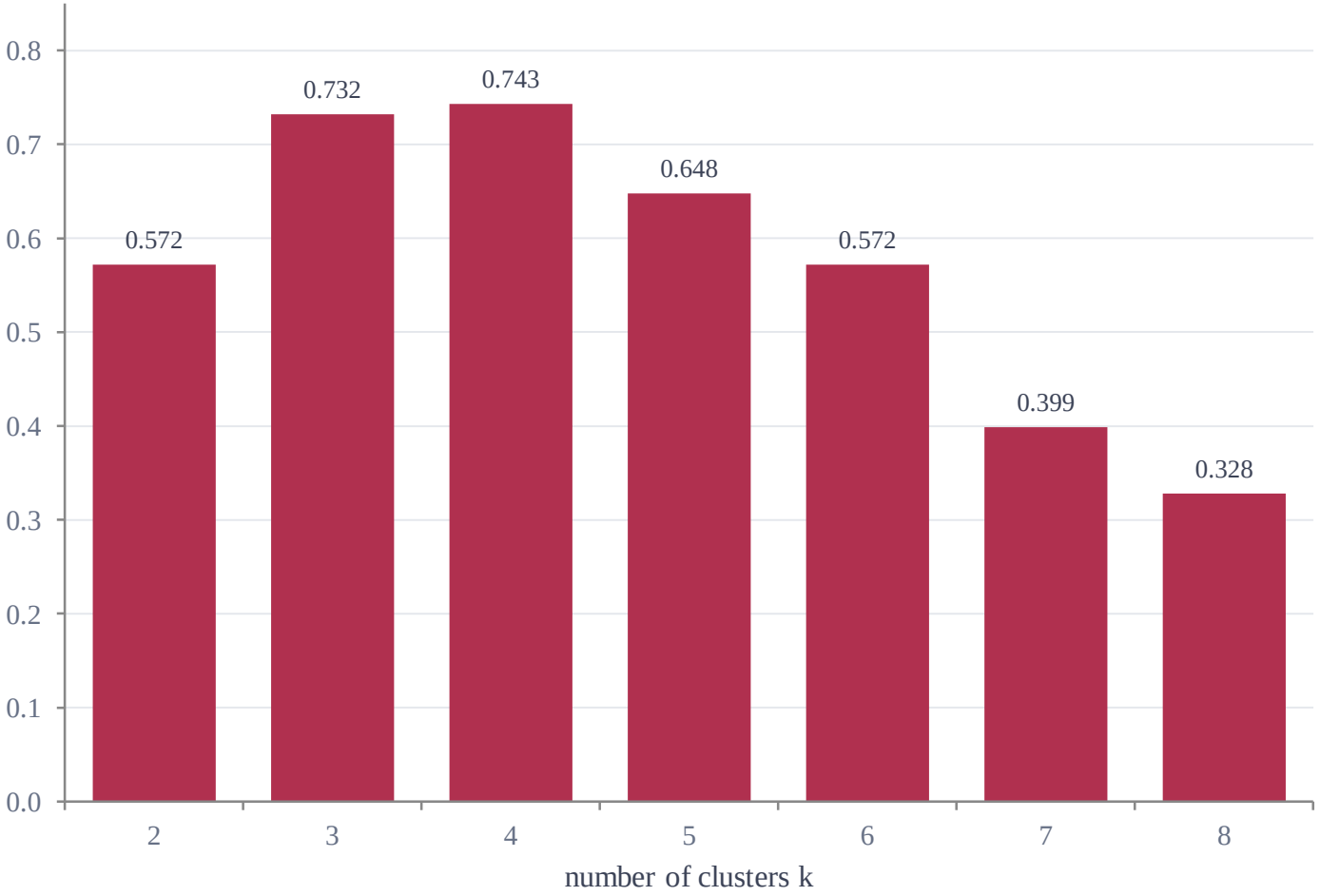
k 2: 0.572
Decent, because merging two pairs of groups still gives separated halves.

k 4: best on both
Highest silhouette 0.743 and lowest Davies Bouldin 0.368.

k 3: a close second
0.732, because clusters 0 and 3 are near each other and merging them costs little.

k 8: 0.328
Splitting real groups in half destroys separation.

Chart: average silhouette by number of clusters



1 **A clear peak**
k equals 4 has the highest bar, matching the elbow.

2 **Rule of thumb**
Above 0.7 suggests strong structure, 0.5 to 0.7 reasonable, below 0.25 weak or no real clusters.

3 **Peak, not just height**
Always compare across k. A high silhouette at k equals 2 alone can simply mean one dominant split.

Checking against the hidden truth and per cluster quality

PYTHON

```python
from sklearn.metrics import adjusted_rand_score, silhouette_samples
print("Agreement with hidden truth (ARI):", round(adjusted_rand_score(true_labels, km.labels_), 4))
sil = silhouette_samples(X, km.labels_)
for c in range(4):
    print(f"cluster {c}: mean silhouette {sil[km.labels_ == c].mean():.3f}  weakest point {sil[km.labels_ == c].min():.3f}")
```

1

2

1 Adjusted Rand index

Compares two groupings while ignoring label names, corrected so random labels score about 0 and identical groupings score 1. Only possible because we simulated the truth.

2 Per point silhouettes

silhouette_samples returns one value per point; averaging within each cluster shows which clusters are weakest.

Output: nearly perfect recovery of the true groups

OUTPUT

```
Agreement with hidden truth (ARI): 0.9867
cluster 0: mean silhouette 0.654          weakest point 0.075
cluster 1: mean silhouette 0.805          weakest point 0.622
cluster 2: mean silhouette 0.844          weakest point 0.642
cluster 3: mean silhouette 0.668          weakest point 0.011
```

ARI 0.987

KMeans recovered the hidden groups almost exactly, with only a few border points swapped.

Cluster 1: 0.805

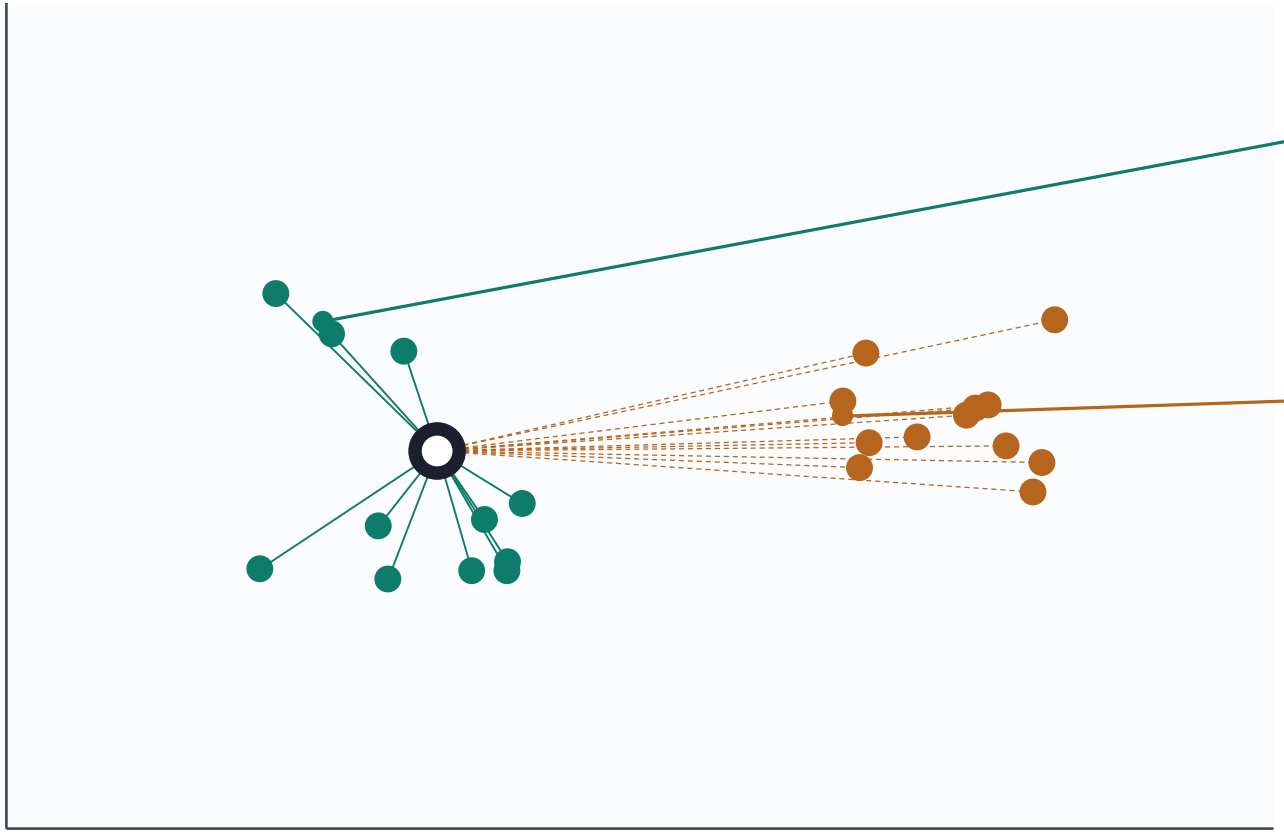
Isolated and compact: every point is confidently placed, the weakest at 0.622.

Cluster 3: weakest point 0.011

A point almost exactly on the border with cluster 0. Clusters 0 and 3 have the lowest averages because they are neighbours.

Illustration: cohesion and separation for one point

The silhouette of the starred point compares its typical distance to its own cluster with its typical distance to the nearestother cluster.



Solid green lines: cohesion

Average distance to its own cluster. Short lines mean a tight fit.

Dashed orange lines: separation

Average distance to the nearest other cluster. Long lines mean clear separation.

Silhouette near 0.7

Separation is much larger than cohesion, so this point sits comfortably inside the green cluster.

A realistic problem: customer segmentation

We now simulate 600 shoppers from three behavioural segments and see whether clustering can rediscover them.

1 Three hidden segments

Loyal: about 12 visits a month, last seen 4 days ago. Occasional: 5 visits, 21 days. Lapsing: 1.5 visits, 45 days.

2 Income adds little

Segments differ only slightly in income (46,000, 40,000 and 37,000 pounds) but income varies hugely within each segment, by about 12,000 pounds.

3 The trap

Income is measured in thousands of pounds while visits are single digits. Unscaled distances will be almost entirely income.

4 The question

Will KMeans find the behavioural segments, or will it split customers by income, which is mostly noise?

5 Business value

Correct segments let a retailer reward the loyal, nudge the occasional and win back the lapsing, each with a different offer.

6 What to watch

The adjusted Rand index with and without scaling, then the cluster profiles.

Simulating three customer segments

PYTHON

```python
rng = np.random.default_rng(5)
seg = {"loyal": (46000, 12, 4), "occasional": (40000, 5, 21), "lapsing": (37000, 1.5, 45)}
parts = []
for name, (income, visits, recency) in seg.items():
    k = 200
    parts.append(pd.DataFrame({"income_gbp": rng.normal(income, 12000, k).round(0),
        "visits_per_month": rng.normal(visits, 1.5, k).clip(0),
        "days_since_visit": rng.normal(recency, 4, k).clip(0), "truth": name}))
cust = pd.concat(parts, ignore_index=True)
feats = ["income_gbp", "visits_per_month", "days_since_visit"]
print(cust[feats].describe().loc[["mean", "std", "min", "max"]].round(1))
```

1 **Segment recipes**

Each segment has a typical income, visit rate and recency.

2 **200 customers each**

Normal noise around each recipe. clip stops visits and days from going below zero.

3 **Combine and summarise**

concat stacks the three segments; we keep the truth column only for checking.

Output: wildly different scales

OUTPUT

	income_gbp	visits_per_month	days_since_visit
mean	40994.0	6.2	23.8
std	12101.1	4.5	17.2
min	3085.0	0.0	0.0
max	78311.0	16.2	56.5

Means
Income averages about 41,000; visits about 6; days since visit about 24.

Spreads
Income spreads by 12,101 pounds, visits by just 4.5. To an unscaled distance, income is thousands of times larger.

Maxima
Income up to 78,311 against at most 16 visits.

KMeans with and without scaling

PYTHON

```python
from sklearn.preprocessing import StandardScaler
raw_lab = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(cust[feats])
Z = StandardScaler().fit_transform(cust[feats])
sc_lab = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Z)
print("ARI without scaling:", round(adjusted_rand_score(cust.truth, raw_lab), 4))
print("ARI with scaling:   ", round(adjusted_rand_score(cust.truth, sc_lab), 4))
```

1

2

3

1 Unscaled

KMeans directly on raw income, visits and days.

2 Scaled

StandardScaler gives every feature mean 0 and spread 1 first.

3 Compare with truth

Adjusted Rand index against the hidden segment labels.

Output: scaling turns failure into success

```
OUTPUT
ARI without scaling: 0.0135
ARI with scaling:            0.98
```

Without scaling: 0.0135

Essentially random agreement. KMeans split customers into low, middle and high income bands, ignoring behaviour entirely.

With scaling: 0.98

Almost perfect recovery of the three behavioural segments. Same algorithm, same data, one preprocessing step.

Illustration: the same customers clustered two ways

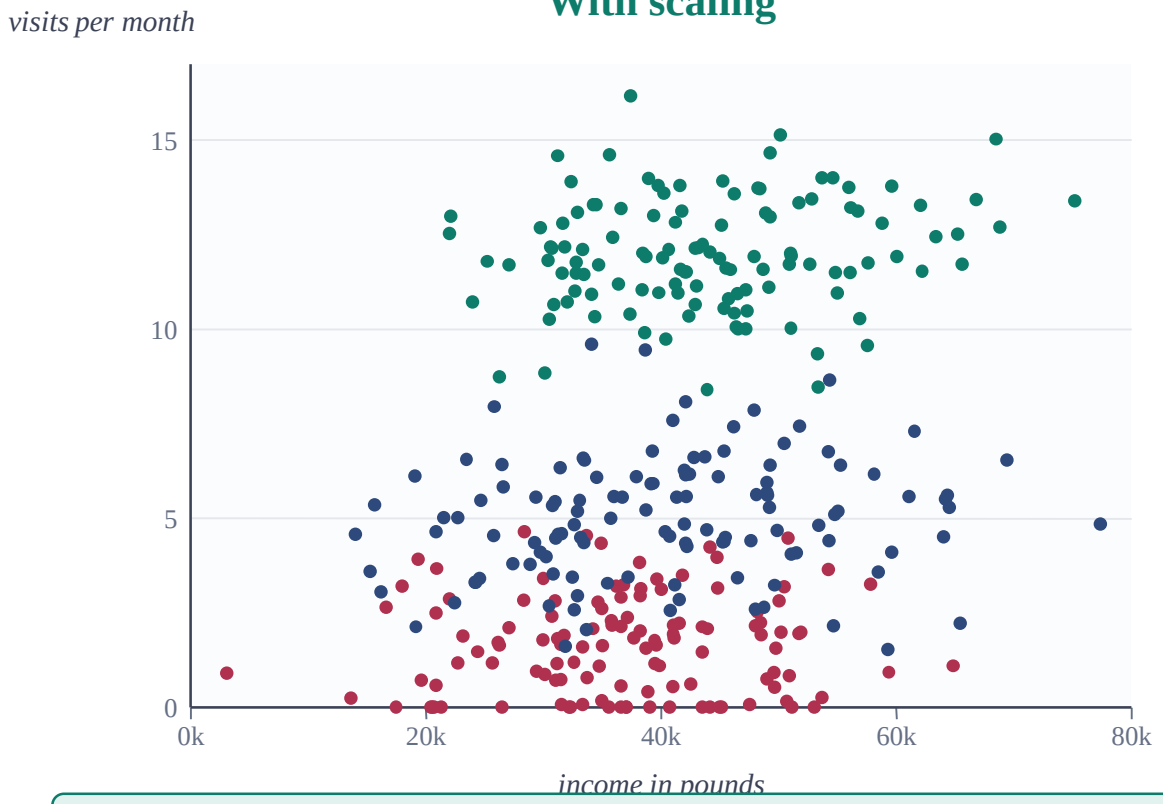
Income against visits per month for 360 customers, coloured by the cluster KMeans assigned.

Without scaling



Vertical stripes: clusters are income bands. Behaviour is ignored.

With scaling



Horizontal layers: clusters follow visit frequency, the real segments.

Profiling the scaled clusters

PYTHON

```python
cust["cluster"] = sc_lab
profile = cust.groupby("cluster")[feats].mean().round(1)
profile["size"] = cust.cluster.value_counts().sort_index()
print(profile)
print(pd.crosstab(cust.cluster, cust.truth))
```

1. **Attach labels**
   Store each customer's cluster as a new column.
2. **Cluster profiles**
   Mean of each feature per cluster, in original units, plus the cluster size. This is how clusters get explained.
3. **Cross tabulation**
   Counts of true segment by cluster. Possible only because the data are simulated.

Output: three clean, interpretable segments

OUTPUT

	income_gbp	visits_per_month	days_since_visit	size
cluster				
0	36974.9	1.6	45.6	199
1	45411.2	12.0	4.4	199
2	40601.7	5.1	21.5	202

truth	lapsing	loyal	occasional
cluster			
0	199	0	0
1	0	198	1
2	1	2	199

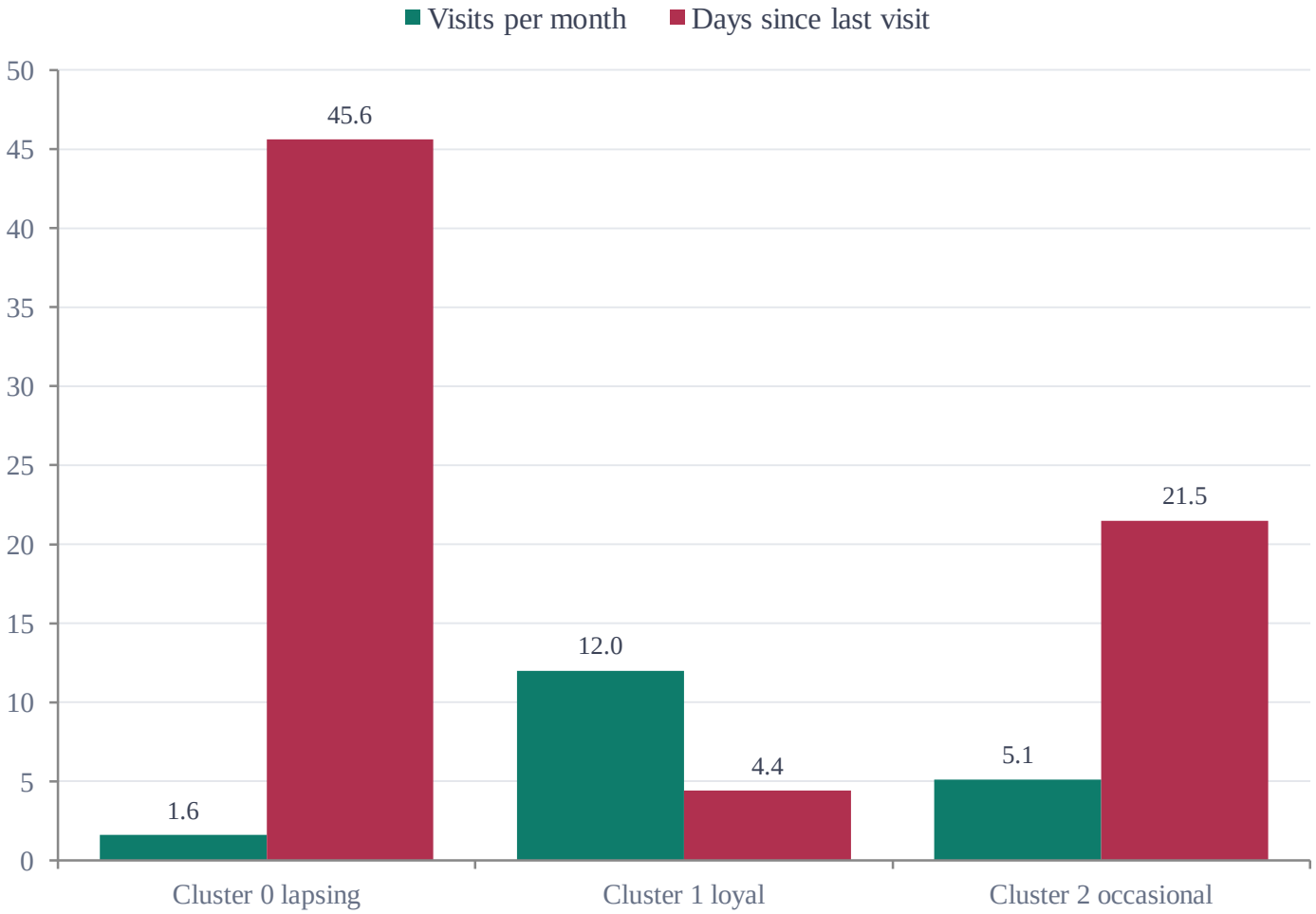
Cluster 0: lapsing
1.6 visits a month, last seen 46 days ago, slightly lower income.

Cluster 1: loyal
12 visits a month, seen 4 days ago. The most valuable customers.

Cluster 2: occasional
About 5 visits a month, three weeks since the last visit.

Only 4 customers misplaced
The diagonal like pattern of the cross tabulation confirms the near perfect ARI.

Chart: behaviour profile of each cluster



1. **Mirror images**
   Visits and recency move in opposite directions across clusters, which is exactly what defines the segments.

2. **Income omitted**
   Income means of 37,000, 45,000 and 41,000 pounds differ much less relative to their spread, so they barely define the clusters.

3. **A profile chart is the deliverable**
   Stakeholders rarely want centroids; they want a picture of how each group behaves.

Turning clusters into action

1 Name each cluster

Give each cluster a short, memorable name from its profile: Loyal, Occasional, Lapsing. Numbers are arbitrary and change between runs.

2 Describe with original units

Report means or medians in pounds, visits and days, never in standardised units.

3 Check sizes

A cluster holding 1 percent of customers may be an outlier group, interesting but rarely a campaign.

4 Validate externally

Do the clusters differ on outcomes they were not built from, such as spend or churn? That is strong evidence they are real.

5 Check stability

Re run with different seeds or subsamples. Segments that dissolve are not a sound basis for decisions.

6 Plan for drift

Behaviour changes over time. Re cluster periodically and track how customers move between segments.

Initialisation: why n_init and smart seeding matter

1 Starting points matter

KMeans only finds a local minimum. Bad starting centres can leave two centres inside one true group and none in another.

2 Smart seeding, the default

scikit learn's default initialisation picks the first centre at random, then favours points far from the centres already chosen.

3 Random seeding

init set to random picks k random points as centres, which more often starts badly.

4 Several runs

n_init repeats the whole algorithm with different starts and keeps the run with the lowest inertia.

5 Fix random_state

Makes the chosen starting points, and therefore the result, reproducible.

6 Bigger k, bigger risk

With many clusters, poor local minima become more common, so the benefit of smart seeding and more starts grows.

Comparing seeding strategies over 20 single runs

PYTHON

```python
for label, extra in [("smart seeding", {}), ("random seeding", {"init": "random"})]:
    runs = [KMeans(n_clusters=8, n_init=1, random_state=s, **extra).fit(X).inertia_
            for s in range(20)]
    print(f"{label:<15} best {min(runs):8.1f}  worst {max(runs):8.1f}  spread {np.ptp(runs):7.1f}")
```

1 2 3

1 Two strategies

An empty settings dictionary keeps the default smart seeding; the second sets init to random.

2 Single runs

n_init set to 1 exposes each start's luck. k equals 8 on the 4 group data makes local minima likely.

3 Best, worst, spread

np.ptp returns the range, the maximum minus the minimum.

Output: smart seeding finds better solutions

OUTPUT

smart seeding	best	624.2	worst	720.9	spread	96.7
random seeding	best	641.5	worst	733.8	spread	92.3

Smart: best 624.2
Lower inertia than any random start achieved, and a better worst case of 720.9.

Random: best 641.5
Even the luckiest of 20 random starts is worse than smart seeding's best.

Both vary
Spreads near 95 show any single run is a gamble. Use n_init of 10 or more in practice.

Hierarchical clustering: building a tree of merges

1 Bottom up

Agglomerative clustering starts with each point as its own cluster and repeatedly merges the two closest clusters until one remains.

2 Linkage defines closest

The distance between two groups can be measured several ways; this choice, the linkage, shapes the result.

3 The dendrogram

The sequence of merges forms a tree. The height of each join is the distance at which the merge happened.

4 Cut to get clusters

Draw a horizontal line across the dendrogram: the branches it crosses are the clusters. A higher cut gives fewer clusters.

5 No k needed up front

You can inspect the whole hierarchy and choose where to cut afterwards.

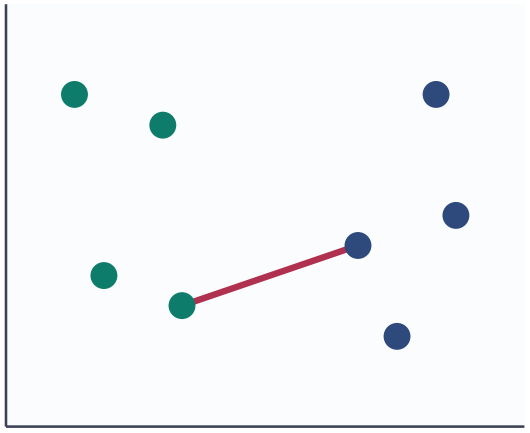
6 Cost

Needs all pairwise distances, so memory and time grow with the square of the number of points: fine for thousands, not millions.

Illustration: four linkage rules

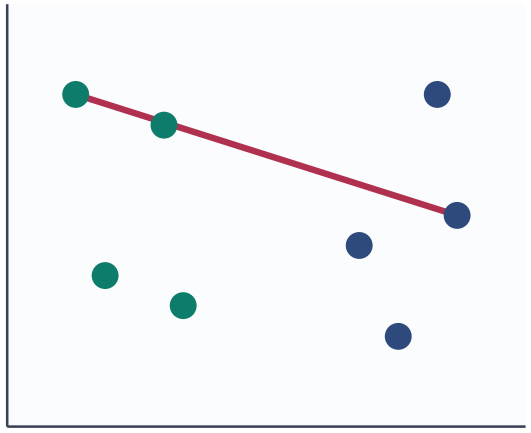
Two groups, one question: how far apart are they? Each linkage answers differently.

Single



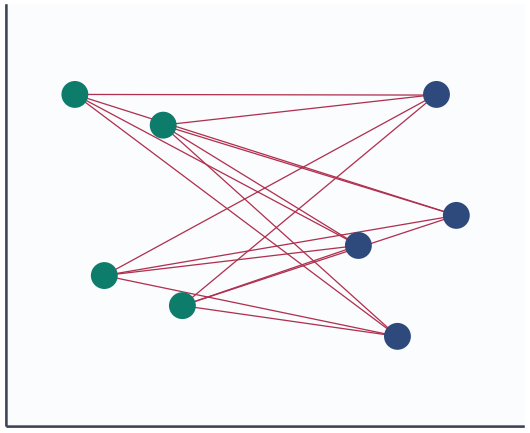
Closest pair of points. Can chain thin bridges of points into one long cluster.

Complete



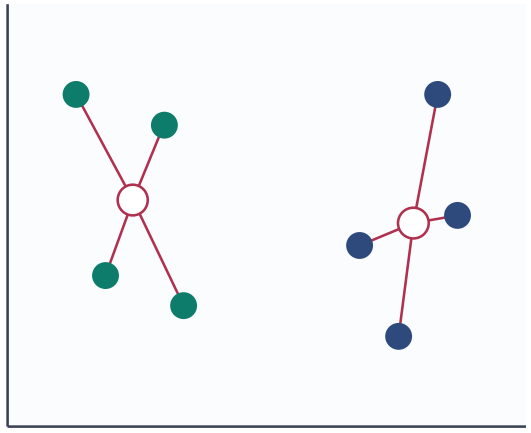
Farthest pair of points. Prefers compact clusters of similar diameter.

Average



Average over all pairs. A compromise between single and complete.

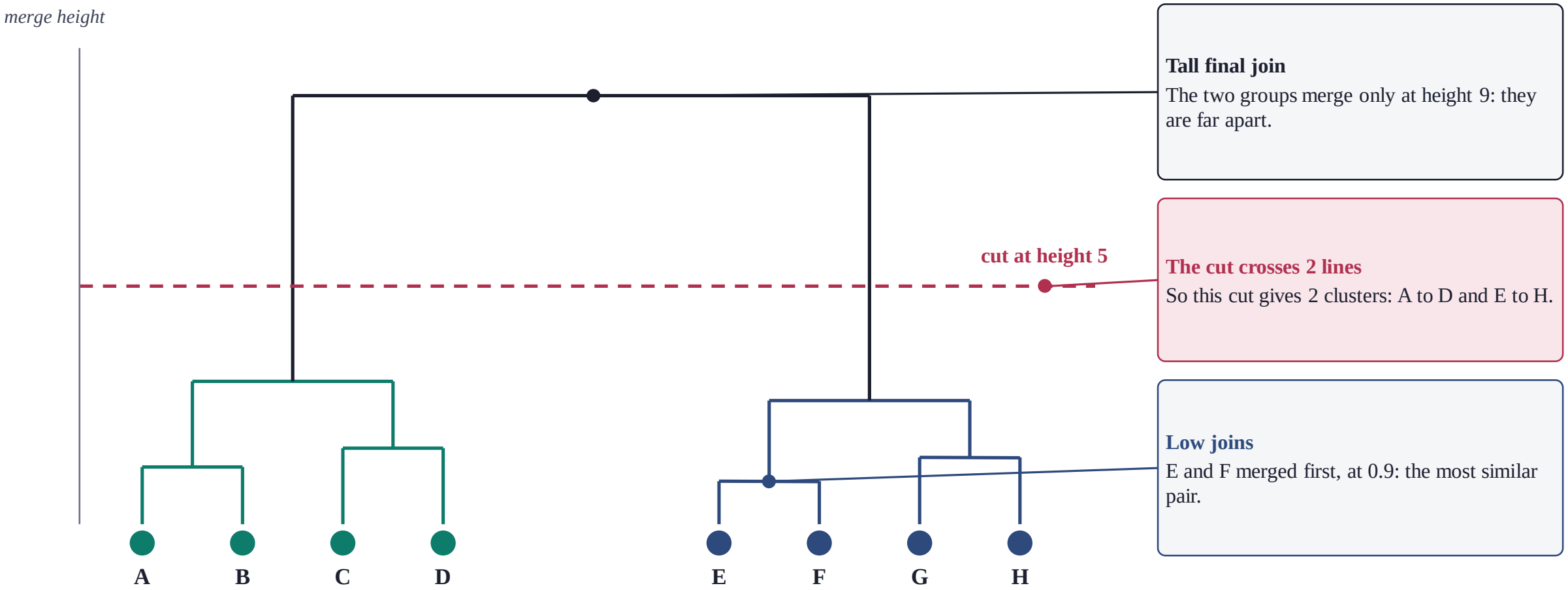
Ward



Merge that increases total within cluster variance least. The default; behaves like KMeans.

Illustration: reading a dendrogram

Illustrative merges of eight points. Joins higher up happened at larger distances; the dashed cut line decides the clusters.



Comparing linkages on blobs and on moons

PYTHON

```python
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_moons
Xm, ym = make_moons(n_samples=400, noise=0.06, random_state=0)
print("linkage    blobs ARI   moons ARI")
for link in ["ward", "complete", "average", "single"]:
        a = AgglomerativeClustering(n_clusters=4, linkage=link).fit_predict(X)
        b = AgglomerativeClustering(n_clusters=2, linkage=link).fit_predict(Xm)
        print(f"{link:<9} {adjusted_rand_score(true_labels, a):>9.3f} {adjusted_rand_score(ym, b):>11.3f}")
```

1

2

3

1 Two datasets

The four blobs from before and two interlocking half moons, a shape that defeats centroid methods.

2 Four linkages

ward, complete, average and single.

3 Score both

Adjusted Rand index against the known groups for each dataset.

Output: no linkage wins everywhere

OUTPUT

linkage	blobs ARI	moons ARI
ward	0.987	0.239
complete	0.982	0.285
average	0.991	0.329
single	0.713	1.000

Ward: great on blobs, poor on moons

0.987 on round groups, only 0.239 on the curved shapes.

Average: best on blobs

0.991, but still poor on moons at 0.329.

Single: perfect on moons

1.000 on moons, because it follows chains of nearby points along each curve, yet only 0.713 on blobs.

Building the merge table with scipy

PYTHON

```python
from scipy.cluster.hierarchy import linkage, fcluster
Zl = linkage(Z, method="ward")
print("Shape of merge table:", Zl.shape)
print("Five tallest merge heights:", np.flip(Zl[:, 2])[:5].round(2))
for h in [10, 20, 40]:
    groups = fcluster(Zl, t=h, criterion="distance")
    print(f"cut at height {h:>2}: {len(np.unique(groups))} clusters")
```

1

2

3

1 Ward linkage on customers

scipy's linkage returns the full merge history of the scaled customer data.

2 The merge table

One row per merge: the two clusters joined, the merge height and the new cluster's size.

3 Cut at three heights

fcluster with criterion distance cuts the tree at a chosen height.

Output: a large gap points to three clusters

OUTPUT

```
Shape of merge table: (599, 4)
Five tallest merge heights: [42.21 21.85 18.57 14.78 13.09]
cut at height 10: 6 clusters
cut at height 20: 3 clusters
cut at height 40: 2 clusters
```

599 merges

600 customers need 599 merges to become one cluster.

Tallest joins: 46.7, 30.4, then 8.3

A huge jump between the second and third tallest merges. Cutting anywhere in that gap gives three clusters.

Heights 10 and 20: 3 clusters

Stable across a wide range of cuts: strong evidence for three segments.

Height 40: 2 clusters

Above 30.4 two segments have merged.

DBSCAN: clusters as dense regions

1 Two settings

eps is a neighbourhood radius; min_samples is how many points, including itself, must fall inside it.

2 Core points

A point with at least min_samples neighbours within eps. Cores are the heart of clusters.

3 Border points

Not core themselves but within eps of a core point. They join that core's cluster.

4 Noise points

Neither core nor near a core. Labelled minus 1 and assigned to no cluster at all.

5 Clusters grow by reachability

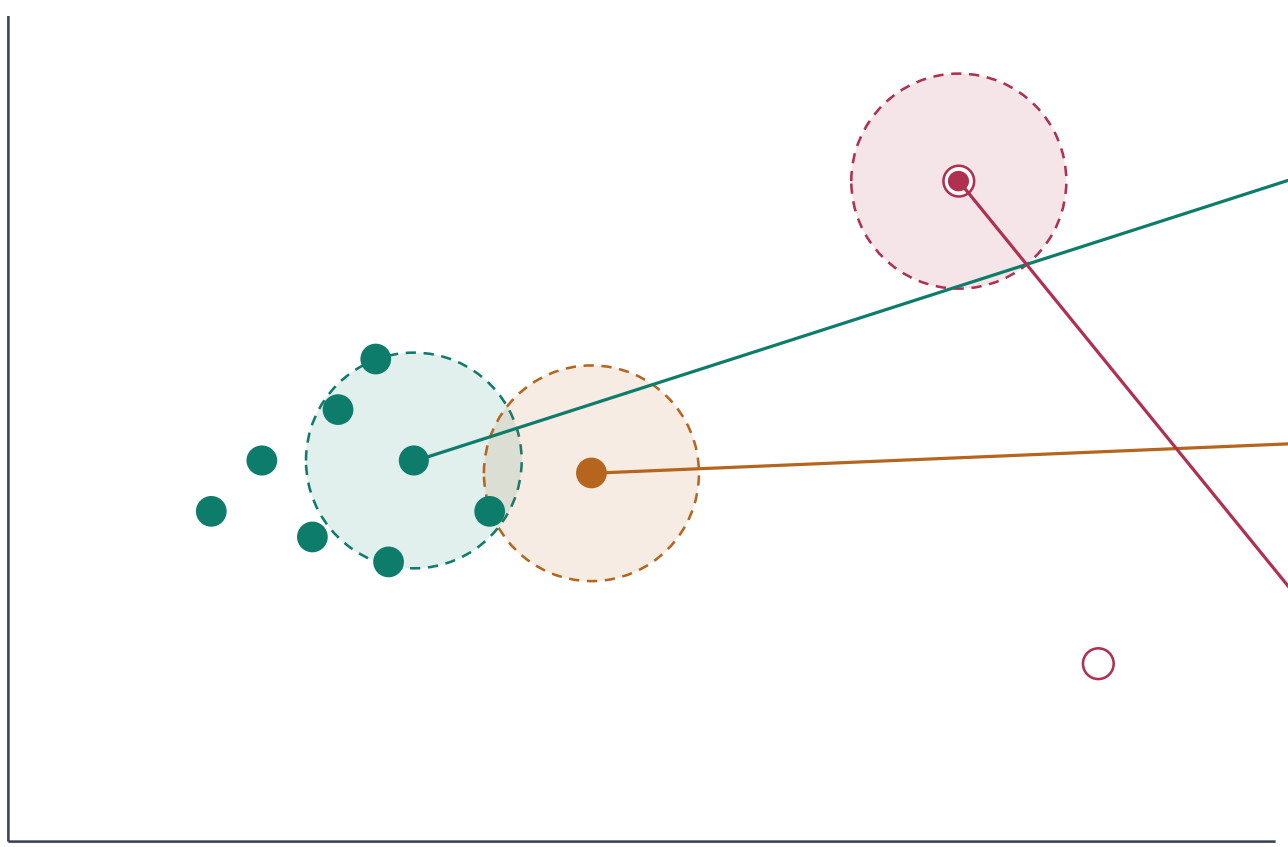
Cores within eps of each other link up, so clusters can snake into any shape.

6 No k required

The number of clusters emerges from the density. The hard part is choosing eps.

Illustration: core, border and noise points

Illustrative points with min_samples set to 4. Circles show the eps radius around three example points.



Core point (green)
Its circle holds at least 4 points, so it anchors a cluster.

Border point (orange)
Too few neighbours to be core, but inside a core's circle, so it joins the cluster.

Noise (hollow red)
Isolated. DBSCAN labels it minus 1 instead of forcing it into a cluster.

KMeans versus DBSCAN on two moons

PYTHON

```python
from sklearn.cluster import DBSCAN
km_m = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(Xm)
db = DBSCAN(eps=0.2, min_samples=5).fit(Xm)
print("KMeans ARI on moons:", round(adjusted_rand_score(ym, km_m), 4))
print("DBSCAN ARI on moons:", round(adjusted_rand_score(ym, db.labels_), 4))
print("DBSCAN clusters found:", len(set(db.labels_[db.labels_ >= 0])))
print("Points labelled noise:", int((db.labels_ < 0).sum()))
```

1 **KMeans with k equals 2**

The right number of clusters, but the wrong shape assumption.

2 **DBSCAN**

eps 0.2 and min_samples 5. No number of clusters is given.

3 **Compare**

ARI for both, then DBSCAN's cluster count, excluding noise, and its noise count.

Output: density beats distance to a centre

```
OUTPUT
KMeans ARI on moons: 0.2738
DBSCAN ARI on moons: 1.0
DBSCAN clusters found: 2
Points labelled noise: 0
```

KMeans: 0.274
It cuts the moons with a straight line, mixing the tips of each moon into the wrong cluster.

DBSCAN: 1.0
Perfect recovery of both curved shapes.

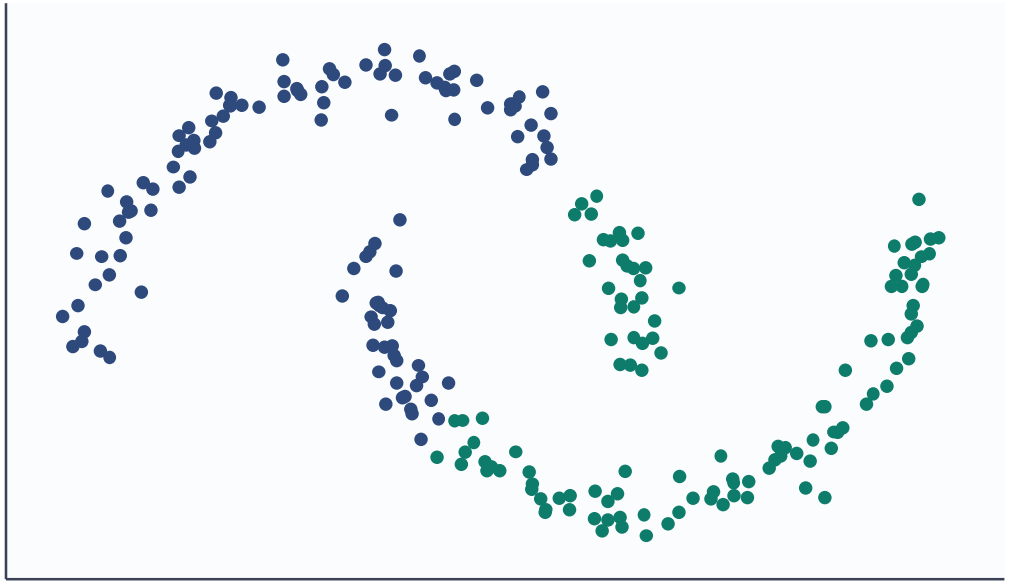
2 clusters found
Discovered from the data, never specified.

0 noise points
With this clean data and eps, every point is in a dense region.

Illustration: the moons through two algorithms

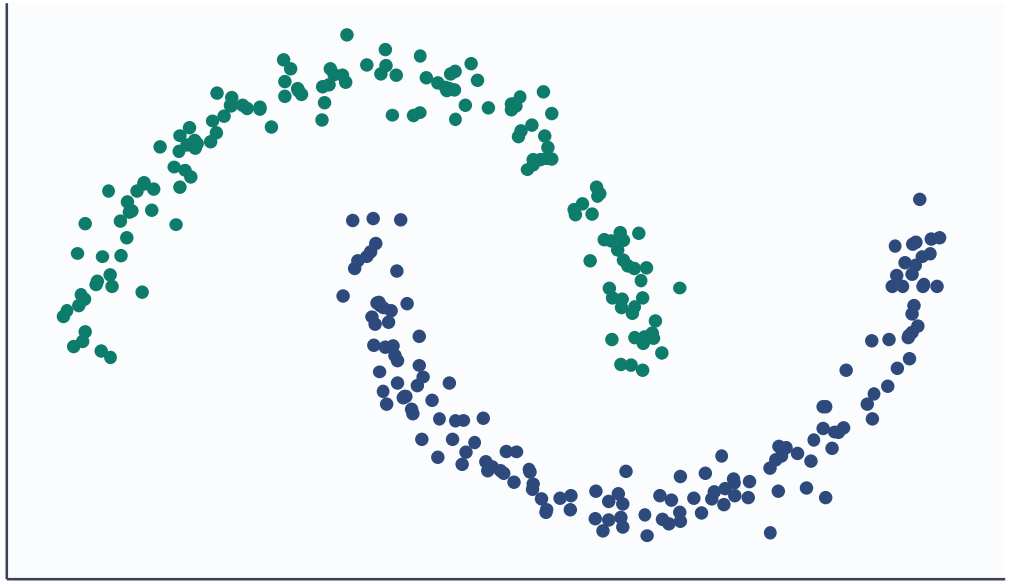
The real cluster assignments from the code above.

KMeans, k equals 2



ARI 0.274
A straight boundary halfway between two centres slices both moons.

DBSCAN, eps 0.2



ARI 1.000
Clusters follow the chains of dense points along each moon.

How sensitive is DBSCAN to eps?

PYTHON

```
print("  eps  clusters  noise")
for eps in [0.05, 0.1, 0.15, 0.2, 0.3, 0.5]:
        lab = DBSCAN(eps=eps, min_samples=5).fit_predict(Xm)
        clusters = len(set(lab[lab >= 0]))
        print(f"{eps:>5}  {clusters:>8}  {int((lab < 0).sum()):>5}")
```

1. **Six radii**
   From 0.05 to 0.5 with min_samples fixed at 5.
2. **Count clusters and noise**
   Noise points carry the label minus 1, so non negative labels are clusters and negative labels are noise.

Output: too small fragments, too large merges

OUTPUT

eps	clusters	noise
0.05	28	218
0.1	4	10
0.15	2	0
0.2	2	0
0.3	1	0
0.5	1	0

eps 0.05: 28 clusters, 218 noise
The radius is smaller than typical gaps between points, so most points look isolated.

eps 0.1: 4 clusters
Closer, but each moon is still broken in two.

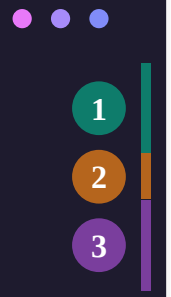
eps 0.15 to 0.2: correct
Two clusters, no noise.

eps 0.3 and above: 1 cluster
The radius now bridges the gap between the moons and everything merges.

Choosing eps with nearest neighbour distances

PYTHON

```python
from sklearn.neighbors import NearestNeighbors
dist, _ = NearestNeighbors(n_neighbors=5).fit(Xm).kneighbors(Xm)
kth = np.sort(dist[:, 4])
for q in [50, 80, 90, 95, 99]:
    print(f"{q}th percentile of 5th neighbour distance: {np.percentile(kth, q):.3f}")
```



1 Neighbour distances

For every point, find the distances to its 5 nearest points, the first being itself at distance 0.

2 The fifth neighbour

Column index 4 is the farthest of the five, matching min_samples of 5. Sorting gives the k distance curve.

3 Percentiles

Where most points' fifth neighbour lies tells us a radius that keeps most points in dense regions.

Output: almost every point has 5 neighbours within 0.15

OUTPUT

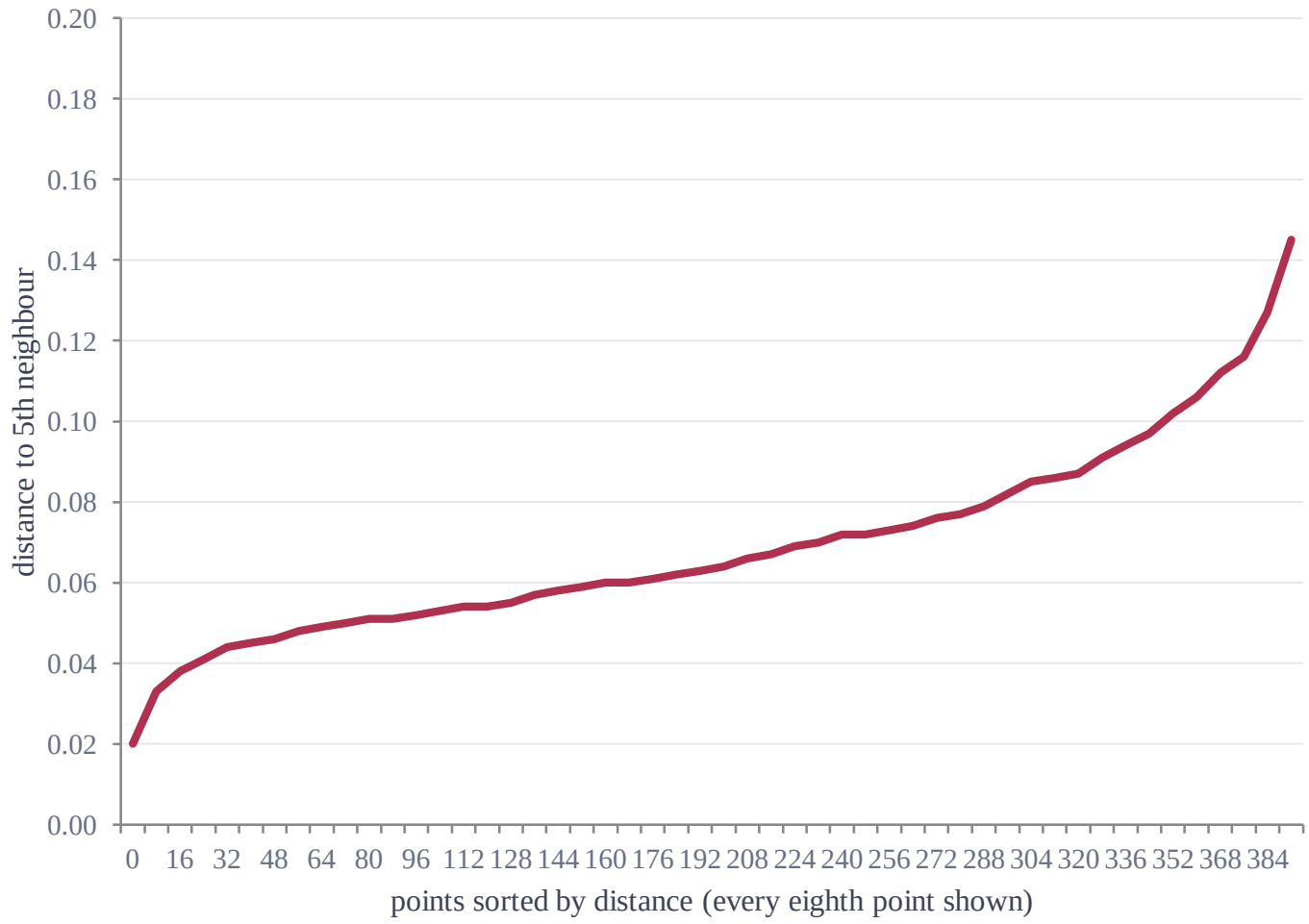
```
50th percentile of 5th neighbour distance: 0.064
80th percentile of 5th neighbour distance: 0.087
90th percentile of 5th neighbour distance: 0.105
95th percentile of 5th neighbour distance: 0.118
99th percentile of 5th neighbour distance: 0.149
```

Median 0.064
Half of all points have five neighbours within 0.064.

95th percentile 0.118
An eps just above this keeps 95 percent of points as potential cores.

99th percentile 0.149
This explains why eps 0.15 produced zero noise. Choose eps near the knee of this curve, not far beyond it.

Chart: the sorted fifth neighbour distance curve



1. **A slow climb**
   Most points have their fifth neighbour within a tight band of distances.

2. **The knee**
   At the far right the curve bends upward: those are the sparsest points, candidates for noise.

3. **Heuristic**
   Set eps near the knee. Here that is about 0.12 to 0.15, consistent with the sweep.

Clustering real data: 178 Italian wines

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
wine = load_wine()
Wz = StandardScaler().fit_transform(wine.data)
wk = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Wz)
print("Wine samples:", wine.data.shape)
print("ARI versus cultivar:", round(adjusted_rand_score(wine.target, wk), 4))
pca = PCA(n_components=2).fit(Wz)
print("Variance explained by 2 components:", pca.explained_variance_ratio_.round(3))
print(pd.crosstab(wk, wine.target, rownames=["cluster"], colnames=["cultivar"]))
```

1 The wine data

13 chemical measurements of wines from three cultivars, shipped with scikit learn.

2 Scale and cluster

Measurements span very different ranges, so scaling is essential before KMeans with k equal to 3.

3 Validate

ARI against the cultivar, the variance captured by two principal components for plotting, and a cross tabulation.

Output: chemistry alone rediscovers the grape variety

OUTPUT

```
Wine samples: (178, 13)
ARI versus cultivar: 0.8975
Variance explained by 2 components: [0.362 0.192]
cultivar    0   1   2
cluster
0           0  65   0
1           0   3  48
2          59   3   0
```

ARI 0.898
Without ever seeing the labels, KMeans groups wines largely by cultivar.

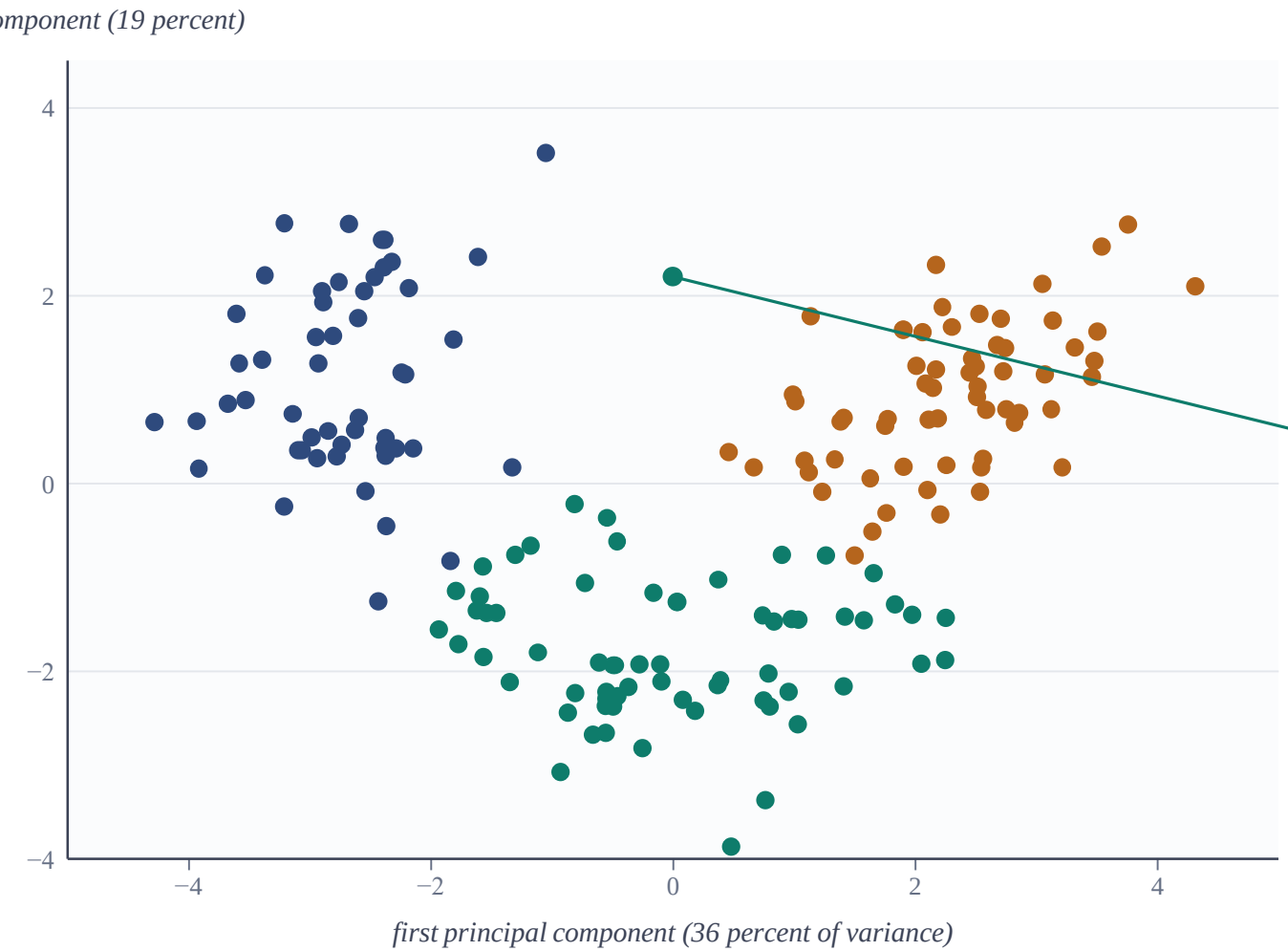
Two components: 55 percent
Two principal components capture 36 plus 19 percent of the variation, enough for a faithful picture.

Cluster 0: all 65 from cultivar 1
A pure cluster.

Six wines misplaced
Cultivar 1 is split 65, 3 and 3: a few of its wines resemble the other varieties chemically.

Illustration: wine clusters in two principal components

Each dot is one wine, positioned by the first two principal components of its 13 scaled measurements, coloured by KMeans cluster.



Three clear clouds

The clusters occupy distinct regions even in this two dimensional shadow of 13 dimensions.

A curved arrangement

The middle cluster sits between the others, where the few misplaced wines are found.

PCA is for viewing

Clustering used all 13 scaled features. PCA only flattens them for the picture.

Assigning new customers to segments

PYTHON

```
scaler = StandardScaler().fit(cust[feats])
seg_model = KMeans(n_clusters=3, n_init=10, random_state=0).fit(scaler.transform(cust[feats]))
newcomers = pd.DataFrame({"income_gbp": [52000, 31000], "visits_per_month": [11, 2],
                          "days_since_visit": [5, 40]})
print("Assigned clusters:", seg_model.predict(scaler.transform(newcomers)))
print("Distance to each centroid:")
print(seg_model.transform(scaler.transform(newcomers)).round(2))
```

1 2 3

1 Keep the scaler

New customers must be scaled with the same statistics the model was trained on, so the scaler is stored alongside the model.

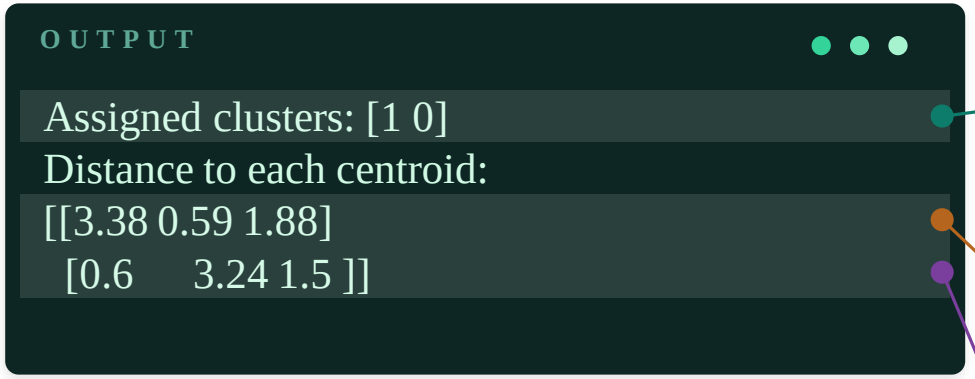
2 Two newcomers

One visits often and recently; one rarely and long ago.

3 predict and transform

predict returns the nearest centre; transform returns the distance to every centre, a useful measure of fit.

Output: one loyal customer, one lapsing



Clusters 1 and 0
From the profile slide, cluster 1 is Loyal and cluster 0 is Lapsing.

Newcomer 1
Distance 0.59 to the loyal centre against 1.88 and 3.38 to the others: a confident assignment.

Newcomer 2
Distance 0.6 to the lapsing centre. A win back offer is appropriate.

Choosing a clustering algorithm

	KMeans	Agglomerative	DBSCAN
Need to choose	k, the number of clusters	Linkage, then k or a cut height	eps and min_samples
Cluster shapes	Round, similar sizes	Depends on linkage; single follows chains	Any shape
Handles noise	No, every point is assigned	No, every point is assigned	Yes, labels outliers as noise
Scales to large data	Very well	Poorly, all pairwise distances	Well with spatial indexing
Assigns new points	Yes, predict	No, refit needed	No, refit needed
Best for	Segmentation with a known k	Exploring hierarchy, small data	Irregular shapes, anomaly detection

Common pitfalls in cluster analysis

1 Forgetting to scale

As the customer example showed, an unscaled large unit feature can single handedly decide every cluster.

2 Irrelevant features

Every feature votes on similarity. Noise features blur real groups; choose features for the question being asked.

3 Finding clusters in noise

Clustering algorithms always return clusters, even on random data. Check silhouette and stability before believing them.

4 Over trusting metrics

Silhouette favours round, separated clusters and can rank a meaningful grouping below a useless one.

5 Comparing label numbers

Cluster 0 in one run is not cluster 0 in another. Use ARI or cross tabulations to compare runs.

6 Categorical data

Euclidean distance on one hot columns behaves poorly. Consider specialised methods or dissimilarity measures for mixed data.

A cluster analysis workflow

1

Purpose

Decide what the groups will be used for; it determines the features.

2

Prepare

Select features, handle outliers, encode and scale.

3

Explore k

Elbow, silhouette and Davies Bouldin, or a dendrogram.

4

Fit and compare

Try KMeans, a linkage or two and DBSCAN where shapes may be irregular.

5

Validate

Stability across seeds and samples; external outcomes.

6

Profile and deploy

Name segments, report original units, keep the scaler with the model.

Check your understanding

KMeans on raw features gives an adjusted Rand index of 0.01 against known segments. After scaling it gives 0.98. Why?

A Scaling adds information

B A large unit feature dominated distances

C KMeans needs integers

D ARI ignores unscaled data

Answer B and why

Distances add up squared differences across features, so income measured in thousands of pounds swamped visits per month. Scaling gives every feature comparable influence, letting the behavioural differences that define the segments drive the clustering.

Check your understanding

Your data contain two long curved groups and scattered outliers. Which algorithm is most suitable?

A KMeans

B Ward linkage

C DBSCAN

D Logistic regression

Answer C and why

DBSCAN follows dense chains of points into any shape and labels isolated points as noise. On the moons data it achieved an ARI of 1.0 where KMeans managed 0.274. Logistic regression is supervised and cannot cluster at all.

Bringing it all together

Six sections, one way of thinking: define the question, respect the data, measure honestly, and explain what the model learned.

1 Foundations

Split before anything, pipelines against leakage, baselines to beat, cross validation for choices, the bias variance trade off everywhere.

2 Linear regression

A weighted sum with interpretable coefficients. Diagnose with residuals, protect against collinearity with ridge, bend with polynomial features.

3 Logistic regression

A weighted sum read as log odds and squashed into probabilities. Choose thresholds and metrics from the cost of each error.

4 Decision trees

Greedy impurity reducing splits that read as rules, capture interactions, ignore scaling and overfit without limits.

5 Tuning

Validation curves, pruning, grid and random search, then one honest test evaluation and nested cross validation for reporting.

6 Clustering

Distance or density defines similarity, scaling decides outcomes, and usefulness is the final judge of a segmentation.

Every headline result in this tutorial

Section	Model	Data	Key result
Foundations	Scaled logistic pipeline	Exam pass or fail	Accuracy 0.94 against a 0.70 baseline
Linear	LinearRegression	800 simulated houses	Test R squared 0.927, MAE about 14,700 pounds
Linear	Polynomial degree 2	Fuel economy curve	Test R squared 0.768 against negative for a line
Logistic	Scaled LogisticRegression	Breast cancer	Test accuracy 0.986, ROC AUC 0.998
Trees	Unrestricted tree	Breast cancer	Training 1.000, test 0.923: overfitting
Tuning	Grid searched tree	Breast cancer	Test 0.951; nested estimate 0.928
Clustering	KMeans, scaled	Customer segments	ARI 0.98 against 0.01 unscaled
Clustering	DBSCAN	Two moons	ARI 1.000 against 0.274 for KMeans

Where to go next

1

Practise

Rerun every snippet, then change one setting at a time and predict the output before running it.

2

Ensembles

Random forests and gradient boosting, built from the trees you now understand.

3

Feature engineering

Interactions, transformations and domain features often beat model changes.

4

Model explanation

Permutation importance, partial dependence and SHAP values for any model.

5

Probabilistic clustering

Gaussian mixture models give soft cluster memberships.

6

Production

Saving pipelines, monitoring drift and retraining on schedule.